ICPC

icpcBoard

ICPC Algorithm Reference

竞赛算法模板与调试速查

Nara-F

目录

1 竞赛常用内容 6

1.1 通用代码骨架与多测清空 6
1.2 数据范围与复杂度速判 7
1.3 快速输入输出 7
1.4 编译、运行与 Sanitizer 8
1.5 调试宏与断言 9
1.6 随机数与数据生成 9
1.7 暴力程序与正确性基准 10
1.8 自动对拍与失败保留 11
1.9 文件与标准流切换 12
1.10 WA、RE、TLE、MLE 与 UB 排查 13
1.11 边界数据检查清单 13
1.12 整数溢出与浮点精度 14
1.13 赛中提交与三人协作流程 15
1.14 Python 调试与随机对拍 15
1.15 中级数据结构学习路线 17

2 STL 库使用 20

2.1 顺序容器：`vector` 与 `array` 20
2.2 `string` 20
2.3 `pair`、`tuple` 与结构化绑定 21
2.4 两端序列与适配器：`deque`、`stack`、`queue` 21
2.5 堆：`priority_queue` 22
2.6 有序集合：`set`、`multiset` 22
2.7 有序映射：`map` 22
2.8 哈希容器：`unordered_set`、`unordered_map` 23
2.9 `bitset` 24
2.10 排序、二分与边界 24
2.11 去重、排列与区间算法 25
2.12 数值算法与 Lambda 25
2.13 迭代器与半开区间 25

3 基础算法与搜索 27

3.1 复杂度与枚举 27
3.2 前缀和与差分 28
3.3 二分查找与二分答案 29
3.4 双指针与滑动窗口 30
3.5 离散化 32
3.6 分治与归并计数 33
3.7 贪心与区间调度 34
3.8 DFS 深度优先搜索 35
3.9 BFS 广度优先搜索 36
3.10 回溯与剪枝 37
3.11 折半搜索 38
3.12 0-1 BFS 39

3.13 常用构造题思路 40
4 动态规划 42
4.1 DP 入门：状态、转移与顺序 42
4.2 0-1 背包 44
4.3 完全背包与多重背包 46
4.4 最长上升子序列（LIS） 48
4.5 最长公共子序列（LCS） 50
4.6 区间 DP 52
4.7 树形 DP 54
4.8 状态压缩 DP 56
4.9 数位 DP 58
4.10 DAG 上的 DP 60
4.11 编辑距离 63
4.12 树形 DP 与换根 64
4.13 概率与期望 DP 66
5 数据结构 69
5.1 栈与队列 69
5.2 单调栈 71
5.3 单调队列 73
5.4 堆与优先队列 75
5.5 并查集 76
5.6 树状数组 78
5.7 线段树（区间加、区间和） 81
5.8 ST 表（静态 RMQ） 84
5.9 分块（区间加、区间和） 86
5.10 树状数组：区间加与区间和 88
5.11 DFS 序与子树展平 90
5.12 树链剖分 92
5.13 线段树：区间加、赋值与求和 95
5.14 DSU on Tree（树上启发式合并） 99
5.15 单点修改线段树与自定义合并 103
5.16 启发式合并（Small-to-Large） 106
5.17 线段树合并（Segment Tree Merge） 108
5.18 主席树（可持久化线段树） 112
5.19 虚树 116
5.20 带权并查集 121
5.21 离线树状数组 123
5.22 莫队算法 127
5.23 笛卡尔树 131
5.24 李超线段树 133
5.25 差分 + GCD 线段树 137
5.26 扫描线 + 线段树 140
5.27 线段树 + 矩阵乘法 143
5.28 线段树上二分 146
5.29 倍增 + 单调栈 148
5.30 可撤销并查集 150

5.31 线段树分治（动态连通性） 153
5.32 P2 进阶：树上莫队、Segment Tree Beats 与 LCT 156
6 图论 162
6.1 拓扑排序 162
6.2 Dijkstra 单源最短路 163
6.3 Bellman–Ford 与负环判断 166
6.4 Floyd 全源最短路 168
6.5 Kruskal 最小生成树 170
6.6 二分图判定 172
6.7 二分图最大匹配（增广路） 174
6.8 强连通分量（Tarjan） 176
6.9 割点与桥（Tarjan） 178
6.10 最近公共祖先（倍增 LCA） 180
6.11 树的直径 183
6.12 最大流（Dinic） 185
6.13 欧拉路径与回路 187
6.14 2-SAT 189
6.15 最小费用最大流 191
6.16 树上差分 + LCA 194
7 数学 197
7.1 GCD 与扩展 GCD 197
7.2 快速幂 198
7.3 素数筛 200
7.4 质因数分解 202
7.5 欧拉函数 203
7.6 模逆元 205
7.7 线性同余方程 206
7.8 中国剩余定理（非互质模数） 208
7.9 组合数 210
7.10 容斥原理 211
7.11 矩阵快速幂 213
7.12 高斯消元 215
7.13 公平组合游戏与 SG 函数 217
7.14 质因数分解、约数个数与约数和 219
8 字符串 221
8.1 字符串哈希 221
8.2 前缀函数与 KMP 223
8.3 Z 函数（扩展 KMP） 225
8.4 Manacher 227
8.5 Trie 字典树 229
8.6 AC 自动机 231
8.7 后缀数组 234
9 计算几何 237
9.1 点、向量与基本运算 237
9.2 叉积与方向判断 239
9.3 直线交点与线段相交 240

9.4 距离与投影 242
9.5 多边形面积 244
9.6 点在多边形内 245
9.7 凸包 247
9.8 旋转卡壳：凸包直径 249
9.9 圆与直线、圆与圆 252
9.10 整数几何方向模板 254
10 题意关键词索引 256
10.1 上机、语言与 STL 256
10.2 序列、搜索、DP 与数据结构 256
10.3 图与树 258
10.4 数学、字符串与几何 259
10.5 五秒模型核对 260

第一章

竞赛常用内容

1.1 通用代码骨架与多测清空

问题：将单组逻辑放进 `solve()`，入口只负责读取测试组数并循环调用，便于复用和隔离状态。多测总成本是各组复杂度之和；优先使用局部容器，谨慎清空大型全局数组。

思路：

- 单测时保留 `int testCases = 1`，不要读取不存在的测试组数。
- 多测时取消读取 `testCases` 的注释；先确认题面是否给出 T。
- 优先把数组、图、DP 表定义为 `solve()` 内局部变量，每组自动重新构造。
- 大型全局数组只清空本组访问过的范围；时间戳法适合频繁清空布尔或编号状态。
- 每组输出格式统一在 `solve()` 内处理，题目要求时加 `Case #x:`。

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：无；从标准输入读取一组数据。返回：无。效果：计算并写出当前测试用例答案。
void solve() {
    int n;
    cin >> n;

    // 状态尽量放在 solve 内：每组测试结束后自动销毁，不会串组。
    vector<ll> a(n + 1); // 本模板约定输入下标为 1..n
    for (int i = 1; i <= n; ++i) cin >> a[i];

    // 在这里编写单组答案。
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int testCases = 1;
    // cin >> testCases; // 仅当题面明确给出测试组数时启用
    while (testCases--) solve();
    return 0;
}
```

易错点：

1. 把样例中的多组数据误认为题目输入含 T，会从第一项开始读错。
2. `vector::clear()` 只把大小变为 0；重新 `resize` 后不能假设旧内存内容有意义。
3. 全局邻接表、答案、计数器、时间戳和静态局部变量都可能跨组残留。
4. 对最大数组每组完整 `memset`，可能把本可通过的算法拖成 TLE。

参考：

- OI Wiki：C++ 语法基础
- OI Wiki：常见错误

1.2 数据范围与复杂度速判

问题： 依据 n, m, W, T 和总和约束估算时间与峰值内存，再选择算法。清空、建图、排序、递归栈和多测开销都要计算；常数和评测机差异使经验表只能初筛。

思路：

- 多测必须估算 $\sum n$、$\sum m$，不能只看单组上限。
- 图算法用 $n + m$ 估算；背包看 nW；状压看 $n2^n$ 或 $n^2 2^n$。
- 内存按“元素个数 × 单元素字节数”计算，并加上容器、递归栈和多份数组。
- 约 10^8 次简单整数操作已可能接近常见时限；哈希、字符串复制、堆操作常数更大。
- 用最坏数据本地计时只能辅助判断，不能替代理论复杂度。

公式与速查：

```
常见候选（只作一秒级 C++ 的粗略起点）：
n <= 20           O(2^n)、O(n * 2^n)
n <= 40           O(2^(n/2)) 折半搜索
n <= 500          O(n^3) 需关注常数
n <= 5,000        O(n^2) 需关注常数和内存
n <= 200,000      O(n log n)、O(n sqrt(n)) 视常数而定
n,m <= 1,000,000 O(n)、O(n+m)

内存 MiB = 元素个数 * sizeof(元素) / 1024 / 1024
例如：2^20 * 20 个 long long 约为 160 MiB。
```

易错点：

1. $O(n^2)$ 的二维 `long long` 数组常先 MLE，再谈不上 TLE。
2. 把取模、平衡树、哈希或大整数运算都当成一个同常数的“基本操作”。
3. 忽略清空数组、建图、排序或输入输出，它们也计入总复杂度。
4. 只根据样例规模选算法；评测按题目给出的最大范围构造数据。

参考：

- OI Wiki：复杂度简介
- OI Wiki：常见错误

1.3 快速输入输出

问题： 非交互题在首次 IO 前关闭同步并解绑 `cin`，统一使用 C++ 流、以 `\n` 换行。交互题要及时刷新；`getline` 与格式化读取混用时需处理残留换行。

思路：

- 在任何 IO 前调用 `ios::sync_with_stdio(false); cin.tie(nullptr);`。
- 关闭同步后统一使用 `cin/cout`，不要混用 `scanf/printf`。
- 普通换行用 `'\n'`；`endl` 会额外刷新缓冲区。
- 交互题必须在每次询问后 `cout << flush`，不能照搬非交互写法。
- 输出大量元素时先拼接或顺序写入，不要在循环中无必要地刷新。

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false); // 必须在第一次 cin/cout 操作前调用
    cin.tie(nullptr);            // 非交互题解除 cin 对 cout 的自动刷新

    int n;
    cin >> n;
    vector<long long> a(n);
```

```
    for (long long& value : a) cin >> value;

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    cout << '\n'; // 不需要立即显示时，用换行而不是 endl
    return 0;
}
```

易错点：

1. 关闭同步后混用 C 与 C++ 输入或混用两套输出，顺序可能异常。
2. 交互题忘记刷新会等待超时；普通题频繁 `endl` 又可能拖慢输出。
3. `cin >> s` 跳过空白；要读取含空格整行时使用 `getline`，并处理残留换行。
4. 输入失败后变量不会得到有效新值；读到 EOF 的题应以 `while (cin >> x)` 控制。

参考：

- OI Wiki：读入、输出优化
- OI Wiki：C++ 输入输出

1.4 编译、运行与 Sanitizer

问题：用 `-O2` 版模拟提交，用带警告和 Sanitizer 的调试版查越界与未定义行为。先修警告，再测样例、边界和随机数据；检查版较慢，不用于时限评估。

思路：

- 普通版开启 C++17、优化和警告；让本地标准尽量接近评测机。
- 调试版加入 `-g`、`-DDEBUG`、`-fsanitize=address,undefined` 和栈帧选项。
- 用输入重定向复现测试，必要时用 `time` 粗测最坏数据。
- 先修编译警告，再运行样例、边界数据和随机对拍。
- Sanitizer 报告从第一处错误和调用栈开始看；后续错误可能只是连锁反应。

命令：

```
# 接近提交环境的优化版
g++ main.cpp -std=c++17 -O2 -Wall -Wextra -Wshadow -o main

# 本地查越界和未定义行为；不要用它测正式时限
g++ main.cpp -std=c++17 -O1 -g -DDEBUG \
  -fsanitize=address,undefined -fno-omit-frame-pointer -o main_dbg

./main < sample.in > sample.out
./main_dbg < fail.in
time ./main < maximal.in > /dev/null
```

易错点：

1. 只在 `-O0` 下测试；未定义行为可能在 `-O2` 下表现不同。
2. 把 Sanitizer 的额外时间、内存开销当成正式程序复杂度。
3. 忽略警告中的窄化、符号比较、变量遮蔽和未初始化使用。
4. 本地编译器版本、标准或整数宽度与评测机不同，造成“本地过、提交错”。

参考：

- OI Wiki：命令行编译与调试
- OI Wiki：编译优化

1.5 调试宏与断言

问题： 把日志集中写到 `cerr`，用 `DEBUG` 控制是否编译；用 `assert` 检查下标、区间和内部不变量。断言不能代替合法输入分支，也不要把必要操作写进断言。

思路：

- 本地用 `-DDEBUG` 开启日志，提交代码不定义 `DEBUG`。
- 在二分边界、图下标、DP 状态范围和数据结构不变量处放断言。
- 错误诊断信息写 `cerr`；正式答案只写 `cout`。
- 先打印最小必要状态：位置、关键变量、容器大小，不要倾倒全部大数组。
- `assert` 失败表示实现或推理错误；若条件来自不可信输入，应正常分支处理。

```cpp
#include <bits/stdc++.h>
using namespace std;

#ifdef DEBUG
template <class... Args>
// 参数：任意可流输出的调试值。返回：无。效果：按顺序写入标准错误。
void debugLog(Args&&... args) {
    // 折叠表达式按参数顺序输出，调试格式集中在一个入口维护。
    ((cerr << forward<Args>(args) << ' '), ...);
    cerr << '\n';
}
#define DBG(...) debugLog("[", #__VA_ARGS__, "] =", __VA_ARGS__)
#else
#define DBG(...) ((void)0)
#endif

// 参数：prefix[0..n] 与 1-based 闭区间 [left,right]。返回：区间和。效果：断言边界并写调试日志。
long long rangeSum(const vector<long long>& prefix, int left, int right) {
    // 调用约定：prefix[0..n]，查询原数组闭区间 [left, right]。
    assert(1 <= left && left <= right);
    assert(right < static_cast<int>(prefix.size()));
    DBG(left, right, prefix[right], prefix[left - 1]);
    return prefix[right] - prefix[left - 1];
}
```

易错点：

1. 调试输出写入 `cout` 会造成 WA；提交前也不要靠手工搜索删除日志。
2. 宏参数含逗号表达式或副作用时容易产生意外；`DBG(++x)` 会真的修改 `x`。
3. 定义 `NDEBUG` 后标准 `assert` 会被移除，不能把必要计算写进断言表达式。
4. 日志本身可能使 TLE 或改变时序，只保留能缩小错误范围的信息。

参考：

- OI Wiki：常见技巧
- OI Wiki：命令行编译与调试

1.6 随机数与数据生成

问题： 用 `<random>` 的引擎和分布生成可控测试，并保存种子以复现失败。均匀整数范围应由分布指定；随机化不能替代正确性证明或复杂度分析。

思路：

- 从命令行读取固定种子；未提供时才使用时钟种子，并把种子输出到 `cerr`。
- 用 `uniform_int_distribution<ll>(left,right)`，不要用 `rng() % range` 追求严格均匀。
- 随机排列使用 `shuffle`；C++17 已移除 `random_shuffle`。

- 生成图时明确是否允许自环、重边、断开；树可令每个 `v` 随机连接 `[1,v-1]`。
- 纯随机以外，固定加入最小值、最大值、全相等、单调、星形和链形等结构。

```cpp
#include <bits/stdc++.h>
using namespace std;

using ull = unsigned long long;

int main(int argc, char** argv) {
    // 对拍时由脚本传入 seed；同一 seed 必须生成同一测试。
    ull seed = argc >= 2 ? stoull(argv[1])
                         :
chrono::steady_clock::now().time_since_epoch().count();
    mt19937_64 rng(seed);
    cerr << "seed = " << seed << '\n';

    // 参数：闭整数区间 [left,right]。返回：均匀随机整数。效果：推进 mt19937_64 状态。
    auto randomInt = [&](long long left, long long right) {
        return uniform_int_distribution<long long>(left, right)(rng);
    };

    int n = static_cast<int>(randomInt(1, 12)); // 小规模便于暴力校验
    vector<int> permutation(n);
    iota(permutation.begin(), permutation.end(), 1);
    shuffle(permutation.begin(), permutation.end(), rng); // 原地打乱，元素仍是
1..n

    cout << n << '\n';
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << permutation[i];
    }
    cout << '\n';
    return 0;
}
```

易错点：

1. 失败后不记录种子，导致错误数据无法再次生成。
2. `rand() * rand()` 可能溢出且分布差；`rng() % k` 也可能有模偏差。
3. 只造平均随机数据，覆盖不到空集、重复值、极值和退化结构。
4. 生成器自身不满足题面约束，会让对拍结论失去意义。

参考：

- OI Wiki：随机函数
- OI Wiki：出题·造数据

1.7 暴力程序与正确性基准

问题：对小规模输入编写结构简单、容易确认的暴力解，作为优化算法的差分基准。暴力可指数级，但必须覆盖合法边界并与被测程序使用相同输入定义。

思路：

- 先写数学定义最直接对应的判定，再写优化程序；两者避免复用核心函数。
- 暴力只需支持生成器的小范围，不必支持题面最大数据。
- 返回完整答案；多解或浮点题需写等价比较器，不能简单文本 `diff`。
- 对优化程序的每个可疑性质，增加能打破该性质的构造数据。
- 发现差异后先保存输入，再分别检查暴力与正解，不能默认暴力必对。

```cpp
#include <bits/stdc++.h>
```

```cpp
using namespace std;

using ll = long long;

// 示例基准：枚举所有子集，求不超过 capacity 的最大价值。
// 仅用于 n 很小的对拍；下标统一为 0..n-1。
// 参数：0-based 平行重量/价值数组及容量。返回：子集枚举得到的最大价值。效果：只适用于 n<=24 的小数据，O(2^n n)。
ll bruteKnapsack(const vector<int>& weight,
                 const vector<ll>& value,
                 int capacity) {
    int n = static_cast<int>(weight.size());
    assert(n <= 24); // 防止误拿暴力跑正式上限
    ll answer = 0;
    for (unsigned long long mask = 0; mask < (1ULL << n); ++mask) {
        // 每个二进制位独立表示一件物品是否选中。
        int totalWeight = 0;
        ll totalValue = 0;
        for (int i = 0; i < n; ++i) {
            if (mask & (1ULL << i)) {
                totalWeight += weight[i];
                totalValue += value[i];
            }
        }
        if (totalWeight <= capacity) answer = max(answer, totalValue);
    }
    return answer;
}
```

易错点：

1. 暴力与正解复制了同一个转移或辅助函数，二者可能稳定地产生同一错误。
2. 暴力数组越界、溢出或误解题意，会把正确程序判成错误。
3. 枚举上界用 `1 << n` 可能溢出，应使用 `1ULL << n` 并限制 n。
4. 只比较答案值而忽略无解、多解、顺序无关或误差判定规则。

参考：

- OI Wiki：枚举
- OI Wiki：常见技巧·对拍

1.8 自动对拍与失败保留

问题：生成器、暴力程序和正式解在同一输入上运行并比较输出，可快速发现反例。对拍成本取决于单轮三程序耗时；保存种子和失败输入，先缩小反例再定位。

思路：

- 先分别编译 `gen.cpp`、`brute.cpp`、`main.cpp`，保证三者无编译警告。
- 脚本给生成器传递递增种子，任一步骤非零退出都视为失败。
- 用 `diff -Z -B` 只在题目允许忽略空白差异时比较；严格格式题直接用 `diff`。
- 失败时保留 `fail.in`、双方输出和 seed，再用 Sanitizer 单独复现。
- 多解、集合输出、浮点误差题应改写 checker，把两个输出都按题意验证。

命令：

```sh
#!/usr/bin/env bash
set -u

# 用法：先编译出 ./gen ./brute ./main，再执行 bash stress.sh。
for seed in $(seq 1 100000); do
  ./gen "$seed" > stress.in || exit 1
```

```
  ./brute < stress.in > brute.out || exit 1
  ./main < stress.in > main.out || exit 1

  if ! diff -Z -B brute.out main.out > /dev/null; then
    cp stress.in fail.in
    echo "Wrong answer, seed = $seed"
    diff -u brute.out main.out
    exit 1
  fi
done

echo "Accepted all generated tests"
```

易错点：

1. 程序 RE 后仍比较旧输出文件，可能误报一致；脚本必须检查退出码。
2. `diff` 不理解“任意合法方案”；此类题必须按题意写 checker。
3. 对拍全是同一种随机分布，跑再多轮也未必覆盖退化边界。
4. 并行对拍共用同名临时文件会互相覆盖；每个进程需独立目录或文件名。

参考：

- OI Wiki：常见技巧·对拍
- OI Wiki：出题·造数据

1.9 文件与标准流切换

问题：本地可在宏控制下重定向输入输出文件，提交时仍使用标准流。检查运行目录和文件名；重定向应发生在任何 IO 之前，避免评测机找不到文件。

思路：

- 本地首选 `./main < sample.in > sample.out`，源码无需修改。
- 比赛明确要求文件 IO 时，严格使用题面给出的文件名和大小写。
- 若使用条件编译，只把本地文件切换放在 `#ifndef ONLINE_JUDGE` 中。
- 检查 `freopen` 返回值；本地文件不存在时立即报错。
- 调试信息仍写 `cerr`，它不会进入重定向到答案文件的 `stdout`。

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
#ifndef ONLINE_JUDGE
    // 本地可用；若评测平台不定义 ONLINE_JUDGE，提交前应删除这段。
    if (freopen("input.txt", "r", stdin) == nullptr) return 1;
    if (freopen("output.txt", "w", stdout) == nullptr) return 1;
#endif

    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long value;
    while (cin >> value) cout << value << '\n';
    return 0;
}
```

易错点：

1. 假设所有 OJ 都定义 `ONLINE_JUDGE`；不确定时不要把 `freopen` 留在提交代码里。
2. 文件名大小写、扩展名或相对路径错误，导致读不到输入。
3. 把输入输出文件写反，或用写模式打开输入文件造成数据被截断。
4. 使用 shell 重定向时把 `2>` 与 `>` 混淆，调试信息进入答案文件。

参考：

- OI Wiki：常见技巧
- OI Wiki：C++ 输入输出

1.10 WA、RE、TLE、MLE 与 UB 排查

问题：按现象分别检查建模/边界、非法访问、渐进复杂度/常数、峰值内存和未定义行为。先用最小失败输入复现，再结合编译警告、Sanitizer 与计时定位。

思路：

- CE：在与评测相同标准下编译，查看第一条错误，确认提交语言和入口函数。
- WA：手算最小反例，核对输入含义、开闭区间、无解输出、取模与浮点格式。
- RE：用失败输入跑 Sanitizer，检查数组下标、递归深度、迭代器和除数。
- TLE/OLE：确认循环会推进、输出量有限，再重新计算最坏复杂度和总测试规模。
- MLE：计算峰值而非单数组大小，关注 `vector<vector<...>>`、DP 多份表和按值传参。

排查命令。

```
# 1. 带警告编译；2. 用检查版复现；3. 查看峰值时间和内存（GNU/Linux）。
g++ main.cpp -std=c++17 -O2 -Wall -Wextra -Wshadow -o main
g++ main.cpp -std=c++17 -O1 -g \
  -fsanitize=address,undefined -fno-omit-frame-pointer -o main_dbg
./main_dbg < fail.in
/usr/bin/time -v ./main < maximal.in > /dev/null
```

易错点：

1. 看到 RE 就盲目加大数组；根因也可能是负下标、栈溢出或除零。
2. 随机修改代码后失败样例消失，就误以为问题已修复；UB 可能只改变表现。
3. 只测样例，未覆盖最大规模、不可达、重复边、负数和空答案。
4. 诊断 TLE 时只看主算法，忽略重复清空、容器复制和调试输出。

参考：

- OI Wiki：题型概述·评测结果
- OI Wiki：常见错误
- OI Wiki：编译优化·未定义行为

1.11 边界数据检查清单

问题：根据约束逐项构造最小、最大、重复、单调、全相等、不可达和答案极值等输入。边界测试用于验证实现假设，复杂度仍需单独证明。

思路：

- 规模：$n = 0$（若允许）、$n = 1$、最小合法值、最大 n, m, T、总规模上限。
- 数值：全 0、全负、全相等、最小/最大值、答案刚好越过 `int`、乘法溢出。
- 序列：递增、递减、回文、重复集中、答案在首尾、空答案和唯一答案。
- 图树：孤点、断开、重边、自环（若允许）、链、星、完全图、源点即终点。
- 区间：单点、全区间、端点相接、左右边界、开闭含义；多测检查上一组残留。

检查清单。

```
[ ] 手算并通过最小合法输入
[ ] 测试空答案、无解、唯一解、多解
[ ] 测试全相等、严格递增、严格递减
[ ] 测试 0、负数、最大值与 long long 溢出边缘
[ ] 测试下标 0/1、n-1/n、区间端点和源点等于终点
[ ] 测试断开图、重边、自环、链、星（按题面允许范围）
```

```
[ ] 测试两组连续输入，第二组比第一组小
[ ] 最大数据下检查时间、内存和递归深度
[ ] 核对输出大小写、空格、换行、精度和 Case 编号
```

易错点：

1. 构造了题面不允许的数据并据此“修复”，反而破坏正确逻辑。
2. 只测 $n = 1$，未测答案在最后一个元素或最后一条边的情形。
3. 多测只用相同大小，未发现第二组变小时的残留数组内容。
4. 随机数据代替结构数据；链、星、全相等等退化形状出现概率极低。

参考：

- OI Wiki：常见错误
- OI Wiki：出题·数据与样例

1.12 整数溢出与浮点精度

问题： 根据最大中间结果选择整数类型，并在乘法前提升类型；累加、平方和及哨兵运算都要检查范围。浮点比较按误差模型选绝对/相对误差，避免随意固定 epsilon。

思路：

- 和、距离、计数优先用 `long long`；乘法写 `1LL * a * b`。
- 可能超过约 9×10^{18} 的整数中间值用 `__int128`，输出需自行转换。
- 无穷大取 `numeric_limits<ll>::max()/4`，加法前先排除不可达状态。
- 浮点比较用 `abs(a-b) <= eps * max(1,abs(a),abs(b))` 的混合误差。
- 输出按题意使用 `fixed << setprecision(k)`；几何 EPS 应结合坐标范围调整。

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using i128 = __int128_t;
using u128 = __uint128_t;

// 参数：待比较实数及相对/绝对容差 eps。返回：混合误差下是否近似相等。效果：无副作用。
bool almostEqual(long double a, long double b,
                 long double eps = 1e-12L) {
    // 同时照顾接近 0 的绝对误差和大数的相对误差。
    return fabsl(a - b) <= eps * max({1.0L, fabsl(a), fabsl(b)});
}

// 参数：有符号 128 位整数。返回：十进制文本。效果：无副作用，支持最小负值。
string toString(i128 value) {
    if (value == 0) return "0";
    bool negative = value < 0;
    // 先加一再取负，避免直接对 i128 最小值取负导致溢出。
    u128 magnitude = negative ? static_cast<u128>(-(value + 1)) + 1
                              : static_cast<u128>(value);
    string result;
    while (magnitude > 0) {
        result.push_back(static_cast<char>('0' + magnitude % 10));
        magnitude /= 10;
    }
    if (negative) result.push_back('-');
    reverse(result.begin(), result.end());
    return result;
}

// 参数：两个 long long 因子。返回：精确的 128 位乘积。效果：先扩宽再相乘，避免 64 位乘法溢出。
```

```cpp
i128 safeProduct(ll a, ll b) {
    return static_cast<i128>(a) * b; // 转换必须发生在乘法之前
}
```

易错点：

1. `long long product = a * b;` 中 `a,b` 为 `int` 时，乘法可能已溢出。
2. 有符号整数溢出是 UB，不能依赖“自然回绕”。
3. 用 `a == b` 判断计算所得浮点数，或所有问题固定使用同一个 EPS。
4. `NaN` 与任何数（包括自身）比较都不相等；异常值应用 `isfinite` 检查。

参考：

- OI Wiki：变量·整数与浮点类型
- OI Wiki：常见错误
- OI Wiki：编译优化·未定义行为

1.13 赛中提交与三人协作流程

问题： 三人协作时约定接口、变量/编号口径和提交负责人，避免多人同时改同一文件。提交前确认编译标准、样例、边界、多测清理与输出格式，并保留可回滚版本。

思路：

- 开场快速读题，记录题意确定度、预估难度、算法、实现者和卡点；优先稳题。
- 维护纸面或离线状态表：未读、在读、待证明、编码、测试、已交、待修。
- 上机前写清状态、关键公式和边界；机器优先给已证明、可快速 AC 的实现。
- 提交前由另一人核对题号、语言、复杂度、清空、整数范围、样例和文件内容。
- 收到 WA/RE/TLE 后先保留版本与失败假设，再决定调试、重写或暂时换题。
- 封榜期按“通过概率 × 题目收益 ÷ 预计时间”排序，不因 sunk cost 长时间单点卡题。

提交前检查。

```
[ ] 提交的是正确题号、正确文件、正确语言标准
[ ] 已删除本地 freopen，或确认评测定义 ONLINE_JUDGE
[ ] 调试信息不进入 stdout，交互题每次询问都 flush
[ ] 复杂度覆盖最大 n、m、T 和所有测试总规模
[ ] long long / __int128、INF 加法、取模和浮点精度正确
[ ] 多测状态已清空；数组、字符串、图的下标边界正确
[ ] 样例、最小数据、无解数据、手造边界均通过
[ ] 队友复述了算法并审过最危险的三处代码
```

易错点：

1. 三人同时围着一份代码调试，导致无人继续读题或准备下一题。
2. 未证明就抢机器编码，写完才发现模型或复杂度不成立。
3. WA 后立即重复提交同一份代码，产生无信息增益的罚时。
4. 只口头同步进度；长时间后队友不知道谁在做什么、哪题可接手。
5. 提交窗口选错题号、语言或旧文件，这是最不值得的罚时。

参考：

- OI Wiki：ICPC/CCPC 赛事与赛制
- OI Wiki：题型概述

1.14 Python 调试与随机对拍

适用范围： 本地验证一个已编译的 C++ 程序；适用于输入规模可由独立暴力程序求解的小数据。 **能做什么：** 固定种子生成用例、运行正解、检测超时/非零返回码、按空白切分比较输出，并保存可复现失败输入。 **输入 / 输出：** 被测可执行文件读取 `n` 和一行 `n` 个整数，输出最大连续子数组和；脚本参数为已编译

程序路径和可选迭代次数，失败时写入 `failed.in`。 **复杂度：** 每轮暴力 $O(n^2)$ 加一次子进程启动和运行时间；错误答案缩小最多额外运行 $O(n^2)$ 次子进程（数组上限为 12）。

独立暴力实现不能复用正解的核心逻辑。对拍答案不一致时，脚本逐项尝试删除并保留仍能复现答案差异的用例；超时或非零退出直接保存原始输入与错误，不继续缩小。失败输入可单独复跑。随机测试可发现错误，不能代替证明。

```python
import random
import subprocess
import sys
from pathlib import Path

# 参数：整数列表。返回：枚举所有非空子数组所得最大和。效果：无副作用，O(n^2)。
def brute(a):
    best = None
    for left in range(len(a)):
        total = 0
        for right in range(left, len(a)):
            total += a[right]
            best = total if best is None else max(best, total)
    return best

# 参数：已编译程序路径和整数列表。返回：(完整输入文本, stdout tokens, 错误描述)。效果：启动子进程，超时 1 秒。
def run(program, a):
    data = f"{len(a)}\n{' '.join(map(str, a))}\n"
    try:
        proc = subprocess.run([program], input=data, text=True, capture_output=True, timeout=1)
    except subprocess.TimeoutExpired:
        return data, [], "timeout"
    if proc.returncode != 0:
        return data, proc.stdout.split(), f"return code {proc.returncode}: {proc.stderr}"
    return data, proc.stdout.split(), ""

# 参数：程序路径、固定随机种子和轮数。返回：通过/失败状态。效果：失败时写 failed.in 与 failed.txt。
def stress(program, seed=20261009, rounds=300):
    rng = random.Random(seed)
    for case_id in range(rounds):
        n = rng.randint(1, 12)
        a = [rng.randint(-20, 20) for _ in range(n)]
        data, got, error = run(program, a)
        expected = str(brute(a))
        if error or got != [expected]:
            # 答案不一致时逐项删除并保留仍不一致的近似最小样例；超时/非零退出直接保存原始输入。
            small = a[:]
            changed = True
            while not error and changed and len(small) > 1:
                changed = False
                for i in range(len(small)):
                    trial = small[:i] + small[i + 1:]
                    trial_data, trial_got, trial_error = run(program, trial)
                    if trial_error or trial_got != [str(brute(trial))]:
                        small, data, got, error = trial, trial_data, trial_got, trial_error
                        changed = True
                        break
            Path("failed.in").write_text(data)
            Path("failed.txt").write_text(
```

```
                f"seed={seed} case={case_id} input={small}
expected={brute(small)} "
                f"actual={got} error={error}\n"
            )
            print("FAIL: see failed.in and failed.txt")
            return False
    print(f"PASS: {rounds} cases, seed={seed}")
    return True

if __name__ == "__main__":
    if len(sys.argv) < 2:
        raise SystemExit("usage: python3 stress.py ./solution [rounds]")
    ok = stress(sys.argv[1], rounds=int(sys.argv[2]) if len(sys.argv) > 2
else 300)
    raise SystemExit(0 if ok else 1)
```

常用 Python 调试/枚举工具：

工具	用法与边界
`assert condition, message`	检查内部不变量；优化模式可能移除断言，不要用它校验不可信输入
`print(repr(x), file=sys.stderr)`	显示空格、换行等不可见字符；不会污染被测程序 stdout
`breakpoint()` / `pdb`	在本地交互检查变量、单步执行；提交前移除断点
`itertools.product/permutations/combinations`	小规模枚举笛卡尔积、排列、组合；规模按指数/阶乘增长，先估算数量
整数 token 比较	输出为整数时用 `stdout.split()` 再逐 token 比较，忽略空白差异
`math.isclose(a,b, rel_tol=..., abs_tol=...)`	浮点答案用容差核对；容差按题目精度要求设定
多解构造题	单独写 validator 检查格式、范围、互异性和约束，不比较与参考答案完全相同

本节脚本只示范整数最大子段和，且子段必须非空，因此全负数组的答案是最大元素。实际题目应替换生成器、独立暴力和输出验证器。

1.15 中级数据结构学习路线

适用范围： 已能写循环、数组、函数和递归，准备学习 ICPC 区域赛中级数据结构及组合建模的读者。**能做什么：** 依据题目操作、维护对象和数据范围选择专题，按“理解、实现、识别、组合”推进。**输入 / 输出：** 输入自己的基础和一道题的约束；输出可验证的模型、维护信息及训练顺序。本节是学习导航，不是算法模板，没有统一的运行复杂度。

目标是应对区域赛中可能成为第 4～6 道解出题的中级数据结构问题：经典结构往往足够，关键在于发现能把问题转成什么操作，以及每个节点应保存什么信息。题目序号不等于难度，实际解题顺序也因队伍而异。以 GCD 线段树、树上 DSU 这两类问题为训练动机即可；这里不把具体年份、赛站、题号或奖牌线当作已经核实的事实。

零基础从哪里开始。“数据结构”就是组织数据并支持指定操作的方式。例如每次逐项求和要 $O(n)$；预先保存前缀和可以快速查询，但单点变化会影响许多前缀。树状数组和线段树在修改与查询之间取得平衡。“维护信息”是每个区间、集合或子树中保存的摘要；如果左右摘要不能快速合并，就不能只因为题面出现“区间”而套线段树。

先依次掌握数组与 `vector`、排序与离散化、前缀和与差分、递归与 DFS、普通并查集、树状数组、普通线段树、DFS 序和 LCA。遇到尚未学过的名词先补这一层；不必先学 Splay 或 LCT。

十二个重点专题。 星级表示本路线中的学习收益，优先级表示学习先后；两者不是赛站命中率预测。

专题	先修与典型任务	推荐	安排
树上 DSU	DFS 序、频次数组；所有子树的颜色数、众数	★★★★★	与 P0 启发式合并一起学
启发式合并	集合、树遍历；小集合并入大集合	★★★★★	P0
线段树合并	动态开点值域树；合并子树权值分布	★★★★★	P1
树链剖分 HLD	DFS、线段树；路径修改与查询	★★★★★	P0
主席树	值域计数、版本；区间及树上第 k 小	★★★★★	P1
带权并查集	普通并查集；相对差值、关系与矛盾	★★★★★	P0
离线树状数组	排序、前缀计数；二维偏序、区间统计	★★★★★	P0
莫队算法	分块、频次增删；静态区间频次统计	★★★★☆	P0
笛卡尔树	单调栈；最值划分与子数组贡献	★★★★☆	P1
李超线段树	直线、线段树；直线最值与 DP 转移	★★★★☆	P1
可撤销并查集	按大小合并、操作栈；恢复历史连通状态	★★★★☆	P2，与时间分治一起学
虚树	DFS 序、LCA、树形 DP；多询问少量关键点	★★★★☆	P1

执行顺序。P0 的六项是 HLD、带权并查集、莫队、扫描线、离线树状数组、启发式合并；树上 DSU 作为启发式思想的并列重点，不要漏掉。先学离线树状数组和启发式合并，再学莫队与树上 DSU；掌握线段树、LCA 后完成 HLD 和扫描线。P1 的五项是主席树、线段树合并、笛卡尔树、虚树、李超线段树。P0 与 P1 合计十一项，和上面的“十二专题”不是同一张清单：扫描线属于组合应用，树上 DSU 单列，可撤销并查集放在 P2。

P2 再推进可撤销并查集与线段树分治、树上莫队、Segment Tree Beats、Link-Cut Tree。先完成线段树分治的完整模板，再阅读其余 P2 专题的入门与适用边界。目标为铜牌到银牌阶段时，优先减少 P0、P1 中的模型盲点；CF 1900～2200 可用作个人训练的参考范围，不能直接换算奖牌，也不要求先刷满某个分数段。

普通结构的组合应用同样是重点。

题面操作	转换后维护的信息	入口
区间加、区间 GCD	差分的点修改；首项与其余差分的 GCD	差分 + GCD 线段树
矩形并面积、覆盖长度	扫描方向的宽度乘当前覆盖长度	扫描线 + 线段树
局部状态转移改变、动态路径计数	按顺序相乘的转移矩阵	线段树 + 矩阵
找区间第一个达标位置	用区间摘要排除整段，再向下找	线段树上二分
多条树路径的总贡献	端点与 LCA 处作差，子树汇总	树上差分 + LCA
逆序对、二维偏序	排序处理一维，树状数组查询另一维	离线树状数组
反复跳到下一个更大元素	单调栈建后继，再倍增跳转	倍增 + 单调栈
加删边与连通性查询	边的存活时间区间；分治中合并与撤销	线段树分治

每题都写一张模型卡。先写“对象是数组、子树、路径还是动态图”，再写“是否允许离线、修改了什么、查询什么”。随后写出不变量、合并或增删公式、复杂度和失败前提。例如“区间不同数”不能直接相加左右答案：相同颜色会重复。静态询问可用最后出现位置的离线树状数组，也可用莫队维护频次；若题目改为子树统计，又可以利用 DFS 序或树上 DSU。这才是从关键词走向模型识别。

每个专题的三个验收层次。

1. 实现：不看代码完成模板，手算一个例子，再与暴力对拍；能解释每个字段、复杂度及非法输入边界。
2. 识别：做不带算法标签的变式，在写代码前说明为何选它、其他候选卡在哪里。做错后记录的是漏掉的题意信号，而不是只记录模板名。
3. 组合：让模板接上差分、离线排序、DFS 序、LCA 或数学恒等式；亲自设计重复值、单点、空集合、负数、祖先端点等反例。

训练时每个专题先做基础操作题，再做改变查询信息的变式，最后做隐藏模型的混合题。完成的标志是能独立推导且通过对拍；仅能默写代码不算完成。各专题自检部分的分层练习给出具体改造方向，不以未经核实的题号或评分代替训练目标。

第二章

STL 库使用

本章按竞赛中最常用的 C++17 功能组织，供选型与查错。`n` 表示元素数，区间均为左闭右开 `[first,last)`。示例统一包含 `<bits/stdc++.h>`（GNU C++17）；若使用标准库实现，应按代码补齐对应头文件。容器操作后按其失效规则重新取得迭代器。

2.1 顺序容器：vector 与 array

`vector<T>` 是连续存储的动态数组，适合默认的序列存储、随机访问和尾部增删。下标、首尾访问 O(1)，尾插均摊 O(1)，中间插删 O(n)。扩容会使所有迭代器、指针和引用失效；未扩容的尾插使 `end()` 失效。`reserve(n)` 只预留容量，`resize(n)` 才改变元素数。`array<T,N>` 是固定长度数组，元素连续，大小编译期确定，不支持增删；适合小型固定表。

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a{4, 1, 3};
    a.reserve(8);                  // 预留容量，不改变 size
    a.push_back(2);
    sort(a.begin(), a.end());
    array<int, 3> fixed{1, 2, 3};
    cout << a[0] + fixed[2] << '\n';
}
```

常用成员：`size()`、`empty()`、`front()`、`back()` 均为 O(1)；`pop_back()` 删除末元素 O(1)，空时不可调用；`clear()` 清空元素但容量通常保留；`erase(it)` 删除单个元素并返回新位置，后缀移动 O(n)；`swap(other)` 交换内容/容量，通常 O(1)。`erase` 会使删除点及之后的迭代器失效，扩容会使全部失效。

操作	作用与复杂度	常见用途/边界
`vector<T> a(n)` / `a(n,x)`	创建 n 个默认值 / x，O(n)	创建后下标有效；空 vector 不可写 `a[0]`
`push_back(x)` / `emplace_back(args...)`	末尾插入，均摊 O(1)	emplace 原位构造；扩容时全体迭代器失效
`reserve(n)`	预留容量，不改变 size	估计规模时减少扩容；不能因此访问 `a[n]`
`resize(n,x)`	调整 size，新元素填 x；最坏 O(n)	需要真正创建可下标访问的槽位
`insert(pos,x)` / `erase(pos)`	中间修改 O(n)	pos 必须来自本 vector；返回新的迭代位置

避免用 `vector<bool>` 需要真实 `bool&` 或连续 `bool` 存储的场景；它是位压缩特化。二维 `vector` 常用于不规则邻接表；规则网格也可用一维下标 `r * width + c` 改善局部性。

2.2 string

`string` 是可变长字符序列，提供随机访问，常用于文本、数字串和字符状态。`size`、下标访问 O(1)，查找和拼接通常与处理字符数成正比。`getline(cin,s)` 读取一整行（此前若用 `>>`，先处理残留换行）；`find(x)` 返回首匹配位置，找不到返回 `string::npos`；`substr(pos,len)` 返回新字符串，越界起点会抛出异常。

`stoi/stoll` 将前缀转换为整数，非法输入抛异常，超范围也抛异常。修改导致扩容时指针、引用和迭代器可能失效。字符串下标是字节位置，不是 Unicode 字符位置。

操作	行为
`s += t` / `s.append(t)`	追加字符或字符串；总成本与追加长度相关
`s.find(t,pos)`	从 pos 起找子串；找不到是 `string::npos`，不可当作普通负数位置用
`s.substr(pos,len)`	复制子串；pos 必须不大于 size，len 可超出尾部并自动截断
`stoi` / `stoll`	将十进制前缀转 int/long long；可用位置参数确认是否消费了全部输入

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s = "contest";
    s += "!";
    auto pos = s.find("test");
    if (pos != string::npos) cout << s.substr(pos, 4) << '\n';
}
```

2.3 pair、tuple 与结构化绑定

`pair<A,B>` 表示两个关联值，常用于坐标、边 `(邻点, 权值)` 和排序键；比较按 `first` 再按 `second` 字典序进行。`tuple` 可装多个异质字段，但字段命名不直观；字段较多且语义稳定时优先自定义 `struct`。结构化绑定要求分量数量匹配。

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<pair<int,int>> edges{{2, 7}, {1, 4}};
    sort(edges.begin(), edges.end()); // 先按 first，再按 second
    auto [to, weight] = edges.front();
    auto record = make_tuple(3, string("Ada"), true);
    cout << to << ' ' << weight << ' ' << get<1>(record) << '\n';
}
```

2.4 两端序列与适配器：deque、stack、queue

`deque<T>` 支持两端均摊 O(1) 插入删除及 O(1) 随机访问；适合单调队列和两端操作。它不是连续内存，不能把 `data()` 当作数组接口。两端插入通常不使已有元素引用失效，但迭代器规则较严格，结构修改后宜重取。

`stack<T>` 提供后进先出，只用 `push/pop/top`；`queue<T>` 提供先进先出，只用 `push/pop/front/back`。二者是容器适配器，`pop` 返回 `void`，应先读取元素再弹出。读取空容器的顶部或队首是错误。

适配器	读取	插入/删除	典型用途
`stack`	`top()`	`push()` / `pop()`	括号匹配、DFS 显式栈、单调栈
`queue`	`front()`、`back()`	`push()` / `pop()`	BFS、按到达顺序处理任务
`priority_queue`	`top()`	`push()` / `pop()`	每步取当前最小/最大候选

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```cpp
int main() {
    deque<int> d{2, 3}; d.push_front(1); d.pop_back();
    stack<int> st; st.push(5); int x = st.top(); st.pop();
    queue<int> q; q.push(x); cout << q.front() << '\n';
}
```

2.5 堆：priority_queue

`priority_queue<T>` 默认是最大堆：`top()` 为最大值，插入和删除堆顶 O(log n)，读取 O(1)。最小堆可用 `greater<T>`。适合反复取当前最大/最小值；若要删除任意元素或遍历有序全体，选择其他结构。自定义比较器需满足严格弱序。

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    priority_queue<int> maxHeap;
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int x : {4, 1, 7}) { maxHeap.push(x); minHeap.push(x); }
    cout << maxHeap.top() << ' ' << minHeap.top() << '\n';
}
```

2.6 有序集合：set、multiset

`set<T>` 保存互异键，`multiset<T>` 保存可重复键；二者按比较器维护有序，查找、插入 O(log n)，按键删除 multiset 的全部 k 个匹配项为 O(log n+k)。适合需要动态维护有序元素并查前驱/后继的场景。迭代器为双向迭代器，`lower_bound(x)` 返回首个不小于 `x` 的元素，`upper_bound(x)` 返回首个大于 `x` 的元素；不存在时返回 `end()`。插删通常只使被删元素迭代器失效。

`set::erase(x)` 删除键为 x 的元素，`multiset` 中会删除所有等于 x 的元素；若只删一个重复项，先 `find` 再按迭代器删除。不要对集合迭代器调用 `sort`。

操作	`set` / `multiset` 行为
`insert(x)`	O(log n) 插入；set 忽略重复键，multiset 保留
`find(x)` / `count(x)`	查键 O(log n)；multiset 统计重复数 O(log n + k)，k 为重复个数
`lower_bound(x)` / `upper_bound(x)`	O(log n) 取首个 >=x / >x 的迭代器
`erase(it)` / `erase(x)`	按迭代器删一个 O(log n)；按键删除 multiset 全部匹配项 O(log n+k)

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    multiset<int> s{2, 2, 5};
    auto it = s.find(2);
    if (it != s.end()) s.erase(it); // 只删除一个 2
    auto next = s.lower_bound(3);
    if (next != s.end()) cout << *next << '\n';
}
```

2.7 有序映射：map

`map<K,V>` 按键有序，每个键唯一；查找、插入和删除 O(log n)。适合键域稀疏、需要有序遍历或前驱后继查询的映射。`m[key]` 不存在时会插入默认值；只读查询用 `find`，避免意外插入。迭代器按键升序，插入通常不使其失效，删除只使被删项迭代器失效。

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    map<string, int> count;
    ++count["AC"];
    auto it = count.find("WA");
    if (it == count.end()) cout << "missing\n";
    for (const auto& [key, value] : count) cout << key << ':' << value << '\n';
}
```

常用成员：

操作	说明
`find(key)`	O(log n)，缺失时返回 `end()`；只读查询优先用它
`at(key)`	查值但不插入；键不存在时抛出 `out_of_range`
`m[key]`	键不存在时插入默认构造的值；适合频次累加
`insert({key,value})` / `emplace(key,value)`	插入键值；重复键不会覆盖既有值
`erase(key)` / `erase(it)`	按键删除或删除迭代器所指项；后者只删一项
`lower_bound` / `upper_bound`	O(log n) 返回首个 `>= key` / `> key` 的迭代器

2.8 哈希容器：unordered_set、unordered_map

哈希容器平均查找、插入和删除 O(1)，最坏 O(n)，不维护顺序；适合只需按键快速查找且不要求有序遍历的场景。扩容 rehash 会使所有迭代器失效（引用和指针仍有效）；插入可能触发 rehash。可用 `reserve` 预留元素数减少扩容。键类型必须可哈希且支持相等比较。

操作	说明
`find(key)`	返回迭代器，缺失时为 `end()`；只读检查优先用它
`count(key)` / `contains(key)`	判存在；`contains` 从 C++20 才提供，C++17 用 `find`
`reserve(n)`	预计元素总量，减少 rehash；参数是元素数而非桶数
`load_factor()` / `max_load_factor(x)`	查看/调整平均装载因子；调参不能保证对抗恶意碰撞

自定义键需要提供哈希函数并保持“相等的键哈希值相同”。哈希不是安全性边界；攻击性输入可能退化。整数键可用随机种子的 `splitmix64` 扰动降低被构造碰撞的风险，但这不是密码学保证。

```cpp
#include <bits/stdc++.h>
using namespace std;
struct SplitMixHash {
    // 参数：64 位整数 x。返回：种子扰动后的哈希值。效果：只读进程级随机种子。
    size_t operator()(uint64_t x) const noexcept {
        static const uint64_t seed = chrono::steady_clock::now().time_since_epoch().count();
        x += seed + 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return static_cast<size_t>(x ^ (x >> 31));
    }
};
int main() {
    unordered_map<uint64_t, int, SplitMixHash> id;
    id.reserve(1000);
    id[123456789ULL] = 7;
    cout << id.at(123456789ULL) << '\n';
}
```

2.9 bitset

`bitset<N>` 在编译期固定 N 位，适合小规模布尔状态、集合位运算和快速计数。与 `vector<bool>` 不同，长度不能运行时改变。按位与或异或、取反和移位按机器字批量处理；`count()` 返回 1 的位数。下标 0 是最低位，移位超出范围的位被丢弃。

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    bitset<8> a(string("00101101"));
    bitset<8> b = a << 1;
    cout << (a & b).count() << ' ' << b << '\n';
}
```

2.10 排序、二分与边界

`sort(first,last)` 要求随机访问迭代器，原地升序排序 O(n log n)，不稳定；降序可传 `greater<>`，自定义记录用严格弱序比较器。`stable_sort` 保留等价元素原相对顺序，通常使用 O(n) 额外空间。`lower_bound/upper_bound` 要求区间按相同比较规则有序，返回迭代器，比较次数 O(log n)；对 `vector` 总体 O(log n)，对 `set` 应调用成员 `lower_bound` 才能得到 O(log n)。

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a{4, 1, 2, 2};
    sort(a.begin(), a.end());
    auto lo = lower_bound(a.begin(), a.end(), 2);
    auto hi = upper_bound(a.begin(), a.end(), 2);
    cout << (lo - a.begin()) << ' ' << (hi - a.begin()) << '\n';
}
```

常用补充：`nth_element(first,nth,last)` 平均 O(n)，把第 `nth` 元素放在排序后应处的位置，两侧各自满足分界但内部无序；`min_element/max_element` 返回极值迭代器，空区间返回 `last`；`binary_search` 只返回是否存在，`equal_range` 返回 `[lower_bound,upper_bound)`。`set/map` 需要二分时调用成员 `lower_bound/upper_bound`，复杂度 O(log n)；对其迭代器使用泛型二分会因逐步移动而退化。

常用算法速查（`n` 为区间长度）：

算法	前提与结果	复杂度 / 副作用
`sort` / `stable_sort`	随机访问区间；稳定版保留等价项相对次序	O(n log n)，原地重排 / 稳定排序有足够内存时 O(n log n)，内存不足时最坏 O(n log² n)
`nth_element(first,nth,last)`	`nth` 处成为全序第 `nth` 个元素，两边只保证分界	平均 O(n)，重排；若 `nth==last`，不解引用该迭代器
`lower_bound` / `upper_bound`	已按同一比较器排序；分别给出 `>=x` / `>x` 的首位置	O(log n) 比较；vector 随机访问总体 O(log n)
`binary_search` / `equal_range`	有序区间；前者测存在，后者给出等值段	O(log n) 比较
`min_element` / `max_element`	返回极值迭代器，空区间返回 `last`	O(n)，不修改区间
`reverse` / `fill`	分别反转、写入同一个值	O(n)，原地修改
`count` / `count_if`	统计等值项 / 满足谓词项	O(n)，不修改区间
`copy` / `transform`	输出区须足够大；`back_inserter(out)` 可追加	O(n)，写目标区间
`unique`	只压缩相邻重复项，返回新逻辑末尾	O(n)，重排但不改变容器 size；常与 `erase` 配合

`sort` 自定义比较器必须满足严格弱序，例如按键排序时比较 `a.key < b.key`，不能用 `<=`。对于 `set/map`，泛型 `lower_bound` 迭代器移动可能为 O(n)，应使用容器成员的 O(log n) 查找。

2.11 去重、排列与区间算法

`unique(first,last)` 把相邻重复元素压到前部并返回新逻辑末尾，不会缩短容器；先排序才能全局去重，再用 `erase` 真正缩短。`next_permutation` 把序列改成字典序下一个排列；到最大排列时会重置为最小排列并返回 `false`，故从排序序列开始可枚举所有不同排列。`reverse` 原地反转双向迭代器范围 O(n)；`fill(first,last,value)` 用同一值覆盖范围 O(n)；`count/count_if` 分别统计等值项/谓词为真的项，均 O(n)。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a{3, 1, 1, 2};
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    do { for (int x : a) cout << x; cout << '\n'; }
    while (next_permutation(a.begin(), a.end()));
}
```

2.12 数值算法与 Lambda

`accumulate(first,last,init)` 左折叠区间，结果类型由 `init` 决定；求 `long long` 和应写 `0LL`。`iota` 用递增值填充区间。

函数	结果	提醒
`accumulate(first,last,init)`	左折叠 / 累加	返回类型由 init 决定；求 long long 和用 `0LL`
`partial_sum(first,last,out)`	输出每个前缀累计值	输出空间须足够；可用 `back_inserter` 追加
`iota(first,last,start)`	依次填 start、start+1…	需 `<numeric>`；常用于生成下标序列
`gcd(a,b)` / `lcm(a,b)`	C++17 求最大公约数 / 最小公倍数	lcm 乘法仍可能溢出

`gcd(a,b)` 求最大公约数，`lcm(a,b)` 求最小公倍数（C++17 `<numeric>`）；结果类型取决于参数类型，乘法中间值仍可能溢出。Lambda 可捕获局部变量并作为比较器或局部逻辑；按值捕获复制，按引用捕获会修改原变量且不可超出其生命周期。`[&]` 捕获所有使用到的局部变量引用，需确认调用期间对象仍存活。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> a(5); iota(a.begin(), a.end(), 1);
    long long sum = accumulate(a.begin(), a.end(), 0LL);
    int limit = 3;
    int count = count_if(a.begin(), a.end(), [limit](int x) { return x <=
limit; });
    cout << sum << ' ' << count << '\n';
}
```

2.13 迭代器与半开区间

迭代器连接容器和泛型算法，范围统一写 `[first,last)`；`end()` 是尾后哨兵，不可解引用。`begin()/end()` 获取边界通常 O(1)，空范围满足 `begin()==end()`。`vector`、`array`、`deque`、`string` 迭代器支持随

机访问；`set/map` 为双向迭代器。`sort`、`nth_element` 要求随机访问；`reverse` 要求双向；`find`、`count` 可用于前向区间。

`it + k`、`last-first` 只适用于随机访问迭代器且区间须来自同一容器。`next(it,k)` 返回移动后的副本，`advance(it,k)` 原地移动；随机访问容器 O(1)，双向/前向容器按步数 O(k)。不要保存跨越容器结构修改仍未确认有效性的迭代器。

常见边界：所有标准算法按半开区间操作；`end()` 不可解引用；`erase`、扩容和 rehash 后检查迭代器有效性；`lower_bound` 前先确认排序规则一致；注意整数累加类型是否足够。

第三章

基础算法与搜索

3.1 复杂度与枚举

适用范围： 枚举有限状态/子集；子集和用 O(2^n) 空间时间。输入 vector，输出所有子集和；位掩码只适用于 n 小且移位不溢出的范围。**能做什么：** 枚举所有子集和，或列出一个掩码的全部非空子掩码。**输入 / 输出：** 输入 0-based long long 数组或非负掩码；返回子集和数组或非空子掩码列表。 **复杂度：** O(2^n).

用途： 先估算可执行次数，再选择枚举对象：约 10^8 次简单操作通常已接近上限。直接枚举为 $O(n^k)$，位掩码枚举全部子集为 $O(2^n)$、空间 $O(1)$；只枚举固定大小组合可减少无效状态。适用于数据范围较小或作为对拍暴力。

思路：

- `mask` 的第 `i` 位表示是否选择下标 `i`，适合 `n <= 25` 左右的全子集枚举。
- 用 `mask & (1 << i)` 判断元素是否被选择；位数较多时改用 `1LL << i`。
- 枚举某个集合的非空子集时，使用 `(sub - 1) & mask`。
- 若只需固定大小的集合，可用 `__builtin_popcount(mask)` 筛选。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

// 枚举 n 个元素的全部子集，并计算每个子集的元素和。
// 参数：a 为 0-based 整数数组。返回：每个掩码对应的子集和，空集和为 0。效果：无副作用；要求 a.size() 可安全作为位掩码宽度。
vector<long long> enumerateSubsetSums(const vector<long long>& a) {
    int n = static_cast<int>(a.size());
    // a 使用 0-based；sums[mask] 表示恰好选择 mask 中元素的总和。
    vector<long long> sums(1ULL << n, 0);
    for (unsigned long long mask = 1; mask < (1ULL << n); ++mask) {
        // 取一个已选位，复用删去该位后的答案，避免每个子集重新扫描 a。
        int bit = __builtin_ctzll(mask);
        sums[mask] = sums[mask ^ (1ULL << bit)] + a[bit];
    }
    return sums;
}

// 枚举 mask 的所有非空子集。
// 参数：非负集合掩码 mask。返回：mask 的所有非空子掩码。效果：无副作用。
vector<int> nonemptySubmasks(int mask) {
    vector<int> result;
    // 每次删去 sub 的最低位并只保留 mask 中的位；0 不在返回结果中。
    for (int sub = mask; sub; sub = (sub - 1) & mask) {
        result.push_back(sub);
    }
    return result;
}
```

易错点：

1. `1 << n` 在 `n >= 31` 时可能溢出，应使用 `1ULL << n`。
2. 全部 `mask` 再枚举其全部子集的总复杂度是 $O(3^n)$，不是 $O(2^n)$。
3. 不要只看大 O；字符串复制、哈希和递归的常数明显更大。

参考：

- OI Wiki：枚举
- OI Wiki：复杂度

3.2 前缀和与差分

适用范围： 静态区间和与区间加差分；前缀查询 O(1)，差分恢复 O(n)。输入按约定下标数组及合法闭区间，输出区间和或恢复后的数组。 **能做什么：** 计算静态区间和、批量区间加后恢复数组、查询二维矩形和。 **输入 / 输出：** 1D 使用 1-based 闭区间；2D 使用 1-based 闭矩形；返回区间和或恢复数组。 **复杂度：** O(n) build / O(1) query.

用途： 前缀和把静态区间和查询化为两个前缀之差，预处理 $O(n)$、单次查询 $O(1)$、空间 $O(n)$。差分把离线区间加转为两个端点修改，单次修改 $O(1)$，最后 $O(n)$ 还原。若修改与查询交错，应改用树状数组或线段树。

思路：

- 一维区间 `[l,r]` 的和为 `prefix[r] - prefix[l - 1]`。
- 对 `[l,r]` 全部加 `value`：`diff[l] += value`，`diff[r+1] -= value`。
- 二维矩形使用左上、右下坐标，容斥四个二维前缀。
- 数值总和可能超过 `int`，数组统一使用 `long long`。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct PrefixSum1D {
    // 输入与查询均为 1-based；prefix[0] 是空前缀哨兵。
    vector<ll> prefix;

    // 参数：含 0 下标哨兵的 1-based 数组 a[1..n]。返回：构造前缀和对象。效果：O(n) 初始化，仅保存汇总。
    explicit PrefixSum1D(const vector<ll>& a) {
        int n = static_cast<int>(a.size()) - 1; // a[1..n]
        prefix.assign(n + 1, 0);
        for (int i = 1; i <= n; ++i) prefix[i] = prefix[i - 1] + a[i];
    }

    // 参数：1-based 闭区间 [left,right]。返回：该区间元素和。效果：只读；要求区间在构造数组范围内。
    ll query(int left, int right) const {
        // 返回闭区间 [left, right]，调用者需保证 1 <= left <= right <= n。
        return prefix[right] - prefix[left - 1];
    }
};

struct Difference1D {
    int n;
    vector<ll> diff;

    // 参数：含哨兵的 1-based 初始数组。返回：差分对象。效果：O(n) 初始化，后续 add 记录闭区间加法。
    explicit Difference1D(const vector<ll>& a)
        : n(static_cast<int>(a.size()) - 1), diff(n + 2, 0) {
        for (int i = 1; i <= n; ++i) diff[i] = a[i] - a[i - 1];
    }

```

```cpp
    // 参数：1-based 闭区间及增量 value。返回：无。效果：记录区间加法，须在 build 前调用。
    void add(int left, int right, ll value) {
        // 闭区间 [left, right] 加 value；diff[n+1] 承接右端撤销标记。
        diff[left] += value;
        diff[right + 1] -= value;
    }

    // 参数：无。返回：应用所有已记录区间加法后的 1-based 数组。效果：只读，不清空差分状态。
    vector<ll> build() const {
        // 所有离线修改完成后再还原；result 仍为 1-based。
        vector<ll> result(n + 1, 0);
        for (int i = 1; i <= n; ++i) result[i] = result[i - 1] + diff[i];
        return result;
    }
};

struct PrefixSum2D {
    // a 的有效区域为 [1..n] x [1..m]，第 0 行和第 0 列是容斥哨兵。
    vector<vector<ll>> prefix;

    // 参数：1-based 矩阵，0 行/列作为哨兵。返回：构造二维前缀和对象。效果：O(nm) 初始化。
    explicit PrefixSum2D(const vector<vector<ll>>& a) {
        int n = static_cast<int>(a.size()) - 1;
        int m = static_cast<int>(a[0].size()) - 1;
        prefix.assign(n + 1, vector<ll>(m + 1, 0));
        for (int i = 1; i <= n; ++i) {
            for (int j = 1; j <= m; ++j) {
                prefix[i][j] = a[i][j] + prefix[i - 1][j]
                    + prefix[i][j - 1] - prefix[i - 1][j - 1];
            }
        }
    }

    // 参数：1-based 闭矩形 [x1..x2]×[y1..y2]。返回：矩形元素和。效果：只读。
    ll query(int x1, int y1, int x2, int y2) const {
        // 返回闭矩形 [x1..x2] x [y1..y2]，调用者需保证坐标顺序合法。
        return prefix[x2][y2] - prefix[x1 - 1][y2]
            - prefix[x2][y1 - 1] + prefix[x1 - 1][y1 - 1];
    }
};
```

易错点：

1. 预留 `0` 行、`0` 列以及 `n+1` 差分哨兵，避免边界特判。
2. 二维容斥最后要把重复减去的左上区域加回来。
3. 差分适用于“全部修改后统一还原”，不能直接回答中途查询。

参考：

- OI Wiki：前缀和与差分

3.3 二分查找与二分答案

适用范围：已排序查找或单调判定函数的整数答案；二分 O(log 范围) 次判定。输入有效边界与单调谓词，输出首个/末个满足边界；无解须由调用方预先界定。**能做什么：**在排序序列找边界，或对单调谓词求首个/最后一个真值。**输入 / 输出：**输入有效边界和假转真/真转假的单调谓词；无解分别返回 right+1/left-1。**复杂度：** O(log n) predicate checks.

用途：在有序序列查找边界，或答案存在“左侧均可行、右侧均不可行”等单调性时使用。维护一真一假的边界，每次检查中点并舍弃一半区间。整数二分需 $O(\log R)$ 次检查、空间 $O(1)$；总复杂度还要乘 `check` 的复杂度。

思路：

- `firstTrue` 返回闭区间 `[left,right]` 中第一个满足 `check` 的整数。
- 调用前必须保证至少存在一个可行答案；否则自行增加无解判断。
- 求“最后一个可行值”可把判定取反，或使用对应的 `lastTrue`。
- 中点写成 `left + (right - left) / 2`，避免两端相加溢出。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

template <class Check>
// 参数：闭区间边界及单调谓词（假到真）。返回：首个为真的值；无解返回 right+1。效果：调用谓词 O(log范围) 次。
ll firstTrue(ll left, ll right, Check check) {
    // 闭区间内要求答案存在，且 check 呈 false...false,true...true。
    while (left < right) {
        // 向下取整；check(mid) 为真时 mid 仍可能是首个真值，不能丢弃。
        ll mid = left + (right - left) / 2;
        if (check(mid)) right = mid;
        else left = mid + 1;
    }
    return left;
}

template <class Check>
// 参数：闭区间边界及单调谓词（真到假）。返回：最后一个为真的值；无解返回 left-1。效果：调用谓词 O(log范围) 次。
ll lastTrue(ll left, ll right, Check check) {
    // 闭区间内要求答案存在，且 check 呈 true...true,false...false。
    while (left < right) {
        // 向上取整保证只剩两个数时 mid != left，否则真分支会死循环。
        ll mid = left + (right - left + 1) / 2;
        if (check(mid)) left = mid;
        else right = mid - 1;
    }
    return left;
}
```

易错点：

1. 二分答案的关键是判定函数单调，不能只因答案是数字就二分。
2. 求最后一个可行值时中点必须向上取整，否则可能死循环。
3. 浮点二分通常固定迭代次数；不要直接套整数的 `+1/-1`。

参考：

- OI Wiki：二分

3.4 双指针与滑动窗口

适用范围：窗口可由双指针单向维护时适用；和阈值模板要求数组元素非负且 `target>0`。输入数组及目标，输出最长长度或最短长度；不存在时返回 0。**能做什么：**求最长互异子数组，或非负数组中和至少为目标的最短子数组。**输入 / 输出：**输入整数数组及 target；返回长度；最短窗口不存在时返回 0，`target<=0` 时非空数组返回 1、空数组返回 0。 **复杂度：** O(n).

用途：当区间随右端扩张后，合法性关于左端具有单调性时，用左右指针维护连续区间；每个元素至多进出窗口一次，时间 $O(n)$、空间由维护信息决定。常用于非负数组和、至多含若干种元素、去重窗口；有负数时“和越长越大”不成立。

思路：

- 右指针每次加入一个元素，再不断移动左指针直到窗口合法。
- 求最长合法区间时，在收缩完成后更新答案。
- 求最短满足区间时，应在条件成立期间更新答案并收缩。
- 计数类窗口常用 `unordered_map` 或定长频次数组维护。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

// 最长无重复连续子数组长度，元素值无需预先离散化。
// 参数：整数数组。返回：元素互异的最长连续子数组长度。效果：无副作用，O(n)。
int longestDistinctSubarray(const vector<int>& a) {
    // a 使用 0-based；每轮收缩后窗口 [left, right] 始终不含重复值。
    unordered_map<int, int> count;
    int answer = 0;
    for (int left = 0, right = 0; right < static_cast<int>(a.size()); ++right) {
        ++count[a[right]];
        // 新加入值重复时持续移走左端，直到窗口重新合法。
        while (count[a[right]] > 1) {
            if (--count[a[left]] == 0) count.erase(a[left]);
            ++left;
        }
        answer = max(answer, right - left + 1);
    }
    return answer;
}

// 非负数组中，和至少为 target 的最短非空连续子数组长度；不存在返回 0。
// 参数：非负数组 a 与目标和 target。返回：和至少为 target 的最短非空子数组长度；不存在返回 0。效果：无副作用，O(n)。
int minLengthAtLeast(const vector<long long>& a, long long target) {
    if (target <= 0) return a.empty() ? 0 : 1;
    // 仅适用于非负数组；非负性保证收缩左端不会漏过更短可行窗口。
    long long sum = 0;
    int answer = static_cast<int>(a.size()) + 1;
    for (int left = 0, right = 0; right < static_cast<int>(a.size()); ++right) {
        sum += a[right];
        // 条件成立时先记录当前闭区间，再移走左端继续尝试缩短。
        while (sum >= target) {
            answer = min(answer, right - left + 1);
            sum -= a[left++];
        }
    }
    return answer == static_cast<int>(a.size()) + 1 ? 0 : answer;
}
```

易错点：

1. 必须证明移动某一端不会错过答案，即窗口条件具有单调性。
2. `while` 收缩和 `if` 收缩含义不同，多数窗口需要连续收缩。
3. 用哈希表计数时，计数归零后及时删除，避免种类数判断错误。

参考：

- OI Wiki：双指针

3.5 离散化

适用范围： 把可比较值映射为保持大小次序的紧凑整数编号；排序 O(n log n)。输入原值，输出 0-based 编号/去重值表；仅适用于查询构造时收录过的值；对未收录值，`id` 返回插入位置而非查找失败信号。 **能做什么：** 把离线值映射为排序去重后的紧凑编号。 **输入 / 输出：** 构造时输入全体候选值；id 仅要求返回 0-based 插入位置，value(id) 要求有效编号，size 返回不同值数。 **复杂度：** O(n log n).

用途： 只关心数值的大小、相等关系，而原值范围很大时，将出现过的值排序去重后映射到连续编号。建表 $O(n \log n)$，每次查询 $O(\log n)$，空间 $O(n)$。适用于树状数组、线段树和扫描线；若数值间距离有意义，不能直接用名次替代。

思路：

- 收集所有可能参与查询、修改的坐标，再统一排序去重。
- `lower_bound` 得到从 `0` 开始的编号；需要 1-based 时再加一。
- 可用 `values[id]` 从编号恢复原值。
- 动态出现且无法预收集的值，应考虑平衡树或动态线段树。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

template <class T>
struct Discretizer {
    // values[id] 保存第 id 小的不同原值；编号统一为 0-based。
    vector<T> values;

    // 参数：所有将参与查询的原值。返回：初始化后的离散化表。效果：排序并去重，O(n log n)。
    // 参数：所有后续查询可能出现的值。返回：初始化对象。效果：排序去重，O(n log n)。
    explicit Discretizer(vector<T> allValues) : values(move(allValues)) {
        sort(values.begin(), values.end());
        // unique 只压缩有效前缀；erase 同步缩小编号域。
        values.erase(unique(values.begin(), values.end()), values.end());
    }

    // 参数：原值。返回：排序去重后的 0-based 插入位置；调用者应保证该值已收录。效果：只读。
    int id(const T& value) const {
        // value 应在建表输入中出现；否则返回的只是它的插入位置。
        return static_cast<int>(lower_bound(values.begin(), values.end(), value)
                                - values.begin());
    }

    // 参数：有效 0-based 编号。返回：对应原值常量引用。效果：只读。
    const T& value(int id) const {
        return values[id];
    }

    // 参数：无。返回：去重值数量。效果：只读，O(1)。
    int size() const {
        return static_cast<int>(values.size());
    }
};
```

易错点：

1. 忘记 `unique` 后的 `erase` 会使重复值获得错误的空间规模。
2. 线上读到新坐标后再加入，会导致已有编号整体变化。
3. 区间长度不能用离散编号之差代替，应使用原坐标之差。

参考：

- OI Wiki：离散化

3.6 分治与归并计数

适用范围： 可将问题拆为两个子区间并合并时适用；归并 O(n log n)。输入可随机访问数组，输出排序结果或逆序对数。 **能做什么：** 归并排序并统计逆序对。 **输入 / 输出：** 输入数组及半开区间；返回逆序对数并原地排序相应范围。 **复杂度：** O(n log n).

用途： 分治把问题拆成互不重叠的同类子问题，递归求解后合并。典型归并排序与逆序对计数：左右两半已排序时，若右元素更小，它与左侧剩余元素都构成逆序对。时间 $O(n \log n)$，辅助空间 $O(n)$，递归栈 $O(\log n)$。

思路：

- 调用 `countInversions(a)` 后，数组会被排序并返回逆序对数量。
- 逆序对可能达到 $n(n-1)/2$，返回值使用 `long long`。
- 合并阶段维持左右区间已排序这一不变量。
- 其他分治题需明确“如何拆分”和“如何在线性或较低复杂度内合并”。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：数组、同长临时区及半开下标区间 [left,right)。返回：该区间逆序对数。效果：原地排序 a[left..right)，递归 O(n log n)。
ll mergeCount(vector<ll>& a, vector<ll>& temp, int left, int right) {
    // 全函数统一使用左闭右开区间 [left, right)。
    if (right - left <= 1) return 0;
    int mid = left + (right - left) / 2;
    ll answer = mergeCount(a, temp, left, mid)
              + mergeCount(a, temp, mid, right);

    int i = left, j = mid, k = left;
    // 不变量：两半已排序，temp[left..k) 已是合并后的最小前缀。
    while (i < mid && j < right) {
        if (a[i] <= a[j]) {
            temp[k++] = a[i++];
        } else {
            temp[k++] = a[j++];
            // 当前右值小于左半当前值，也小于左半剩余的全部 mid-i 个值。
            answer += mid - i;
        }
    }
    while (i < mid) temp[k++] = a[i++];
    while (j < right) temp[k++] = a[j++];
    copy(temp.begin() + left, temp.begin() + right, a.begin() + left);
    return answer;
}

// 参数：可修改整数数组。返回：逆序对数量。效果：将 a 升序重排，O(n log n)。
ll countInversions(vector<ll>& a) {
    vector<ll> temp(a.size());
    return mergeCount(a, temp, 0, static_cast<int>(a.size()));
}
```

易错点：

1. 本模板采用左闭右开区间 `[left,right)`，递归和合并边界必须一致。
2. 相等元素不是逆序对，比较应写 `<=`。

3. 若每层合并都新建大数组，额外分配会增大常数。

参考：

- OI Wiki：递归与分治

3.7 贪心与区间调度

适用范围： 半开区间 `[left,right)` 的无权最大不相交选择；相接端点允许兼选。输入区间及调用方提供的原始 id，输出所选原始 id。复杂度 O(n log n)。 **能做什么：** 按结束位置贪心选择最多个互不重叠区间。 **输入 / 输出：** 输入 `Interval{left,right,id}` 列表；返回被选区间原始 `id`，空输入返回空向量。 **复杂度：** O(n log n).

用途： 贪心每步选择当前看来最优且可证明不损失全局最优的决策。典型最多不相交区间：按右端点升序，依次选择起点不早于上个终点的区间。排序 $O(n \log n)$、扫描 $O(n)$、空间 $O(n)$。套用前需用交换论证或反证证明选择安全。

思路：

- 本模板把区间视为半开区间 `[left,right)`，端点相接允许同时选择。
- 返回被选区间在原输入中的下标，便于恢复方案。
- 若题意为闭区间且端点重合算相交，把条件改成 `left > lastRight`。
- 带权区间不能用此贪心，应使用按终点排序的动态规划。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Interval {
    long long left, right;
    int id;
};

// 参数：半开区间列表，id 由调用者提供。返回：所选区间的原始 id。效果：复制并排序输入，O(n log n)。
vector<int> maximumNonOverlapping(vector<Interval> intervals) {
    // 输入按半开区间 [left, right)；返回原输入 id，而不是排序后位置。
    // 比较器参数：两个区间。返回：先按 right、再按 left 升序的严格次序；无外部副作用。
    sort(intervals.begin(), intervals.end(), [](const Interval& a, const Interval& b) {
        if (a.right != b.right) return a.right < b.right;
        return a.left < b.left; // 同终点时固定次序，便于复现与检查
    });

    vector<int> chosen;
    long long lastRight = numeric_limits<long long>::lowest();
    for (const auto& interval : intervals) {
        // 选择右端最早的可行区间，为后续选择留下最大空间。
        if (interval.left >= lastRight) {
            chosen.push_back(interval.id);
            lastRight = interval.right;
        }
    }
    return chosen;
}
```

易错点：

1. “看起来合理”不是证明；不同目标可能需要按左端点、长度或其他关键字排序。
2. 明确区间开闭，否则端点相等时是否冲突会写反。
3. 最多数量与最大权值是不同问题，后者通常需要 DP。

参考：

- OI Wiki：贪心

3.8 DFS 深度优先搜索

适用范围： 图遍历、可达性和连通分量；邻接表 DFS O(V+E)。输入 1..n 编号图和起点，输出访问序或分量；无向图分量代码按无向邻接表使用。 **能做什么：** 从起点 DFS 遍历及无向图连通分量划分。 **输入 / 输出：** 输入 1..n 邻接表和起点；返回访问序或分量列表。 **复杂度：** O(V+E).

用途： DFS 沿一条分支走到底再回退，适合连通块、可达性、环检测及需要回溯的状态搜索。邻接表图遍历时间 $O(n+m)$、空间 $O(n+m)$，递归栈最坏 $O(n)$。一般图必须记录访问状态；深链可能爆栈，可改显式栈。

思路：

- `dfsFrom(start)` 返回起点所在连通块的访问顺序。
- 无向图应加入双向边；有向图只加入题目给出的方向。
- 搜整张图时，对所有未访问点分别调用 DFS，可得到连通块或 DFS 森林。
- 需要退出时刻、子树信息时，在递归调用返回后处理。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数：1..n 编号图且 1<=start<=n。返回：从起点可达点的 DFS 顺序。效果：只读图，O(V+E)。
vector<int> dfsFrom(int start, const vector<vector<int>>& graph) {
    // graph 与节点均为 1-based；环、重边与边方向由建图方式决定。
    int n = static_cast<int>(graph.size()) - 1;
    vector<char> visited(n + 1, false);
    vector<int> order;

    // 参数：当前顶点 u。返回：无。效果：递归标记可达点并追加到外层 order。
    function<void(int)> dfs = [&](int u) {
        // 必须在递归邻点前标记，防止沿环或重边再次进入 u。
        visited[u] = true;
        order.push_back(u);
        for (int v : graph[u]) {
            if (!visited[v]) dfs(v);
        }
    };

    dfs(start);
    return order;
}

// 参数：无向图邻接表，点编号 1..n。返回：各连通分量的点集。效果：只读图，O(V+E)。
vector<vector<int>> connectedComponents(const vector<vector<int>>& graph) {
    // 仅用于无向图连通分量；有向图不能据此求强连通或弱连通。
    int n = static_cast<int>(graph.size()) - 1;
    vector<char> visited(n + 1, false);
    vector<vector<int>> components;
    for (int start = 1; start <= n; ++start) {
        if (visited[start]) continue;
        components.push_back({});
        stack<int> st;
        st.push(start);
        // 显式栈同样在入栈时标记，保证每个点最多入栈一次。
        visited[start] = true;
        while (!st.empty()) {
            int u = st.top();
            st.pop();
```

```
                components.back().push_back(u);
                for (int v : graph[u]) {
                    if (!visited[v]) {
                        visited[v] = true;
                        st.push(v);
                    }
                }
            }
        }
        return components;
}
```

易错点：

1. 一般图进入节点时就要标记，返回时才标记会在环上无限递归。
2. 递归深度接近 10^5 时有爆栈风险。
3. 有向图的“弱连通”不能直接用原图 DFS 得到。

参考：

- OI Wiki：DFS（图论）
- OI Wiki：DFS（搜索）

3.9 BFS 广度优先搜索

适用范围： 无权图最短路；BFS O(V+E)。输入邻接表和起点，输出距离/父节点；不可达距离为 –1，路径恢复对不可达点返回空。 **能做什么：** 求无权图单源最短距离并恢复路径。 **输入 / 输出：** 输入 1..n 邻接表和源/目标；不可达距离为 –1，路径恢复返回空向量。 **复杂度：** O(V+E).

用途： BFS 用队列按层扩展状态，在每条边代价相同的图中可求起点到各点的最少边数，并记录前驱还原路径。邻接表实现时间 $O(n+m)$、空间 $O(n+m)$。边权不全相等时不能直接使用；权值仅为 0/1 时可用双端队列优化。

思路：

- `distance[v] == -1` 表示不可达，首次入队时记录距离和前驱。
- 无向边加入两次，有向边只加入一次。
- 从多个起点求最近距离时，把所有起点以距离 0 同时入队。
- 用 `restorePath` 根据 `parent` 还原起点到终点的最短路径。

代码：

```
#include <bits/stdc++.h>
using namespace std;

struct BFSResult {
    vector<int> distance;
    vector<int> parent;
};

// 参数：1-based 单位边权图及一个或多个源点。返回：每点最短边数与父节点；不可达距离为 -1。效果：O(V+E)，只读图。
BFSResult bfs(const vector<vector<int>>& graph, const vector<int>& sources)
{
    // graph 与节点为 1-based；仅保证单位边权图上的最少边数。
    int n = static_cast<int>(graph.size()) - 1;
    vector<int> distance(n + 1, -1), parent(n + 1, -1);
    queue<int> q;
    for (int source : sources) {
        if (distance[source] != -1) continue;
        // 所有源点同时处于第 0 层；重复源点只入队一次。
        distance[source] = 0;
```

```
            q.push(source);
        }

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : graph[u]) {
                // 首次入队即得到最短距离并标记，避免同层其他点重复入队。
                if (distance[v] != -1) continue;
                distance[v] = distance[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
        return {distance, parent};
}

// 参数：目标点及 BFS 的 parent/distance 结果。返回：起点到目标的点序列；不可达返回空。效果：只读结果，O(路径长度)。
vector<int> restorePath(int target, const BFSResult& result) {
    // 不可达返回空数组；否则沿 parent 回到某个源点并反转为正向路径。
    if (result.distance[target] == -1) return {};
    vector<int> path;
    for (int v = target; v != -1; v = result.parent[v]) path.push_back(v);
    reverse(path.begin(), path.end());
    return path;
}
```

易错点：

1. 节点应在入队时标记；出队才标记会导致重复入队。
2. 普通 BFS 只保证“边数最少”，不保证任意带权图的距离最短。
3. 多源 BFS 的所有源点都应初始化为距离 0。

参考：

- OI Wiki：BFS（搜索）
- OI Wiki：BFS（图论）

3.10 回溯与剪枝

适用范围：解空间可逐步构造且能安全剪枝时适用；最坏指数时间。输入元素与目标，输出一个可行子集或空结果；剪枝条件必须不排除有效解。**能做什么：**用回溯搜索一个满足目标和的子集。**输入 / 输出：**输入非负数组与目标；返回一个解的元素值，未找到返回空；剪枝基于非负性。 **复杂度：** 指数级最坏情况.

用途：回溯按决策树枚举方案，尝试一个选择、递归、再撤销；剪枝提前排除不可能或不优的分支。最坏仍为指数或阶乘复杂度，空间通常为递归深度。适合规模小的排列、组合和约束满足；应优先安排限制强的决策并设计可证明安全的剪枝。

思路：

- 模板示例从输入元素中选取一个子集使和为 `target`，每个位置至多选一次，找到一组即停止。
- 仅当所有数非负时，`sum > target` 和“剩余和不足”剪枝才成立。
- 先排序，使相同数字相邻，从而跳过同层重复选择。
- 若要枚举全部方案，找到答案后保存并继续搜索，不要立即返回。

代码：

```
#include <bits/stdc++.h>
using namespace std;

// 找到一个和恰为 target 的子集，返回其元素；要求 a 中元素非负。
```

```cpp
// 参数：整数数组与目标和。返回：一组和为 target 的元素值；无解返回空。效果：复制输入并指数搜索；适用非负数剪枝条件。
vector<int> subsetSumBacktracking(vector<int> a, long long target) {
    // a 必须全为非负数；返回任意可行子集，空数组也可能是合法答案。
    sort(a.rbegin(), a.rend()); // 大数优先，通常更早触发剪枝
    int n = static_cast<int>(a.size());
    vector<long long> suffix(n + 1, 0);
    // suffix[i] 是尚未考虑的 a[i..n-1] 总和，用于剩余和不足剪枝。
    for (int i = n - 1; i >= 0; --i) suffix[i] = suffix[i + 1] + a[i];

    vector<int> path, answer;
    // 参数：下一候选下标与当前和。返回：是否找到解。效果：修改外层 path/answer，成功时保留答案。
    function<bool(int, long long)> dfs = [&](int index, long long sum) {
        if (sum == target) {
            answer = path;
            return true;
        }
        // 后两个剪枝依赖元素非负；允许负数时必须删除。
        if (index == n || sum > target || sum + suffix[index] < target) return false;

        for (int i = index; i < n; ++i) {
            // 只跳过同层重复值，不影响下一层选择另一个相同元素。
            if (i > index && a[i] == a[i - 1]) continue;
            path.push_back(a[i]);
            if (dfs(i + 1, sum + a[i])) return true;
            // 当前选择失败，必须恢复 path 后才能尝试同层下一个选择。
            path.pop_back();
        }
        return false;
    };

    dfs(0, 0);
    return answer;
}
```

易错点：

1. 递归返回后必须完整撤销对路径、计数器和标记数组的修改。
2. 去重条件是“同一递归层跳过相同选择”，不能误删不同位置的合法使用。
3. 不满足前提的剪枝会漏解，例如数组含负数时不能用当前和超界剪枝。

参考：

- OI Wiki：回溯法
- OI Wiki：剪枝

3.11 折半搜索

适用范围： n 约 40、元素非负且上限 `limit>=0` 的子集和最大化；两半枚举约 O(2^(n/2)) 空间。**能做什么：** 两半枚举合并，求不超过上限的最大子集和。 **输入 / 输出：** 输入 long long 数组及非负上限；返回不超过上限的最大子集和，允许空集，故总有解且答案至少为 0。 **复杂度：** 时间 O(n·2^(n/2))，空间 O(2^(n/2))。

用途： 折半搜索把 n 个决策分成两半，各自枚举约 $2^{n/2}$ 个状态，再排序、二分或双指针合并。适合 $n \approx 40$ 的子集和、计数和最优化；时间通常 $O(2^{n/2}n)$，空间 $O(2^{n/2})$。它以额外内存换取指数规模的平方根级下降。

思路：

- `maximumSubsetSumNotExceeding` 求不超过 `limit` 的最大子集和。

- 模板要求元素非负且 `limit>=0`；因此空集总是可行。若允许负数，仍可枚举，但剪枝和返回约定需调整。
- 两半分别生成子集和，右半排序后对每个左半和做 `upper_bound`。
- 若要求方案，子集和需同时保存对应掩码。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：数组与半开下标区间 [left,right)。返回：该半区间所有子集和。效果：无副作用，空间 O(2^k)。
vector<ll> allSubsetSums(const vector<ll>& a, int left, int right) {
    // a 使用 0-based，枚举左闭右开区间 [left, right) 的全部子集和。
    vector<ll> sums{0};
    for (int i = left; i < right; ++i) {
        int oldSize = static_cast<int>(sums.size());
        // 只遍历扩展前的旧状态，否则本轮元素会被重复选择。
        for (int j = 0; j < oldSize; ++j) sums.push_back(sums[j] + a[i]);
    }
    return sums;
}

// 参数：非负整数数组及 limit>=0。返回：不超过 limit 的最大子集和，允许空集故结果至少为 0。效果：O(2^(n/2) log 2^(n/2))。
ll maximumSubsetSumNotExceeding(const vector<ll>& a, ll limit) {
    // 元素要求非负；返回不超过 limit 的最大子集和，默认空集给出 0。
    int mid = static_cast<int>(a.size()) / 2;
    vector<ll> leftSums = allSubsetSums(a, 0, mid);
    vector<ll> rightSums = allSubsetSums(a, mid, static_cast<int>(a.size()));
    sort(rightSums.begin(), rightSums.end());

    ll answer = 0;
    for (ll sum : leftSums) {
        if (sum > limit) continue;
        // upper_bound 指向首个过大值，其前驱才是可配对的最大右半和。
        auto it = upper_bound(rightSums.begin(), rightSums.end(), limit - sum);
        if (it != rightSums.begin()) answer = max(answer, sum + *prev(it));
    }
    return answer;
}
```

易错点：

1. `2^(n/2)` 个和可能占用大量内存，元素和也可能溢出 `int`。
2. `upper_bound(x)` 指向第一个大于 `x` 的值，取可行值前必须先判断是否为 `begin()`。
3. 分组范围统一使用左闭右开，避免漏掉或重复中点元素。

参考：

- OI Wiki：双向搜索 / Meet in the middle

3.12 0-1 BFS

适用范围： 边权只为 0 或 1 的最短路；双端队列 O(V+E)。输入 1..n 编号图和起点，输出距离；不可达值为 INF。 **能做什么：** 在 0/1 权图求单源最短距离。 **输入 / 输出：** 输入 n、1..n 编号 0/1 权图及源点；返回距离数组，不可达为 INF。 **复杂度：** O(V+E).

用途与复杂度。 边权只取 0 或 1 的单源最短路可用双端队列在线处理：松弛到 0 边的新点放队首，1 边放队尾。时间 $O(n+m)$，空间 $O(n)$；一般非负边权应使用 Dijkstra。

思路与用法。 邻接表按 1-based 点编号，权值必须是 0 或 1。队列允许同一顶点多次入队；每次弹出时用当前最短距离松弛邻边。

代码：

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Edge01 {
    int to;
    int weight; // 必须为 0 或 1
};

// 参数：1..n 点的 0/1 权有向图与起点。返回：最短距离，未达点为 INF。效果：只读图，O(V+E)。
vector<int> zeroOneBfs(int n, const vector<vector<Edge01>>& graph, int source) {
    const int INF = numeric_limits<int>::max() / 4;
    vector<int> dist(n + 1, INF); // 点编号 1..n；不可达保持 INF
    deque<int> work;
    dist[source] = 0;
    work.push_front(source);

    while (!work.empty()) {
        int u = work.front();
        work.pop_front();
        for (const Edge01& edge : graph[u]) {
            int candidate = dist[u] + edge.weight;
            if (candidate >= dist[edge.to]) continue;
            dist[edge.to] = candidate;
            // 0 权边不增加距起点的层数，优先处理；1 权边排在后面。
            if (edge.weight == 0) work.push_front(edge.to);
            else work.push_back(edge.to);
        }
    }
    return dist;
}
```

易错点。 权值出现 2 或负数时该策略不再保证最短路；只在松弛成功后入队；不要在入队时永久标记 visited，因为后续可能找到更短距离。

3.13 常用构造题思路

适用范围： 输出一个满足条件的对象，且可从不变量、奇偶性、极值或递推关系直接拼出候选解。 **能做什么：** 用奇偶/模类分类、排序交错、分组配对、反向构造或归纳递推设计答案；图构造先核对度数和连通性。 **输入 / 输出：** 先明确实例参数和可行条件，再输出构造对象；本节代码构造 n 个互异正整数，使其和为 S。 **复杂度：** 示例 O(n) 时间、O(n) 空间。

常见构造策略应对应可验证的题面信号：

思路	适用信号与小例	限制与检查
不变量（奇偶/模类）	每步同时将两个数各加 1，总和增加 2，所以总和奇偶不变；若初始总和为偶数，目标奇数不可达	只给出不可能性判定，需从操作定义证明不变量确实保持
奇偶分组	需排列 1..n 且避免相邻差 1；n>=4 时先放偶数再放奇数并检查两个拼接边界	n<4 单独处理；仍须逐项检查相邻差

思路	适用信号与小例	限制与检查
分组配对	需让配对和相同；将 1..2k 配为 (i, 2k+1-i)，每对和为 2k+1	检查每个数恰用一次；额外范围/配对限制需另证
反向构造	目标最后一步容易确定；从目标状态删除一个元素倒推前态	倒推必须终止且每步可逆/保持可行
归纳递推	构造树可从单点开始，每次添加一个新顶点并连一条边；连通且无环保持，边数始终为顶点数减一	该扩展构造只覆盖树；额外度数限制需单独验证
图的度数/连通性	题目要求给定度数序列或连通图；先检查度数和为偶数	度数和偶数仅为必要条件；还需验证度数上界及连通性

这些方法依赖具体限制，不能只凭模式猜测；给出不变量证明，并对小规模枚举核验。

例：构造 n>=1 个互不相同的正整数且总和为 S。可行当且仅当 `S >= n(n+1)/2`（假设 n>=1）。从 `[1,2,...,n]` 开始，把多余量全部加到最大元素；它仍大于其余元素，因此互异性保持。下面的验算枚举小 n、S，检查可行性和输出性质。

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数：正整数个数 n 与目标和 S。返回：构造数组；不可行返回空。效果：无副作用，O(n)。
vector<long long> constructDistinctPositive(int n, long long sum) {
    if (n <= 0 || sum < 1LL * n * (n + 1LL) / 2) return {};
    vector<long long> a(n);
    iota(a.begin(), a.end(), 1LL);
    a.back() += sum - 1LL * n * (n + 1LL) / 2;
    return a;
}

// 参数：无。返回：进程状态 0 表示所有小规模构造验算通过。效果：断言检查可行性、互异、正数和总和。
int main() {
    for (int n = 1; n <= 8; ++n) {
        for (long long sum = 1; sum <= 80; ++sum) {
            auto a = constructDistinctPositive(n, sum);
            bool possible = sum >= 1LL * n * (n + 1LL) / 2;
            assert(static_cast<int>(a.size()) == (possible ? n : 0));
            if (possible) {
                set<long long> unique(a.begin(), a.end());
                assert(static_cast<int>(unique.size()) == n);
                assert(*min_element(a.begin(), a.end()) > 0);
                assert(accumulate(a.begin(), a.end(), 0LL) == sum);
            }
        }
    }
}
```

第四章

动态规划

4.1 DP 入门：状态、转移与顺序

1. 它是做什么的

本例解决“从一列数中选若干个，不能选相邻位置，求最大和”的问题。输入 `value[1..n]`，输出最大和及一组达到最大和的 1-based 下标。允许一个都不选，因此所有数为负时答案为 0。模板只适用于相邻约束和加法权值这一具体模型；其他题目需重新定义状态和转移。

令 `dp[i]` 表示只看前 `i` 个位置、允许空选时的最大和。对位置 `i`，要么不选它，答案沿用 `dp[i-1]`；要么选它，此时 `i-1` 不能选，候选值是 `dp[i-2]+value[i]`。初始化 `dp[0]=0`，`dp[1]=max(0,value[1])`。计算每个状态花费 $O(1)$，总时间 $O(n)$，保存 DP 表及输入需要 $O(n)$ 空间；恢复下标另需至多 $O(n)$ 时间和空间。

2. 变量在做什么

名称	含义
`value`	1-based 权值数组，`value[1]` 是第一个数；`value[0]` 不使用。
`n`	由 `value.size()-1` 得到的元素数；空输入约定为仅含占位的 `{0}`。
`dp[i]`	前 `i` 个位置允许空选时的最大总和，不是“必须选第 `i` 个”的最优值。
`chosen`	回溯过程中收集的 1-based 下标；最终反转为递增顺序。
`maximumSum`	`dp[n]`，即允许空选的最大和。

状态初始化为 0 表示空选可行。若多个方案同样最优，回溯遇到 `dp[i]==dp[i-1]` 时固定跳过 `i`，所以只保证返回其中一组，不保证特定的字典序或元素数最少。

3. 函数在做什么

函数	参数与返回	行为及前提
`maximumNonAdjacentSum`	`value[1..n]` 的 `long long` 权值；返回 `LineDPResult`。	返回最大和及一组互不相邻的 1-based 下标。允许空选；空输入 `{0}` 返回和 0、空下标。输入向量不修改。

`LineDPResult` 包含 `maximumSum` 和 `chosenIndices`。函数每次调用都新建 DP 表，状态不会跨调用保留。

4. 怎么调用

准备一个首位占位的向量，调用函数，再读取总和和下标。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct LineDPResult {
    ll maximumSum;
    vector<int> chosenIndices; // 1-based
};
```

```cpp

// 参数：value[1..n] 的 long long 权值。返回：最大和与一组 1-based 互不相邻下标；允许空选。效果：O(n) 时间、空间。
LineDPResult maximumNonAdjacentSum(const vector<ll>& value) {
    // value 使用 1-based；dp[i] 表示前 i 个位置允许空选时的最大和。
    int n = static_cast<int>(value.size()) - 1; // value[1..n]
    vector<ll> dp(n + 1, 0);
    if (n >= 1) dp[1] = max(0LL, value[1]);
    for (int i = 2; i <= n; ++i) {
        // 不选 i 继承 dp[i-1]；选 i 时前一个可选前缀只能到 i-2。
        dp[i] = max(dp[i - 1], dp[i - 2] + value[i]);
    }

    vector<int> chosen;
    // 从 dp[n] 逆推一组最优下标；相等时固定选择“不取 i”。
    for (int i = n; i >= 1;) {
        if (i == 1) {
            if (dp[i] > 0) chosen.push_back(1);
            break;
        }
        if (dp[i] == dp[i - 1]) {
            --i;
        } else {
            chosen.push_back(i);
            i -= 2;
        }
    }
    reverse(chosen.begin(), chosen.end());
    return {dp[n], chosen};
}
```

可在业务函数中直接使用以下输入和断言：权值 `[5,1,4]` 时选择位置 1 和 3，总和为 9。

```cpp
vector<ll> value = {0, 5, 1, 4};
LineDPResult result = maximumNonAdjacentSum(value);
assert(result.maximumSum == 9);
assert((result.chosenIndices == vector<int>{1, 3}));
```

5. 要改哪里

- 若允许空选，应保留当前 `max(0LL, value[1])` 初始化；若必须至少选一个，`dp` 不能全置 0，需用不可达哨兵并重写边界与恢复逻辑。
- 若不能相邻的限制改成至少间隔 `k` 个位置，选择 `i` 时依赖的前缀会改变为 `i-k-1`，同时处理下标越界的基例。
- 若要统计最优方案数，状态需同时维护最优值和方案数，并在两条转移相等时累加计数。
- 若只要最大和且不恢复方案，可用常数个变量滚动保存前两状态；当前接口返回下标，仍需保留 DP 信息或改用前驱记录。

6. 自检

输入 `[5,1,4]`，最优选择 `{1,3}`，和为 9。单元素 `[-3]` 返回 0 和空下标；单元素 `[6]` 返回 6 和 `{1}`。全负序列允许空选，答案仍为 0。重复权值 `[2,2,2]` 的最大和为 4，返回的一组下标应互不相邻且总权值确为 4。

7. 坑

1. 把 `dp[i]` 误解为“必须选 i”；这会导致初始化和回溯条件不匹配，先固定状态定义再写转移。
2. 允许空选却把 `dp[1]` 设成负数；全负输入会产生错误答案，应将其与 0 取最大。
3. 输入按 0-based 提供却直接传入；函数约定首位是占位，真实元素从下标 1 开始。
4. 只按 `dp[i]` 值恢复却忽略相等情况；平局时本实现跳过当前位置，得到的仍是最优方案。

8. 一句话原理

每个位置只需比较“不选它”和“选它并接上前一个不相邻前缀”的最优值，再沿同一转移回溯方案。

参考：

- OI Wiki：动态规划基础
- OI Wiki：记忆化搜索

4.2 0-1 背包

1. 它是做什么的

给定 `n` 件物品，每件有非负整数重量与 `long long` 价值，且每件最多选一次，求容量不超过 `capacity` 时的最大价值。模板另提供恰好装满版本：返回能恰好达到容量的最大价值，无解时返回空 `optional`。重量和价值数组按 0-based 下标对应，长度必须相同；容量非负且 `O(n*capacity)` 的时间和 `O(capacity)` 的内存可接受。

普通版本允许不装满，空选价值为 0；恰好版本只把重量 0 设为可达。处理每件物品时容量倒序，防止当前物品在同一轮被重复使用。两种接口每件物品最多更新 `capacity+1` 个状态，时间 $O(nC)$、额外空间 $O(C)$，其中 `C=capacity`。返回值和状态运算须能放入 `long long`；exact 版所有可达状态的最优值还须严格大于 `NEG_INF`，否则应另设可达标记。

2. 变量在做什么

名称	含义
`weight[i]`、`value[i]`	第 `i` 件物品的重量和值，`i` 从 0 到 `n-1`；两向量必须一一对应。
`capacity`	容量上限或恰好要求的重量，必须非负。
`dp[j]`（普通版）	处理过的物品中，总重量不超过 `j` 的最大价值。所有状态初值 0，表示可以不选。
`dp[j]`（恰好版）	当前恰好组成重量 `j` 的最大价值；仅 `dp[0]=0` 初始可达，其余用 `NEG_INF` 标记不可达。
`NEG_INF`	安全的负哨兵；不可达状态参与加法会产生虚假方案，所以转移前先判断。

倒序遍历的关键是更新 `dp[j]` 时，`dp[j-weight[i]]` 仍是处理第 `i` 件物品之前的状态。重量为 0 时，每个 `j` 仍只处理当前物品一次；正价值可加一次，负价值可不选。

3. 函数在做什么

函数	参数与返回	行为及前提
`zeroOneKnapsack`	平行的重量和值数组、非负容量；返回 `ll`。	返回重量不超过容量的最大价值，允许不装满；数组长度须相等，重量非负。
`exactZeroOneKnapsack`	同上；返回 `optional<ll>`。	求恰好装满的最大价值；无法凑出容量时返回 `nullopt`。

两个函数都不改输入，每次调用都重新建 DP 数组。重量必须为非负整数；若某个重量大于容量，它在对应转移循环中自然不会被选中。容量数组大小和计算结果须在可用整数范围内。

4. 怎么调用

准备平行数组并调用所需版本。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

```

```cpp
// 参数：平行的重量/价值数组与容量；每件至多取一次。返回：不超过容量的最大价值。效果：O(nC)，要求容量非负。
ll zeroOneKnapsack(const vector<int>& weight,
                   const vector<ll>& value,
                   int capacity) {
    // 物品为 0-based；dp[j] 表示总重量不超过 j 的最大价值。
    vector<ll> dp(capacity + 1, 0);
    for (int i = 0; i < static_cast<int>(weight.size()); ++i) {
        // 容量倒序保证右侧 dp[j-weight[i]] 尚未使用当前物品。
        for (int j = capacity; j >= weight[i]; --j) {
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    return dp[capacity];
}

// 恰好装满时的最大价值；不可达返回 nullopt。
// 参数：平行的重量/价值数组及非负容量；每件至多取一次。返回：恰好装满的最大价值，无解为空。效果：O(nC)。
optional<ll> exactZeroOneKnapsack(const vector<int>& weight,
                                  const vector<ll>& value,
                                  int capacity) {
    const ll NEG_INF = numeric_limits<ll>::lowest() / 4;
    // 除 dp[0] 外均不可达，因而该版本要求最终重量恰好为 capacity。
    vector<ll> dp(capacity + 1, NEG_INF);
    dp[0] = 0;
    for (int i = 0; i < static_cast<int>(weight.size()); ++i) {
        for (int j = capacity; j >= weight[i]; --j) {
            // 不可达状态不能参与加法，否则会制造不存在的方案。
            if (dp[j - weight[i]] != NEG_INF) {
                dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
            }
        }
    }
    if (dp[capacity] == NEG_INF) return nullopt;
    return dp[capacity];
}
```

以下输入中两件物品分别为 `(重量 2,价值 3)`、`(重量 3,价值 4)`。容量不超过 3 的最优价值为 4；恰好装满 3 也为 4；恰好装满 4 无解。

```cpp
vector<int> weight = {2, 3};
vector<ll> value = {3, 4};
assert(zeroOneKnapsack(weight, value, 3) == 4);
auto exact = exactZeroOneKnapsack(weight, value, 3);
assert(exact.has_value() && *exact == 4);
assert(!exactZeroOneKnapsack(weight, value, 4).has_value());
```

5. 要改哪里

- 若只需要最大价值且容量允许空余，调用普通版本；若必须刚好达到给定总重，调用 exact 版本，不能把两种初始化混用。
- 若问题变成完全背包（物品可重复选），状态转移方向需改为容量正序；只改循环方向会改变每件物品的使用次数语义。
- 若只问是否可行，可把价值状态换成布尔状态并用逻辑或转移；若还要求方案数，应改变状态聚合方式。
- 若要恢复具体选取物品，需要额外保存前驱/选择信息，当前接口只返回价值，不返回方案。
- 若重量可为负数或容量极大，当前按整数容量逐格枚举的表示不适用，应改用其他建模或算法。

6. 自检

物品 `(2,3)`、`(3,4)` 且容量 3 时，普通版和 exact 版都得到 4；容量 4 的 exact 版无解。容量为 0 且没有零重量物品时，两个版本均返回 0。单件零重量、价值 5 时，容量 0 的两版都只能把它选一次，答案为 5；这也可检查倒序循环在零重量时没有无限循环或重复选取。负价值物品不会提升普通版答案，因为允许不选。

7. 坑

1. 0-1 背包容量正序遍历；会让当前物品被重复使用，应倒序更新。
2. exact 版本把所有状态初始化为 0；这会把无法达到的重量误当成可达，除 `dp[0]` 外应使用不可达哨兵。
3. 对不可达状态直接加价值；先判断来源状态不等于 `NEG_INF`。
4. 误以为普通版本必须装满；它求的是重量不超过容量，空选可行。
5. 重量和值数组错位或长度不同；第 `i` 项必须代表同一件物品，并确保两数组等长。

8. 一句话原理

逐件物品倒序更新容量状态，让每次转移只使用上一轮结果，从而保证每件物品至多进入方案一次。

参考：

- OI Wiki：背包 DP

4.3 完全背包与多重背包

1. 它是做什么的

完全背包允许每种物品取任意非负件，多重背包给每种物品一个数量上限。两个接口都求总重量不超过 `capacity` 时的最大价值，允许空选。重量、价值数组按 0-based 对应；多重背包另有同长度的非负数量数组。完全背包要求重量为正，否则正价值零重物品可被无限取，最优值无界。所有重量和容量为非负整数，数组长度需匹配。

完全背包用容量正序转移，时间 $O(nC)$、额外空间 $O(C)$。多重背包将每类数量拆成若干 0-1 组，再倒序更新。令 `K` 为拆出的组数，时间为 $O(CK+n+K)$，空间为 $O(C+K)$（代码显式保存所有拆分组），其中 `K=O(\sum_i \log(count[i]+1))`。拆分组的重量和值以及 DP 结果都须在整数类型范围内。

2. 变量在做什么

名称	含义
`weight[i]`、`value[i]`	第 `i` 类物品单件重量和值，按 0-based 对应。
`count[i]`	多重背包中第 `i` 类最多可取的件数，必须非负。
`capacity`	总重量上限，必须非负。
`dp[j]`	已处理物品下，重量不超过 `j` 时的最大价值；初值 0 表示可留空。
`items`	多重背包二进制拆分后形成的 `(组重量, 组价值)` 列表；每组只能选一次。
`remaining`、`group`、`take`	拆分当前物品时的剩余数量、当前二进制组大小，以及本轮实际取出的数量。

完全背包的正序让 `dp[j-weight[i]]` 可包含当前物品，因此允许重复取；拆分后的组则必须倒序，避免同一组被重复使用。多重背包每类的组数量之和恰好等于其上限，最后一组取完剩余数量。

3. 函数在做什么

函数	参数与返回	行为及前提
`completeKnapsack`	平行重量/价值数组及容量；返回 `ll`。	求不超过容量的最大价值；重量须为正，两个数组长度相同。每次调用重新建 DP，不修改输入。

函数	参数与返回	行为及前提
`multipleKnapsack`	平行重量/价值/数量数组及容量；返回 `ll`。	每类最多取 `count[i]` 件，通过二进制分组转成 0-1 物品后求最大价值；三数组长度相同，重量、数量非负。每次调用重建状态。

4. 怎么调用

根据物品是否可重复选择调用对应函数。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：平行重量/价值数组与容量；每件可无限取。返回：不超过容量的最大价值。效果：O(nC)，重量须为正。
ll completeKnapsack(const vector<int>& weight,
                    const vector<ll>& value,
                    int capacity) {
    // weight 必须为正；dp[j] 表示容量上限为 j 的最大价值。
    vector<ll> dp(capacity + 1, 0);
    for (int i = 0; i < static_cast<int>(weight.size()); ++i) {
        // 正序使 dp[j-weight[i]] 可含当前物品，从而允许重复选择。
        for (int j = weight[i]; j <= capacity; ++j) {
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    return dp[capacity];
}

// 参数：重量、价值、数量上限数组与容量。返回：不超过容量的最大价值。效果：二进制拆分，O(CΣlog count)。
ll multipleKnapsack(const vector<int>& weight,
                    const vector<ll>& value,
                    const vector<int>& count,
                    int capacity) {
    // 每个二进制组视为一件只能选择一次的 0-1 物品。
    vector<pair<int, ll>> items;
    for (int i = 0; i < static_cast<int>(weight.size()); ++i) {
        int remaining = count[i];
        for (int group = 1; remaining > 0; group <<= 1) {
            // 最后一组取全部 remaining，确保拆分数量之和恰为 count[i]。
            int take = min(group, remaining);
            items.push_back({weight[i] * take, value[i] * take});
            remaining -= take;
        }
    }

    vector<ll> dp(capacity + 1, 0);
    for (auto [groupWeight, groupValue] : items) {
        // 拆分后的每组只能使用一次，因此容量必须倒序。
        for (int j = capacity; j >= groupWeight; --j) {
            dp[j] = max(dp[j], dp[j - groupWeight] + groupValue);
        }
    }
    return dp[capacity];
}
```

以下两类物品分别为 `(重量 2,价值 3)` 和 `(重量 3,价值 4)`。完全背包容量 6 可取三件第一类，重量 6、价值 9，比两件第二类的价值 8 更优。多重背包若第一类最多 2 件、第二类最多 1 件，容量 7 时可全取，价值 10。

```cpp
vector<int> weight = {2, 3};
vector<ll> value = {3, 4};
```

```
assert(completeKnapsack(weight, value, 6) == 9);
vector<int> count = {2, 1};
assert(multipleKnapsack(weight, value, count, 7) == 10);
```

5. 要改哪里

- 物品可无限使用时用完全背包；有有限数量上限时用多重背包；每件最多一次时应使用 0-1 背包。
- 完全背包重量必须为正。若重量 0 且价值为正，问题无有限最优值；若价值非正可忽略该类，但本模板没有为此作特殊处理。
- 多重背包只支持非负件数；`count[i]=0` 表示该类不进入拆分列表。
- 若容量很大而总价值较小，可改为“价值为状态、最小重量为状态值”的 DP；需重写状态和转移，不能仅调整循环方向。
- 要恢复选取数量，需要记录每个拆分组是否选中并将结果映射回原类别；当前接口只返回最大价值。

6. 自检

完全背包中重量和值分别为 `(2,3)`、`(3,4)`，容量 6 时取三件第一类，得到 9；这能区分完全背包与 0-1 背包。多重背包数量 `{2,1}`、容量 7 时三件都可选，重量 7、价值 10。数量全为 0 时答案为 0；容量为 0 时正重量物品均不能选。完全背包如传入零重量应判为输入违反前提，不应期待有限答案。

7. 坑

1. 完全背包容量倒序，导致每类只取一次；它应正序，以复用本轮状态。
2. 多重背包拆分后容量正序，导致单个分组可被重复用；拆分组是 0-1 物品，应倒序。
3. 拆分时遗漏最后的剩余件数；每轮 `take=min(group,remaining)`，才能让各组总数等于上限。
4. 完全背包允许零重量正价值物品；可无限取使答案无界，需排除此输入或另行定义。
5. 拆分后 `weight*take`、`value*take` 或累计结果溢出；按题目范围选择足够宽的类型并预先核算上界。

8. 一句话原理

完全背包用正序让当前物品参与后续状态，多重背包先把有限件数拆成 0-1 组再倒序更新。

参考：

- OI Wiki：背包 DP

4.4 最长上升子序列（LIS）

1. 它是做什么的

给定可比较序列，求一条最长严格递增子序列并返回它在原序列中的下标。子序列保持原有先后次序，但不要求连续。输入下标为 0-based；空输入返回空下标向量。模板用 `lower_bound` 实现严格递增，若要非下降子序列则需改变查找规则。

算法为每个元素二分查找一次，时间 $O(n \log n)$；前驱、尾值和尾下标数组共需 $O(n)$ 空间，输出也至多 $O(n)$。要求元素可由 `<` 严格弱序比较；模板不修改输入。

2. 变量在做什么

名称	含义
`a[i]`	输入第 `i` 个元素，`i` 从 0 开始。
`tailValue[len]`	当前已见元素中，长度为 `len+1` 的严格递增子序列所能达到的最小末值。
`tailIndex[len]`	对应 `tailValue[len]` 在原序列中的下标。
`previous[i]`	以 `a[i]` 结尾的当前构造链中，前一个元素的原数组下标；不存在时为 `-1`。

名称	含义
`position`	`a[i]` 在 `tailValue` 中第一个不小于它的位置，也是它更新的递增子序列长度减一。
`indices`	从最终尾下标沿 `previous` 逆向收集、再反转得到的答案。

`tailValue` 不是最终子序列：替换某个末值只是在保留该长度的更优延伸可能。实际方案通过 `tailIndex` 和 `previous` 追溯，因此输出下标严格递增，且对应值严格递增。

3. 函数在做什么

函数	参数、返回与行为
`longestIncreasingSubsequenceIndices<T>`	任意可严格比较的只读向量；返回 `vector<int>`。返回一条最长严格递增子序列的 0-based 下标；输入不改，空向量返回空向量。相同长度的多种答案中返回哪一条由更新和平局顺序决定。
`lower_bound`	在已排序的 `tailValue` 中查找首个 `>= a[i]` 的位置。`tailValue` 按严格递增维护；相等值会替换末尾而不增加长度，因此得到严格递增序列。

4. 怎么调用

将可比较序列传给模板，按返回下标读取元素。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

template <class T>
// 参数：可严格比较序列。返回：一个最长严格递增子序列的 0-based 下标。效果：无副作用，O(n log n)。
vector<int> longestIncreasingSubsequenceIndices(const vector<T>& a) {
    // a 与返回下标均为 0-based；本模板恢复严格上升子序列。
    int n = static_cast<int>(a.size());
    // tailValue[len] 是长度 len+1 的严格上升子序列的最小末值。
    vector<T> tailValue;
    // tailIndex 与 tailValue 同步，保存该末值来自的原数组下标。
    vector<int> tailIndex;
    vector<int> previous(n, -1);

    for (int i = 0; i < n; ++i) {
        // lower_bound 找首个 >= a[i] 的位置；改成 upper_bound 即求非下降。
        int position = static_cast<int>(lower_bound(tailValue.begin(),
                                                    tailValue.end(), a[i])
                                        - tailValue.begin());
        if (position == static_cast<int>(tailValue.size())) {
            tailValue.push_back(a[i]);
            tailIndex.push_back(i);
        } else {
            tailValue[position] = a[i];
            tailIndex[position] = i;
        }
        // 记录长度少一的最优末尾下标，供最终方案沿前驱恢复。
        if (position > 0) previous[i] = tailIndex[position - 1];
    }

    vector<int> indices;
    if (!tailIndex.empty()) {
        for (int i = tailIndex.back(); i != -1; i = previous[i]) {
            indices.push_back(i);
        }
        reverse(indices.begin(), indices.end());
    }
```

```
    return indices;
}
```

以下序列 `[3,1,2,2,4]` 的实现会返回 `{1,3,4}`，对应值 `{1,2,4}`，长度为 3：

```
vector<int> a = {3, 1, 2, 2, 4};
vector<int> indices = longestIncreasingSubsequenceIndices(a);
assert((indices == vector<int>{1, 3, 4}));
assert(a[indices[0]] < a[indices[1]] && a[indices[1]] < a[indices[2]]);
```

5. 要改哪里

- 若要求非下降子序列，将 `lower_bound` 换成 `upper_bound`，允许相等值接在已有序列后面。
- 若只需要 LIS 长度，可不保存 `tailIndex` 和 `previous`，只维护尾值数组；当前接口需要恢复下标，不能删掉前驱记录。
- 若要返回值而不是下标，可用返回下标在原数组中取值；若需自定义“递增”关系，必须让二分比较和尾值维护采用一致的严格弱序。
- 若需要统计 LIS 方案数或所有 LIS，单个最小末值状态不足，需要改成更丰富的 DP/数据结构状态。

6. 自检

对 `[3,1,2,2,4]`，输出下标应严格递增，值为严格递增的 `{1,2,4}`，长度 3。空序列返回空；单元素 `[7]` 返回一个有效下标 `{0}`。全相同的 `[5,5,5]` 的严格 LIS 长度应为 1；若把查找改为 `upper_bound`，则对应的非下降版本长度为 3。

7. 坑

1. 把子序列误当成连续子数组；LIS 可跳过中间元素，但下标必须递增。
2. 严格递增使用 `upper_bound`；相等值会被错误地接到序列末尾，应使用 `lower_bound`。
3. 直接把 `tailValue` 当作答案；它是各长度的最小末值集合，未必能对应同一条链，应沿前驱恢复。
4. 将返回下标当成 1-based；本函数输入与输出都按 0-based。
5. 输入类型没有一致的严格比较关系；二分查找要求比较规则有效且与尾值顺序一致。

8. 一句话原理

维护每种长度可达到的最小末值来二分更新，并用前驱下标把最优长度对应的真实子序列链接回来。

参考：

- OI Wiki：最长上升子序列

4.5 最长公共子序列（LCS）

1. 它是做什么的

给定两个序列，求一个最长公共子序列：结果元素按原顺序同时出现在两个输入中，但不要求连续。函数返回其中一条实际元素序列；存在多条最优解时不保证选哪一条。输入可以是任意支持相等比较的类型，空序列也合法。

令 `n=a.size()`、`m=b.size()`。二维 DP 填表需 $O(nm)$ 时间和 $O(nm)$ 空间，回溯方案另需 $O(n+m)$ 时间；输入不修改。模板返回序列本身，不能只把 DP 压成一行而仍按当前方式恢复方案。若只求长度，才可改用滚动数组压到 $O(\min(n,m))$ 空间。

2. 变量在做什么

名称	含义
`a`、`b`	两个 0-based 输入序列。
`n`、`m`	两个输入长度。
`dp[i][j]`	`a` 的前 `i` 项与 `b` 的前 `j` 项的 LCS 长度；多开一行一列表示空前缀，初值为 0。

名称	含义
`result`	回溯时逆序收集的公共元素，最后反转成原顺序。
`i`、`j`	回溯中当前前缀长度；对应比较的是 `a[i-1]` 与 `b[j-1]`。

若末项相等，模板将它们一起匹配并接在 `dp[i-1][j-1]` 的解后；否则在两种舍弃一个末项的可能中沿 DP 值不减小的一侧前进。相等时优先向上，因此并列 LCS 的选择受这一规则影响。

3. 函数在做什么

函数	参数、返回与行为
`longestCommonSubsequence<T>`	两个支持 == 的只读向量；返回 `vector<T>`。 返回一个最长公共子序列，元素顺序与两输入一致；空输入时返回空向量，不修改输入。元素类型还需可复制以写入结果。
`reverse`	反转已收集的结果。 将从末尾回溯得到的逆序元素恢复为正序；模板内部使用。

4. 怎么调用

直接传入两个同类向量，按返回序列读取。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

template <class T>
// 参数：支持相等比较的两序列。返回：一个最长公共子序列。效果：无副作用，O(nm) 时间与空间。
vector<T> longestCommonSubsequence(const vector<T>& a, const vector<T>& b) {
    // a、b 为 0-based；dp[i][j] 表示两序列前 i、j 个元素的 LCS 长度。
    int n = static_cast<int>(a.size());
    int m = static_cast<int>(b.size());
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    // i、j 递增，保证左、上、左上三个依赖状态已经计算。
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1]) {
                // 最优公共子序列可把这对相同末字符一起接到左上状态。
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                // 末字符不同时，至少舍弃一侧末字符，取两种可能的较优值。
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    vector<T> result;
    // 从右下角逆推；值相等走左上，否则走不减小 dp 的一侧。
    for (int i = n, j = m; i > 0 && j > 0;) {
        if (a[i - 1] == b[j - 1]) {
            result.push_back(a[i - 1]);
            --i;
            --j;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            --i;
        } else {
            --j;
        }
    }
    reverse(result.begin(), result.end());
    return result;
}
```

以下字符串序列用字符向量表示，公共子序列为 `a,c,e`：

```
vector<char> a = {'a', 'b', 'c', 'd', 'e'};
vector<char> b = {'a', 'c', 'e'};
vector<char> result = longestCommonSubsequence(a, b);
assert((result == vector<char>{'a', 'c', 'e'}));
```

5. 要改哪里

- 若只需 LCS 长度，可仅保留两行 DP，并根据依赖滚动更新；当前代码需要完整表来回溯具体序列。
- 若题目要求最长公共子串（连续片段），相等时的状态需从 `dp[i-1][j-1]` 延续，不等时应清零，不能使用本模板的取最大转移。
- 若需统计最优方案数或枚举所有 LCS，当前一个长度值和单一路径回溯不足，须改变状态及恢复方式。
- 若两输入类型不同，需将模板改为适当的比较器或统一元素类型；当前接口要求两个参数都是同一 `T` 类型的向量并支持 `==`。

6. 自检

`a=[a,b,c,d,e]`、`b=[a,c,e]` 时，LCS 是 `[a,c,e]`，长度 3。任一输入为空时返回空；两个序列完全相同则返回该序列；两个序列没有共同元素时也返回空。对重复元素输入，结果仍须是两边都能按顺序找到的序列，不能只检查长度。

7. 坑

1. 把 LCS 当成连续子串；子序列允许跳过元素，子串不允许。
2. DP 下标混淆前缀长度与原数组下标；状态 `[i][j]` 比较的是 `a[i-1]`、`b[j-1]`。
3. 不相等时直接令当前值为 0；LCS 应取上方和左方最大值。
4. 只返回 DP 长度却以为拿到了方案；当前函数返回的是元素序列，长度可用 `result.size()` 取得。
5. `n*m` 二维表过大导致内存不足；只需长度时改成滚动数组，若需恢复则考虑适合的分治恢复算法。

8. 一句话原理

用两个前缀的 LCS 长度作状态，按末元素是否相同递推，再从右下角沿保持最优值的方向恢复一条序列。

参考：

- OI Wiki：动态规划基础

4.6 区间 DP

1. 它是做什么的

本模板解决链上相邻石子合并：每次把相邻两堆合为一堆，代价等于这两堆总重量，求合成一堆的最小总代价。输入为 0-based 权值向量，输出最小费用；空输入或只有一堆时返回 0。若题目允许环形合并、合并代价不同或要求其他目标，需调整建模与转移。

`dp[left][right]` 表示闭区间合成一堆的最小代价。枚举区间内最后一次切分位置，左右分别先合并，再加上当前区间总和。前缀和构造为 $O(n)$；区间长度、起点和分割点三重循环，时间 $O(n^3)$；二维 DP 与前缀和共需 $O(n^2)$ 空间。实现使用 `long long` 和 `INF` 哨兵，需保证前缀和、候选费用不溢出且有效答案小于 `INF`。

2. 变量在做什么

名称	含义
`stones[i]`	第 `i` 堆的初始权值，`i` 从 0 到 `n-1`。
`prefix[i]`	前 `i` 堆的权值和；`prefix[0]=0`。
`dp[left][right]`	合并连续闭区间 `[left,right]` 为一堆的最小总代价；单点区间初值为 0。
`length`	当前枚举区间长度，从 2 增至 `n`。

名称	含义
`split`	最后一次合并前左右区间的分界，范围 `left..right-1`。
`intervalSum`	当前区间权值和，也就是把左右两堆最后合并时的新增费用。
`INF`	初始化尚未求出的多元素区间的候选最小值。

按长度递增保证 `[left,split]` 与 `[split+1,right]` 已经算好。前缀和用 `prefix[right+1]-prefix[left]` 在常数时间取区间权值。

3. 函数在做什么

函数	参数与返回	行为及前提
`minimumMergeCost(stones)`	`long long` 权值向量；返回 `ll`。	返回链式相邻合并的最小总费用。输入不修改；空向量和单元素向量返回 0。中间前缀和及 DP 值须可由 `ll` 表示，且有效费用不能与 `INF` 哨兵冲突。

函数每次调用都重新构建前缀和与 DP 表；模板没有返回合并顺序或树形结构的接口。

4. 怎么调用

把初始堆权值放入向量并调用函数。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

// 参数：相邻合并权值序列。返回：每次合并相邻段、代价为段和时的最小总费用。效果：O(n^3)，空/单元素费用为 0。
ll minimumMergeCost(const vector<ll>& stones) {
    // stones 使用 0-based；dp[left][right] 表示闭区间合成一堆的最小代价。
    int n = static_cast<int>(stones.size());
    if (n == 0) return 0;
    vector<ll> prefix(n + 1, 0);
    for (int i = 0; i < n; ++i) prefix[i + 1] = prefix[i] + stones[i];

    // dp[i][i]=0；按长度递增保证转移所需的两个子区间已经算完。
    vector<vector<ll>> dp(n, vector<ll>(n, 0));
    for (int length = 2; length <= n; ++length) {
        // 所有长度更短的区间都已完成，当前转移可直接读取子区间。
        for (int left = 0; left + length <= n; ++left) {
            int right = left + length - 1;
            dp[left][right] = INF;
            ll intervalSum = prefix[right + 1] - prefix[left];
            // split 将 [left,right] 分成 [left,split] 与 [split+1,right]。
            for (int split = left; split < right; ++split) {
                dp[left][right] = min(dp[left][right],
                    dp[left][split] + dp[split + 1][right] + intervalSum);
            }
        }
    }
    return dp[0][n - 1];
}
```

输入 `[1,2,3,4]` 的最优费用为 19：先合并 `1+2=3`，再合并 `3+3=6`，最后合并 `6+4=10`，总费用 `3+6+10`。

```cpp
vector<ll> stones = {1, 2, 3, 4};
assert(minimumMergeCost(stones) == 19);
assert(minimumMergeCost(vector<ll>{}) == 0);
```

5. 要改哪里

- 若求最大总代价，将区间状态的初值改为负无穷并将 `min` 改成 `max`；还需核对代价定义是否仍是区间和。
- 若合并代价不是当前区间权值和，替换 `+ intervalSum` 为题目定义的代价函数。
- 环形石子合并可将序列复制一遍，在长度至多为原 `n` 的区间中枚举起点，并在所有合法起点结果中取最优；当前函数只计算线性链。
- 若题目具备可证明的四边形不等式/决策单调性，可进一步优化分割点枚举；不能仅因状态形似就跳过 `split` 范围。
- 若需要输出合并顺序，应另存每个区间的最优 `split` 并递归恢复；当前接口只返回费用。

6. 自检

`[1,2,3,4]` 的最小费用为 19。空序列和单堆 `[8]` 均为 0。两堆 `[2,5]` 只能合并一次，答案为 7。三堆 `[1,2,3]` 可先合并 `1+2`，费用 `3+6=9`；也可先合并 `2+3`，费用 `5+6=11`，因此答案为 9。检查小规模时可枚举所有相邻合并顺序作对照。

7. 坑

1. 区间长度未从小到大枚举；转移读取的子区间可能尚未计算。
2. 分割点写到 `right`；右侧 `[split+1,right]` 将为空，合法范围应为 `left <= split < right`。
3. 把区间 DP 当成任意子集合并；题意要求相邻合并，状态必须是连续闭区间。
4. 前缀和右端写成 `prefix[right]`；`prefix[t]` 表示前 `t` 项，闭区间和应减 `prefix[left]`、取 `prefix[right+1]`。
5. 数值范围超出 `long long` 或答案碰到 `INF`；需依据输入上界选择安全类型/哨兵，并确保加法不溢出。

8. 一句话原理

将每个连续区间的最后一次合并位置作为分割点，组合已求出的左右最优值并加上整段合并费用。

参考：

- OI Wiki：区间 DP

4.7 树形 DP

1. 它是做什么的

本模板求无向树的最大权独立集：选出的节点中任意两点不能有边相连，最大化所选权值之和。输入是 1-based 连通无环树 `tree` 和 `weight[1..n]`，可指定根；返回根状态中的最大值。允许空集，因此若所有权值为负，答案为 0。若输入是森林，当前函数只遍历指定根所在连通分量。

每个节点在 DFS 中处理一次，每条树边被访问两次，时间 $O(n)$；DP 数组和递归调用栈共需 $O(n)$ 空间。递归深度最坏可达 `n`，长链可能超过运行环境的栈限制。要求根在 `1..n`，邻接表和权值数组至少有 `n+1` 个元素。

2. 变量在做什么

名称	含义
`tree[u]`	与节点 `u` 相邻的节点编号；无向边两端都需记录。
`weight[u]`	节点 `u` 的权值，编号从 1 开始，0 号位不用。
`root`	DFS 的起点，默认是 1。
`dp[u][0]`	在 `u` 的子树中不选 `u` 时，最大独立集权值。
`dp[u][1]`	在 `u` 的子树中选 `u` 时，最大独立集权值。
`parent`	DFS 中 `u` 的父节点；跳过这条边，避免在无向树中返回祖先。

转移为 `dp[u][0]=Σmax(dp[v][0],dp[v][1])`、`dp[u][1]=weight[u]+Σdp[v][0]`，其中 `v` 是孩子。DFS 后序保证先得到孩子状态，再合并到父节点。

3. 函数在做什么

函数	参数、返回与行为
`maximumWeightIndependentSet`	1-based 邻接表、权值向量、合法根（默认 1）；返回 `ll`。 返回根覆盖的整棵树上的最大权值独立集和；允许空集，不改输入。输入必须是连通无向树，数组需覆盖 `1..n`。
内部递归 `dfs(u,parent)`	当前节点与父节点；返回 `void`。 后序遍历并写入 `dp[u][0/1]`；只遍历根所在连通分量，正常由公开函数启动。

函数每次调用都会新建 DP 状态。权值及状态累加必须能放入 `long long`。

4. 怎么调用

用 `n+1` 大小的邻接表和权值数组，按无向边双向加边，然后调用函数。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：1..n 编号无向树。返回：不选相邻点时的最大权值和。效果：O(n)，权值按本函数参数约定给出。
ll maximumWeightIndependentSet(const vector<vector<int>>& tree,
                               const vector<ll>& weight,
                               int root = 1) {
    // tree 与 weight 均为 1-based；输入应为一棵连通无向树。
    int n = static_cast<int>(tree.size()) - 1;
    vector<array<ll, 2>> dp(n + 1, {0, 0});

    // 参数：当前点与父点。返回：无。效果：递归聚合子树状态到 dp[u]。
    function<void(int, int)> dfs = [&](int u, int parent) {
        // dp[u][0/1]：u 不选/选时，其整棵子树内可取得的最大权值。
        dp[u][0] = 0;
        dp[u][1] = weight[u];
        for (int v : tree[u]) {
            if (v == parent) continue;
            dfs(v, u); // 后序处理保证子树状态先于父状态完成
            // 不选 u 可自由选孩子最优状态；选 u 时所有孩子都必须不选。
            dp[u][0] += max(dp[v][0], dp[v][1]);
            dp[u][1] += dp[v][0];
        }
    };

    dfs(root, 0);
    return max(dp[root][0], dp[root][1]);
}
```

三节点树中，1 与 2、3 相连，权值分别为 `5,4,3`。选两个叶子得到 7，比只选根的 5 大：

```cpp
vector<vector<int>> tree(4);
tree[1] = {2, 3};
tree[2] = {1};
tree[3] = {1};
vector<ll> weight = {0, 5, 4, 3};
assert(maximumWeightIndependentSet(tree, weight) == 7);
```

5. 要改哪里

- 若节点权值改变，修改 `weight` 即可重新计算；此实现每次从头运行，不维护动态更新。

- 若要恢复选中的节点，需要记录每个节点状态下孩子取了哪个状态，再从根沿记录回溯；当前接口只返回最优和。
- 若改成“相邻必须满足某种代价/状态约束”，需扩展 DP 状态和孩子合并规则，不能只替换权值。
- 若输入是森林，需对每个连通分量分别调用或改成遍历所有根，并把各分量最优值相加；当前只启动一次根 DFS。
- 若树可能是极深链且递归栈不可靠，应将 DFS 改为显式栈后序处理。

6. 自检

三节点星形树权值 `[5,4,3]` 得 7。单节点正权 `[6]` 得 6；单节点负权 `[-6]` 因允许空集得 0。链 `1-2-3` 权值 `[4,10,4]` 时应选两端，得到 8，而不能选相邻的中间点与端点。每条无向边只建立一次逻辑连接但需在邻接表两端各记录一次。

7. 坑

1. 无向 DFS 没有跳过父节点；会沿边递归回去，应传 `parent` 并跳过。
2. 选了 `u` 仍给孩子取 `max(dp[v][0],dp[v][1])`；孩子必须不选，只能加 `dp[v][0]`。
3. 将权值/树按 0-based 构造；模板从节点 1 开始，0 号位是哨兵。
4. 输入含多个连通分量却只调用一次；未访问分量不会进入结果，需按森林处理。
5. 长链递归栈溢出；若约束允许深度达 `n`，考虑迭代后序遍历。

8. 一句话原理

把每个子树分成“根不选”和“根选”两种状态，利用选根时孩子必须不选这一约束自底向上合并。

参考：

- OI Wiki：树形 DP

4.8 状态压缩 DP

1. 它是做什么的

本模板求有向或无向带权图中的旅行商回路：从指定起点出发，每个节点恰好访问一次，最后回到起点，返回最小费用。`cost[u][v]` 是从 `u` 到 `v` 的边权；缺边必须用 `INF` 表示。无可行回路时返回 `INF`。节点和起点均为 0-based，要求 `n>=1`、起点在范围内、矩阵为 `n×n`。

用位掩码表示访问集合，状态数约 `n*2^n`。转移枚举当前终点和下一个未访问点，时间 $O(n^2 2^n)$；DP 空间 $O(n2^n)$，另有输入矩阵 $O(n^2)$。`1<<n` 必须在 `int` 位宽内且数组内存可分配；实际可处理规模通常更受内存限制。所有有效状态费用须小于 `INF`，相加中间值不能溢出 `long long`。

2. 变量在做什么

名称	含义
`cost[u][v]`	从节点 `u` 到 `v` 的费用；有向图可不对称，无边用 `INF`。
`start`	起点下标，默认 0；位掩码的初始集合只含该点。
`mask`	位集合；第 `u` 位为 1 表示节点 `u` 已访问。
`dp[mask][u]`	从 `start` 出发，访问集合恰为 `mask` 并停在 `u` 的最小路径费用。
`nextMask`	将未访问节点 `v` 加入后得到的新集合。
`all`	`1<<n`，所有掩码的数量；完整访问集合是 `all-1`。
`INF`	无边和不可达状态哨兵；代码仅将恰好等于它的值视作不可达。

初始化仅 `dp[1<<start][start]=0`。转移只扩展未访问节点；最后从完整集合的每个终点补上回到起点的边。无向图需在矩阵两方向都填边权；有向图按方向填值。

3. 函数在做什么

函数	参数与返回	行为及前提
`travelingSalesmanCycle`	方阵 `cost` 和合法 0-based 起点（默认 0）；返回 `ll`。	返回最小回路费用；不存在完整回路时返回 `INF`。输入不修改。矩阵至少有 `n` 行且每行 `n` 列，`n>=1`。

函数每次调用都会新建全部 DP 状态。对于 `n=1`，实现会把 `cost[0][0]` 当作回起点的边；若该项为 `INF` 则返回 `INF`。

4. 怎么调用

建立方阵，缺边位置设成 `INF`，再给出起点。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

// 参数：方阵 cost 与起点下标。返回：从起点出发访问每点一次并回起点的最小费用；无可行环返回 INF。效果：O(2^n n^2)，n 须较小。
ll travelingSalesmanCycle(const vector<vector<ll>>& cost, int start = 0) {
    // 节点为 0-based；cost[u][v] 可表示有向边，无边必须设为 INF。
    int n = static_cast<int>(cost.size());
    int all = 1 << n;
    // dp[mask][u]：从 start 出发，恰好访问 mask 中节点并停在 u 的最小代价。
    vector<vector<ll>> dp(all, vector<ll>(n, INF));
    dp[1 << start][start] = 0;

    for (int mask = 0; mask < all; ++mask) {
        if (!(mask & (1 << start))) continue;
        for (int u = 0; u < n; ++u) {
            // u 必须已访问且状态可达；INF 状态不能参与加法。
            if (!(mask & (1 << u)) || dp[mask][u] == INF) continue;
            for (int v = 0; v < n; ++v) {
                if (mask & (1 << v)) continue;
                if (cost[u][v] == INF) continue;
                // 只向未访问 v 扩展，因此每个节点在路径中恰好出现一次。
                int nextMask = mask | (1 << v);
                dp[nextMask][v] = min(dp[nextMask][v],
                                      dp[mask][u] + cost[u][v]);
            }
        }
    }

    ll answer = INF;
    // 最后补上 u -> start；若仍为 INF，表示不存在哈密顿回路。
    for (int u = 0; u < n; ++u) {
        if (dp[all - 1][u] != INF && cost[u][start] != INF) {
            answer = min(answer, dp[all - 1][u] + cost[u][start]);
        }
    }
    return answer;
}
```

下面的有向图中，最优回路 `0→1→2→0` 的费用为 `1+2+3=6`：

```cpp
vector<vector<ll>> cost = {
    {INF, 1, 10},
    {10, INF, 2},
    {3, 10, INF}
};
assert(travelingSalesmanCycle(cost, 0) == 6);
```

5. 要改哪里

- 若只求访问全部节点的最短路径、不要求回起点，最终在 `dp[all-1][u]` 中取最小值，不补 `cost[u][start]`。
- 若起点可任选，需分别以每个起点计算或修改初始化和终点闭合规则；当前默认固定从 `start` 出发。
- 若图是无向的，构造 `cost[u][v]` 时同时设置 `cost[v][u]`；当前转移本身按有向费用读取。
- 若要恢复访问顺序，需记录每个 `dp[nextMask][v]` 的前驱状态/终点，当前函数只返回费用。
- 若 `n` 稍大到 `2^n*n` 不可存储，换用剪枝搜索、近似算法或利用问题的特殊结构；改成更宽位掩码只能解决移位范围，不能降低指数级内存。

6. 自检

示例矩阵的最优环 `0→1→2→0` 费用为 6，反方向路线费用为 30。若删去 `2→0` 边，则没有闭合回路，结果为 `INF`。`n=1` 时结果等于对角边费用；空图不合法。再检查所用 `INF` 大于所有合法路线费用且累加过程不溢出。

7. 坑

1. 缺边用 0 表示；0 可能是合法边权，模板约定缺边必须用 `INF`。
2. 把终点最小路径当作回路；必须确认最后一条 `u→start` 边存在并计入费用。
3. 对 `INF` 状态直接加边权；先跳过不可达状态和缺边，避免溢出伪造可达值。
4. 位掩码移位超出 `int` 范围；约束需保证 `1<<n` 有效且状态数组可分配。
5. 估算内存时只看 `2^n`；还要乘终点数 `n`，并考虑 DP 单元大小和输入矩阵。

8. 一句话原理

用集合掩码和当前终点描述已走过的路径，对每个未访问点扩展一次，最后把完整路径闭合回起点。

参考：

- OI Wiki：状压 DP

4.9 数位 DP

1. 它是做什么的

本模板统计闭区间 `[left,right]` 中，十进制数位和等于 `targetSum` 的非负整数个数。`countDigitSumUpTo(n,s)` 统计 `[0,n]`，负上界或负目标和返回 0；区间函数要求 `0<=left<=right`。数字按普通十进制表示，前导零只用于补齐递归位数，不改变数位和，也不会把同一整数重复计数。

设上界有 `D` 位，目标和为 `S`。每个位置、已用和与上界限制状态至多计算一次，每个状态枚举至多 10 个数字，时间 $O(D(S+1)\cdot 10)$；记忆表空间 $O(D(S+1))$，递归深度 $O(D)$。计数用 `long long`，若题目要求取模或计数可能超出类型范围，须修改转移。

2. 变量在做什么

名称	含义
`digits`	`n` 的十进制字符串，从最高位到最低位处理。
`position`	当前处理的位下标，范围 `0..length`。
`sum`	已处理位的数位和；大于 `targetSum` 时可剪枝。
`tight`	当前已选前缀是否仍等于上界前缀；为真时当前位最大只能取上界数字。
`memo[position][sum]`	仅针对 `tight=false` 状态缓存的补全方案数；受限状态随上界前缀变化，不共用该缓存。
`upper`	当前位可选择的最大数字：受限时为上界当前位，否则为 9。
`answer`	当前状态所有合法下一位选择的计数总和。

到达末位后，只有累计和恰好等于目标才贡献 1。下一位仍受限的条件是“原状态受限且当前数字取到了原上界位”。前导零自然作为普通的 0 转移，因此数字 0 被计入：当目标和为 0 时，`[0,n]` 至少包含数字 0。

3. 函数在做什么

函数	参数与返回	行为及前提
`countDigitSumUpTo`	`long long` 上界和整数目标和；返回 `ll`。	返回 `[0,n]` 中符合条件的个数。`n<0` 或目标和负时返回 0；每次调用新建 memo。
内部递归 `dfs(position,sum,tight)`	当前位、已用数位和、上界限制标记；返回 `ll`。	递归枚举本位可选数字，并只缓存不受限状态；由前缀函数内部调用。
`countDigitSumInRange`	非负闭区间及目标和；返回 `ll`。	用两个前缀计数相减得到区间计数；要求 `0<=left<=right`，函数自身不校验区间顺序。

这两个函数不修改输入。`countDigitSumInRange` 会各调用一次前缀计数，复杂度为两次同阶计算。

4. 怎么调用

统计 `[0,n]` 用前缀函数；统计闭区间用范围函数。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 参数：非负上界 n 与目标数位和。返回：[0,n] 中数位和等于目标的整数个数。效果：O(位数×targetSum)。
ll countDigitSumUpTo(ll n, int targetSum) {
    // 统计闭区间 [0,n]；n<0 或目标和为负时统一返回 0。
    if (n < 0 || targetSum < 0) return 0;
    string digits = to_string(n);
    int length = static_cast<int>(digits.size());
    vector<vector<ll>> memo(length, vector<ll>(targetSum + 1, -1));

    // position 从最高位向低位；tight 表示当前前缀是否仍贴住 n 的上界。
    // 参数：当前数位位置、已用数位和及是否受上界限制。返回：满足目标数位和的补全数。效果：写入 memo，避免重复状态计算。
    function<ll(int, int, bool)> dfs = [&](int position, int sum, bool tight) -> ll {
        if (sum > targetSum) return 0;
        if (position == length) return sum == targetSum;
        // 仅非受限状态与更高位选择无关，可以跨路径复用记忆化值。
        if (!tight && memo[position][sum] != -1) return memo[position][sum];

        int upper = tight ? digits[position] - '0' : 9;
        ll answer = 0;
        for (int digit = 0; digit <= upper; ++digit) {
            // 只有原状态受限且本位取到当前上界，下一位才继续受限。
            answer += dfs(position + 1, sum + digit,
                          tight && digit == upper);
        }
        if (!tight) memo[position][sum] = answer;
        return answer;
    };

    return dfs(0, 0, true);
}

// 参数：非负闭区间 [left,right] 与目标数位和。返回：区间内符合条件的整数个数。效果：调用两次 countDigitSumUpTo。
ll countDigitSumInRange(ll left, ll right, int targetSum) {
    // 用前缀计数相减得到闭区间 [left,right]；left-1 由前缀函数处理负数。
    return countDigitSumUpTo(right, targetSum)
         - countDigitSumUpTo(left - 1, targetSum);
}
```

`[0,20]` 中数位和为 2 的数是 `2`、`11`、`20`，共 3 个；`[10,22]` 中是 `11`、`20`，共 2 个。

```
assert(countDigitSumUpTo(20, 2) == 3);
assert(countDigitSumInRange(10, 22, 2) == 2);
assert(countDigitSumInRange(0, 0, 0) == 1);
```

5. 要改哪里

- 若统计范围是 `[left,right]`，保留前缀相减，但确保边界非负且 `left<=right`。
- 若约束涉及相邻位、是否出现过某数字或数字长度，需增加相应状态维度；当前 `position,sum,tight` 只记录数位和与上界关系。
- 若前导零对约束有影响（例如统计实际位数或禁止前导模式），需加入 `started` 状态，区分尚未开始构造数字与数字中的 0。本模板的数位和统计不需要该状态。
- 若结果要求模 `M`，在每次累加后取模，并确保 memo 的未计算标记不与合法值混淆。
- 若计数上界超出 `long long` 可表示范围或目标和过大导致表无法分配，应改用更宽计数/字符串上界或重新设计状态范围。

6. 自检

`[0,20]` 中数位和为 2 的数只有 `2`、`11`、`20`，结果 3。`[10,22]` 中只有 `11`、`20`，结果 2。`[0,0]` 且目标和 0 返回 1，说明零会计入；目标和大于上界十进制位数乘 9 时返回 0。单点区间 `[7,7]` 的目标和 7 返回 1，目标和 6 返回 0。

7. 坑

1. `tight` 更新写成只看当前数字是否等于 `upper`；当原来不受限时不能重新变受限，条件必须保留原 `tight`。
2. 把受限状态写进只按 `(position,sum)` 索引的 memo；不同上界前缀可能有不同上界位，代码只缓存 `tight=false`。
3. 把前导零当作实际数位限制；数位和下前导零无影响，但相邻位等条件通常需要 `started` 状态。
4. 区间减法使用 `left-1` 却未处理 0；此实现的前缀函数对负上界返回 0，故 `left=0` 正常。
5. 忘记 0 是否计数；本实现统计 `[0,n]`，目标和为 0 时数字 0 会被计入。

8. 一句话原理

从高位向低位枚举数字，用已用数位和与是否贴住上界描述状态，并复用所有不再受上界约束的子问题。

参考：

- OI Wiki：数位 DP

4.10 DAG 上的 DP

1. 它是做什么的

给定 1-based 有向无环图和源点，函数求源点到每个节点的最长路权值。边权可以为负，也可有重边；无向边不适用。不可达节点的结果为 `NEG_INF`；若图中任意位置存在有向环，拓扑序无法覆盖全部节点，函数返回空向量表示失败。

构造入度并拓扑排序遍历每条边一次，松弛也遍历每条边一次，时间 $O(V+E)$。辅助的入度、队列、拓扑序和距离数组占 $O(V)$ 额外空间；输入邻接表另占 $O(V+E)$。点编号为 `1..n`，要求 `graph` 有 `n+1` 个槽位，源点和每条边终点均合法。距离累加须在 `long long` 范围内，所有有限最长距离须严格大于 `NEG_INF`。

2. 变量在做什么

名称	含义
`graph[u]`	从 `u` 出发的有向边列表，每条 `Edge` 存 `to` 和 `weight`。
`indegree[v]`	进入 `v` 的边数；重边按多条边计数，出队处理时逐条减回。

名称	含义
`q`、`order`	入度为 0 的节点队列及其拓扑序。
`source`	最长路起点，初始距离为 0。
`distance[u]`	当前已确定的源点到 `u` 最长路；`NEG_INF` 表示尚不可达。
`edge`	当前待处理有向边，松弛候选为 `distance[u]+edge.weight`。
`NEG_INF`	不可达哨兵；不可达节点不能参与边权加法。

代码把所有入度为 0 的点都加入队列，所以即使 DAG 有多个连通部分也能检查全图是否有环。拓扑排序长度不足 `n` 时直接失败；若图合法，未从源点可达的其他分量仍保留 `NEG_INF`。

3. 函数在做什么

函数/类型	参数与返回	行为及前提
`Edge`	字段 `to`（终点）和 `weight`（边权）。	表示一条有向边；`to` 必须为 `1..n`。
`longestPathInDAG`	1-based 邻接表及合法源点；返回 `vector<ll>`。	合法 DAG 时返回长度 `n+1` 的距离表，0 号位不用；有环返回空向量。输入不修改。

本函数不返回具体路径，只返回每点最长距离。每次调用重建拓扑序和距离状态；图必须整体是 DAG，即使环与源点不连通也会失败。

4. 怎么调用

按方向建立邻接表，用 `Edge{终点,权值}` 表示边，调用后先检查结果是否为空，再读取距离。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll NEG_INF = numeric_limits<ll>::lowest() / 4;

struct Edge {
    int to;
    ll weight;
};

// 参数：1..n 编号 DAG 邻接表与源点。返回：各点最长路权值；不可达为 NEG_INF。效果：O(V+E)，图必须无环。
vector<ll> longestPathInDAG(const vector<vector<Edge>>& graph, int source) {
    // graph 与节点为 1-based、有向、可含重边和负权，但必须无环。
    int n = static_cast<int>(graph.size()) - 1;
    vector<int> indegree(n + 1, 0);
    for (int u = 1; u <= n; ++u) {
        for (const auto& edge : graph[u]) ++indegree[edge.to];
    }

    queue<int> q;
    // 所有入度为 0 的点同时入队，保证断开的 DAG 也能完成拓扑排序。
    for (int u = 1; u <= n; ++u) {
        if (indegree[u] == 0) q.push(u);
    }
    vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (const auto& edge : graph[u]) {
            if (--indegree[edge.to] == 0) q.push(edge.to);
```

```
        }
    }
    // 拓扑序不含全部节点说明存在有向环；用空数组作为失败状态。
    if (static_cast<int>(order.size()) != n) return {};

    // distance[u] 为 source 到 u 的最长距离；NEG_INF 表示不可达。
    vector<ll> distance(n + 1, NEG_INF);
    distance[source] = 0;
    for (int u : order) {
        // 不可达点不能松弛出边，否则负无穷参与加法会产生伪状态。
        if (distance[u] == NEG_INF) continue;
        for (const auto& edge : graph[u]) {
            distance[edge.to] = max(distance[edge.to],
                                    distance[u] + edge.weight);
        }
    }
    return distance;
}
```

下例中最长路到节点 4 为 `1→3→4`，长度 7；孤立节点 5 不可达，距离为 `NEG_INF`：

```
vector<vector<Edge>> graph(6);
graph[1] = {{2, 3}, {3, 2}};
graph[2] = {{4, 4}};
graph[3] = {{4, 5}};
vector<ll> distance = longestPathInDAG(graph, 1);
assert(distance.size() == 6);
assert(distance[4] == 7);
assert(distance[5] == NEG_INF);
```

5. 要改哪里

- 求 DAG 最短路时，将初值哨兵改为正无穷并把 `max` 松弛改成 `min`；仍须跳过不可达状态。
- 若只需从源点可达部分且想容忍图其他区域有环，需改变环检测范围；当前拓扑排序检查整张图。
- 若要返回具体路径，松弛成功时记录 `edge` 的前驱节点，再从终点反向恢复；当前接口不存前驱。
- 若要统计路径数或最优路径数，状态聚合不再是单个最大距离，需额外维护计数并明确并列最优时的合并规则。
- 若图有环，不能直接使用 DAG DP；可按题意缩点、处理强连通分量，或选择适用于有环图的算法。

6. 自检

示例中 `1→2→4` 权值 7，`1→3→4` 权值 7? 其中第二条为 `2+5=7`，两条并列，结果到 4 为 7。节点 5 无入边但从源点不可达，仍为 `NEG_INF`。若增加边 `4→2`，图出现环，函数应返回空向量；若源点出发的边权是负数，可达节点的距离应保留该负值，不能误当作不可达或 0。

7. 坑

1. 按节点编号直接松弛而非拓扑顺序；后继可能先于前驱处理，应遍历 `order`。
2. 把不可达初始化为 0；负权路径会被错误压过，使用 `NEG_INF` 并跳过不可达点。
3. 只检测源点所在部分是否有环；实现检查全图，任何环都会导致空结果。
4. 将无向边只存一侧或将有向边自动补反向；邻接表必须忠实表达图方向。
5. 把返回空向量当作“所有点不可达”；它表示图不是 DAG，合法 DAG 的不可达节点仍在长度 `n+1` 的结果中并取 `NEG_INF`。

8. 一句话原理

按拓扑序确保所有前驱先完成，再把每条边的最长距离候选传播到后继。

参考：

- OI Wiki：DAG 上的 DP

4.11 编辑距离

1. 它是做什么的

本函数计算把字符串 `a` 变成 `b` 所需的最少操作数，每次插入、删除或替换一个字符的代价都是 1。输入是两个 `std::string`，返回 `int` 距离。实现按 `string` 的 `char` 单位比较；UTF-8 多字节字符会被当作多个字节处理，若题目按 Unicode 字符计数需先解码。

设长度为 `n`、`m`，每个前缀状态只做常数次转移，时间 $O(nm)$。滚动数组保存第二个字符串的各前缀，额外空间为 $O(m)$；代码不会交换字符串以压缩到较短一侧。输入不修改。

2. 变量在做什么

名称	含义
`a`、`b`	源字符串与目标字符串，比较下标为 0-based。
`dp[j]`	当前已处理的 `a` 前缀与 `b` 的前 `j` 个字节之间的编辑距离。
`i`、`j`	当前源前缀长度和目标前缀长度；字符下标分别为 `i-1`、`j-1`。
`diagonal`	覆写当前格之前的左上旧值，即前一行的 `dp[j-1]`。
`oldAbove`	覆写前的上方旧值，即前一行的 `dp[j]`。
`dp[j-1]`	本行已更新的左侧状态。

初始化 `dp[j]=j` 表示空串变成 `b` 的前 `j` 个字符需要 `j` 次插入；每行开头将 `dp[0]=i`，表示将 `a` 的前 `i` 个字符删空。字符相同直接继承左上值；否则从删除、插入、替换三种操作中取最小值并加 1。

3. 函数在做什么

函数	参数与返回	行为及前提
`editDistance(a,b)`	两个只读 `string`；返回 `int`。	返回单位插入/删除/替换代价下的最小编辑次数；空字符串合法，不改输入。

函数每次调用都新建滚动数组。字符串长度和最终距离需能由 `int` 表示。

4. 怎么调用

直接传入源字符串与目标字符串，返回值是最少操作次数。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数：两字符串。返回：插入、删除、替换代价均为 1 的最小编辑次数。效果：O(|a||b|) 时间、O(|b|) 空间。
int editDistance(const string& a, const string& b) {
    // dp[j] 表示当前 a 前缀与 b 的前 j 个字符之间的编辑距离。
    vector<int> dp(b.size() + 1);
    iota(dp.begin(), dp.end(), 0); // 空串变成 b 前 j 字符需要 j 次插入

    for (int i = 1; i <= static_cast<int>(a.size()); ++i) {
        int diagonal = dp[0]; // 覆盖前保存左上角的旧状态
        dp[0] = i; // b 为空时只能删除 a 的 i 个字符
        for (int j = 1; j <= static_cast<int>(b.size()); ++j) {
            int oldAbove = dp[j];
            if (a[i - 1] == b[j - 1]) {
                dp[j] = diagonal;
            } else {
                dp[j] = 1 + min({oldAbove, dp[j - 1], diagonal});
            }
            diagonal = oldAbove;
        }
```

```
    }
    return dp[b.size()];
}
```

`kitten` 变为 `sitting` 至少需要 3 次操作；空串变为 `abc` 需要 3 次插入：

```
assert(editDistance("kitten", "sitting") == 3);
assert(editDistance("", "abc") == 3);
assert(editDistance("abc", "abc") == 0);
```

5. 要改哪里

- 若插入、删除、替换代价不同，转移应分别加上对应代价，不再统一 `+1`。
- 若只允许插入/删除，可将问题转为最长公共子序列长度 `|a|+|b|-2*LCS(a,b)`；当前实现还允许替换。
- 若要输出具体编辑脚本或对齐方案，需保存前驱/完整 DP 表；滚动版本只返回距离。
- 若要处理 UTF-8 文本并按字符计算，先将字符串解码为码点序列，再在该序列上运行等价 DP。

6. 自检

`kitten` 到 `sitting` 的距离为 3。相同字符串距离为 0；空串到长度为 `k` 的字符串距离为 `k`，反向也为 `k`。单字符 `a` 到 `b` 距离为 1。可用对称性检查交换两输入后距离不变，因为这三种单位操作的代价对称。

7. 坑

1. 更新 `dp[j]` 前没保存旧值；上方旧值和本行左值混用会破坏转移，需用 `oldAbove` 与 `diagonal` 保存状态。
2. 空前缀初始化为 0；`dp[j]=j` 与每行 `dp[0]=i` 分别代表插入和删除代价。
3. 把 `dp[j-1]` 当成左上旧值；它已是本行新值，左上值应单独保存在 `diagonal`。
4. 期望 UTF-8 字符按一个字符处理；本函数实际比较字节，须先解码。
5. 认为长字符串也适合本算法；空间虽为线性，时间仍是 $O(nm)$，大输入可能过慢。

8. 一句话原理

滚动维护两个前缀的编辑距离，并在每个字符对上从删除、插入、替换三种最后操作中取最小代价。

4.12 树形 DP 与换根

1. 它是做什么的

本模板计算无权树中每个顶点到所有顶点的距离和。顶点编号为 `1..n`，边无方向且每条边长度视为 1；`solve()` 返回长度 `n+1` 的向量，`answer[u]` 位于下标 `u`，0 号位不用。要求 `n>=1`，输入恰为一棵树。

先以 1 为根做 DFS，计算子树大小和根 1 的距离和；再沿每条父子边换根一次。每个点和边处理常数次，构造及添加 `n-1` 条边为 $O(n)$，`solve()` 为 $O(n)$；邻接表、状态数组和递归栈共需 $O(n)$ 空间。答案最大可达 $O(n^2)$，使用 `long long` 存储。

2. 变量在做什么

名称	含义
`n`	树的顶点数，节点编号为 `1..n`。
`graph[u]`	无向邻接表；每条输入边在两个端点各记录一次。
`subtreeSize[u]`	初次以 1 为根时，`u` 子树中的节点数，包含 `u`。
`answer[u]`	以 `u` 为根时到全体顶点的距离和。
`depth`	第一遍 DFS 中节点到固定根 1 的边数。
`parent`	DFS 父节点；跳过该邻接点以免沿树边返回。

从父节点 u 换根到孩子 v 时，v 子树内的 subtreeSize[v] 个点距离减 1，其余 n-subtreeSize[v] 个点距离加 1，因此 answer[v]=answer[u]+n-2*subtreeSize[v]。第一遍累加得到 answer[1]，第二遍沿树逐个转移。

3. 函数在做什么

函数	参数与返回	行为及前提
RerootDistanceSum(n)	正顶点数；构造对象。	分配 1-based 邻接表及状态数组，答案初值为 0。
addEdge(u,v)	树中两个合法相邻端点；返回 void。	将无向边加入邻接表两端；所有边须在 solve 前添加，输入需保持为树。
内部 firstDfs(u,parent,depth)	当前点、父点和深度；返回 void。	填子树大小并累加根 1 的距离和；由 solve() 调用。
内部 rerootDfs(u,parent)	当前根候选及父点；返回 void。	根据子树大小把父答案转移到孩子，遍历全部节点；由 solve() 调用。
solve()	无参数；返回 vector<ll>。	运行两遍 DFS 并返回内部答案副本。每个对象仅调用一次；重复调用会再次累加 answer[1]，结果错误。

addEdge 不检查端点、重复边或环；调用者须提供合法树。递归深度最坏为 n。

4. 怎么调用

构造对象，加入所有无向边，再调用一次 solve() 并用节点编号访问结果。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct RerootDistanceSum {
    int n;
    vector<vector<int>> graph;
    vector<int> subtreeSize;
    vector<ll> answer;

    // 参数：树的顶点数 n>=1。返回：清零的换根 DP 对象。效果：分配 1..n 邻接表及状态数组。
    explicit RerootDistanceSum(int n)
        : n(n), graph(n + 1), subtreeSize(n + 1), answer(n + 1, 0) {}

    // 参数：树中相邻顶点 u、v。返回：无。效果：向无向邻接表加入一条边。
    void addEdge(int u, int v) {
        graph[u].push_back(v);
        graph[v].push_back(u); // 无向树
    }

    // 参数：当前点、父点和深度。返回：无。效果：计算子树大小与初始根答案。
    void firstDfs(int u, int parent, int depth) {
        subtreeSize[u] = 1;
        answer[1] += depth; // 第一遍累加以 1 为根的距离和
        for (int v : graph[u]) {
            if (v == parent) continue;
            firstDfs(v, u, depth + 1);
            subtreeSize[u] += subtreeSize[v];
        }
    }

    // 参数：当前根及其父点。返回：无。效果：将父点答案转移到子点并遍历整棵树。
    void rerootDfs(int u, int parent) {
        for (int v : graph[u]) {
            if (v == parent) continue;
```

```
            answer[v] = answer[u] + n - 2LL * subtreeSize[v];
            rerootDfs(v, u);
        }
    }

    // 参数：无；须已加入全部树边且仅对新对象调用一次。返回：answer[1..n] 的各根距离和。效果：O(n)，写入并返回内部答案副本。
    vector<ll> solve() {
        firstDfs(1, 0, 0);
        rerootDfs(1, 0);
        return answer; // answer[1..n] 有效
    }
};
```

路径 `1-2-3` 的三个距离和依次为 3、2、3：

```
RerootDistanceSum solver(3);
solver.addEdge(1, 2);
solver.addEdge(2, 3);
vector<ll> answer = solver.solve();
assert(answer.size() == 4);
assert(answer[1] == 3 && answer[2] == 2 && answer[3] == 3);
```

5. 要改哪里

- 若边 `(u,v)` 长度为 `w`，跨边转移改为 `answer[v]=answer[u]+w*(n-2*subtreeSize[v])`。若按节点权重求加权距离和，则应将子树节点数替换为子树权重总和，并用全树权重总和替换 `n`。
- 若要统计每点距离平方和或其他距离函数，通常需扩展子树聚合量及转移式；当前状态只支持无权距离和。
- 若要对森林求结果，需分别处理每个连通分量并设定每个分量的节点数；当前以节点 1 启动，要求整图连通。
- 多组数据应为每组新建对象；不要在同一对象上再次 `solve()`，也不要在第一次求解后继续加边。
- 若树很深且递归栈有限，将两遍 DFS 改为迭代遍历并按逆序完成子树聚合。

6. 自检

路径 `1-2-3` 的距离和是 `[3,2,3]`。单节点树 `n=1` 返回 `answer[1]=0`。星形树中心 1 连三个叶子时，中心答案为 3，每个叶子的答案为 5。可检查总距离和对称：所有 `answer[u]` 之和应为所有无序点对距离总和的两倍。

7. 坑

1. 无向边只加入一侧；DFS 会漏点或结果错误，每条边应调用一次 `addEdge(u,v)`，由接口负责双向记录。
2. 误用 0-based 节点；对象的有效下标为 `1..n`，`answer[0]` 不代表节点。
3. 在同一对象重复 `solve()`；第一遍会再次累加根 1 的距离和，须每个对象只求解一次。
4. 把无权换根公式用于加权树；边长改变后必须重新推导跨边增量。
5. 输入含环或不连通；DFS 只跳父节点，非树输入可能递归循环或遗漏分量。

8. 一句话原理

先算一个根的全树距离和与各子树大小，再用跨边时“子树内减一、子树外加一”在线性时间传播到所有根。

4.13 概率与期望 DP

1. 它是做什么的

模型是每次独立、等概率地抽取 `n` 种卡片之一，抽到已收集种类仍会继续抽，求集齐全部种类所需抽取次数的期望。输入为正整数 `n`，返回 `double`。它不处理不同种类概率不等、依赖历史的抽取过程或其他一般状态转移问题。

状态 `e[i]` 表示已集齐 `i` 种后还需抽取次数的期望。下一次抽取必消耗 1 次，并以 `(n-i)/n` 概率进入 `i+1`；若抽到旧种则留在 `i`，将这一项移至等式左边得到 `e[i]=(1+pNew*e[i+1])/pNew`。循环计算 `n` 个状态，时间 $O(n)$；保存 `n+1` 个双精度值，空间 $O(n)$。期望使用浮点数近似，需按题目精度比较或输出。

2. 变量在做什么

名称	含义
`n`	卡片种类数，调用者必须保证 `n>0`。
`e[i]`	已收集 `i` 种之后的剩余抽取次数期望；边界 `e[n]=0`。
`i`	当前已收集种类数，从 `n-1` 递减到 0。
`pNew`	下一次抽到一种尚未收集种类的概率 `(n-i)/n`，在循环范围内严格大于 0。

状态只依赖 `e[i+1]`，从集齐状态向空集状态倒推。抽到旧种的概率为 `i/n`，其期望仍是 `e[i]`，解方程时已被吸收到分母中，不能再简单把旧种情况丢掉。

3. 函数在做什么

函数	参数与返回	行为及前提
`couponCollectorExpectation`	正整数种类数；返回 `double`。	返回独立均匀抽取直到集齐的期望次数；不修改外部状态。调用者需保证 `n>0`，函数本身不检查。

函数内部新建期望数组，因此每次调用互不影响。较大的 `n` 下，结果为近似浮点数而非精确分数。

4. 怎么调用

传入种类数，按题目要求设置浮点输出精度。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数：不同卡片种类数 n>0。返回：等概率独立抽取直到集齐的期望次数。效果：O(n) 时间、空间。
double couponCollectorExpectation(int n) {
    // e[i] 是已收集 i 种时还需抽取次数的期望，边界 e[n]=0。
    vector<double> e(n + 1, 0.0);
    for (int i = n - 1; i >= 0; --i) {
        double pNew = static_cast<double>(n - i) / n;
        // 一步必然消耗，再以 pNew 前进；重复抽到旧种的概率已移项。
        e[i] = (1.0 + pNew * e[i + 1]) / pNew;
    }
    return e[0];
}
```

只有一种卡片时必抽一次；两种卡片时期望为 `2 + 1 = 3`：

```cpp
assert(fabs(couponCollectorExpectation(1) - 1.0) < 1e-9);
assert(fabs(couponCollectorExpectation(2) - 3.0) < 1e-9);
```

5. 要改哪里

- 若每种卡片的抽取概率不相等，仅记录已收集种类数 `i` 不足以确定下一步概率；状态需记录具体已收集集合（或能区分剩余种类概率的等价信息），再按该状态的新增概率写方程并处理自环。
- 若一次抽取可能得到多种卡片、或抽取结果依赖历史，需重新定义状态和概率转移。
- 若要求方差等更高阶统计量，需为状态增加相应的二阶矩/统计量转移；当前函数只输出期望。
- 若需查询多个种类数 `n` 的答案，当前表只对应一个固定 `n`，不能直接跨 `n` 复用。可预处理调和数 `H[n]=H[n-1]+1/n`，再用 `n*H[n]` 回答各个 `n`。

6. 自检

`n=1` 时期望为 1；`n=2` 时先抽到第一种必花 1 次，之后抽到另一种平均再花 2 次，总期望 3。递推也等价于 `e[i]=n/(n-i)+e[i+1]`，因此 `e[0]=n(1+1/2+...+1/n)`。输入必须为正；`n=0` 不符合模型，不能把函数返回 0 当作有意义答案。

7. 坑

1. 直接计算 `1+pNew*e[i+1]` 而不除以 `pNew`；会忽略抽到已收集种类后仍要继续的自环期望。
2. 忘记边界 `e[n]=0`；集齐后不再需要抽取。
3. 用整数类型保存期望或中间概率；会截断小数，应使用 `double`。
4. 输入概率不均匀仍套用 `(n-i)/n`；此概率只适用于每种等可能。
5. 用精确相等比较浮点结果；验证和判题应使用容差或按题目要求格式化输出。

8. 一句话原理

对每个已收集种类数写出一步期望方程，将停留在当前状态的自环概率移项后从满集向空集递推。

第五章

数据结构

本章代码默认使用 C++17。除特别说明外，数组下标从 1 开始；调用查询、删除操作前应保证容器非空、区间合法。字典树统一收录在第 8 章字符串中。

5.1 栈与队列

1. 它是做什么的

栈、队列和双端队列是按位置约束插入、删除顺序的序列容器：栈后进先出，队列先进先出，双端队列允许在两端操作。括号匹配、撤销操作常用栈；广度优先搜索和按到达顺序处理任务常用队列；需要两端进出时用 `deque`。

本例接收整数序列 `a`，分别将其压入局部栈和队列，再逐个读取并弹出；它还演示向双端队列两端插入和从前端删除。函数返回 `void`，不修改 `a`，也不输出数据；读取出的值目前被丢弃，实际题目要在取值处加入处理逻辑。三种容器在本例中均不要求元素有序或互异。

复杂度：构造容器并处理长度为 `k` 的序列，整体时间 $O(k)$；一次 `push`、`pop`、`top`、`front`、`back` 或 `empty` 均为 $O(1)$。`deque` 两端插入和删除为 $O(1)$。存放 `k` 个元素需要 $O(k)$ 空间；这里统计额外空间的渐进量级。

2. 变量在做什么

名称	含义
`a`	只读输入序列；按从前到后的顺序依次入栈、入队。
`st`	`stack<int>` 栈适配器；只能查看和删除栈顶，先入的元素后取出。
`q`	`queue<int>` 队列适配器；从队首查看和删除，先入的元素先取出。
`dq`	`deque<int>` 双端队列；示例先在前端放 `1`、再在后端放 `2`，随后从前端删除 `1`。
`x`	每轮取出的元素；必须在 `pop()` 前读取，当前示例没有保留或输出它。

这些容器按元素顺序工作，没有节点编号或自定义下标。`stack` 和 `queue` 不提供迭代器，也不支持随机访问；若需要遍历全部元素，应逐次读取并弹出，或选择适合的其他容器。

3. 函数在做什么

函数	参数与返回	行为及前提
`stackQueueExample(const vector<int>& a)`	接收整数向量常量引用；返回 `void`。	创建三个局部容器并演示入栈、入队和双端队列操作；不改变 `a`，函数结束时容器销毁。当前取出的值被丢弃，没有可见输出。
`push`	插入一个元素；返回 `void`。	栈和队列从各自的后端加入；`deque` 可用 `push_front` 或 `push_back` 指定一端。
`top` / `front` / `back`	无参数；返回相应端元素的引用。	只能在容器非空时读取；示例将元素复制到 `x` 后再弹出。

函数	参数与返回	行为及前提
`pop` / `pop_front` / `pop_back`	无参数；返回 `void`。	删除指定端元素，不返回被删值；调用前必须确认非空。
`empty` / `size`	无参数；分别返回是否为空、元素个数。	可用于安全循环和检查容器状态。

题目通常不需要调用演示函数；直接在自己的逻辑中声明并使用相应容器即可。若调用本例函数，`a` 中的元素只会被消费演示，不会产生答案。

4. 怎么调用

先选符合处理顺序的容器；插入元素；每轮先判断非空，再读取对应端并处理元素，最后调用 `pop` 删除。若要两端操作，使用 `deque`。本例完整模板如下：

```

#include <bits/stdc++.h>
using namespace std;

// 参数 a 是待演示的整数序列；返回 void，局部容器消费并展示栈/队列/双端队列顺序，不改 a。
    void stackQueueExample(const vector<int>& a) {
    // 适配器不暴露迭代器；以下循环都先读顶端/队首，再 pop。
    stack<int> st;
    for (int x : a) st.push(x); // 入栈顺序与 a 相同，弹出顺序因此相反
    while (!st.empty()) {
        int x = st.top();
        st.pop();
        (void)x;  // 在此处理：顺序与 a 相反
    }

    queue<int> q;
    for (int x : a) q.push(x); // 队列按到达顺序服务
    while (!q.empty()) {
        int x = q.front();
        q.pop();
        (void)x;  // 在此处理：顺序与 a 相同
    }

    deque<int> dq;
    dq.push_front(1);
    dq.push_back(2);
    if (!dq.empty()) dq.pop_front();
}
```

下面的片段可放在业务函数内。对序列 `4 7 9`，栈依次取出 `9 7 4`，队列依次取出 `4 7 9`；两端队列操作后剩余 `[2]`：

```
vector<int> a = {4, 7, 9};
stack<int> st;
for (int x : a) st.push(x);
while (!st.empty()) {
    int x = st.top();
    // 处理 x
    st.pop();
}

queue<int> q;
for (int x : a) q.push(x);
while (!q.empty()) {
    int x = q.front();
    // 处理 x
    q.pop();
}

```

```
deque<int> dq;
dq.push_front(1);
dq.push_back(2);
dq.pop_front();
// 结果：dq 中只剩 2
```

5. 要改哪里

- 需要后进先出时保留 `stack`，处理逻辑放在读取 `top()` 之后、`pop()` 之前；例如括号匹配时在这里检查并配对。
- 需要先进先出时保留 `queue`，在读取 `front()` 后处理；广度优先搜索通常在入队时标记节点已发现，避免重复入队。
- 需要从两端插入或删除时改用 `deque` 的 `push_front`、`push_back`、`pop_front`、`pop_back`。栈和队列适配器不提供另一端接口。
- 若需要按下标读取、排序或遍历而不消费元素，换用 `vector` 或 `deque` 等支持相应操作的容器；`stack` / `queue` 不适合这些需求。
- 多组数据时，局部容器在函数结束时自动销毁；若容器定义在外层并跨组复用，每组开始前需清空，避免残留元素。

6. 自检

取 `a = [4, 7, 9]`：逐个压入栈后，弹出顺序应为 `9, 7, 4`；逐个入队后，出队顺序应为 `4, 7, 9`。对双端队列先执行 `push_front(1)`、`push_back(2)`，再执行 `pop_front()`，应只剩 `2`。单元素序列压入任一容器后只读取一次；空序列则两个处理循环都不进入。重复值（如 `[5, 5]`）仍按位置入出，结果仍各有两个 `5`，容器不会自动去重。

7. 坑

1. 空容器调用 `top()`、`front()`、`back()` 或 `pop()` 会导致未定义行为；先用 `empty()` 检查，或在 `while (!empty())` 中操作。
2. 先 `pop()` 再读取会丢失待处理元素；先复制/处理 `top()` 或 `front()`，再删除。
3. 误以为 `pop()` 会返回元素；它只负责删除，需先用 `top()`、`front()` 或 `back()` 取值。
4. 需要遍历或随机访问时使用 `stack` / `queue`；它们是适配器，只暴露受限接口，应改选支持目标操作的容器。
5. 显式栈模拟 DFS 时，子节点的入栈次序会影响弹出访问次序；若要维持递归 DFS 的访问顺序，通常按期望访问顺序的反向压入。

8. 一句话原理

栈、队列和双端队列通过限制允许操作的端点，分别实现后进先出、先进先出和两端进出。

参考：

- OI Wiki：栈
- OI Wiki：队列

5.2 单调栈

1. 它是做什么的

给定一个静态整数数组，若要对每个位置找右侧第一个严格更小的元素，可以从左向右扫描并用单调栈在线性时间内完成。模板返回的是元素的下标；找不到时为 `-1`。它不修改输入数组，也不处理动态插入、删除或区间更新。

预处理：无。单次调用时间：$O(n)$，其中 n 是数组长度；每个下标只入栈、出栈各至多一次。额外空间：$O(n)$，用于答案和栈。每次入栈、出栈本身为 $O(1)$。

2. 变量在做什么

变量	含义
`a`	只读输入数组，位置从 `0` 到 `n-1`。
`n`	`a.size()` 转成的整数，表示元素个数。
`answer`	与 `a` 等长；`answer[j]` 存位置 `j` 右侧首个严格更小元素的位置，暂时未找到或最终不存在时为 `-1`。
`st`	保存尚未找到答案的位置下标。栈中下标从底到顶递增，对应的数组值单调不减。
`i`	当前扫描到的下标；`st.top()` 是当前正在等待答案的位置。

例如 `a = [3, 1, 2, 1, 4]`。扫描到位置 `1` 的值 `1` 时，位置 `0` 的值 `3` 被弹出并得到答案 `1`。值相等时不弹出，所以位置 `1` 不会把位置 `3` 当作“严格更小”；结束后 `answer = [1, -1, 3, -1, -1]`。

3. 函数在做什么

函数	参数与返回	作用和前提
`firstSmallerOnRight(const vector<int>& a)`	参数是任意长度的只读整数数组；返回等长 `vector<int>`。	从左向右扫描。每当 `a[i] < a[st.top()]`，位置 `i` 就是栈顶下标右侧第一个严格更小的位置，记录后弹栈；最后仍在栈中的下标没有答案，保留 `-1`。可直接处理空数组。

没有需要调用者自行实现的辅助函数；通常只需调用公开函数并读取返回数组。函数不保留跨调用状态，同一输入可重复调用。

4. 怎么调用

先准备数组，再调用函数；返回数组与输入逐项对应。下例可放在业务函数中：

```
vector<int> a = {3, 1, 2, 1, 4};
vector<int> next = firstSmallerOnRight(a);
// next 为 {1, -1, 3, -1, -1}
```

模板代码如下，代码片段本身不含 `main`，需要在调用处提供数组：

```

#include <bits/stdc++.h>
using namespace std;

// 参数 a 为 0-based 数组；返回每项右侧首个严格更小位置的下标，不存在为 -1；不改 a。
    vector<int> firstSmallerOnRight(const vector<int>& a) {
    // a 与答案均为 0-based；answer[i] 是右侧首个严格更小值的下标。
    int n = (int)a.size();
    vector<int> answer(n, -1);
    stack<int> st;  // 栈中下标递增，且对应值从栈底到栈顶单调不减

    for (int i = 0; i < n; ++i) {
        // 严格更小才弹；改成 <= 即改为“右侧首个小于等于”。
        while (!st.empty() && a[i] < a[st.top()]) {
            answer[st.top()] = i;
            st.pop();
        }
        st.push(i);
    }
    return answer;
}
```

5. 要改哪里

- 求右侧首个严格更大位置：把弹栈条件改为 `a[i] > a[st.top()]`。

- 求右侧首个小于或等于的位置：把条件改为 `a[i] <= a[st.top()]`。相等值会在此时被弹出，因此较早位置会把当前相等位置记为答案。
- 求左侧最近元素：反向扫描，并相应调整栈中下标顺序与答案定义；不能只改一个比较符号。
- 需要返回元素值而不是位置时，可在确定答案处使用 `a[i]`；若要计算距离，则用 `i - j`（当前下标减去被弹出下标 `j`），并先判断答案是否为 `-1`。

6. 自检

对 `a = [2, 2, 1]`，结果应为 `[2, 2, -1]`：相等的 2 不算严格更小，而 1 是前两个位置的答案。对单元素 `[7]`，结果为 `[-1]`；空数组返回空数组。扫描结束时栈中每个下标右侧都没有更小值，这些位置的答案应继续为 `-1`。

7. 坑

- 重复值答案不对：严格与非严格条件混淆。严格更小用 `<`，小于等于用 `<=`。
- 得到的是值却拿来当位置：栈中保存的是下标，确定答案时记录的也是下标。
- `-1` 被用于计算距离或访问数组：先判断该位置是否存在答案，再计算 `answer[j] - j`。
- 误以为栈里最终只剩最小值：栈保存的是尚未找到右侧更小值的候选位置，多个位置可以同时留在栈中。

8. 一句话原理

从左向右扫描时，新位置一旦比栈顶位置的值小，就成为该栈顶位置等待已久的第一个严格更小元素。

参考：

- OI Wiki：单调栈

5.3 单调队列

1. 它是做什么的

给定长度为 `n` 的整数数组 `a` 和窗口长度 `k`，本模板返回每个连续长度为 `k` 的闭区间最小值。仅当 `1 <= k <= n` 时有答案，结果长度为 `n-k+1`；否则返回空数组。对固定窗口，时间复杂度为 $O(n)$，队列辅助空间为 $O(k)$，另需 $O(n-k+1)$ 保存返回结果。初始化没有独立预处理阶段，单次调用处理每个元素一次；每个下标最多入队、出队各一次，所以摊还总操作数为 $O(n)$，单次循环可能弹出多个元素。

它也可用于某些 DP：当每个状态只需查询一段随状态移动的候选区间最优值，且候选值能按下标顺序维护时，可用单调队列代替逐项扫描。若候选区间不单调移动，或候选优劣不能由队尾淘汰规则保证，则不能直接套用。

2. 变量在做什么

变量	含义
`a`	输入数组，0-based；函数不会修改它。
`k`	窗口长度，合法范围是 `1..n`。
`i`	当前扫描到的数组下标，也是当前窗口的右端点。
`dq`	候选下标的双端队列；下标递增，对应值严格递增。队首是当前窗口最小值候选。
`answer`	按窗口右端点递增收集的结果；`answer[j]` 对应闭区间 `[j, j+k-1]`。

处理到 `i` 时，当前窗口是 `[i-k+1, i]`。队首下标不大于 `i-k` 就已过期；从队尾删除值大于或等于 `a[i]` 的下标，因为新元素更靠右且不大于旧元素，旧元素之后不可能再成为最小值。相等时保留较新的下标，它能留在窗口更久。

3. 函数在做什么

函数	参数、返回与作用
`slidingWindowMinimum(a, k)`	参数为 0-based 整数数组和窗口长度；返回各合法窗口的最小值，不修改 `a`。`k <= 0` 或 `k > a.size()` 时返回空数组。队列在调用内创建，不保留跨调用状态。

4. 怎么调用

调用时传入数组和窗口长度，直接使用返回数组；数组为空或窗口不合法时，结果为空。下面片段可放在业务函数内：

```
vector<int> a = {4, 2, 2, 5, 1};
vector<int> ans = slidingWindowMinimum(a, 3);
// ans 为 {2, 2, 1}，依次对应 [4,2,2]、[2,2,5]、[2,5,1]
```

模板代码如下：

```

#include <bits/stdc++.h>
using namespace std;

// 参数 a 为 0-based 数组、k 为窗口长度；返回各窗口最小值，k<=0 或 k>n 时返回空表；不改 a。
    vector<int> slidingWindowMinimum(const vector<int>& a, int k) {
    // a 为 0-based；每个答案对应闭区间 [i-k+1, i]。
    int n = (int)a.size();
    vector<int> answer;
    if (k <= 0 || k > n) return answer;

    // dq 保存候选下标：下标递增，对应值严格递增，队首永远是窗口最小值。
    deque<int> dq;
    for (int i = 0; i < n; ++i) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front();  // 移出窗口
        // 相等也弹出旧下标，保留有效期更长的新下标；求最大值时反转不等号。
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) answer.push_back(a[dq.front()]);
    }
    return answer;
}
```

5. 要改哪里

- 求窗口最大值：队列对应值改为严格递减，把队尾条件 `a[dq.back()] >= a[i]` 改为 `a[dq.back()] <= a[i]`，队首读取方式不变。
- 若要保留相等值中较早的下标，可将最小值的队尾条件改成 `>`；此时队列值非递减，答案仍正确，但可能多保留候选。
- 变长窗口或其他 DP 候选区间，需要按题目定义修改过期条件和队尾淘汰条件；只有当淘汰后旧候选永远不可能胜出时，才能删除它。若需要同时查询多个不同窗口且窗口端点不按顺序前进，应换用适合的数据结构。

6. 自检

对 `a = {4,2,2,5,1}`、`k = 3`，答案应为 `{2,2,1}`；第二个窗口出现重复最小值，队列会舍弃较早的 `2`，但最小值仍为 `2`。`k=1` 时结果应逐项等于原数组；`k=n` 时只返回全数组最小值；空数组、`k=0` 或 `k>n` 均返回空数组。扫描过程中可检查队列下标递增、所有下标均在当前窗口内、对应值严格递增。

7. 坑

- 结果少一个或区间错位：窗口是闭区间，只有 `i >= k-1` 才产生答案；第 `j` 项对应 `[j,j+k-1]`。
- 最小值偶尔错误：队尾条件方向写反，导致队首不再是窗口最小值候选；最小值应弹出所有值大于或等于当前值的候选。

- 旧元素影响当前答案：先移除所有过期下标，再读取队首。
- 用值而非下标入队：无法判断候选何时离开窗口；队列应保存数组下标，取值时再访问 `a[dq.front()]`。

8. 一句话原理

每个新元素淘汰它之后永远不可能更优的旧候选，队首因此始终给出当前窗口最小值。

参考：

- OI Wiki：单调队列

5.4 堆与优先队列

1. 它是做什么的

堆适合在一批候选元素中反复取得当前优先级最高或最低者，同时允许新元素加入。C++ `priority_queue` 默认是大根堆，`top()` 返回最大值；指定 `greater<T>` 后成为小根堆。常用于贪心选择、Dijkstra 和 Top-K。它支持查看堆顶、插入和删除堆顶，不支持高效删除任意元素或直接修改堆中元素。

本模板把输入向量中的整数依次放入小根堆，再按从小到大顺序取出，返回一个新向量，不修改输入。初始化采用逐个 `push`，因此处理 `n` 个元素整体为 $O(n \log n)$；不能把此实现的建堆成本写成线性。单次 `top()` 为 $O(1)$，`push` / `pop` 为 $O(\log n)$，空间为 $O(n)$。若使用支持范围建堆的接口，复杂度会不同，但本模板没有这样做。

2. 变量在做什么

名称	含义
`a`	输入整数序列，以常量引用传入，函数不改它。
`minHeap`	小根优先队列；每次 `top()` 是尚未取出的最小值。
`sorted`	新建的结果向量，按弹出次序收集所有元素。
`x`	当前从 `a` 读入并加入堆的整数。

堆不暴露元素下标，也不保证除堆顶外的元素有序。相同值可以重复存在；弹出的值按非降序排列，但相等元素之间没有可观察的先后差异。

3. 函数在做什么

函数	参数与返回	行为及前提
`heapOrder(const vector<int>& a)`	接收只读整数向量；返回 `vector<int>`。	将所有值压入局部小根堆，逐个弹出并返回升序副本；空输入返回空向量。
`push(x)`	插入一个元素，返回 `void`。	堆调整后仍保持比较器定义的优先级关系。
`top()`	无参数；返回堆顶元素的常量引用。	仅堆非空时可调用。
`pop()`	无参数；返回 `void`。	删除堆顶，不返回其值；需要该值时先保存 `top()`。
`empty()` / `size()`	无参数；分别返回空状态和元素数量。	可用于安全取堆顶的循环条件。

4. 怎么调用

需要最小值优先时声明 `priority_queue<T, vector<T>, greater<T>>`；最大值优先可用默认 `priority_queue<T>`。循环中先保存 `top()`，再 `pop()`。本模板代码如下：

```
1
2 #include <bits/stdc++.h>
3 using namespace std;
```

```cpp

// 参数 a 为待排序整数序列；返回按升序弹出的副本，不改 a。
    vector<int> heapOrder(const vector<int>& a) {
    // greater<int> 使 top() 为当前最小值；若去掉则默认是大根堆。
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int x : a) minHeap.push(x);

    vector<int> sorted;
    while (!minHeap.empty()) {
        sorted.push_back(minHeap.top());
        minHeap.pop();
    }
    return sorted;
}
```

片段可放在业务函数内。输入 `[8, 3, 5, 3]` 时，返回值为 `[3, 3, 5, 8]`：

```cpp
vector<int> a = {8, 3, 5, 3};
vector<int> result = heapOrder(a);
// result: 3 3 5 8; a 仍为 8 3 5 3
```

5. 要改哪里

- 要取最大值，改用默认大根堆；要按其他优先级排序，换元素类型或提供比较器，并明确比较器定义的“优先”方向。
- 要处理 `(距离, 点编号)` 等状态，可存 `pair`；`greater<pair<...>>` 按字典序比较，先比较距离，再比较点编号。
- 若在 Dijkstra 等算法中更新同一状态，不能原地修改堆内记录。可把新状态再次 `push`，取出时与当前最佳状态比较，跳过过期项；注意这种懒删除可能让堆暂时大于有效状态数。
- 要删除任意元素或频繁修改优先级，考虑维护索引的堆或其他数据结构；普通 `priority_queue` 没有对应接口。
- 若只想取得前 `k` 个值，可在读取至多 `k` 次后停止，不必把剩余元素全部弹出；堆仍需先容纳本模板插入的全部 `n` 个元素。

6. 自检

对 `[8, 3, 5, 3]`，小根堆连续弹出应为 `3, 3, 5, 8`，元素总数保持为 4，重复的 `3` 不会丢失。单元素 `[6]` 返回 `[6]`；空向量返回空向量。若去掉 `greater<int>`，同一输入应按 `8, 5, 3, 3` 弹出，借此确认比较器方向。

7. 坑

1. 默认 `priority_queue` 是大根堆；求最小值时忘记 `greater<T>` 会让结果顺序反过来。
2. 对空堆调用 `top()` 或 `pop()` 会导致未定义行为；先检查 `empty()`。
3. `pop()` 只删除，不返回元素；在删除前保存 `top()`。
4. 误以为堆内所有元素都已排序；只有堆顶有优先级保证，完整排序需反复弹出或使用排序算法。
5. 自定义比较器方向弄反会改变堆顶含义；用两个可区分元素手动核对比较关系。

8. 一句话原理

堆只维护最高优先级元素位于顶端，因此能快速查看它，并在每次插入或删除后用对数时间恢复这一结构。

参考：

- OI Wiki：堆

5.5 并查集

1. 它是做什么的

并查集维护元素只合并、不拆分的分组关系，支持判断是否同组、查询代表元和集合大小。元素编号为 `1..n`。构造初始化时间与空间为 $O(n)$；路径压缩配合按大小合并后，`find`、`unite`、`same`、`size` 的均摊时间为 $O(\alpha(n))$，其中 α 是反 Ackermann 函数。它不能任意拆分集合，也不直接维护集合内部路径等信息。

2. 变量在做什么

变量	含义
`parent[x]`	元素 `x` 的父节点；根满足 `parent[x] == x`。
`sizeOf[x]`	仅当 `x` 是根时有效，表示该根代表的集合大小。
`n`	构造参数和最大合法编号；底层数组长为 `n+1`，下标 0 不作为元素使用。
`a`, `b`, `x`	操作涉及的元素编号，必须在 `1..n`。

3. 函数在做什么

函数	契约
`DSU(n)`	初始化 `1..n` 为 `n` 个单元素集合；构造参数应为非负数。
`find(x)`	要求 `x` 在 `1..n`；返回所在集合的根，并通过路径压缩修改访问路径上的 `parent`。
`unite(a,b)`	要求两参数均在 `1..n`；合并所在集合，成功返回 `true`，原已连通返回 `false`；成功时修改根的 `parent` 和 `sizeOf`。
`same(a,b)`	返回两元素是否同组；查询会调用 `find`，因此可能压缩路径。
`size(x)`	返回 `x` 所在集合大小；查询会调用 `find` 并可能压缩路径。

4. 怎么调用

创建对象后，对每条需要加入连通关系的边调用 `unite`；再用 `same` 或 `size` 查询：

```
DSU dsu(4);
dsu.unite(1, 2);
dsu.unite(2, 3);
// dsu.same(1, 3) 为 true; dsu.size(2) 为 3
// dsu.unite(1, 3) 返回 false，因为二者已经连通
```

原模板：

```

#include <bits/stdc++.h>
using namespace std;

struct DSU {
    // parent[x] 指向父节点；仅根满足 parent[x]==x，且只有根的 sizeOf 有效。
    vector<int> parent, sizeOf;

    // 参数 n 是元素上界；建立编号 1..n 的单元素集合，初始化内部 parent/sizeOf。
    explicit DSU(int n) : parent(n + 1), sizeOf(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    // 参数 x 是 1..n 的元素；返回其集合根，并对访问路径做路径压缩。
    int find(int x) {
        if (parent[x] == x) return x;
        // 路径压缩保持集合不变，只把查询路径上的点直接挂到根。
        return parent[x] = find(parent[x]);
    }

```

```
21      // 参数 a、b 是 1..n 的元素；合并两集合，成功返回 true、原已连通返回 false；修改根和集合大小。
22      bool unite(int a, int b) {
23          a = find(a);
24          b = find(b);
25          if (a == b) return false;
26          if (sizeOf[a] < sizeOf[b]) swap(a, b);  // 始终把小树挂到大树
27          parent[b] = a;
28          sizeOf[a] += sizeOf[b];
29          return true;
30      }
31
32      // 所有公开查询都先压到根，避免调用方误把非根当代表。
33      // 参数 a、b 是 1..n 元素；返回是否同属一集合，查询会通过 find 压缩路径。
34      bool same(int a, int b) { return find(a) == find(b); }
35      // 参数 x 是 1..n 元素；返回其所在集合大小，查询会通过 find 压缩路径。
36      int size(int x) { return sizeOf[find(x)]; }
37  };
```

5. 要改哪里

- 若改成 0-based 编号，需同步调整数组长度、初始化范围和每个接口的合法下标。
- 若需维护集合聚合信息，在成功合并时将两个根的状态合并到新根；非根上的旧聚合值不再有效。
- 若需要按后进先出顺序撤销已记录的合并，可使用回滚并查集并记录每次修改；它只能按操作历史撤销，不能任意拆分某个集合。常见回滚实现不做路径压缩，以便恢复修改。
- 多组数据可每组重新构造对象；普通并查集不提供清空或撤销接口。

6. 自检

初始 `DSU(3)` 中 `same(1,2)` 为 `false`、`size(1)` 为 1。`unite(1,2)` 返回 `true` 后两点同组且大小为 2；再次 `unite(2,1)` 返回 `false`，大小仍为 2。再合并 2 和 3 后，三个元素应同组、`size(3)` 为 3。

7. 坑

- 直接改 `parent[a]`：`a` 可能不是根；先用 `find` 取得两个根。
- 读取非根的 `sizeOf`：集合大小只保证存于根节点。
- 把 `unite` 的返回值当新根：它只表示本次是否真正合并。
- 需要拆分却使用普通并查集：已合并的信息无法恢复；回滚也只支持按记录顺序撤销，不支持任意拆分。

8. 一句话原理

每个集合由根代表，合并时小树挂到大树上，查询时压缩访问路径。

参考：

- OI Wiki：并查集

5.6 树状数组

1. 它是做什么的

树状数组维护长度 `n>=1` 的 1-based 数组的可加减信息，支持单点增加、前缀和与区间和。本模板另提供 `lowerBound`：在每个位置的频次非负时，找前缀和首次达到目标的位置。更新与每次查询均为 $O(\log n)$，空间为 $O(n)$；构造全零结构的初始化为 $O(n)$，若逐点插入初始值则为 $O(n \log n)$。它适合和、频次等可做加减的量，不直接支持任意区间最值或区间赋值。

2. 变量在做什么

变量	含义
`n`	最大有效下标；合法位置为 `1..n`。

变量	含义
`tree[i]`	Fenwick 分块和，覆盖 `(i-lowbit(i), i]`，其中 `lowbit(i) = i & -i`。
`index`	`add` 的更新位置，或 `prefixSum` 的查询右端点；前者必须至少为 1，后者可为 0。
`left`, `right`	`rangeSum` 的闭区间端点。若 `left > right`，实现返回 0。
`target`	`lowerBound` 要达到的前缀和目标。
`accumulated`	`lowerBound` 二进制提升过程中，已确认小于 `target` 的前缀和。

下标从 1 开始；`tree[0]` 不参与更新。`lowerBound(target)` 返回最小的 `i`，使 `prefixSum(i) >= target`；没有满足位置时返回 `n+1`。本实现对 `target <= 0` 返回 1。

3. 函数在做什么

函数	参数、返回与作用
`Fenwick(n)`	创建最大下标为 `n` 的全零结构，内部数组长度为 `n+1`。
`add(index, delta)`	将位置 `index` 增加 `delta`，修改覆盖该位置的分块；要求 `1 <= index <= n`。
`prefixSum(index)`	返回闭区间 `[1,index]` 的和，`index` 可为 `0..n`；不修改结构。
`rangeSum(left, right)`	返回闭区间和 `prefixSum(right)-prefixSum(left-1)`；要求端点在有效范围内，空区间返回 0。
`lowerBound(target)`	在各点值非负时返回最小满足前缀和至少为 `target` 的下标；无解为 `n+1`。不修改结构。

4. 怎么调用

先构造容量，再用 `add` 记录每个位置的增量；前缀和与区间和可以随时查询。`lowerBound` 只适用于各点频次非负的情形，例如频次统计：

```
Fenwick fw(5);
fw.add(1, 2);
fw.add(3, 1);
fw.add(5, 2);
// fw.prefixSum(3) == 3, fw.rangeSum(2, 5) == 3
// fw.lowerBound(4) == 5, fw.lowerBound(6) == 6 (无解, n+1)
```

模板代码如下：

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Fenwick {
    int n;
    // 仅使用 1..n; tree[i] 保存 (i-lowbit(i), i] 这一段的和。
    vector<ll> tree;

    // 参数 n 是正的最大下标；创建全零 1-based 树状数组，初始化内部 tree。
    explicit Fenwick(int n) : n(n), tree(n + 1, 0) {}

    // 参数 index 属于 1..n、delta 为增量；返回 void，修改覆盖 index 的 Fenwick 节点。
    void add(int index, ll delta) {
```

```
        // index 必须 >=1；向父区间跳转，直到超过 n。
        for (int i = index; i <= n; i += i & -i) tree[i] += delta;
    }

    // 参数 index 属于 0..n；返回原数组闭区间 [1,index] 的和，不改结构。
    ll prefixSum(int index) const {
        // 查询闭区间 [1,index]；每次删掉当前下标最低位的 1。
        ll result = 0;
        for (int i = index; i > 0; i -= i & -i) result += tree[i];
        return result;
    }

    // 参数 left/right 为合法 1-based 闭区间；返回当前区间和，不修改结构。
    ll rangeSum(int left, int right) const {
        if (left > right) return 0;
        return prefixSum(right) - prefixSum(left - 1);
    }

    // 频次数组非负时，返回最小 i 使 prefixSum(i)>=target；无解返回 n+1。
    // 参数 target 为目标前缀频次；要求各点频次非负，返回最小满足前缀和>=target 的下标，无解为 n+1。
    int lowerBound(ll target) const {
        int index = 0;
        ll accumulated = 0;
        int step = 1;
        while ((step << 1) <= n) step <<= 1;
        // 二进制提升：index 始终是“前缀和仍小于 target”的最大已知下标。
        for (; step; step >>= 1) {
            int next = index + step;
            if (next <= n && accumulated + tree[next] < target) {
                index = next;
                accumulated += tree[next];
            }
        }
        return index + 1;
    }
};
```

5. 要改哪里

- 单点赋值需先保存旧值，再调用 `add(index, newValue-oldValue)`；直接把新值作为增量会把旧值重复计入。
- 若要维护频次并按累计频次定位，保证每次更新后每个位置的频次仍非负，再调用 `lowerBound`。普通数组值可以为负，但此时前缀和不再单调，不能用该函数二进制提升。
- 想从一组初始值建树，当前结构没有批量 `build` 接口；可逐点 `add`，时间为 $O(n \log n)$，或另行实现线性建树。
- 区间加、区间和等需求通常需要两棵树状数组并重新推导更新公式；区间最值或任意赋值应换用支持相应操作的数据结构。

6. 自检

上面的示例各点值为 `[2,0,1,0,2]`，所以前三项和为 `3`，区间 `[2,5]` 和为 `3`。累计和首次达到 4 的位置是 5；目标 6 大于总和 5，返回 `n+1=6`。单位置更新后可用 `rangeSum(i,i)` 核对；目标为 1 时应返回第一个正频次位置。更新下标 0 会导致循环无法前进，必须始终使用 `1..n`。

7. 坑

- 更新后程序卡住：`add(0, delta)` 中 `i & -i` 为 0，循环不前进；更新位置必须至少为 1。
- 区间和差一位：区间是闭区间，公式要减去 `prefixSum(left-1)`。
- `lowerBound` 给出错误位置：某次减量让某个点频次变成负数，前缀和不再单调；保持所有点频次非负。

- 无解时访问数组越界：目标超过总频次会返回 `n+1`，调用方要先处理这个哨兵值。
- `target <= 0` 的边界与直觉不同：本实现仍返回 1；若题意需要空前缀下标 0，需调整接口约定和实现。

8. 一句话原理

用 `i & -i` 把前缀拆成少量不重叠分块，并在二进制提升时跳过仍达不到目标的整段。

参考：

- OI Wiki：树状数组

5.7 线段树（区间加、区间和）

1. 它是做什么的

这棵线段树维护一个长度为 `n` 的 1-based 数组，支持把闭区间 `[l,r]` 每个数加上 `delta`，以及查询该区间的元素和。两种操作可以交错进行；单点加是 `rangeAdd(i,i,delta)`。适合区间更新和区间查询混合出现的场景，不能直接支持区间赋值或任意其他统计量。

构造对象及开辟数组空间为 $O(n)$；对初值建树为 $O(n)$；一次合法区间加或区间和查询为 $O(\log n)$。空间为 $O(n)$，本实现使用约 `4n` 个节点槽位。每个值及区间和均使用 `long long`，题目数据仍须保证乘法和总和不溢出。

2. 变量在做什么

名称	含义
`n`	数组长度，必须大于 0。有效位置为 `1..n`。
`sum[p]`	线段树节点 `p` 所管理闭区间的当前总和，已计入该节点上的整段修改。
`lazy[p]`	已计入 `sum[p]`、但尚未传给孩子的每项增量。下传后必须清零，避免重复应用。
`p, left, right`	递归内部使用的节点编号和当前闭区间；不是使用者传入数组的零基下标。
`ql, qr`	查询或更新的目标闭区间，要求 `1 <= ql <= qr <= n`。
`delta`	对目标区间每个元素增加的值。
`a`	初始值向量；`a[1]` 到 `a[n]` 对应数组位置，`a[0]` 不参与构建。

内部节点的左右孩子分别代表 `[left,mid]` 和 `[mid+1,right]`。`sum[p]` 已计入本节点收到的更新；祖先尚未下传的标记可能还未反映在子节点中。访问子节点前必须先下传。若整段加 `delta`，总和应增加 `delta * (right-left+1)`。

3. 函数在做什么

函数	参数与返回	行为及调用条件
`SegmentTree(int n)`	长度 `n`；构造对象。	分配并将 `sum`、`lazy` 初始化为零；要求 `n > 0`。
`build(a)`	1-based 初值向量；返回 `void`。	初始化节点和。仅应在刚构造的新对象上调用一次；它不清除已有 `lazy` 标记，带更新状态的对象不能靠再次 `build` 重置，应重新构造对象。
内部 `build(a,p,left,right)`	初值及当前节点区间；返回 `void`。	递归建立 `sum`，不触碰 `lazy`，使用者无需直接调用。
`apply(p,left,right,delta)`	节点区间和增量；返回 `void`。	更新本段和与本段懒标记，供内部调用。

函数	参数与返回	行为及调用条件
`pushDown(p,left,right)`	当前节点及其区间；返回 `void`。	将增量按左右子区间的实际长度下传并清空父标记；叶节点无需下传。
`rangeAdd(l,r,delta)`	合法 1-based 闭区间和增量；返回 `void`。	修改区间内所有元素。
`rangeSum(l,r)`	合法 1-based 闭区间；返回 `long long`。	查询区间和；查询时可能下传懒标记，但不改变逻辑数组的值。
内部重载 `rangeAdd` / `rangeSum`	额外接收 `p,left,right`；更新返回 `void`，查询返回和。	递归实现，使用者调用公开的三参数或双参数接口即可。

4. 怎么调用

令 `n >= 1`，准备长度至少 `n+1` 的向量，使初值放在 `a[1..n]`；构造一个新对象并调用一次 `build(a)`。之后对所有操作使用合法闭区间。模板如下：

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct SegmentTree {
    int n;
    // 节点 p 管理闭区间 [left,right]。
    // sum[p] 已含本节点全部修改；lazy[p] 表示尚未下传给子节点的“每项增量”。
    vector<ll> sum, lazy;

    // 参数 n 为正数组长度；分配区间和与懒标记空间，初始值均为 0。
    explicit SegmentTree(int n) : n(n), sum(4 * n + 4), lazy(4 * n + 4) {}

    // 参数 a 为 1-based 初值；p 管理闭区间 [left,right]，递归写入 sum、不触碰 lazy；区间合法且非空，只应对全零新对象构建。
    void build(const vector<ll>& a, int p, int left, int right) {
        // 输入 a 使用 1-based；递归区间始终是闭区间。
        if (left == right) {
            sum[p] = a[left];
            return;
        }
        int mid = (left + right) / 2;
        build(a, p * 2, left, mid);
        build(a, p * 2 + 1, mid + 1, right);
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    }

    // 参数 a[1..n] 是初值；返回 void，只初始化节点 sum，不清 lazy；须对刚构造的新对象调用一次，已有更新后重建需重新构造对象。
    void build(const vector<ll>& a) { build(a, 1, 1, n); }

    // 参数指定节点区间和增量 delta；返回 void，直接修改该节点 sum/lazy。
    void apply(int p, int left, int right, ll delta) {
        // 整段每项增加 delta，所以区间和增加 delta * 区间长度。
        sum[p] += delta * (right - left + 1);
        lazy[p] += delta;
    }

    // 参数 p 管理闭区间 [left,right]；返回 void，将未下传增量写到子节点并清父标记。
    void pushDown(int p, int left, int right) {
        if (lazy[p] == 0 || left == right) return;
        int mid = (left + right) / 2;
        // 下传必须使用子区间真实长度；完成后清空父标记，避免重复应用。
        apply(p * 2, left, mid, lazy[p]);
        apply(p * 2 + 1, mid + 1, right, lazy[p]);
```

```
        lazy[p] = 0;
    }

    // ql/qr 为待加闭区间，p,left,right 为当前节点；要求区间合法且相交，返回 void 并修改覆盖节点。
    void rangeAdd(int ql, int qr, ll delta, int p, int left, int right) {
        // ql,qr 是待修改闭区间；调用前保证它合法且与当前区间有交集。
        if (ql <= left && right <= qr) {
            apply(p, left, right, delta);
            return;
        }
        pushDown(p, left, right);
        int mid = (left + right) / 2;
        if (ql <= mid) rangeAdd(ql, qr, delta, p * 2, left, mid);
        if (qr > mid) rangeAdd(ql, qr, delta, p * 2 + 1, mid + 1, right);
        // 子树修改后回收信息，保持 sum[p] 等于整个 [left,right] 的和。
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    }

    // 参数 left/right 为合法 1-based 闭区间、delta 为每项增量；返回 void，更新覆盖节点的和及懒标记。
    void rangeAdd(int left, int right, ll delta) {
        rangeAdd(left, right, delta, 1, 1, n);
    }

    // ql/qr 为查询闭区间，p,left,right 为当前节点；返回两区间交集之和，并可能下传懒标记。
    ll rangeSum(int ql, int qr, int p, int left, int right) {
        // 完全覆盖直接返回；部分覆盖前先下传，保证子节点信息最新。
        if (ql <= left && right <= qr) return sum[p];
        pushDown(p, left, right);
        int mid = (left + right) / 2;
        ll result = 0;
        if (ql <= mid) result += rangeSum(ql, qr, p * 2, left, mid);
        if (qr > mid) result += rangeSum(ql, qr, p * 2 + 1, mid + 1, right);
        return result;
    }

    // 参数为合法 1-based 闭区间；返回区间和，查询可能下传内部懒标记。
    ll rangeSum(int left, int right) {
        return rangeSum(left, right, 1, 1, n);
    }
};
```

片段可放在业务函数内。初值 `[1,2,3,4,5]`，将 `[2,4]` 每项加 `3` 后，数组逻辑值为 `[1,5,6,7,5]`，`rangeSum(3,5)` 返回 `18`：

```
vector<ll> a = {0, 1, 2, 3, 4, 5}; // a[0] 不使用
SegmentTree st(5);
st.build(a);
st.rangeAdd(2, 4, 3);
ll answer = st.rangeSum(3, 5); // 18
```

5. 要改哪里

- 单点增量直接调用 `rangeAdd(i,i,delta)`；查询单点也可调用 `rangeSum(i,i)`。
- 若要区间赋值，需要新增赋值标记及其与加法标记的组合规则，不能把赋值当作一次区间加处理。
- 若要维护区间最小值/最大值等统计量，需修改 `sum` 的含义、节点合并方式、整段更新规则和查询返回值；仅改变量名不够。
- 若初值为全零，可不调用 `build`，因为构造函数已将节点初始化为零；有非零初值时必须构建。
- 多组数据或需要彻底重置时，为每组新建 `SegmentTree` 对象。当前 `build` 不清理旧懒标记，不能复用于已经做过更新的对象。

- 空数组不受支持；区间接口没有边界检查，调用者需保证 `1 <= l <= r <= n`。

6. 自检

初值 `[1,2,3,4,5]` 的总和是 `15`。执行 `rangeAdd(2,4,3)` 后，区间 `[2,4]` 的和应为 `18`，全数组和应为 `24`，`rangeSum(3,5)` 为 `18`。再执行 `rangeAdd(3,3,-6)`，位置 3 变为 0，`rangeSum(1,5)` 应为 `18`。单元素数组 `[7]` 上查询 `[1,1]` 得 `7`，加 `-7` 后得 `0`。这些检查也覆盖负增量和端点区间。

7. 坑

1. 把区间增量只加到区间和一次；正确增量是 `delta * 区间长度`。
2. 部分访问前未下传懒标记，子节点记录因此过期；检查递归进入孩子前是否调用 `pushDown`。
3. 下传后没清父标记会重复加；本模板在更新完孩子后将 `lazy[p]` 置零。
4. 把 0-based 区间传给接口；本模板所有公开区间和初值位置都是 1-based 闭区间。
5. 对已更新对象再次 `build` 试图重置；它只写 `sum`，不会清 `lazy`，重置时重新构造对象。

8. 一句话原理

整段更新先记在节点的和与懒标记中，只有递归访问子区间时才下传，从而避免逐个修改区间元素。

参考：

- OI Wiki：线段树

5.8 ST 表（静态 RMQ）

1. 它是做什么的

ST 表用于静态、不可修改的数组，预处理后查询闭区间最大值。本模板的数组按 `1..n` 存储，要求至少一个有效元素；构造时间和空间均为 $O(n \log n)$，每次查询为 $O(1)$。查询将两个长度为 2^j 的区间合并，二者可能重叠，因此运算须同时满足结合律和幂等性；`max`、`min`、`gcd` 都满足这两个条件，区间和不满足幂等性，不能这样查询。数组修改后需重建或改用支持更新的结构。

2. 变量在做什么

变量	含义
`n`	有效元素数，输入 `a` 的下标 `1..n` 有效，`a[0]` 不用。
`levels`	预处理层数，覆盖长度 `2^0` 至 `2^(levels-1)`。
`log2Floor[len]`	`floor(log2(len))`，用于选取不超过查询长度的最大幂次。
`st[j][i]`	闭区间 `[i, i+2^j-1]` 的最大值；只有该区间完全落在数组内的位置才用于查询。
`left`, `right`	`query` 的 1-based 闭区间端点。

对长度 `len=right-left+1`，令 `j=floor(log2(len))`、`blockLen=2^j`。查询合并 `[left,left+blockLen-1]` 与 `[right-blockLen+1,right]`：两段长度相同，第一段从 `left` 开始，第二段从 `right-blockLen+1` 开始；因 `2*blockLen >= len`，它们覆盖查询区间，重叠部分由幂等运算安全重复计算。

3. 函数在做什么

函数	契约
`SparseTableMax(a)`	参数为 1-based 非空静态数组；构造各幂长区间的最大值表，不修改 `a`。
`query(left,right)`	参数为满足 `1<=left<=right<=n` 的闭区间；返回该段最大值，不修改表。

4. 怎么调用

先把数据放入长度 `n+1` 的数组并保留 `a[0]` 作为占位，再构造一次，之后可重复查询：

```
vector<int> a = {0, 4, 2, 7, 1, 5};
SparseTableMax rmq(a);
// rmq.query(2, 5) 为 7; rmq.query(3, 4) 为 7
```

原模板：

```

#include <bits/stdc++.h>
using namespace std;

struct SparseTableMax {
    int n, levels;
    vector<int> log2Floor;
    // st[j][i] 保存闭区间 [i, i+2^j-1] 的最大值；输入 a 为 1-based。
    vector<vector<int>> st;

    // 参数 a 为 1-based 非空静态数组；构造各级区间最大值表，不修改 a。
    explicit SparseTableMax(const vector<int>& a) {
        n = (int)a.size() - 1;
        log2Floor.assign(n + 1, 0);
        // log2Floor[i] = floor(log2(i))，查询时可 O(1) 找到覆盖长度。
        for (int i = 2; i <= n; ++i) log2Floor[i] = log2Floor[i / 2] + 1;

        levels = log2Floor[n] + 1;
        st.assign(levels, vector<int>(n + 1));
        for (int i = 1; i <= n; ++i) st[0][i] = a[i];

        for (int j = 1; j < levels; ++j) {
            int length = 1 << j;
            for (int i = 1; i + length - 1 <= n; ++i) {
                // 长为 2^j 的区间由两个相邻、长为 2^(j-1) 的区间组成。
                st[j][i] = max(st[j - 1][i], st[j - 1][i + length / 2]);
            }
        }
    }

    // 参数为合法闭区间 1<=left<=right<=n；返回该区间最大值，不修改表。
    int query(int left, int right) const {
        // max 可重复贡献：用两个可能重叠的 2^j 区间覆盖 [left,right]。
        int j = log2Floor[right - left + 1];
        return max(st[j][left], st[j][right - (1 << j) + 1]);
    }
};
```

5. 要改哪里

- 求区间最小值时，将构造和查询中的 `max` 都改为 `min`。
- 求 GCD 时将两处合并都改为 `gcd`，并确保运算实现可用。
- 区间和不满足重叠计算的幂等条件，应使用前缀和、树状数组或线段树。
- 数组发生修改后，当前表不会自动更新；可重建整张表，或换用支持修改的数据结构。

6. 自检

数组 `[4,2,7,1,5]` 映射到下标 `1..5` 后，`query(2,5)` 和 `query(3,4)` 都应为 7，单点 `query(4,4)` 应为 1。包含负数的数组 `[-8,-3,-11]` 上，`query(1,3)` 应为 −3，`query(2,3)` 应为 −3；这能检查最大值初始化和重叠区间起点是否正确。长度为 1 时唯一合法查询返回该元素。

7. 坑

- 把 `a[0]` 当有效数据：输入约定从 1 开始。
- 第二段起点写错：应为 `right-2^j+1`；漏掉 `+1` 会错位。
- 把 ST 表用于求和：两段会重叠，重叠元素被重复计入。
- 在数组修改后沿用旧表：查询仍反映构造时的数据。

8. 一句话原理

预存各个 2 的幂长度区间的最大值，再用两个可能重叠的等长区间覆盖查询段。

参考：

- OI Wiki：ST 表

5.9 分块（区间加、区间和）

1. 它是做什么的

本模板把非空 1-based 数组分成约 $\sqrt{n}$ 个块，支持闭区间加一个增量和查询闭区间和。数组长度为 `n` 时，构造耗时、空间均为 $O(n)$；一次 `rangeAdd` 或 `rangeSum` 最多扫描两端散块并处理完整中间块，时间为 $O(\sqrt{n})$。它适合整块更新和查询都容易维护的场景；区间最值、赋值等操作需要重新设计块内状态，不能直接使用本模板。

2. 变量在做什么

变量	含义
`value[i]`	下标 `i` 的基础值，不含所在块的懒标记。实际值是 `value[i] + lazy[belong[i]]`。
`sum[b]`	块 `b` 所有 `value` 的和，同样不含懒标记。
`lazy[b]`	整块统一增加的量。整块实际和为 `sum[b] + lazy[b] * 块长`。
`belong[i]`	1-based 位置 `i` 所属的 0-based 块编号。
`leftEnd[b]`, `rightEnd[b]`	块 `b` 的实际闭区间端点；末块可能比其他块短。
`blockSize`, `blockCount`	目标块长和实际块数，块长取 `max(1, floor(sqrt(n)))`。

输入数组需满足 `a.size() >= 2`，有效元素为 `a[1..n]`，`a[0]` 不使用。元素、懒标记、整块和及其乘积均须能放入 `long long`。所有操作端点必须满足 `1 <= left <= right <= n`。

3. 函数在做什么

函数	参数、返回与作用
`SqrtDecomposition(a)`	用 `a[1..n]` 初始化块，复制数据，不修改输入；要求数组非空且采用 1-based 存储。
`rangeAdd(left,right,delta)`	将闭区间内每项增加 `delta`，修改基础值或完整块懒标记，并同步散块的 `sum`；无返回值。
`rangeSum(left,right)`	返回当前闭区间和；读取散块时补上懒标记，读取完整块时整块计入标记贡献，不修改结构。

4. 怎么调用

先准备一个首项不用的数组，再构造一次；后续更新和查询都调用对象方法。下面片段可放在业务函数内：

```
vector<long long> a = {0, 1, 2, 3, 4, 5};
SqrtDecomposition ds(a);
ds.rangeAdd(2, 4, 10);
// ds.rangeSum(1, 5) == 45; ds.rangeSum(2, 4) == 39
```

模板代码如下：

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

```

```cpp
struct SqrtDecomposition {
    int n, blockSize, blockCount;
    // value[i] 与 sum[b] 均不含 lazy[b]；真实值为 value[i]+lazy[belong[i]]。
    vector<ll> value, sum, lazy;
    // belong 映射 1-based 下标到 0-based 块；块端点均为闭区间。
    vector<int> belong, leftEnd, rightEnd;

    // 参数 a[1..n] 为非空初始数组；构造块和懒标记副本，不修改 a。
    explicit SqrtDecomposition(const vector<ll>& a) {
        // 输入 a[1..n]，a[0] 不使用；要求 n>=1。
        n = (int)a.size() - 1;
        blockSize = max(1, (int)sqrt(n));
        blockCount = (n + blockSize - 1) / blockSize;
        value = a;
        sum.assign(blockCount, 0);
        lazy.assign(blockCount, 0);
        belong.resize(n + 1);
        leftEnd.resize(blockCount);
        rightEnd.resize(blockCount);

        for (int b = 0; b < blockCount; ++b) {
            leftEnd[b] = b * blockSize + 1;
            rightEnd[b] = min(n, (b + 1) * blockSize);
            for (int i = leftEnd[b]; i <= rightEnd[b]; ++i) {
                belong[i] = b;
                sum[b] += value[i];
            }
        }
    }

    // 参数为合法闭区间及每项增量；返回 void，更新散块元素/整块懒标记。
    void rangeAdd(int left, int right, ll delta) {
        // 修改闭区间 [left,right]：散块写 value/sum，完整块只累加 lazy。
        int bl = belong[left], br = belong[right];
        if (bl == br) {
            for (int i = left; i <= right; ++i) value[i] += delta;
            sum[bl] += delta * (right - left + 1);
            return;
        }
        for (int i = left; i <= rightEnd[bl]; ++i) value[i] += delta;
        sum[bl] += delta * (rightEnd[bl] - left + 1);
        for (int b = bl + 1; b < br; ++b) lazy[b] += delta;  // 不改原始块和
        for (int i = leftEnd[br]; i <= right; ++i) value[i] += delta;
        sum[br] += delta * (right - leftEnd[br] + 1);
    }

    // 参数 left/right 为合法 1-based 闭区间；返回当前区间和，不修改结构。
    ll rangeSum(int left, int right) const {
        // 查询闭区间 [left,right]；散块逐项补 lazy，整块一次补标记贡献。
        int bl = belong[left], br = belong[right];
        ll result = 0;
        if (bl == br) {
            for (int i = left; i <= right; ++i) result += value[i] +
lazy[bl];
            return result;
        }
        for (int i = left; i <= rightEnd[bl]; ++i) result += value[i] +
lazy[bl];
        for (int b = bl + 1; b < br; ++b) {
            result += sum[b] + lazy[b] * (rightEnd[b] - leftEnd[b] + 1);
        }
        for (int i = leftEnd[br]; i <= right; ++i) result += value[i] +
lazy[br];
        return result;
```

```
    }
};
```

5. 要改哪里

- 改成区间加、单点查询时，可查询 `value[i] + lazy[belong[i]]`；若还维护其他聚合量，需同步更新散块和完整块状态。
- 增加区间赋值、区间最值等操作时，必须重新定义懒标记和块汇总值的组合及更新规则；不能只把某个循环里的加法替换掉。
- 若采用不同块长，应同时检查两端扫描长度与中间块数量，确保最坏时间仍符合预期。多组数据应为每组重新构造对象；构造函数会初始化全部块状态。

6. 自检

示例初始数组为 `[1,2,3,4,5]`，对 `[2,4]` 加 10 后为 `[1,12,13,14,5]`，总和 45，区间 `[2,4]` 和为 39。可用长度为 1 的区间检查点值，再查询整段核对总和；特意选跨块区间和落在最后一个短块的区间，检查边界端点及实际块长是否正确。

7. 坑

- 整块更新后散块查询少算：`value` 不含 `lazy`，扫描单个元素时必须加上所在块标记。
- 整块查询重复或漏算标记：`sum[b]` 也不含标记，应额外加 `lazy[b] * 块长`。
- 最后一块结果错误：末块长度可能不足 `blockSize`，计算整块贡献应使用 `rightEnd[b]-leftEnd[b]+1`。
- 用 0-based 输入直接调用：构造和操作都约定有效下标从 1 开始，需先调整存储方式或改写映射。

8. 一句话原理

两端不完整的块逐项处理，中间完整块用块和与懒标记一次处理。

参考：

- OI Wiki：分块

5.10 树状数组：区间加与区间和

1. 它是做什么的

`RangeFenwick` 在长度为 `n` 的 1-based 数组上支持闭区间加法和区间求和。它把数组变化表示为差分，再用两棵 5.6 节的 `Fenwick` 维护差分系数。适用于区间加、区间和交错出现；不能直接支持区间赋值或区间最值。

构造两棵全零树状数组为 $O(n)$ 时间和空间。一次区间加和一次区间求和均为 $O(\log n)$；查询前缀同样为 $O(\log n)$。非零初值需逐点调用 `addRange(i,i,a[i])`，总初始化时间为 $O(n \log n)$。数组值及中间乘积使用 `long long`，需保证数值不溢出。本代码依赖 5.6 节的 `Fenwick` 类型及 `ll` 别名；单独摘出代码块不能编译。

2. 变量在做什么

名称	含义
`n`	逻辑数组长度；合法位置为 `1..n`，构造要求正数。
`b1`、`b2`	两棵差分 Fenwick，分别存差分值和乘以位置偏移后的系数。
`x`	前缀右端点，`prefix(x)` 接受 `0..n`。
`l`、`r`	公开区间的左右端点，使用 1-based 闭区间。
`v`	每个被覆盖位置增加的值。

名称	含义
`addPoint(x,v)`	内部更新差分端点；`n+1` 是区间右界之后的哨兵，不写入树。

设 `D[i]=A[i]-A[i-1]`，则 `A[i]` 是 `D[1..i]` 的和。对 `A[l..r]` 加 `v`，令 `D[l]+=v`、`D[r+1]-=v` 即可。前缀数组和由 `x * sum(D[1..x]) - sum((i-1)D[i])` 得出，即 `x*b1.prefixSum(x)-b2.prefixSum(x)`。

3. 函数在做什么

函数	参数与返回	行为及调用条件
`RangeFenwick(size)`	长度 `size`；构造对象。	初始化两棵全零 Fenwick，逻辑数组初值全为 0；要求 `size > 0`。
`addPoint(x,v)`	差分位置 `x` 与增量；返回 `void`。	同步修改两棵树；允许 `x=n+1` 哨兵并忽略该更新。内部辅助函数，通常通过 `addRange` 使用。
`addRange(l,r,v)`	1-based 闭区间及增量；返回 `void`。	通过两个差分端点更新逻辑数组；若 `l>r` 则不做事。非空区间要求 `1<=l<=r<=n`。
`prefix(x)`	`x` 属于 `0..n`；返回 `long long`。	返回逻辑数组 `[1,x]` 的和；`x=0` 返回 0。
`query(l,r)`	1-based 闭区间；返回 `long long`。	返回 `[l,r]` 的和；若 `l>r` 返回 0。非空查询须在数组范围内。
依赖的 `Fenwick::add`	位置 `1..n` 和增量；返回 `void`。	更新单点差分系数；不能对位置 0 调用。
依赖的 `Fenwick::prefixSum`	位置 `0..n`；返回前缀和。	被 `prefix` 调用以取得系数前缀。

4. 怎么调用

先提供 5.6 节的 `Fenwick` 和 `ll` 定义，再构造 `RangeFenwick(n)`。它默认代表全零数组；非零初值按点初始化，然后使用 `addRange` 和 `query`。模板如下：

```
1
2  struct RangeFenwick {
3      // 两棵树保存差分系数，prefix(x)=x*B1.prefix-B2.prefix。
4      int n;
5      Fenwick b1,b2;
6      // 参数 size 为正数组长度；初始化两棵差分 Fenwick，逻辑数组初始为零。
7      explicit RangeFenwick(int size) : n(size), b1(size), b2(size) {}
8      // 参数 x 属于 1..n+1，v 是差分增量；返回 void，更新系数树，n+1 哨兵不写树。
9      void addPoint(int x, ll v) {
10         // 差分在 l 加 v、在 r+1 减 v；n+1 的哨兵不需要写入树中。
11         if (x <= n) {
12             b1.add(x, v);
13             b2.add(x, v * (x - 1));
14         }
15     }
16     // 参数为 1-based 闭区间及增量；返回 void，修改两棵树表示的逻辑数组，空区间无操作。
17     void addRange(int l, int r, ll v) {
18         if (l > r) {
19             return;
20         }
21         addPoint(l, v);
22         addPoint(r + 1, -v);
23     }
24     // 参数 x 属于 0..n；返回逻辑数组 [1,x] 之和，不修改树。
25     ll prefix(int x) const {
26         return b1.prefixSum(x) * x - b2.prefixSum(x);
27     }
28     // 参数为合法闭区间；返回逻辑数组区间和，空区间返回 0。
29     ll query(int l, int r) const {
```

```
        return l > r ? 0 : prefix(r) - prefix(l - 1);
    }
};
```

片段可放在业务函数内。初值 `[1,2,3,4,5]`，对 `[2,4]` 加 2 后数组为 `[1,4,5,6,5]`，查询 `[3,5]` 得 16：

```
RangeFenwick fw(5);
for (int i = 1; i <= 5; ++i) fw.addRange(i, i, i);
fw.addRange(2, 4, 2);
long long answer = fw.query(3, 5); // 16
```

5. 要改哪里

- 单点增加用 `addRange(i,i,v)`；区间和使用 `query(l,r)`。
- 非零初值逐点调用 `addRange(i,i,a[i])`，总计 $O(n \log n)$；本模板没有线性建树接口。
- 区间赋值、区间最值等需求不能只改查询公式；差分及两棵和树的状态不足以维护它们，需换用或设计相应数据结构。
- 空更新区间 `l>r` 会忽略；空查询区间 `l>r` 返回 0。其他下标没有边界检查，调用者须保证合法。
- `r=n` 时 `r+1` 是 `n+1` 哨兵，`addPoint` 会跳过它；不要把哨兵传入底层 `Fenwick::add`。更新位置不能为 0。

6. 自检

构造长度 5 的全零数组，逐点加入 `[1,2,3,4,5]` 后 `query(1,5)` 应为 15。将 `[2,4]` 加 2 后，逻辑值为 `[1,4,5,6,5]`，`query(3,5)` 为 16，`query(1,1)` 仍为 1。`query(2,1)` 返回 0，`addRange(4,3,100)` 不改变数据。再执行 `addRange(5,5,1)`，`query(5,5)` 应为 6，可核对右端点哨兵。

7. 坑

1. 区间加没有转为两个差分端点；应在 `l` 加 `v`、在 `r+1` 减 `v`。
2. 忘记偏移树 `b2` 或写错 `x-1`，前缀公式会出错；核对 `x*b1.prefixSum(x)-b2.prefixSum(x)`。
3. `r=n` 时把 `n+1` 传给底层 Fenwick；底层合法下标只有 `1..n`，本实现会忽略哨兵。
4. 初始化非零数组后遗漏逐点加入；构造函数只创建全零逻辑数组。
5. 复制此结构时漏掉 5.6 节的 `Fenwick` / `ll`；需将依赖一并提供。

8. 一句话原理

把区间加转换为差分数组的两个端点变化，再用两棵 Fenwick 将差分前缀还原成原数组前缀和。

参考：

- OI Wiki：树状数组

5.11 DFS 序与子树展平

1. 它是做什么的

对一棵静态、连通无向树选定根后，DFS 序把每个节点的子树映射为连续闭区间 `[tin[u],tout[u]]`。因此可把子树查询或修改交给树状数组、线段树等区间结构。建序耗时 $O(n)$，辅助空间为 $O(n)$。本模板每个节点只记录一次，不是 LCA 常用的进入与回溯都记录的欧拉游程。

2. 变量在做什么

变量	含义
`g`	1-based 无向邻接表。
`timer`	已进入 DFS 的节点数；每进入一个节点递增一次。
`tin[u]`	节点 `u` 的进入时间戳。
`tout[u]`	`u` 子树遍历完成时的时间戳，与 `tin[u]` 构成闭区间。

变量	含义
`order[t]`	DFS 时间戳 `t` 对应的节点编号，即 `order[tin[u]] == u`。
`u, p`	当前节点及父节点；根调用时父节点传 0。

两种映射用途不同：若要把每个节点的点值放进线段树或树状数组，应按 `tin[u]` 作为位置写入；若要逐个枚举子树节点，则遍历 `t=tin[u]..tout[u]`，并用 `order[t]` 取回节点编号。不能把时间戳 `t` 当成点编号。

例如边为 `1-2`、`1-3`、`2-4`、`2-5`，根为 1 且按邻接顺序访问时，`order` 为 `[1,2,4,5,3]`；节点 2 的区间 `[2,4]` 对应节点 2、4、5。

3. 函数在做什么

函数	契约
`DfsOrder(n)`	分配图和序列数组；点编号范围为 `1..n`，应传正点数。
`addEdge(u,v)`	参数为合法树点编号；向 `g` 双向加入一条边，修改邻接表，无返回值；每条无向边只调用一次。
`dfs(u,p)`	从 `u` 开始遍历，写入 `tin`、`tout`、`order` 并增加 `timer`，无返回值；输入必须是树且不能形成递归环。根的 `p` 传 0，整张树由调用方指定根后调用一次。

4. 怎么调用

先构造对象并加入全部 `n-1` 条边，再从选定根调用 `dfs(root,0)`。之后点值按 `tin` 放入区间结构，或通过 `order` 枚举子树：

```
DfsOrder d(5);
d.addEdge(1, 2); d.addEdge(1, 3);
d.addEdge(2, 4); d.addEdge(2, 5);
d.dfs(1, 0);
// tin[2]=2、tout[2]=4; order[2..4] 为 2,4,5
// 将点值 value[u] 写入序列位置 tin[u]; 枚举子树 2 则读取 order[2..4]
```

原模板：

```

struct DfsOrder {
    vector<vector<int>> g;
    vector<int> tin,tout,order;
    int timer=0;
    // 参数 n 为树点数；分配 1-based 图与 DFS 序数组，timer 初始为 0。
    explicit DfsOrder(int n)
        : g(n + 1), tin(n + 1), tout(n + 1), order(n + 1) {}
    // 参数 u、v 是 1..n 树点；返回 void，向 g 双向加入边；建 DFS 序前每条树边调用一次。
    void addEdge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    // 参数 u 是当前子树根、p 是父点（根传 0）；返回 void，写 tin/tout/order/timer；输入须为树。
    void dfs(int u, int p) {
        // 进入时分配唯一时间戳，递归结束时的 timer 即子树最后一个时间戳。
        tin[u] = ++timer;
        order[timer] = u;
        for (int v : g[u]) {
            if (v != p) {
                dfs(v, u);
            }
```

```
        }
        tout[u] = timer;
    }
};
```

5. 要改哪里

- 更换根或重新计算 DFS 序：创建新对象，重新加入全部边，再调用 `dfs(newRoot,0)`。旧对象的 `timer` 和时间戳已被使用；不必在它上面清空图。
- 子树点值求和或更新：把 `value[u]` 放在位置 `tin[u]`，对闭区间 `[tin[u],tout[u]]` 操作。
- 需要枚举子树中的原节点编号：对 `t` 遍历该闭区间，再读取 `order[t]`。这是枚举映射，不等同于直接把 DFS 位置当作节点编号。
- 需要 LCA 的欧拉游程：另建会在进入及回溯时记录节点的序列，当前 `order` 每点只出现一次。

6. 自检

按示例树从根 1 遍历，应得到 `tin[1]=1`、`tin[2]=2`、`tin[4]=3`、`tin[5]=4`、`tin[3]=5`，并有 `tout[2]=4`。每个节点的 `tout[u]-tin[u]+1` 应等于其子树节点数；节点 2 的区间位置 `2..4` 映回节点应为 `2,4,5`。

7. 坑

- 把区间端点当半开区间：本模板的 `[tin,tout]` 两端都包含。
- 把 `order` 位置当作节点编号：应使用 `order[t]` 映回节点。
- 对一般图直接运行：代码只跳过父节点，没有 visited 标记；输入需为树。
- 重用已遍历对象：`timer` 和数组不会自动重置；重根或重新建序时新建对象并重新加边。

8. 一句话原理

DFS 完整遍历完一个节点的子树后才离开它，因此子树节点的进入时间连续排列。

5.12 树链剖分

1. 它是做什么的

树链剖分把一棵 1-based 连通无向树上两点间的路径拆成 $O(\log n)$ 个连续位置区间，供线段树等结构逐段处理。建树剖分耗时 $O(n)$、空间 $O(n)$；本模板的 `forPath` 产生 $O(\log n)$ 段。若每段在线段树上查询或更新一次，单次路径操作为 $O(\log^2 n)$。回调收到的区间端点闭合，但区间顺序不保证等于从 `u` 到 `v` 的路径顺序；可用于可交换聚合或各段独立更新。需要方向敏感的非交换运算时，必须额外记录方向并按路径顺序合并。

先看复杂度从哪里来：若沿树路径逐点向父亲走，最坏要访问 $O(n)$ 个点；DFS 序只能把子树展平，不能让任意路径成为单段。HLD 选最大子树孩子为重儿子，第二次 DFS 优先走它，使重链连续。沿路径向上跨一条轻边时，父点的子树规模至少是轻儿子子树规模的两倍；反向从父点走向轻儿子时，子树规模至少减半。因此每条根到点路径经过 $O(\log n)$ 条轻边，任意两点路径也只跨 $O(\log n)$ 条重链。

小例子：边为 `1-2,1-3,3-4,3-5`，以 1 为根，3 是 1 的重儿子，2 是轻儿子；若 4 被选为 3 的重儿子，重链为 `1-3-4`。查询 2 到 5 会得到覆盖点 2、1、3、5 的若干闭区间。

2. 变量在做什么

变量	含义
`g[u]`	无向邻接表；通过 `addEdge` 加边，所有边必须在 `build` 前加入。
`parent[u]`,`depth[u]`	以指定根定根后的父节点和深度，根深度为默认初值 0。
`sz[u]`	以 `u` 为根的子树大小。
`heavy[u]`	`u` 的最大子树孩子；无孩子时为 `-1`。

变量	含义
`top[u]`	`u` 所在重链的链头。
`pos[u]`	剖分后的 1-based 连续位置；同一重链连续，子树也对应连续区间。
`timer`	分配 `pos` 时递增的全局位置计数器；对象构造时为 0，`build` 会重置。

点编号和剖分位置是两个不同维度：路径区间传给维护结构时使用 `pos`，建树时遍历点编号，令 `base[pos[u]]=weight[u]` 即可，不需要逆映射；若要从位置取回点编号，再另建 `vertexAt[pos[u]]=u`。

3. 函数在做什么

函数	参数、返回与作用
`HLD(nodeCount)`	分配 `1..nodeCount` 的邻接表和数组；点数应为正。
`addEdge(u,v)`	要求两个合法不同点；向邻接表双向加入一条边，须在 `build` 前且每条边只加一次。
`dfs1(u,p)`	内部递归，设置父亲、深度、子树大小及重儿子，返回子树大小；在合法树上由 `build` 调用。
`dfs2(u,head)`	内部递归，设置链头和位置，并优先遍历重儿子；无返回值。
`build(root)`	完成剖分；要求所有边已加入，`root` 在 `1..n`。会清空旧剖分数组并重建，要求输入仍为连通树；可重复调用或更换根。省略参数时根为 1。
`forPath(u,v,use)`	将路径拆成闭区间并逐段调用 `use(l,r)`；回调返回值被忽略，需通过捕获外部对象实现查询或更新。区间参数是剖分位置，次序不承诺对应原路径方向。

4. 怎么调用

用 `addEdge` 加完边，再调用 `build`；随后让 `forPath` 的回调把区间交给线段树。下例为链 `1-2-3`，路径查询得到的位置区间 `[1,3]`：

```
HLD hld(3);
hld.addEdge(1, 2);
hld.addEdge(2, 3);
hld.build();
// l/r 为剖分位置闭区间；回调输出端点，不修改 hld。
hld.forPath(1, 3, [&](int l, int r) { cout << l << " " << r << "\n"; });
// 输出: 1 3
```

模板代码如下：

```

struct HLD {
    // heavy 子树最大者优先同链；任意树路径被拆成 O(log n) 个连续区间。
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> parent,depth,sz,heavy,top,pos;
    // nodeCount 为点数（须 >=1）；分配 1-based 邻接表和剖分数组，不修改调用方点权。
    explicit HLD(int nodeCount)
        : n(nodeCount), g(nodeCount + 1), parent(nodeCount + 1),
          depth(nodeCount + 1), sz(nodeCount + 1),
          heavy(nodeCount + 1, -1), top(nodeCount + 1), pos(nodeCount + 1) {
        assert(n >= 1);
    }
    // u、v 为 1..n 的树点；返回 void，向 g 双向加入一条边；必须在 build 前且每条边只加一次。
    void addEdge(int u, int v) {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
```

```
        g[u].push_back(v);
        g[v].push_back(u);
    }
    // u 为当前子树根、p 为父点；返回子树大小，并写 parent/depth/sz/heavy；输入必须是树。
    int dfs1(int u, int p) {
        parent[u] = p;
        sz[u] = 1;
        heavy[u] = -1;
        int best = 0;
        for (int v : g[u]) {
            if (v == p) continue;
            depth[v] = depth[u] + 1;
            int sub = dfs1(v, u);
            sz[u] += sub;
            if (sub > best) {
                best = sub;
                heavy[u] = v;
            }
        }
        return sz[u];
    }
    // u 为子树根、head 为其链头；返回 void，按重儿子优先写 top/pos 并递增 timer。
    void dfs2(int u, int head) {
        top[u] = head;
        pos[u] = ++timer;
        if (heavy[u] != -1) dfs2(heavy[u], head);
        for (int v : g[u]) {
            if (v != parent[u] && v != heavy[u]) dfs2(v, v);
        }
    }
    // root 为 1..n 的定根；返回 void，清空旧剖分并重建所有数组；要求 g 是连通树。
    void build(int root = 1) {
        assert(1 <= root && root <= n);
        timer = 0;
        fill(parent.begin(), parent.end(), 0);
        fill(depth.begin(), depth.end(), 0);
        fill(sz.begin(), sz.end(), 0);
        fill(heavy.begin(), heavy.end(), -1);
        fill(top.begin(), top.end(), 0);
        fill(pos.begin(), pos.end(), 0);
        dfs1(root, 0);
        dfs2(root, root);
        assert(timer == n);
    }
    // u/v 为已 build 的树端点；use(l,r) 接收 1-based 闭区间且返回值被忽略；区间方向不保证，返回 void。
    template<class F>
    void forPath(int u, int v, F use) const {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        while (top[u] != top[v]) {
            if (depth[top[u]] < depth[top[v]]) swap(u, v);
            use(pos[top[u]], pos[u]);
            u = parent[top[u]];
        }
        if (depth[u] > depth[v]) swap(u, v);
        use(pos[u], pos[v]);
    }
};
```

若要实际查询点权路径和，先提供 5.7 节的 `SegmentTree`，再将点权放到剖分位置上。接着上面的三点链，在业务函数中继续写：

```
vector<ll> weight = {0, 10, 20, 30}, base(4);
for (int u = 1; u <= 3; ++u) base[hld.pos[u]] = weight[u];
SegmentTree st(3);
st.build(base);
ll pathSum = 0;
// l/r 为剖分位置闭区间；将该段和累加到 pathSum，返回 void。
hld.forPath(1, 3, [&](int l, int r) { pathSum += st.rangeSum(l, r); });
// pathSum == 60
```

子树查询可直接使用 `[pos[u], pos[u]+sz[u]-1]`，同样交给数组维护结构处理。

5. 要改哪里

- 点权路径查询或更新：把回调区间 `[l,r]` 交给线段树处理，线段树下标使用 `pos`。
- 边权路径查询：约定每条边的权值放在较深端点对应的位置；当两个端点落在同一条链时，最终区间要从 `pos[LCA]+1` 开始，排除代表 LCA 的位置。跨链段的链头位置代表的是与父亲相连的边，应保留。
- 非交换聚合（如字符串拼接、矩阵乘法）：当前回调区间无方向信息且分段顺序不保证，需改写分解接口，同时维护从两端收集的区间和方向；不能直接按回调顺序合并。
- 多测或更换根：边加入后可再次调用 `build(root)`；它会清空并重建剖分数组。外部线段树也须按新 `pos` 重排。

区分模型：路径加、路径和或路径最值通常使用 HLD 加线段树；只有子树操作时 DFS 序更简单。路径上的字符串拼接、矩阵乘法等不可交换操作，需要记录每段方向并按原路径顺序合并，当前 `forPath` 不提供方向。

6. 自检

在链 `1-2-3` 上构建后，`forPath(1,3,...)` 应只回调一次，区间为 `[1,3]`；`forPath(2,2,...)` 应得到单点区间。再用树 `1-2, 1-3` 测 `forPath(2,3,...)`，回调区间覆盖点 2、1、3 对应的位置，不能遗漏 LCA。边权版本还要确认同链查询排除了 LCA 的位置。

7. 坑

- 结果像用了旧树：在加完边之前调用了 `build`；先完整建图再剖分。
- 深链上递归深度可达 n，栈不足时需将两次 DFS 改为显式栈。
- 重建后外部权值看似错乱：`build` 会改变 `pos`，需按新位置重建外部维护结构。
- 路径和/最值适用、路径拼接错误：回调区间顺序不保证路径方向；方向敏感操作需同时维护区间方向和两端的合并顺序。
- 边权答案多一条或少一条：最终同链段包含 LCA；将该段左端改为 `pos[LCA]+1`。

8. 一句话原理

最大子树孩子优先进入同一条链；每次跨轻边时子树规模至少减半，所以任一路径只跨 $O(\log n)$ 条链，每条链映射成连续区间。

5.13 线段树：区间加、赋值与求和

1. 它是做什么的

`AssignAddSegmentTree` 在长度为 `n` 的 1-based 数组上支持三种操作：把闭区间每项加上一个值、把闭区间每项设成同一个值、查询闭区间和。适合两种更新交错出现的场景；数组值、区间和及乘积必须能放进 `long long`。

构造节点空间为 $O(n)$，调用 `build` 初始化为 $O(n)$；一次区间加、区间赋值或区间求和均为 $O(\log n)$；空间为 $O(n)$，节点数组约为 `4n`。要求 `n>=1`，公开操作的区间须满足 `1<=l<=r<=n`，接口不检查这些条件。

2. 变量在做什么

名称	含义
`n`	数组长度；位置从 1 到 `n`。
`tr[p].sum`	节点 `p` 管辖区间的当前总和，包含本节点尚未下传的更新效果。
`tr[p].hasSet`	是否存在等待下传的赋值操作；赋值为 0 时也必须为真。
`tr[p].setValue`	`hasSet` 为真时，表示该节点区间每项待赋成的值。
`tr[p].add`	独立的待下传增量；若赋值标记存在，之后的增量会合并进 `setValue`，不另存此标记。
`p,l,r`	递归内部的节点编号和它管理的闭区间。
`ql,qr`	更新或查询目标闭区间；所有接口采用 1-based 下标。
`assign`	内部更新参数；真表示赋值，假表示加法。公开接口会替调用者设置它。

标记表示对整段尚未传给孩子的操作。组合顺序遵循时间先后：新赋值会覆盖旧赋值和旧加法；有赋值标记时再加 `delta`，应把 `delta` 加到待赋值数上；没有赋值标记时，加法累积在 `add` 中。向下传递时先赋值，再加法。

3. 函数在做什么

函数	参数与返回	行为及调用条件
`AssignAddSegmentTree(size)`	正数组长度；构造对象。	创建初值为零、无标记的节点空间。
`build(a)`	1-based 初值 `a[1..n]`；返回 `void`。	仅初始化 `sum`，不清懒标记；在新对象上调用，重置需重新构造。
内部 `build(p,l,r,a)`	初值和当前区间；返回 `void`。	递归写叶值并合并区间和，不重置标记；无需直接调用。
`applySet(p,l,r,value)`	当前节点和赋值值；返回 `void`。	更新区间和，设置赋值标记并清除加法标记。
`applyAdd(p,l,r,delta)`	当前节点和增量；返回 `void`。	更新区间和；依据是否已有赋值标记，把增量合并到赋值值或独立加法标记。
`push(p,l,r)`	当前节点及闭区间；返回 `void`。	先把赋值、再把加法下传到左右孩子，并清除父标记；内部调用。
`rangeAdd(l,r,value)`	合法闭区间和增量；返回 `void`。	将每个区间元素增加 `value`。
`rangeSet(l,r,value)`	合法闭区间和赋值值；返回 `void`。	将每个区间元素覆盖为 `value`，包括 0。
`rangeSum(l,r)`	合法闭区间；返回 `long long`。	返回区间和；部分查询可能下传标记。
内部 `update` / `query`	当前节点、目标区间和操作参数；更新返回 `void`，查询返回区间和。	递归处理；通常使用上述公开接口，不直接调用。

4. 怎么调用

构造正长度的新对象；若有非零初值，准备 `a[1..n]` 并调用一次 `build(a)`。之后按需要调用 `rangeSet`、`rangeAdd` 和 `rangeSum`。模板如下：

```
1
2  #include <bits/stdc++.h>
3  using namespace std;
4  using ll = long long;
5
6  struct AssignAddSegmentTree {
7      struct Node {
8          ll sum = 0;
```

```
        ll add = 0;
        ll setValue = 0;
        bool hasSet = false; // 赋值为 0 也有效，不能用 setValue==0 表示“无标记”
    };
    int n;
    vector<Node> tr;

    // 参数 size 为正数组长度；分配初始零值节点，标记均无效。
    explicit AssignAddSegmentTree(int size) : n(size), tr(4 * size + 4) {}

    // 参数 p 管理闭区间 [l,r]、value 为赋值；返回 void，覆盖该节点和并设置赋值标记。
    void applySet(int p, int l, int r, ll value) {
        // 覆盖整个节点：清除旧加法，并留下待下传的赋值。
        tr[p].sum = value * (r - l + 1);
        tr[p].setValue = value;
        tr[p].add = 0;
        tr[p].hasSet = true;
    }
    // 参数 p 管理闭区间 [l,r]、delta 为增量；返回 void，修改该节点和并合并加法/赋值标记。
    void applyAdd(int p, int l, int r, ll delta) {
        tr[p].sum += delta * (r - l + 1);
        // 赋值标记仍在时把增量合并到赋值值，否则累积为独立加法标记。
        if (tr[p].hasSet) tr[p].setValue += delta;
        else tr[p].add += delta;
    }
    // 参数 p 管理闭区间 [l,r]；返回 void，按赋值后加法顺序下传标记并清理父标记。
    void push(int p, int l, int r) {
        if (l == r) return;
        int m = (l + r) / 2;
        // 操作顺序必须是先赋值后加法，才能保持时间顺序。
        if (tr[p].hasSet) {
            applySet(p * 2, l, m, tr[p].setValue);
            applySet(p * 2 + 1, m + 1, r, tr[p].setValue);
            tr[p].hasSet = false;
        }
        if (tr[p].add != 0) {
            applyAdd(p * 2, l, m, tr[p].add);
            applyAdd(p * 2 + 1, m + 1, r, tr[p].add);
            tr[p].add = 0;
        }
    }
    // 参数 a 为 1-based 初值，p 管理闭区间 [l,r]；返回 void，递归建立节点和。
    void build(int p, int l, int r, const vector<ll>& a) {
        if (l == r) {
            tr[p].sum = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        tr[p].sum = tr[p * 2].sum + tr[p * 2 + 1].sum;
    }
    // 参数 a[1..n] 为初值；返回 void，只建立节点和、不清懒标记；须在新对象上初始化，重置需重新构造对象。
    void build(const vector<ll>& a) { build(1, 1, n, a); }

    // 参数 p/l/r 为当前节点，ql/qr 为合法修改区间，value 为操作值、assign 选择赋值或加法；返回 void 并更新节点。
    void update(int p, int l, int r, int ql, int qr, ll value, bool assign)
{
        if (ql <= l && r <= qr) {
            if (assign) applySet(p, l, r, value);
            else applyAdd(p, l, r, value);
```

```
            return;
        }
        push(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) update(p * 2, l, m, ql, qr, value, assign);
        if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, value, assign);
        tr[p].sum = tr[p * 2].sum + tr[p * 2 + 1].sum;
    }
    // 参数为合法闭区间及增量；返回 void，将区间每项增加 value 并修改树状态。
    void rangeAdd(int l, int r, ll value) { update(1, 1, n, l, r, value,
false); }
    // 参数为合法闭区间及赋值；返回 void，将区间每项设为 value 并修改树状态。
    void rangeSet(int l, int r, ll value) { update(1, 1, n, l, r, value,
true); }

    // 参数 p/l/r 为当前节点，ql/qr 为合法查询区间；返回交集和，并可能下传懒标记。
    ll query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tr[p].sum;
        push(p, l, r);
        int m = (l + r) / 2;
        ll result = 0;
        if (ql <= m) result += query(p * 2, l, m, ql, qr);
        if (qr > m) result += query(p * 2 + 1, m + 1, r, ql, qr);
        return result;
    }
    // 参数为合法闭区间；返回区间和，查询可能下传内部标记。
    ll rangeSum(int l, int r) { return query(1, 1, n, l, r); }
};
```

片段可放在业务函数内。初值 `[1,2,3,4,5]`，先将 `[2,4]` 赋成 7，再给 `[3,5]` 加 2，数组变成 `[1,7,9,9,7]`，`rangeSum(2,5)` 返回 32：

```
vector<ll> a = {0, 1, 2, 3, 4, 5};
AssignAddSegmentTree st(5);
st.build(a);
st.rangeSet(2, 4, 7);
st.rangeAdd(3, 5, 2);
ll answer = st.rangeSum(2, 5); // 32
```

5. 要改哪里

- 单点操作用 `[i,i]` 区间即可，例如 `rangeSet(i,i,v)` 或 `rangeAdd(i,i,d)`。
- 若只需要区间加与区间和，可使用更简单的 5.7 节模板；若还需要区间最值，须重新设计节点聚合和更新逻辑。
- 要增加新的更新类型，须定义其对 `sum` 的影响、懒标记的存法，以及它与现有赋值和加法标记的时间顺序组合；不能只增加一个接口函数。
- 初始数组全零时，构造后的逻辑值已为零，可跳过 `build`。非零初值需在新对象上构建。多测重置或重复初始化时重新构造对象，因为 `build` 不会清理旧标记。
- `n` 必须为正数，区间须合法；实现不做边界检查。还需保证所有中间乘法和区间和不溢出 `long long`。

6. 自检

对初值 `[1,2,3,4,5]`，先 `rangeSet(2,4,7)` 后为 `[1,7,7,7,5]`；再 `rangeAdd(3,5,2)` 后为 `[1,7,9,9,7]`，全数组和为 33，`rangeSum(2,5)` 为 32。再把 `[1,5]` 赋为 0，查询 `[1,5]` 必须得到 0，这能检查“赋值为零”由 `hasSet` 区分的行为。单元素数组 `[6]` 执行加 `-2` 后查询得 4；随后赋值 0 得 0。

7. 坑

1. 用 `setValue==0` 判断是否有赋值标记；0 是合法赋值，必须看 `hasSet`。
2. 有待下传赋值时仍把新加法记入独立 `add`；本实现将它合并到 `setValue`，保证时间顺序正确。
3. 下传时先加后赋值，会把较新的加法覆盖掉；应先下传赋值，再下传加法。

4. 用 `build` 重建一个已操作的对象；该函数只写节点和、不清标记，应重新构造对象后初始化。
5. 更新或查询越界、使用 0-based 区间；公开接口无边界保护，必须传入合法 1-based 闭区间。

8. 一句话原理

线段树用赋值标记表达覆盖、用加法标记表达增量，并按操作发生顺序合并和下传，从而在对数时间内维护区间和。

5.14 DSU on Tree（树上启发式合并）

1. 它是做什么的

给定一棵静态有根树和每个节点的颜色，本模板一次求出所有节点子树中的不同颜色数。颜色可以是负数或很大的整数，模板会先离散化。输入是 1-based 连通无向树 `tree`、颜色数组 `color[1..n]` 和根 `root`；`solve()` 返回长度 `n+1` 的数组，`answer[u]` 是节点 `u` 的子树答案，0 号位不用。

朴素地为每个点单独 DFS 其子树，一条链会扫描 $n+(n-1)+\cdots=O(n^2)$ 个节点；仅把子树展平为区间并不能消除重复扫描。DSU on Tree 的关键是保留最大子树的统计表，轻子树算完答案后清空，稍后再作为轻子树重新加入。每条轻边都会让子树规模至少翻倍，因此每个节点最多重新加入 $O(\log n)$ 次。DSU on Tree 负责安排哪些节点加入或移出当前集合，桶、树状数组或线段树负责维护集合里的统计。本模板用颜色频次桶统计不同颜色数。它与普通并查集不同：这里没有合并集合代表元的操作，保留的是重儿子已经算好的频次状态。适合离线回答固定根下的子树统计，不直接支持在线改色、动态换根或路径查询。

预处理 DFS 序为 $O(n)$，颜色排序与压缩为 $O(n\log n)$。统计中节点的加入、删除共 $O(n\log n)$ 次；本模板每次频次变化为 $O(1)$，所以总时间为 $O(n\log n)$、额外空间为 $O(n)$，包括递归栈。若改用每次增删为 $O(\log n)$ 的统计结构，统计总时间就会变成 $O(n\log^2 n)$。这些界限还要求每个节点的答案能在对应代价内读出。

2. 变量在做什么

变量	含义
`tree`、`color`	输入树和颜色的常量引用；被引用对象必须比求解器活得更久。
`n`、`root`	节点数与固定根；要求 `n>=1`、`1<=root<=n`。
`parent[u]`、`subtreeSize[u]`	定根后的父节点与子树大小。
`heavy[u]`	子树最大的那个儿子；叶子为 0。大小并列时选最先遇到的一个即可。
`tin[u]`、`tout[u]`	子树在 DFS 序上的闭区间 `[tin[u],tout[u]]`。
`order[i]`	DFS 位置 `i` 对应的节点编号，满足 `order[tin[u]]==u`。
`compressed[u]`	节点 `u` 的颜色编号，范围为 `0..颜色种类数-1`。
`frequency[c]`、`distinct`	当前活跃集合中颜色编号 `c` 的次数，以及次数非零的颜色数。
`answer[u]`、`timer`	各子树答案，以及预处理时分配 DFS 位置的计数器。
`keep`	当前子树算完后是否保留频次；不是永久标记，也不表示跳过该子树。

例如边按 `1-2`、`1-3`、`2-4`、`2-5` 加入，根为 1，DFS 顺序是 `order[1..5]=[1,2,4,5,3]`。子树 2 对应位置 `[2,4]`，映回节点是 `{2,4,5}`。枚举它时先取 `order[i]`，再读该节点颜色；不能把位置 `i` 直接当节点号。

统计结构的下标由统计维度决定：这里是 `frequency[compressed[u]]`；若按深度计数，则用深度作桶或线段树的下标。只有维护“DFS 位置上的点值”时才把 `tin[u]` 用作线段树位置，不能把所有统计都套到 DFS 下标上。

3. 函数在做什么

函数	参数、返回与作用
`DSUOnTree(tree,color,root)`	保存输入引用并分配内部数组；树必须连通无环，颜色数组覆盖 `1..n`，构造时不修改输入。
`prepare(u,p)`	内部预处理；`u` 是当前点，`p` 是父点，根传 0；返回子树大小，并写父节点、DFS 区间和重儿子。
`build()`	无参数、无返回值；清空旧预处理状态，再以 `root` 建立 DFS 序等信息。只做结构预处理，不初始化频次。
`changeColor(x,delta)`	内部把节点 `x` 的颜色次数增减 1，`delta` 为 `+1/-1`；无返回值，修改频次和 `distinct`。压缩与频次数组须已就绪，删除的是当前集合中已有的节点贡献。
`addSubtree(u,add)`	内部枚举 `[tin[u],tout[u]]`，`add=true` 加入、`false` 移除；无返回值，修改频次状态，依赖预处理与频次初始化。
`dfs(u,p,keep)`	内部统计；记录 `u` 子树答案，`keep=true` 保留贡献，`false` 则移除；无返回值，修改答案与活跃集合。只应由 `solve()` 从空状态启动。
`solve()`	无参数；重新预处理、压缩颜色、清空频次后计算全部答案，返回 `vector<int>`；不修改输入，允许重复调用。

业务代码只需构造对象并调用 `solve()`。内部过程在计算 `u` 的答案时按以下顺序执行：

1. 递归处理所有轻儿子，传 `keep=false`；各自记下答案后清空，避免污染兄弟子树。
2. 递归处理重儿子，传 `keep=true`；保留它的整棵子树贡献。
3. 枚举每棵轻子树重新加入，再加入 `u` 自己；重子树已经在集合中，不能再加一次。
4. 此时活跃集合恰好是 `u` 的子树，令 `answer[u]=distinct`。
5. 若本层 `keep=false`，移除 `u` 整棵子树；否则保留给父层继续用。

在上述五点树中，颜色设为 `[3,3,5,5,3]`。根的轻儿子 3 先得到答案 1 后清空。处理重儿子 2 时，先算轻儿子 5 并清空，再保留重儿子 4 的颜色 5；重新加入节点 5 的颜色 3 和节点 2 的颜色 3 后，频次为 `{3:2,5:1}`，得到 `answer[2]=2`。回到根，再加入节点 3 和根，频次为 `{3:3,5:2}`，得到 `answer[1]=2`。

4. 怎么调用

先准备树和颜色，再构造求解器。以下模板统计不同颜色数，无需额外接线段树：

```
#include <bits/stdc++.h>
using namespace std;

struct DSUOnTree {
    int n, root;
    const vector<vector<int>>& tree;
    const vector<int>& color;
    vector<int> compressed, parent, subtreeSize, heavy;
    vector<int> tin, tout, order, frequency, answer;
    int timer = 0, distinct = 0;

    // 输入 tree 为 1-based 连通无向树，color[1..n] 为任意整数颜色；引用对象须长于本对象。
    // 初始化数组；不修改 tree 或 color。要求 n>=1 且 root 在 1..n。
    DSUOnTree(const vector<vector<int>>& tree, const vector<int>& color, int root)
        : n((int)tree.size() - 1), root(root), tree(tree), color(color),
          compressed(n + 1), parent(n + 1), subtreeSize(n + 1), heavy(n + 1, 0),
          tin(n + 1), tout(n + 1), order(n + 1), answer(n + 1) {}

```

```
    // 对 u 子树做一次 DFS 序展平，计算 parent、子树大小和最大子树的重儿子；返回 subtreeSize[u]。
    // p 是 u 的父节点，根调用时 p=0；写入内部预处理数组，不改变输入树。
    int prepare(int u, int p) {
        parent[u] = p;
        subtreeSize[u] = 1;
        tin[u] = ++timer;
        order[timer] = u;
        int largest = 0;
        for (int v : tree[u]) {
            if (v == p) continue;
            int childSize = prepare(v, u);
            subtreeSize[u] += childSize;
            if (childSize > largest) {
                largest = childSize;
                heavy[u] = v;
            }
        }
        tout[u] = timer;
        return subtreeSize[u];
    }

    // 无参数；重新以 root 为根建立 DFS 序区间、父节点、子树大小和重儿子信息；清空旧预处理数组，不修改输入树。
    // 仅完成结构预处理；颜色频次要由 solve() 初始化后才能用于 dfs() 统计。
    void build() {
        timer = 0;
        fill(parent.begin(), parent.end(), 0);
        fill(subtreeSize.begin(), subtreeSize.end(), 0);
        fill(heavy.begin(), heavy.end(), 0);
        fill(tin.begin(), tin.end(), 0);
        fill(tout.begin(), tout.end(), 0);
        prepare(root, 0);
    }

    // 将节点 x 的颜色频次增加 delta（+1 或 -1）；更新 distinct，保持频次非负。
    // x 是 1..n 点号；只修改内部 frequency/distinct。
    void changeColor(int x, int delta) {
        int id = compressed[x];
        if (frequency[id] == 0 && delta == 1) ++distinct;
        frequency[id] += delta;
        if (frequency[id] == 0 && delta == -1) --distinct;
    }

    // 将以 u 为根的整棵子树加入当前频次表或移除；add=true 表示加入。
    // u 必须已由 prepare 建立 DFS 区间；只修改内部频次状态。
    void addSubtree(int u, bool add) {
        int delta = add ? 1 : -1;
        for (int i = tin[u]; i <= tout[u]; ++i) changeColor(order[i], delta);
    }

    // 处理 u 子树；keep=true 时保留该子树颜色频次供父层合并，false 时清空。
    // p 为父节点；结束时 answer[u] 是子树 distinct 数，频次表依 keep 决定是否保留。
    void dfs(int u, int p, bool keep) {
        for (int v : tree[u]) {
            if (v != p && v != heavy[u]) dfs(v, u, false);
        }
        if (heavy[u] != 0) dfs(heavy[u], u, true);
        for (int v : tree[u]) {
            if (v != p && v != heavy[u]) addSubtree(v, true);
        }
        changeColor(u, 1);
        answer[u] = distinct;
```

```
        if (!keep) addSubtree(u, false);
    }

    // 重新预处理并计算所有子树答案；每次调用都会清空并重建结果，允许安全重复调用。
    // 返回 n+1 长度数组，answer[0] 不使用；不修改输入 tree/color。
    vector<int> solve() {
        build();
        fill(answer.begin(), answer.end(), 0);

        vector<int> values(color.begin() + 1, color.begin() + n + 1);
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        for (int u = 1; u <= n; ++u) {
            compressed[u] = lower_bound(values.begin(), values.end(),
color[u]) - values.begin();
        }
        frequency.assign(values.size(), 0);
        distinct = 0;
        dfs(root, 0, true);
        return answer;
    }
};

// 示例：tree 为 1-based 无向邻接表，color[1..n] 可使用任意整数；答案按点号读取。
// DSUOnTree solver(tree, color, 1); vector<int> distinct = solver.solve();
```

下面片段可放入业务函数。注意输入数组的生命周期要覆盖 `solver.solve()`：

```
vector<vector<int>> tree(6);
// u/v 为 1..5 的树点；向捕获的 tree 双向加边，返回 void。
auto addEdge = [&](int u, int v) {
    tree[u].push_back(v);
    tree[v].push_back(u);
};
addEdge(1, 2); addEdge(1, 3);
addEdge(2, 4); addEdge(2, 5);
vector<int> color = {0, 3, 3, 5, 5, 3};
DSUOnTree solver(tree, color, 1);
vector<int> answer = solver.solve();
// answer 为 {0, 2, 2, 1, 1, 1}，0 号位不使用
```

5. 要改哪里

需求	改动
换成颜色次数、众数等统计	改 `changeColor` 的状态维护与 `answer[u]` 的读取方式；不同颜色数的加减规则不能直接当作众数维护。
按深度或权值统计	将统计键改为深度或权值，调整离散化和桶/树的容量；DFS 序仍只负责枚举节点。
换成线段树维护频次	在节点增删时更新对应统计键的叶子，重算单次增删和读答案的代价；不把总时间一律写成 $O(n \log n)$。
换根或多组数据	为新根/新数据构造对象后运行 `solve()`；同一输入上重复 `solve()` 也会完整清空并重算。
维护子树内部边权	可将边放在较深端点上，但对 `u` 子树应排除连接 `u` 与父亲的那条边；不能把原点权统计原样套过去。
深链数据	递归深度可达 `n`；栈不足时需将预处理和统计遍历改为显式栈。

扩展到子树众数时，需同步改变加入、删除和答案读取。若只要求最高频率，可维护 `frequency[color]`、`countAtFrequency[f]` 和 `bestFrequency`：加入时更新频率桶并可能提高最大频率，删除时减少桶计数并在最高频率桶为空时向下移动。`keep=false` 仍须逐节点撤销。若还要并列时最小颜色，可对每个频率

维护有序集合，操作增加 $O(\log n)$ 因子。只求最高频率时频率桶可均摊 $O(1)$。众数扩展不适用于在线改色或路径众数。

模型识别：静态树、每个点都要回答一个子树统计，且统计可以由频率表增删维护时考虑 DSU on Tree；少数子树区间查询可考虑 DFS 序加离线树状数组，在线修改则考虑动态结构。

6. 自检

用上述五点树检查 `order[tin[u]]==u`、`tout[u]-tin[u]+1==subtreeSize[u]`，以及 `heavy[u]` 是否为最大子树的孩子。答案应为 `[2,2,1,1,1]`；若所有颜色相同则每个答案都为 1，若各节点颜色不同则 `answer[u]==subtreeSize[u]`。单点树答案为 1；负颜色、大颜色经压缩后仍应满足这些规律。连续调用两次 `solve()`，结果必须相同。

调试时在 `dfs` 的 `keep=false` 分支结束后，检查本次子树贡献已清除；按此框架调用时应回到空频次状态，`distinct==0`。保留分支返回时，频次则恰好描述该子树。不要在重儿子返回后把全局频次清零。

7. 坑

- 子树统计像用了别的节点：把 DFS 位置当成节点号；必须通过 `order[i]` 映回节点，再查 `compressed`。
- 兄弟子树的答案偏大：轻子树处理后未完整删除；检查 `keep=false` 分支及所有统计变量的清理规则。
- 重子树频次翻倍：保留后又加入了一次；父层只重新枚举轻子树。
- 颜色越界：直接用原始负颜色或大颜色作桶下标；使用离散化编号。
- 改统计量后清空出错：新的状态未必能仅靠减频次还原；增删和清空必须成套设计，不能只替换取答案的一行。

8. 一句话原理

保留最大子树的状态，只重扫轻子树；沿轻边子树大小至少减半，因此每个节点只被反复加入、删除 $O(\log n)$ 次。

5.15 单点修改线段树与自定义合并

1. 它是做什么的

这棵迭代线段树维护固定长度的非空 1-based 序列，支持单点替换和闭区间聚合。合并器 `merge` 必须满足结合律，并有左右两侧都成立的单位元 `identity`；合并器还必须能在 `rangeQuery` 的 `const` 函数中调用。构造时间为 $O(n)$，单点修改和区间查询各需 $O(\log n)$ 次合并，空间为 $O(n)$；这里按摘要的复制、存储和一次 `merge` 均为 $O(1)$ 计；若摘要或合并本身更昂贵，需乘上对应代价。它不支持非结合操作，也没有懒标记，不能直接执行区间更新。

示例用仿射函数 `f(x)=ax+b (mod MOD)` 作摘要。区间元素按下标从左到右依次作用，所以合并 `[left,right]` 时结果是 `right ∘ left`。模板保留查询区间的原始顺序，因而也能处理这种不满足交换律的合并。

2. 变量在做什么

变量	含义
`n`	序列元素个数；输入的有效元素位于 `initial[1..n]`。
`tree`	长度 `2*n` 的紧凑迭代树；叶子 `tree[n+i-1]` 对应原位置 `i`，内部节点合并左右孩子的摘要；非二次幂长度时，部分内部节点不对应原数组上的连续区间。
`identity`	`merge` 的双侧单位元，用于查询左右累积器的初值；示例中为恒等仿射函数。
`merge`	用户提供的结合合并操作；参数顺序有意义，必须可在常量查询中调用。

变量	含义
`leftResult`, `rightResult`	查询期间分别从区间左侧和右侧累积摘要，最后按 `leftResult` 在前、`rightResult` 在后的次序合并。
`Affine{a,b}`	仿射摘要，表示模 `MOD` 下的函数 `a*x+b`，系数存为 `[0,MOD)`。

输入及查询端点都是 1-based 闭区间；`rangeQuery(l,r)` 要求 `1 <= l <= r <= n`。非二次幂长度时，紧凑树的 `tree[1]` 不一定代表原序列整体，需用 `rangeQuery(1,n)` 查询整段。

3. 函数在做什么

函数	参数、返回与作用
`PointSegmentTree(initial, identity, merge)`	复制 `initial[1..n]`，建立树；`initial` 必须非空且为 1-based，构造器用 `assert` 检查 `n>=1`。单位元和合并器需匹配且满足结合律。
`pointSet(pos,value)`	把位置 `pos` 替换为 `value`，重算叶子到根的摘要；要求 `1<=pos<=n`，无返回值。
`rangeQuery(l,r)`	返回闭区间 `[l,r]` 按原下标次序合并的摘要；要求合法非空区间，不修改树。
`normalizeMod(x)`	将任意 `long long` 规范到 `[0,MOD)`。
`ComposeAffine::operator()(left,right)`	返回先执行 `left` 再执行 `right` 的 `right∘left`；两输入系数应已规范化。
`makeAffine(a,b)`	建立 `a*x+b`，并将系数规范到 `[0,MOD)`。
`affineExample()`	演示初始化、点替换、区间查询和代入；返回最终模值。

4. 怎么调用

准备 1-based 的初始摘要、符合代数条件的合并器和单位元，构造后调用 `pointSet` / `rangeQuery`。示例中点 2 被替换为 `7x+2`，区间组合为 `70x+51`；代入 `x=11` 得到 `821`：

```
ll result = affineExample();
// result == 821
```

模板代码如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

template<class T, class Merge>
struct PointSegmentTree {
    int n;
    T identity;
    Merge merge;
    vector<T> tree;

    // initial 为 1-based 非空序列，identity 是 merge 的双侧单位元，merge 满足结合律；复制到内部树并建树，不修改入参。
    explicit PointSegmentTree(const vector<T>& initial, T identity, Merge merge)
        : n((int)initial.size() - 1), identity(identity), merge(merge), tree(2 * n, identity) {
        assert(n >= 1);
        for (int i = 1; i <= n; ++i) tree[n + i - 1] = initial[i];
        for (int i = n - 1; i >= 1; --i) tree[i] = this->merge(tree[i << 1], tree[i << 1 | 1]);
    }

```

```
    // pos 为 1..n 的位置，value 是替换后的元素；返回 void，更新叶子及其祖先摘要，不改其他位置。
    void pointSet(int pos, const T& value) {
        int p = n + pos - 1;
        tree[p] = value;
        while (p > 1) {
            p >>= 1;
            tree[p] = merge(tree[p << 1], tree[p << 1 | 1]);
        }
    }

    // l、r 为 1-based 闭区间且 1<=l<=r<=n；返回按原序聚合的摘要，不修改树。
    T rangeQuery(int l, int r) const {
        int left = l + n - 1, right = r + n;
        T leftResult = identity, rightResult = identity;
        while (left < right) {
            if (left & 1) leftResult = merge(leftResult, tree[left++]);
            if (right & 1) rightResult = merge(tree[--right], rightResult);
            left >>= 1;
            right >>= 1;
        }
        return merge(leftResult, rightResult);
    }
};

constexpr ll MOD = 1000000007;

// x 为任意 long long；返回模 MOD 的 [0,MOD) 代表元。
ll normalizeMod(ll x) {
    x %= MOD;
    if (x < 0) x += MOD;
    return x;
}

struct Affine {
    ll a, b;
};

struct ComposeAffine {
    // left、right 的系数均已归一化；返回先执行 left 再执行 right 的复合 right∘left。
    Affine operator()(const Affine& left, const Affine& right) const {
        return {right.a * left.a % MOD,
                (right.a * left.b + right.b) % MOD};
    }
};

// 原始系数可为任意 long long；返回系数归一化后的 f(x)=ax+b。
Affine makeAffine(ll a, ll b) {
    return {normalizeMod(a), normalizeMod(b)};
}

// 示范仿射函数的顺序组合与点替换；仅修改本地线段树，返回区间函数作用于 x 的结果。
ll affineExample() {
    vector<Affine> functions(4);
    functions[1] = makeAffine(2, 1);
    functions[2] = makeAffine(3, 4);
    functions[3] = makeAffine(5, 6);
    PointSegmentTree<Affine, ComposeAffine> st(functions, makeAffine(1, 0), ComposeAffine{});
    st.pointSet(2, makeAffine(7, 2));
    Affine combined = st.rangeQuery(1, 3);
    ll x = normalizeMod(11);
    return (combined.a * x + combined.b) % MOD;
```

```
}
```

5. 要改哪里

- 维护区间和、最大值等摘要时，替换 `T`、`identity` 和 `merge`，并确保 `merge` 结合、`identity` 左右两侧均为单位元。最小值的单位元须不小于所有可能输入的值（如该类型的最大值），不能随意取 0。
- 维护非交换摘要时，严格保留“左段在前、右段在后”的合并顺序；不要交换 `merge(left,right)` 参数，也不要只用一个累积器改写查询。
- 仿射函数的作用顺序若改为从右向左，应同步重定义合并器和示例的区间语义，不能只交换说明。
- 增加区间修改需要引入懒标记及其与摘要的作用规则，当前模板没有这部分能力，应按操作重新设计。

6. 自检

示例的三项初值是 `2x+1`、`3x+4`、`5x+6`；将第二项替换为 `7x+2` 后，按顺序复合得到 `5(7(2x+1)+2)+6 = 70x+51`，代入 11 为 821。可分别检查单点区间、全区间和修改后查询；再用非二次幂长度（如 `n=3`）核对 `rangeQuery(1,n)`，不要把 `tree[1]` 当成全序列结果。

7. 坑

- 部分区间结果顺序反了：非交换合并时，右侧累积器必须用新块放在已有结果左侧；本模板通过 `merge(tree[--right], rightResult)` 保持顺序。
- 更新后结果异常：合并规则不满足结合律或单位元不匹配，线段树分组将改变结果。
- 整段查询拿错：`n` 非二次幂时 `tree[1]` 不是原序列整体摘要；使用 `rangeQuery(1,n)`。
- 下标越界或访问空叶子：接口按 1-based 编号且要求非空序列；构造输入的 `initial[0]` 不使用，查询端点须合法。

8. 一句话原理

每个节点保存其左段再接右段的摘要，单点替换后沿祖先重算，查询时按原序合并覆盖区间的节点。

5.16 启发式合并（Small-to-Large）

1. 它是做什么的

树上每个点有一个颜色，想知道每个点的子树里有几种颜色。比如父点有颜色 5，两个孩子分别是 5 和 8，那么父点子树的颜色集合是 `{5,8}`，答案为 2。

朴素做法是每个点都重新遍历整棵子树并收集颜色。链状树中，大量点会反复收集几乎相同的节点，工作量很大。

可以让孩子先收好自己的颜色，再把较小的一袋颜色加入较大的一袋。相同颜色只保留一份；合并完成后，大袋就代表父点整棵子树。

2. 变量在做什么

这里有两个容易混淆的量：集合里有多少种颜色，和这袋颜色来自多少个树节点。比如 `{5,5}` 去重后只有一种颜色，但它代表两个节点。模板用颜色集合回答题目，用节点数 `mass` 决定哪一边较小；不能拿不同颜色数代替 `mass`。

名称	含义
`Bag::values`	当前子树的颜色集合，使用 `set<int>` 去重；所有权由 `unique_ptr` 独占。
`Bag::mass`	当前子树包含的节点数，用于选定较小容器；不能用不同颜色数替代。
`n`, `root`	输入树点数与固定根；点号为 `1..n`。
`tree`, `color`	输入树及颜色的常量引用；必须比对象活得更久。

名称	含义
`answer[u]`	根为 `u` 的子树不同颜色数；0 号位不用。

3. 函数在做什么

沿用树 `1-2, 1-3`、颜色 `[5,5,8]`。

1. 叶子 2 形成 `{5}`、质量 1；叶子 3 形成 `{8}`、质量 1。
2. 点 1 自己先形成 `{5}`，收到点 2 的袋子后仍是 `{5}`，质量变成 2。
3. 点 3 的质量 1 袋子并入质量 2 袋子，得到 `{5,8}`、质量 3，答案为 2。

每袋的颜色集合正好是其子树颜色的并集，所以 `size()` 就是不同颜色数。扫描时只把较小质量袋子的元素插入较大袋子，结果集合仍是两边的并集；归纳到根时，每个点的答案都正确。

摊还理由也看节点质量而不是颜色数：每次扫描的小袋元素至多来自较小节点侧。对一个元素项，若重复颜色已在大袋中，该项本次就消失；若作为新颜色留下，它之后再次随袋子搬运时，所在子树质量至少翻倍。因此每个原始节点项最多被扫描 $O(\log n)$ 次；重复颜色不会破坏这个界。

函数	参数与返回	行为及前提
`SmallToLargeDistinct`	tree 为 1-based 连通无向树，color 为颜色数组，root 为合法根。 返回：无（构造函数）。	保存输入引用并分配答案；输入必须比对象活得更久。
`dfs(u,p)`	u 是当前子树根，p 是父点；根调用时 p=0。 返回：Bag，子树所有颜色的集合及其质量。	合并子树颜色集合并写入 answer[u]；集合所有权随返回值交给调用者。
`solve()`	无参数。 返回：vector<int>，下标 1..n 的答案。	重新计算所有子树不同颜色数；允许重复调用且不改输入。

复杂度：本实现按子树节点数（质量）选择容器。使用 `set` 时单次插入为 $O(\log n)$，总时间 $O(n \log^2 n)$、空间 $O(n)$。

若换成期望 $O(1)$ 插入的哈希集合，总时间期望 $O(n \log n)$。

4. 怎么调用

模板实现如下。

```cpp
#include <bits/stdc++.h>
using namespace std;

struct SmallToLargeDistinct {
    struct Bag {
        unique_ptr<set<int>> values;
        int mass = 0; // 对应子树节点数，用于严谨的小并大选择
    };
    int n, root;
    const vector<vector<int>>& tree;
    const vector<int>& color;
    vector<int> answer;

    // 参数：tree 为 1-based 连通无向树，color 为颜色数组，root 为合法根。
    // 返回：无（构造函数）。
    // 效果：保存输入引用并分配答案；输入必须比对象活得更久。
    SmallToLargeDistinct(const vector<vector<int>>& tree,
                         const vector<int>& color, int root)
        : n((int)tree.size() - 1), root(root), tree(tree), color(color),
          answer(n + 1) {
        assert(n >= 1 && 1 <= root && root <= n);
    }
    // 参数：u 是当前子树根，p 是父点；根调用时 p=0。
    // 返回：Bag，子树所有颜色的集合及其质量。
    // 效果：合并子树颜色集合并写入 answer[u]；集合所有权随返回值交给调用者。
```

```
    Bag dfs(int u, int p) {
        Bag current{make_unique<set<int>>(), 1};
        current.values->insert(color[u]);
        for (int v : tree[u]) if (v != p) {
            Bag other = dfs(v, u);
            // 质量而非不同颜色数决定小侧；这样重复颜色也有明确的摊还收费。
            if (current.mass < other.mass) swap(current, other);
            current.values->insert(other.values->begin(), other.values->end());
            current.mass += other.mass;
        }
        answer[u] = (int)current.values->size();
        return current;
    }
    // 参数：无参数。
    // 返回：vector<int>，下标 1..n 的答案。
    // 效果：重新计算所有子树不同颜色数；允许重复调用且不改输入。
    vector<int> solve() {
        fill(answer.begin(), answer.end(), 0);
        Bag result = dfs(root, 0);
        (void)result; // 根集合所有权在本次调用结束时释放，只保留 answer。
        return answer;
    }
};
```

最小完整调用示例，树为 `1-2,1-3`，颜色为 `[5,5,8]`：

```
vector<vector<int>> tree(4);
tree[1] = {2, 3}; tree[2] = {1}; tree[3] = {1};
vector<int> color = {0, 5, 5, 8};
SmallToLargeDistinct solver(tree, color, 1);
auto answer = solver.solve();
assert(answer[1] == 2 && answer[2] == 1 && answer[3] == 1);
```

5. 要改哪里

先确认摘要支持逐元素插入，并明确重复元素如何处理；只支持整体合并的 DP 摘要不能直接套用。可以先统计不同颜色，再用 `unordered_map<颜色,频次>` 求众数，最后替换成自定义摘要并重新证明搬运次数。

DSU on Tree 会清理临时表来回答子树询问；这里把集合所有权交给父节点，孩子集合被消费，只在 `answer` 中留下孩子答案。线段树合并则把单个元素换成值域树节点，适合值域统计或第 k 小。

6. 自检

再检查非叶合并：两个子树质量分别为 2 和 7，即使两边的颜色集合大小都是 2，也必须让质量 2 的集合并入质量 7 的集合，合并后 `mass=9`；答案仍取合并后集合大小。重复调用 `solve()` 会重新建临时袋子，因此上述例子再次得到 `[2,1,1]`。

7. 坑

不要按不同颜色数而非 `mass` 选小侧；重复颜色会破坏翻倍证明。交换 `Bag` 后继续使用新的 `current` 所有权，不能再访问已转移的容器。递归必须以父节点过滤邻接边；输入有环时会无限递归。`set` 插入为 $O(\log n)$，换哈希容器后复杂度是期望界且需考虑退化风险。

8. 一句话原理

始终把较小质量侧逐项加入较大侧，使仍存活的元素每次搬运后面对的子树质量至少翻倍。

5.17 线段树合并（Segment Tree Merge）

1. 它是做什么的

有两个多重集合 {3} 和 {1,3}，合并后是 {1,3,3}：值 1 出现一次，值 3 出现两次。我们还想查某个值出现几次，或找第 k 小。值域线段树把每个值放在固定值域的一片叶子里，叶子记录频次。

若每次都逐个取出集合元素再插入，集合很大时会重复做许多工作。尤其树上每个子树都维护一份权值集合，父子集合常有大量相同值。

这里让两棵树的同值区间递归合并：对应节点的频次相加，空分支直接借用另一边已有的节点。

2. 变量在做什么

先分清树节点编号和权值：root 是节点池中的下标，不是原树的点号；叶子所在的区间才表示某个权值。编号 0 代表整片区间都没有节点。再分清“合并结果”和“两个原集合”：本模板会消费第二棵树，合并后只剩一棵有效树，不能把旧根继续当独立集合用。

名称	含义
lo, hi	所有集合共享的闭值域；中点计算要求 hi-lo 不溢出 int。
tr	动态节点池；下标是节点编号，tr[0] 是全零空节点。
Node::left/right	当前值域区间左右半边的根编号；0 表示该侧为空。
Node::sum	当前区间的总频次；必须非负且不超过 INT_MAX。

3. 函数在做什么

用 {3} 与 {1,3} 手算。

1. 值域 [1,8] 的两棵根总频次分别为 1 和 2。
2. 左半边 [1,4] 中，值 1 只在第二棵树出现，值 3 在两棵树都出现。

   递归把值 1 的非空节点接入结果，并把值 3 的叶子计数相加。
3. 结果频次为 {1:1,3:2}，所以第 1 小是 1、第 2 小是 3，count(3)=2。

每个叶子唯一对应一个值；对应区间的频次相加，等于把两个集合中每个值的出现次数相加。内部节点的左右区间互不重叠，因此递归完成后每个值都恰好被合并一次。遇到空根时直接返回另一根，等价于加上全零频次。

函数	参数与返回	行为及前提
SegmentTreeMerge	lo<=hi 是权值闭区间，reserveNodes 为预留节点数。 返回：无（构造函数）。	节点 0 为空；预留容量仅影响常数，不改变结果。
add(root,l,r,pos,delta)	root 是旧频次树根；l、r 是当前值域区间；pos 是修改值；delta 是计数增量。 返回：int，更新后的根。	要求 pos 属于值域且更新后频次非负；修改节点池并更新根到叶路径。
add(root,pos,delta)	root 是旧频次树根（可为 0）；pos 属于 lo..hi；delta 是计数增量。 返回：int，更新后的根。	只修改内部节点池。
merge(a,b,l,r)	a、b 是待合并根（可有一个为 0）；l、r 是值域区间；非空根须互异且节点不共享。 返回：int，结果根。	破坏性合并频次；结果存回 a 对应的节点，b 不再可独立使用。
merge(a,b)	a、b 来自本池同一值域且节点完全不共享。 返回：int，唯一有效的合并根，旧集合不可再独立使用。	破坏性合并两个不共享节点的频次树。
count(root,l,r,pos)	root 是当前频次树根；l、r 是值域区间；pos 是待查询值。 返回：int，该值的出现次数，root 可为 0。	只读，不修改结构。
count(root,pos)	root 是频次树根；pos 属于 lo..hi。 返回：int，该值的出现次数，root 可为 0。	只读，不修改结构。

函数	参数与返回	行为及前提
`kth(root,l,r,k)`	root 是频次树根，l、r 为值域闭区间，k 为 1-based 排名。返回：int，全局第 k 小值，k 非法时断言失败。	只读，不修改结构。
`kth(root,k)`	root 是频次树根；k 是从 1 开始且不超过总计数的排名。返回：int，根的第 k 小值。	只读，不修改结构。

复杂度：`add` 为 $O(\log \sigma)$。一次破坏性合并访问第二棵树的非空节点及重叠分支。若总共创建 P 个节点、外部调用合并 M 次，这批合并总成本为 $O(P+M)$。

每次两个非空节点相遇都会让第二个节点退出独立可用的森林。空节点返回次数至多是递归分叉数的常数倍再加 M。

常见做法是每点先建一条值域路径，再沿原树合并 $n-1$ 次，创建与合并共 $O(n\log(\sigma+1))$；空间为已创建节点数。`sum`、计数、节点池下标都须在 `int` 范围内。

4. 怎么调用

模板实现如下。

```cpp
#include <bits/stdc++.h>
using namespace std;

struct SegmentTreeMerge {
    struct Node { int left = 0, right = 0, sum = 0; };
    int lo, hi;
    vector<Node> tr;
    // 参数：lo<=hi 是权值闭区间，reserveNodes 为预留节点数。
    // 返回：无（构造函数）。
    // 效果：节点 0 为空；预留容量仅影响常数，不改变结果。
    SegmentTreeMerge(int lo, int hi, int reserveNodes = 0)
        : lo(lo), hi(hi) {
        assert(lo <= hi);
        tr.reserve(max(1, reserveNodes));
        tr.push_back(Node{});
    }
    // 参数：root 是旧频次树根；l、r 是当前值域区间；pos 是修改值；delta 是计数增量。
    // 返回：int，更新后的根。
    // 效果：要求 pos 属于值域且更新后频次非负；修改节点池并更新根到叶路径。
    int add(int root, int l, int r, int pos, int delta) {
        if (!root) { root = (int)tr.size(); tr.push_back(Node{}); }
        tr[root].sum += delta;
        assert(tr[root].sum >= 0);
        if (l == r) return root;
        int m = l + (r - l) / 2;
        if (pos <= m) tr[root].left = add(tr[root].left, l, m, pos, delta);
        else tr[root].right = add(tr[root].right, m + 1, r, pos, delta);
        return root;
    }
    // 参数：root 是旧频次树根（可为 0）；pos 属于 lo..hi；delta 是计数增量。
    // 返回：int，更新后的根。
    // 效果：只修改内部节点池。
    int add(int root, int pos, int delta = 1) {
        assert(lo <= pos && pos <= hi);
        return add(root, lo, hi, pos, delta);
    }
    // 参数：a、b 是待合并根（可有一个为 0）；l、r 是值域区间；非空根须互异且节点不共享。
    // 返回：int，结果根。
    // 效果：破坏性合并频次；结果存回 a 对应的节点，b 不再可独立使用。
    int merge(int a, int b, int l, int r) {
        if (!a) return b;
        if (!b) return a;
```

```
        tr[a].sum += tr[b].sum;
        if (l != r) {
            int m = l + (r - l) / 2;
            tr[a].left = merge(tr[a].left, tr[b].left, l, m);
            tr[a].right = merge(tr[a].right, tr[b].right, m + 1, r);
        }
        return a;
    }
    // 参数：a、b 来自本池同一值域且节点完全不共享。
    // 返回：int，唯一有效的合并根，旧集合不可再独立使用。
    // 效果：破坏性合并两个不共享节点的频次树。
    int merge(int a, int b) {
        assert(a == 0 || b == 0 || a != b); // merge(a,a) 会重复累计，调用契约禁止它
        return merge(a, b, lo, hi);
    }
    // 参数：root 是当前频次树根；l、r 是值域区间；pos 是待查询值。
    // 返回：int，该值的出现次数，root 可为 0。
    // 效果：只读，不修改结构。
    int count(int root, int l, int r, int pos) const {
        if (!root) return 0;
        if (l == r) return tr[root].sum;
        int m = l + (r - l) / 2;
        return pos <= m ? count(tr[root].left, l, m, pos)
                        : count(tr[root].right, m + 1, r, pos);
    }
    // 参数：root 是频次树根；pos 属于 lo..hi。
    // 返回：int，该值的出现次数，root 可为 0。
    // 效果：只读，不修改结构。
    int count(int root, int pos) const {
        assert(lo <= pos && pos <= hi);
        return count(root, lo, hi, pos);
    }
    // 参数：root 是频次树根，l、r 为值域闭区间，k 为 1-based 排名。
    // 返回：int，全局第 k 小值，k 非法时断言失败。
    // 效果：只读，不修改结构。
    int kth(int root, int l, int r, int k) const {
        assert(root && 1 <= k && k <= tr[root].sum);
        if (l == r) return l;
        int m = l + (r - l) / 2;
        int leftSum = tr[tr[root].left].sum;
        if (k <= leftSum) return kth(tr[root].left, l, m, k);
        return kth(tr[root].right, m + 1, r, k - leftSum);
    }
    // 参数：root 是频次树根；k 是从 1 开始且不超过总计数的排名。
    // 返回：int，根的第 k 小值。
    // 效果：只读，不修改结构。
    int kth(int root, int k) const { return kth(root, lo, hi, k); }
};
```

最小调用示例，可直接放在 `main` 函数体内：合并 `{3}` 与 `{1,3}`，结果第 2 小为 3。

```
SegmentTreeMerge st(1, 8);
int a = st.add(0, 3);
int b = st.add(0, 1);
b = st.add(b, 3);
int merged = st.merge(a, b);
assert(st.count(merged, 3) == 2);
assert(st.kth(merged, 1) == 1 && st.kth(merged, 2) == 3);
```

5. 要改哪里

树上后序合并时，先记录孩子答案，再把孩子根交给父亲并将孩子根置 0；本模板不为每棵子树保留版本。先离散化到紧凑值域，再练习频次合并、子树第 k 小，之后再结合差分根或离线删除分析节点重叠。

若需要分裂操作，要另行设计节点所有权。本结构会破坏旧根并复用节点；主席树则复制路径并保留历史版本。

6. 自检

值域 `[1,8]`：A=`{3:1}`、B=`{1:1,3:1}`。先看根总数 `1+2=3`；再看左侧值 1 被保留、值 3 的两份计数相加；最后检查 `kth(1)=1`、`kth(2)=3`、`count(3)=2`。若合并空根与非空根，结果应等于非空根，因为空根没有任何非零计数。

7. 坑

两个根必须来自同一节点池和同一值域，且节点集合不得共享；非空 `merge(a,a)` 会重复累计。合并后第二棵树不可继续独立读取或修改。`sum`、总频次、节点池下标和 `hi-lo` 都受 `int` 范围限制；不要把总频次误当不同值个数。

8. 一句话原理

相同值域的对应节点频次相加，空分支直接复用另一侧节点；节点所有权随破坏性合并转移。

5.18 主席树（可持久化线段树）

1. 它是做什么的

给定一个静态数组，多次询问某个区间内的第 `k` 小值。例如：

```
数组：[40, 10, 30, 20, 10]
询问 [2,5] 的第 3 小：
取出 [10,30,20,10]，排序为 [10,10,20,30]，答案是 20。
```

每次取出区间再排序，可以得到答案，但大量询问会重复排序。主席树先为每个前缀保存一份“各种值出现了多少次”的记录。查询 `[l,r]` 时，用前 `r` 项的记录减去前 `l-1` 项的记录，得到该区间的频次，再寻找第 `k` 小。

这里的“可持久化”，就是**更新后保留旧版本，旧版本仍然可以查询**。如果每个前缀都复制一整份记录，空间会很大；主席树只复制本次更新影响的部分，其他部分由不同版本共享。

本节先讲静态数组区间第 `k` 小。树上路径第 `k` 小使用同样的频次思想，放在基本过程之后理解。

2. 变量在做什么

首先区分两种区间：**数组区间** `[l,r]` 指要查询原数组的哪些位置；**值域区间**指线段树节点负责统计哪些值。本模板的线段树按值划分，节点保存这些值的出现次数。

先把原值排序去重并编号：

```
原值：   10  20  30  40
编号：    1   2   3   4
```

值域节点 `[1,2]` 统计的是值 10 和 20 一共出现多少次，不是原数组前两个位置的信息。

名称	含义
`lo`、`hi`	离散值编号的闭区间，本例为 `[1,4]`。
`root[i]`	前 `i` 个数组元素对应的版本根；由调用方保存，不是类的成员。
`tr`	所有版本共用的节点池，根和孩子都用池内编号表示。
`Node::sum`	当前节点负责的值域内，元素一共出现了多少次。
`Node::left/right`	左右孩子在节点池中的编号。
`previous`、`now`	更新使用的旧节点编号，以及复制后新节点的编号。

名称	含义
`pos`、`delta`	本次修改的值编号，以及频次增量；前缀建树每次加 1。
`k`	要找第几小，从 1 开始，重复值分别计数。

`root[i]` 只保存一个节点编号。不同版本可以共享没有变化的节点，不需要各自复制一整棵树。节点编号 0 表示空树，频次为 0；因此 `root[0]=0` 表示还没有加入任何元素。

3. 函数在做什么

先看“前缀版本”记录了什么。本例的各版本频次如下：

版本	已加入的数组元素	10 的次数	20 的次数	30 的次数	40 的次数
`root[0]`	无	0	0	0	0
`root[1]`	`40`	0	0	0	1
`root[2]`	40, 10	1	0	0	1
`root[3]`	40, 10, 30	1	0	1	1
`root[4]`	40, 10, 30, 20	1	1	1	1
`root[5]`	40, 10, 30, 20, 10	2	1	1	1

怎样建立新版本？

从 `root[2]` 建立 `root[3]` 时，新加入的是 30，即编号 3。只有包含编号 3 的节点计数需要加一：

```
值域 [1,4] → 值域 [3,4] → 值域 [3,3]
```

模板复制这条路径上的节点，并修改新节点。其他分支继续指向旧节点。例如 `[1,2]` 内没有新增元素，它的整棵子树都可以共享。

因此 `root[3]` 能查到新加入的 30，`root[2]` 仍表示原来的两个元素，两版本共享的节点都没有被修改。这就是“路径复制”。

怎样查询区间第 k 小？

查询 `[2,5]` 时，使用 `root[5]` 和 `root[1]`。对同一个值，它在区间内的次数等于“前 5 项中的次数减去前 1 项中的次数”，于是得到 `10:2，20:1，30:1，40:0`。

寻找第 3 小，从值域根开始：

1. 左半值域 `[1,2]`，即 10、20，共有 3 个元素。第 3 小一定在这里，进入两版本的左孩子，`k` 保持为 3。
2. 再看左半值域 `[1,1]`，即 10，有 2 个元素。第 3 小不在这里，跳过这两个元素，进入两版本的右孩子，并令 `k=3-2=1`。
3. 到达编号 2 的叶子，对应原值 20，查询结束。

每层只需判断：**左边有多少个元素，足不足以包含我要找的第 k 个？**两个根同步下降，计数用相应左孩子的 `sum` 作差，并不真的建立一棵区间差树。

函数	参数与返回	行为及前提
`PersistentSegmentTree`	输入值编号范围和可选预留节点数；构造函数无返回值。	建立节点池和空版本 0；预留容量只影响扩容次数。
`add(previous,pos,delta)`	输入旧根、值编号及频次增量；返回新版本根。	复制受影响路径，保留旧版本；前缀建树时使用默认增量 1。
内部 `add`	额外输入当前值域边界 `l,r`；返回新节点编号。	复制当前节点并递归复制受影响孩子，未修改分支继续共享。
`kthRange`	输入 `root[r]`、`root[l-1]` 和排名；返回离散值编号。	通过两个前缀的计数差寻找第 k 小，要求 k 合法。

函数	参数与返回	行为及前提
`kthPath`	输入分别对应 u、v、LCA、parent(LCA) 的四个版本根及排名；返回离散值编号。	用于后述树路径扩展；要求四根的组合构成合法频次。
内部 `kth4`	输入四根、当前值域和排名；返回叶子的离散编号。	计算左侧组合频次，决定下降方向；查询不创建或修改节点。

设不同值有 `σ` 种，树高为 $O(\log(\sigma+1))$。每次加入只复制一条路径，建立全部前缀版本需要 $O(n\log(\sigma+1))$ 时间和空间；一次区间或路径第 k 小查询也只走一条路径，为 $O(\log(\sigma+1))$。离散化另需 $O(n\log n)$ 时间。

4. 怎么调用

依次完成离散化、建立前缀版本、查询和还原原值。先放入下面的完整模板。所有版本根必须来自同一个对象；值编号在 `[lo,hi]` 内，构造范围非空。

```cpp
#include <bits/stdc++.h>
using namespace std;

struct PersistentSegmentTree {
    struct Node { int left = 0, right = 0, sum = 0; };
    int lo, hi;
    vector<Node> tr;

    // 参数：lo<=hi 为离散值域闭区间，reserveNodes 为预留节点数。
    // 返回：无（构造函数）。
    // 效果：建立空版本 0；预留容量只影响扩容次数。
    PersistentSegmentTree(int lo, int hi, int reserveNodes = 0) : lo(lo), hi(hi) {
        assert(lo <= hi);
        tr.reserve(max(1, reserveNodes));
        tr.push_back(Node{});
    }
    // 参数：previous 是旧版本根；l、r 是值域区间；pos 是修改值；delta 是计数增量。
    // 返回：int，新版本根。
    // 效果：要求 pos 属于值域且更新后频次非负；复制并修改根到叶路径，旧版本保持不变。
    int add(int previous, int l, int r, int pos, int delta) {
        int now = (int)tr.size();
        tr.push_back(tr[previous]);
        tr[now].sum += delta;
        assert(tr[now].sum >= 0); // 合法 delta 不得让任一值域区间频次为负
        if (l == r) return now;
        int m = l + (r - l) / 2;
        if (pos <= m) tr[now].left = add(tr[previous].left, l, m, pos, delta);
        else tr[now].right = add(tr[previous].right, m + 1, r, pos, delta);
        return now;
    }
    // 参数：previous 是已有版本根；pos 属于值域；delta 是计数增量。
    // 返回：int，只比旧版本多 delta 个 pos 的新根。
    // 效果：创建新版本节点，previous 对应的旧版本保持不变。
    int add(int previous, int pos, int delta = 1) {
        assert(0 <= previous && previous < (int)tr.size());
        assert(lo <= pos && pos <= hi);
        return add(previous, lo, hi, pos, delta);
    }
    // 参数：ru、rv、rlca、rparent 是路径差分版本根；l、r 是值域区间；k 是从 1 开始的排名。
    // 返回：int，路径第 k 小的离散值。
    // 效果：四个版本根按 +rootU +rootV -rootLca -rootParentLca 组合。
    int kth4(int ru, int rv, int rlca, int rparent, int l, int r, int k) const {
```

```
        int total = tr[ru].sum + tr[rv].sum - tr[rlca].sum -
tr[rparent].sum;
        assert(1 <= k && k <= total);
        if (l == r) return l;
        int m = l + (r - l) / 2;
        int leftCount = tr[tr[ru].left].sum + tr[tr[rv].left].sum
                      - tr[tr[rlca].left].sum - tr[tr[rparent].left].sum;
        if (k <= leftCount)
            return kth4(tr[ru].left, tr[rv].left, tr[rlca].left,
                        tr[rparent].left, l, m, k);
        return kth4(tr[ru].right, tr[rv].right, tr[rlca].right,
                    tr[rparent].right, m + 1, r, k - leftCount);
    }
    // 参数：四根均为本对象版本根，k 从 1 开始。
    // 返回：int，路径第 k 小值，父根为空时传 0。
    // 效果：只读，不修改结构。
    int kthPath(int ru, int rv, int rlca, int rparent, int k) const {
        return kth4(ru, rv, rlca, rparent, lo, hi, k);
    }
    // 参数：rootR、rootBeforeL 分别是前缀 [1..r]、[1..l-1] 的版本根；k 是排名。
    // 返回：int，区间第 k 小值。
    // 效果：只读，不修改结构。
    int kthRange(int rootR, int rootBeforeL, int k) const {
        return kth4(rootR, 0, rootBeforeL, 0, lo, hi, k);
    }
};
```

下面用本节数组验证完整调用。外层大括号让这个例子拥有独立的局部变量：

```
{
    vector<int> a = {0, 40, 10, 30, 20, 10};
    vector<int> values(a.begin() + 1, a.end());
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());

    PersistentSegmentTree pst(1, (int)values.size());
    vector<int> root(a.size(), 0);
    for (int i = 1; i < (int)a.size(); ++i) {
        int pos = int(lower_bound(values.begin(), values.end(), a[i])
                      - values.begin()) + 1;
        root[i] = pst.add(root[i - 1], pos);
    }
    int rank = pst.kthRange(root[5], root[1], 3);
    int answer = values[rank - 1];
    assert(answer == 20);
}
```

查询返回的是编号 2，需要通过 `values[1]` 还原成 20。`add` 返回的新根也必须保存，它不会替调用方修改 `root[i]`。排名必须满足 `1<=k<=r-l+1`。

若输入已经是 `[1,3]` 内的值，可直接建立版本。下面保留这组更短的调用：

```
PersistentSegmentTree pst(1, 3);
int root0 = 0;
int root1 = pst.add(root0, 2);
int root2 = pst.add(root1, 1);
int root3 = pst.add(root2, 3);
assert(pst.kthRange(root3, root0, 2) == 2);
assert(pst.kthRange(root3, root1, 1) == 1); // 区间 [2,3]
```

5. 要改哪里

先掌握数组前缀版本，再考虑改变“每个版本代表什么”。树上路径第 k 小中，可以令 `root[u]` 表示总根到节点 u 的路径频次，包含 u。设 `w=LCA(u,v)`，路径频次由四个版本组合：

$$root[u] + root[v] - root[w] - root[parent[w]].$$

这个式子表示**对应节点的计数相加减**，不是把四个根编号进行整数运算。根到 u、根到 v 的两条路径重复计算了公共部分；LCA 以上点的系数为 `2-1-1=0`，LCA 自身的系数为 `2-1=1`，更深的路径点保留一次。令总根父亲的版本为空根 0，就能统一边界。

例如点权依次为 `2:3，1:1，3:2` 的路径 `2-1-3`，组合后得到 `{1:1,2:1,3:1}`，第 2 小为 2。查询仍根据左侧元素数量选择方向，只是计数由“两根相减”改为“四根组合”。每个值编号的组合频次必须非负，不能随意拿四个历史根拼在一起。

空间要按全部版本估算。若每次复制约 `ceil(log2(σ))+1` 个节点，节点含三个 int，在常见 4 字节 int 环境中，`n=10^5`、约 18 层时节点数据约 22 MB，还需根数组和容器多分配的容量。

若要在线修改原数组，后面的多个前缀都会受影响，不能直接沿用这套静态前缀建树。大量元素同时改变的区间修改，也不属于本模板直接支持的操作。

6. 自检

本例 `[2,5]` 排序为 `[10,10,20,30]`，第 1、2、3、4 小分别是 10、10、20、30；`[1,1]` 的第 1 小为 40，`[5,5]` 的第 1 小为 10。

建完全部版本后，`root[1]` 仍只包含一个 40；每个前缀根的 `sum` 应等于前缀长度。再检查全相等数组、只有一个不同值、单元素区间和路径两端相同的情况。

对拍时随机选合法 `[l,r]` 和 k，复制区间排序，与第 k 项比较；树路径版则用暴力枚举路径上的点权排序。查询旧版本的结果不能因后续 `add` 改变。

7. 坑

- 混淆数组位置和值编号：`root[i]` 的 i 是前缀长度，`add` 的 pos 是值编号。
- 修改了共享旧节点：新版本必须先复制路径节点，否则旧版本也会变化。
- 左端少减一：查询 `[l,r]` 必须使用 `root[r]` 和 `root[l-1]`。
- 把根编号相减：实际相减的是对应节点的 `sum`。
- 把重复值当作一个元素：两个 10 要占两个名次。
- 忘记还原原值：查询返回离散编号，不是原值。
- 树路径公式漏掉 LCA 父根：不能正确抵消全部公共部分。
- 传入非法 k 或负频次：`delta` 更新后每个值频次都须非负，排名须在组合总频次内；断言不能代替调用方保证合法。
- 节点池和计数溢出：当前模板的 `sum`、中间组合频次、节点池下标和 `hi-lo` 均须能放进 int；范围过大时应先离散化。

8. 一句话原理

通过路径复制保存每个前缀的频次树，再用两个前缀的计数差，在值域上逐层定位区间第 k 小。

5.19 虚树

1. 它是做什么的

原树很大，但一次询问只给少量关键点。以树 `1-2, 1-3, 2-6, 6-7, 7-4, 7-5` 为例，关键点 `{4,5,3}` 的虚树保留点 `1,7,4,5,3`；原树的 `1→7` 被压成一条距离为 3 的边。

朴素地在原树上为每次询问跑完整 DFS，会反复访问与关键点无关的点。若只留下关键点，它们之间又可能隔着决定分叉位置的祖先。

虚树补上这些关键点间必要的公共祖先，再把中间未保留的路径压成一条边；这样小问题仍保留原树上真正相关的分叉和距离。

2. 变量在做什么

原树的点号始终不变，虚树只是从中挑出一部分点；`vertices` 是按 DFS 先后排列的原树点号，不是新编号。虚树边也不一定是原树直接相邻边，它连接两个保留点，并记录两点间原树路径的长度。

名称	含义
`n`, `logn`, `timer`	原树点数、倍增层数、DFS 进入计时器。
`g[u]`	原树无向邻接表；必须在 `build` 前完成。
`tin[u]`, `tout[u]`	DFS 进入时间及子树区间末端；祖先判定使用同一轮 DFS 的两个时间。
`depth[u]`	相对预处理根的深度；虚树边权用深度差表示原树距离。
`up[j][u]`	节点 `u` 向上 `2^j` 步的祖先；0 表示越过根。
`VirtualTreeResult::vertices`	本次虚树保留的原树点号，按 DFS 序排列。
`VirtualTreeResult::edges`	`(父点,子点,原树距离)` 三元组；不是原树直接边。
`VirtualTreeResult::root`	虚树根；空询问时为 0。

3. 函数在做什么

用下面这棵树，带星号的是关键点：

```
1
├─2
│ └─6
│   └─7
│     ├─4*
│     └─5*
└─3*
```

第一步，DFS 顺序为 `1,2,6,7,4,5,3`，关键点顺序是 `4,5,3`，相邻 LCA 分别为 7 和 1。

第二步，加入 LCA 并排序去重，保留点按 DFS 序为 `1,7,4,5,3`。

第三步，按上述顺序使用祖先栈。每次先弹出不是当前点祖先的栈顶，再连到剩余栈顶：

当前点	弹出哪些点	连出的虚边	入栈后的状态
1	无	根，无父边	`[1]`
7	无	`1→7`，距离 3	`[1,7]`
4	无	`7→4`，距离 1	`[1,7,4]`
5	4	`7→5`，距离 1	`[1,7,5]`
3	5、7	`1→3`，距离 1	`[1,3]`

点 4 在处理 5 时已经出栈，不会等到处理 3 才出栈。原树点 2、6 不需要保留，它们压缩在距离为 3 的 `1→7` 中。

```
1
├─7 (距离 3)
│ ├─4 (距离 1)
│ └─5 (距离 1)
└─3 (距离 1)
```

正确性来自两点：DFS 序相邻关键点的 LCA 保留了关键点之间发生分叉所需的祖先；栈则把每个点连到当前路径上最近的保留祖先。两端点间没保留的原树点只能是被压缩的路径中间点，故边权 `depth[child]-depth[parent]` 就是原树距离。

去重后有 k 个关键点时，相邻对只有 $k-1$ 个，因此至多补入 $k-1$ 个 LCA；最多保留 $2k-1$ 个点、产生 $2k-2$ 条边。这里的 k 是输入关键点去重后的数量。

函数	参数与返回	行为及前提
`VirtualTree(nodeCount)`	nodeCount 为原树点数。 返回：无（构造函数）。	分配 1-based 图和 LCA 表，要求随后加入 n-1 条树边。
`addEdge(u,v)`	u、v 为 1..n。 返回：void。	向原树加入无向边；必须在 build 前调用。
`dfs(u,p)`	p 是父点。 返回：void。	u 的 DFS 进入/离开时间；写 tin/tout/depth/up[0]。
`build(root)`	root 为原树根。 返回：void。	清空并重建 tin/tout/depth/up；要求原图为连通树。
`isAncestor(u,v)`	u、v 是已 build 的树中合法点号。 返回：bool，u 是否为 v 的祖先（包括 u==v）。	只读，不修改结构。
`lca(u,v)`	u、v 是已 build 的树中合法点号。 返回：int，u、v 的最近公共祖先。	要求已 build 且两点在树中。
`buildVirtual(special)`	special 是本次询问的原树关键点，可重复。 返回：VirtualTreeResult，含关键点及必要 LCA 的虚树，要求已 build。	只读，不修改结构。

复杂度：预处理倍增 LCA 为 $O(n \log n)$。若输入 k 个关键点（可重复），单次询问为 $O(k \log k + k \log n)$：排序关键点、求相邻 LCA、加入至多 $k-1$ 个 LCA、再次排序并用栈建树。

即使调用者已按 DFS 序排序，本模板仍会求 LCA 并对补点后的 `all` 再排序；只有另写复用排序的接口才能省第一次排序。空间 $O(n \log n + k)$。空询问返回空点边、根为 0；单个关键点只产生一个保留点和零条边。

4. 怎么调用

模板实现如下。

```cpp
#include <bits/stdc++.h>
using namespace std;

struct VirtualTreeResult {
    vector<int> vertices;                   // 按 DFS 序排列的原树点号
    vector<tuple<int, int, int>> edges;     // (父点, 子点, 原树距离)
    int root = 0;                           // vertices 非空时为虚树根
};

struct VirtualTree {
    int n, logn, timer = 0;
    vector<vector<int>> g;
    vector<int> tin, tout, depth;
    vector<vector<int>> up;
    // 参数: nodeCount 为原树点数。
    // 返回: 无 (构造函数)。
    // 效果: 分配 1-based 图和 LCA 表，要求随后加入 n-1 条树边。
    explicit VirtualTree(int nodeCount)
        : n(nodeCount), logn(1), g(nodeCount + 1), tin(nodeCount + 1),
          tout(nodeCount + 1), depth(nodeCount + 1) {
        assert(n >= 1);
        while ((1 << logn) <= n) ++logn;
        up.assign(logn, vector<int>(n + 1, 0));
    }
    // 参数: u、v 为 1..n。
    // 返回: void。
    // 效果: 向原树加入无向边；必须在 build 前调用。
    void addEdge(int u, int v) {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
```

```
32      }
33      // 参数: p 是父点。
34      // 返回: void。
35      // 效果: u 的 DFS 进入/离开时间; 写 tin/tout/depth/up[0]。
36      void dfs(int u, int p) {
37          tin[u] = ++timer;
38          up[0][u] = p;
39          for (int j = 1; j < logn; ++j) up[j][u] = up[j - 1][up[j - 1][u]];
40          for (int v : g[u]) if (v != p) {
41              depth[v] = depth[u] + 1;
42              dfs(v, u);
43          }
44          tout[u] = timer;
45      }
46      // 参数: root 为原树根。
47      // 返回: void。
48      // 效果: 清空并重建 tin/tout/depth/up; 要求原图为连通树。
49      void build(int root = 1) {
50          assert(1 <= root && root <= n);
51          timer = 0;
52          fill(tin.begin(), tin.end(), 0);
53          fill(tout.begin(), tout.end(), 0);
54          fill(depth.begin(), depth.end(), 0);
55          for (auto& row : up) fill(row.begin(), row.end(), 0);
56          dfs(root, 0);
57          assert(timer == n);
58      }
59      // 参数: u、v 是已 build 的树中合法点号。
60      // 返回: bool, u 是否为 v 的祖先 (包括 u==v)。
61      // 效果: 只读, 不修改结构。
62      bool isAncestor(int u, int v) const {
63          return tin[u] <= tin[v] && tout[v] <= tout[u];
64      }
65      // 参数: u、v 是已 build 的树中合法点号。
66      // 返回: int, u、v 的最近公共祖先。
67      // 效果: 要求已 build 且两点在树中。
68      int lca(int u, int v) const {
69          if (isAncestor(u, v)) return u;
70          if (isAncestor(v, u)) return v;
71          int x = u;
72          for (int j = logn - 1; j >= 0; --j)
73              if (up[j][x] != 0 && !isAncestor(up[j][x], v)) x = up[j][x];
74          return up[0][x];
75      }
76      // 参数: special 是本次询问的原树关键点, 可重复。
77      // 返回: VirtualTreeResult, 含关键点及必要 LCA 的虚树, 要求已 build。
78      // 效果: 只读, 不修改结构。
79      VirtualTreeResult buildVirtual(vector<int> special) const {
80          VirtualTreeResult result;
81          if (special.empty()) return result;
82          // 参数: a、b 是原树点号。
83          // 返回: bool, a 的 DFS 进入时间是否早于 b。
84          // 效果: 比较器只读 tin; sort 据此原地排序 special。
85          sort(special.begin(), special.end(), [&](int a, int b) { return
tin[a] < tin[b]; });
86          special.erase(unique(special.begin(), special.end()),
special.end());
87          vector<int> all = special;
88          for (int i = 1; i < (int)special.size(); ++i)
all.push_back(lca(special[i - 1], special[i]));
89          // 参数: a、b 是关键点或 LCA 点号。
90          // 返回: bool, a 的 DFS 进入时间是否早于 b。
91          // 效果: 比较器只读 tin; sort 据此原地排序 all。
```

```
        sort(all.begin(), all.end(), [&](int a, int b) { return tin[a] < tin[b]; });
        all.erase(unique(all.begin(), all.end()), all.end());
        result.vertices = all;
        vector<int> st;
        for (int x : all) {
            while (!st.empty() && !isAncestor(st.back(), x)) st.pop_back();
            if (!st.empty())
                result.edges.push_back({st.back(), x, depth[x] - depth[st.back()]});
            st.push_back(x);
        }
        result.root = all.front();
        return result;
    }
};
```

最小调用示例，可直接放在 `main` 函数体内：原树为 `1-2,1-3,2-4,2-5`，关键点 `{4,5,3}`，其虚树应有根 1、五个保留点和四条边。

```
VirtualTree vt(5);
vt.addEdge(1, 2); vt.addEdge(1, 3);
vt.addEdge(2, 4); vt.addEdge(2, 5);
vt.build(1);
auto result = vt.buildVirtual({4, 5, 3});
assert(result.root == 1);
assert(result.vertices.size() == 5 && result.edges.size() == 4);
```

本节七点压缩例也可直接验证；以下局部作用域与上一例独立：

```
{
    VirtualTree compressedTree(7);
    compressedTree.addEdge(1, 2); compressedTree.addEdge(1, 3);
    compressedTree.addEdge(2, 6); compressedTree.addEdge(6, 7);
    compressedTree.addEdge(7, 4); compressedTree.addEdge(7, 5);
    compressedTree.build(1);
    auto small = compressedTree.buildVirtual({4, 5, 3});
    assert((small.vertices == vector<int>{1, 7, 4, 5, 3}));
    assert((small.edges == vector<tuple<int, int, int>>{
        {1, 7, 3}, {7, 4, 1}, {7, 5, 1}, {1, 3, 1}}));
}
```

5. 要改哪里

每次询问的 DP 标记、度数和答案数组只初始化或清理 `vertices` 中至多 `2k-1` 个点；不要为了小询问清空整棵原树。虚树边权是原树距离，不是原树单边，距离 DP 要使用这段距离。

HLD 适合动态路径修改或查询；虚树适合原树大、每次询问关键点少的场景。可继续练习关键点两两路径总长、关键点覆盖或最近关键点 DP。

6. 自检

对本节七点树，关键点 `{4,5,3}` 应补出 7 和 1，按 DFS 序得到 `[1,7,4,5,3]`；虚边为 `1→7`（3）、`7→4`（1）、`7→5`（1）、`1→3`（1）。压缩前后关键点距离不变，例如 4 到 3 都为 5。

第 4 部分保留的五点调用是另一个小例：应补出 2、1，共五个保留点和四条边。空关键点返回空点边且根为 0；单关键点只有一个点、没有边。重复给出同一个关键点不应产生重复虚边。

对拍可随机取几个关键点，比较原树和虚树上的两两距离，并核对虚树每条边都是原树的祖先到后代路径。

7. 坑

关键点重复时先去重；只加入关键点会丢失分叉祖先，必须补入 DFS 序相邻关键点的 LCA。`buildVirtual` 会对输入排序、补 LCA 后再次排序，单次复杂度为 $O(k \log k + k \log n)$，不能只报栈扫描的线性复杂度。

`tout` 与 `tin` 必须来自同一次 build；虚树边代表原树压缩路径。递归 DFS 在深链上可能栈溢出。若使用同根 HLD，`pos[u] <= pos[v] < pos[u]+sz[u]` 也可用于祖先判断；此实现用 `tin/tout`。

8. 一句话原理

关键点按 DFS 序排列后补入相邻 LCA，再用祖先栈连接，删去对本次问题无贡献的原树分支。

5.20 带权并查集

1. 它是做什么的

有若干元素，约束会逐条告诉我们两个元素的数值差，例如 `value[2]-value[1]=5`、`value[3]-value[2]=2`。因此 `value[3]-value[1]=7`；若再要求它等于 6，就与已有关系矛盾。我们也可以查询两个已连通元素的差值。

普通并查集能快速判断两个元素是否属于同一组，却不知道组内相差多少。若每次沿所有已知关系重新计算差值，关系多时会重复走许多边。

带权并查集在并查集的父子边上额外记一个差值。路径缩短时把沿途差值累加，于是既能知道是否同组，也能得到组内相对值。

2. 变量在做什么

要分清元素的绝对值和它相对父亲的差。模板并不保存每个 `value[x]` 的绝对值；它只保存 `value[x]-value[parent[x]]`。根的差为 0 只是把根当作参考原点，绝不表示题目规定根的真实数值是 0。

名称	含义
`parent[i]`	父结点；根的父亲是自身。下标 `1..n` 有效，`0` 是未使用哨兵。
`size[i]`	仅根结点的集合大小，用于按大小合并。
`potential[i]`	`value[i]-value[parent[i]]`；经过 `find(i)` 后变为 `value[i]-value[root]`。

3. 函数在做什么

沿用 `value[2]-value[1]=5`、`value[3]-value[2]=2`，按本模板的实际挂接规则手算：

操作	父指针变化	保存的差值
`unite(1,2,5)`	两组大小相等，将根 1 挂到根 2	`potential[1]=-5`，表示 1 比 2 小 5
`unite(2,3,2)`	2 所在组更大，将根 3 挂到根 2	`potential[3]=2`，表示 3 比 2 大 2
`relation(1,3)`	两点根均为 2	差为 `2-(-5)=7`
`unite(1,3,6)`	已同根，不再合并	已有差是 7，与 6 矛盾，返回 `false`

这份小例的父链较短。若父链是 `x→p→root`，压缩时要把两段差相加：`value[x]-value[root]=(value[x]-value[p])+(value[p]-value[root])`。因此必须先递归处理 p，再用它已经算好的根差更新 x。

记 `dx=value[x]-value[rootX]`、`dy=value[y]-value[rootY]`。同根时约束 `value[y]-value[x]=w` 当且仅当 `dy-dx=w`。

若把 `rootX` 接到 `rootY`，需存 `value[rootX]-value[rootY]=dy-dx-w`；若把 `rootY` 接到 `rootX`，需存 `value[rootY]-value[rootX]=w+dx-dy`。

路径压缩把父链上的差相加，不改变实际关系，因此公式能保持已有约束。

函数	参数与返回	行为及前提
`find(x)`	x 必须在 1..n。 返回：int，代表元。	potential[x] 改成 value[x]-value[root]。
`WeightedDSU(n)`	n 为元素个数。 返回：无（构造函数）。	建立 n 个互不连通的单点集合，初始差值均为 0。
`unite(x,y,w)`	x、y 在 1..n，w 表示 value[y]-value[x]=w。 返回：bool，是否与现有约束相容。	先 find 并压缩路径；异组时加入约束并合并，同组时仅检查是否相容。
`relation(x,y)`	x、y 在 1..n。 返回：optional<long long>，连通时为 value[y]-value[x]，否则为空。	find 可能压缩路径但不改变集合划分。

复杂度：每次合并、查找或查询的均摊时间为 $O(\alpha(n))$，空间 $O(n)$。

4. 怎么调用

完整模板如下。代码中每个函数的参数、返回和副作用契约见其定义前注释。

```cpp
#include <bits/stdc++.h>
using namespace std;

class WeightedDSU {
    vector<int> parent, size;
    // potential[x] = value[x] - value[parent[x]]；根的值为 0 只是参考原点。
    vector<long long> potential;
    // 参数：x 必须在 1..n。
    // 返回：int，代表元。
    // 效果：potential[x] 改成 value[x]-value[root]。
    int find(int x) {
        if (parent[x] == x) return x;
        int p = parent[x];
        parent[x] = find(p);
        potential[x] += potential[p];
        return parent[x];
    }

public:
    // 参数：n 为元素个数。
    // 返回：无（构造函数）。
    // 效果：建立 n 个互不连通的单点集合，初始差值均为 0。
    explicit WeightedDSU(int n)
        : parent(n + 1), size(n + 1, 1), potential(n + 1, 0) {
        assert(n >= 0);
        iota(parent.begin(), parent.end(), 0);
    }
    // 参数：x、y 在 1..n，w 表示 value[y]-value[x]=w。
    // 返回：bool，是否与现有约束相容。
    // 效果：先 find 并压缩路径；异组时加入约束并合并，同组时仅检查是否相容。
    bool unite(int x, int y, long long w) {
        int rx = find(x), ry = find(y);
        long long dx = potential[x], dy = potential[y];
        if (rx == ry) return dy - dx == w;
        if (size[rx] > size[ry]) {
            // value[ry]-value[rx] = w + dx - dy。
            parent[ry] = rx;
            potential[ry] = w + dx - dy;
            size[rx] += size[ry];
        } else {
            // value[rx]-value[ry] = dy - dx - w。
            parent[rx] = ry;
            potential[rx] = dy - dx - w;
            size[ry] += size[rx];
```

```
        }
        return true;
    }
    // 参数：x、y 在 1..n。
    // 返回：optional<long long>，连通时为 value[y]-value[x]，否则为空。
    // 效果：find 可能压缩路径但不改变集合划分。
    optional<long long> relation(int x, int y) {
        if (find(x) != find(y)) return nullopt;
        return potential[y] - potential[x];
    }
};
```

调用示例：

```
WeightedDSU dsu(3);
assert(dsu.unite(1, 2, 5));
assert(dsu.unite(2, 3, 2));
auto d = dsu.relation(1, 3);
assert(d.has_value() && *d == 7);
assert(!dsu.unite(1, 3, 6));
```

5. 要改哪里

模 3 关系要把每个差值归一到 `{0,1,2}`，例如 `norm(t)=((t%3)+3)%3`。先把题意统一写成 `value[y]-value[x]=w (mod 3)`。

然后 `potential`、`w` 和合并公式中的每次加减都取模。食物链等方向关系若只改比较、不统一差值方向，公式仍会错。

普通并查集只问连通性即可；若需要删除约束或回滚，不能在这个带路径压缩的结构上直接删边，应考虑可撤销并查集和时间分治。

6. 自检

加入 `value[2]-value[1]=5`、`value[3]-value[2]=2` 后，查询 1 到 3 应为 7，反向应为 −7；再加入差值 6 应返回矛盾。未连通两点返回空值，同点差值为 0。

分层练习： 入门：食物链或相对距离的三类关系；进阶：带时间戳的坐标约束与矛盾输出；提高：离线删除边、线段树分治和可撤销带权并查集。若关系是在模 3 意义下的类别差，不能直接把本模板的整数加减照搬：`potential`、`w` 和每次加减都要改成模 3 代表元，并按题目定义统一方向。

7. 坑

差值方向固定为 `value[y]-value[x]`；方向写反会使合并式和查询错。每次差值累加、两点相减以及合并公式的中间值，都必须能由 `long long` 表示。

组合应用：差值约束可先离散化元素，再由带权并查集维护；在线判矛盾可用于检查方程组的一致性；与二分答案结合时，每次二分检查一批差值约束。注意该模板不能处理“边删除”或要求路径上每条边的明细。

8. 一句话原理

沿父亲路径累加差值，使并查集同时维护连通性和组内相对值。

5.21 离线树状数组

1. 它是做什么的

有时所有查询都已给出，可以先排序再回答。例如给定点 `(1,2),(2,1),(2,2)`，询问横坐标不超过 2、纵坐标也不超过 2 的点有几个，答案是 3。这里两个坐标条件都要满足。

逐个查询都从头检查所有点或所有区间元素，会反复访问相同数据。若查询之间互不影响，可以把它们按某个坐标排序，让已经处理的数据只扫一次。

先把点和询问按横坐标排序，逐步加入横坐标合格的点；树状数组再统计其中纵坐标合格的点数。这类两个坐标同时满足大小条件的计数，常称为二维偏序。区间不同数和逆序对是后面介绍的另外两个应用。

2. 变量在做什么

离散化把很大或负数坐标换成从 1 开始的紧凑编号；相同坐标必须得到相同编号。二维计数的查询 `id` 用来把排序后的答案放回输入顺序。

本节先按 x 扫描，再用树状数组的 y 前缀统计，所以 BIT 下标代表纵坐标编号，不是点在输入中的顺序。后面的区间不同数改用数组位置作下标；逆序对则改用值的排名。

树状数组可看成维护一串数字：`add(pos,delta)` 改一个位置，`sumPrefix(pos)` 统计从 1 到该位置的总和。区间和用 `sumPrefix(r)-sumPrefix(l-1)` 得到。

名称	含义
`Fenwick::n/bit`	有效位置上界 `n` 与树状数组；`bit` 下标从 1 开始。
`PointXY{x,y}`	一个可重复计数的点坐标。
`QueryXY{x,y,id}`	询问右上界及原输入编号；`id` 须为 `0..q-1`。
`RangeQuery{l,r,id}`	1-based 闭区间及答案编号。
`ys`、`rankOf`	合并点和询问的 y 坐标后离散化；Fenwick rank 为 1-based。
`last[value]`	区间不同数扫描中，当前前缀内该值最后出现的位置。
`values`、`rank`	逆序对压缩值域；相同值映射到同一 rank。

3. 函数在做什么

二维偏序例子取点 `(1,2),(2,1),(2,2)`，查询 `(2,2)`。

1. 按 x 扫描，把所有 `x<=2` 的点加入 BIT。
2. y 排名为 1、2；前缀 2 中累计了三次加一。
3. 读出 3。加入条件保证 `px<=X`，前缀条件保证 `py<=Y`，故每个满足两条件的点恰计一次，重复点按重数计。

区间不同数例子 `a=[7,3,7]`、查询 `[2,3]`。扫到位置 3 时先把 7 在位置 1 的旧标记减掉，再在位置 3 加标记；`sum(3)-sum(1)` 得到位置 2、3 上的两个标记，即两种数。

每种值在当前前缀只有最后一次出现位置有标记，所以区间和正好数出最后位置落在 `[l,r]` 的值。

逆序对例子 `[3,1,2]` 从右向左扫：遇到 2、1 时右侧都没有比它更小的值；遇到 3 时右侧有 1、2 两个更小值，因此总数为 2。查询 `rank-1` 排除相等值，正好对应严格条件 `i<j` 且 `a[i]>a[j]`。

函数	参数与返回	行为及前提
`Fenwick(n)`	n 是最大可用下标；n 可以为 0。 返回：无（构造函数）。	建立全零树状数组。
`add(pos,delta)`	pos 必须在 1..n。 返回：void。	把 bit[pos..] 对应的前缀结构都增加 delta。
`sumPrefix(pos)`	pos 在 0..n。 返回：ll，原数组下标 1..pos 的元素和。	只读，不修改结构。
`rangeSum(l,r)`	l、r 为 1-based 合法闭区间。 返回：ll，该区间和。	只读，不修改结构。
`countDominated`	points 为二维点，queries 的 id 必须为 0..queries.size()-1。 返回：vector<ll>，各询问的支配点数。	只读，不修改结构。
`distinctInRanges`	a 是 1-based 数组，a[0] 不参与；queries 中 id 连续且区间合法。 返回：vector<int>，每个闭区间的不同元素数。	排序 queries 副本并维护局部 BIT；不修改输入数组。

函数	参数与返回	行为及前提
`countInversions(a)`	a 是 0-based 输入数组。 返回：ll，满足 i<j 且 a[i]>a[j] 的逆序对数量。	只读，不修改输入 a。

复杂度：树状数组操作 $O(\log n)$。二维偏序含 y 坐标压缩、点和询问排序，时间 $O((p+q)\log(p+q+1))$，空间 $O(p+q)$。

区间不同数为 $O(n\log(n+1)+q\log(q+1)+q\log(n+1))$，分别对应最后出现位置维护、询问排序和查询，空间 $O(n+q)$。逆序对为 $O(n\log(n+1))$，空间 $O(n)$。

4. 怎么调用

完整模板如下。代码中每个函数的参数、返回和副作用契约见其定义前注释。

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Fenwick {
    int n;
    vector<ll> bit;
    // 参数：n 是最大可用下标；n 可以为 0。
    // 返回：无（构造函数）。
    // 效果：建立全零树状数组。
    explicit Fenwick(int n) : n(n), bit(n + 1, 0) { assert(n >= 0); }
    // 参数：pos 必须在 1..n。
    // 返回：void。
    // 效果：把 bit[pos..] 对应的前缀结构都增加 delta。
    void add(int pos, ll delta) {
        assert(1 <= pos && pos <= n);
        for (int i = pos; i <= n; i += i & -i) bit[i] += delta;
    }
    // 参数：pos 在 0..n。
    // 返回：ll，原数组下标 1..pos 的元素和。
    // 效果：只读，不修改结构。
    ll sumPrefix(int pos) const {
        assert(0 <= pos && pos <= n);
        ll answer = 0;
        for (int i = pos; i > 0; i -= i & -i) answer += bit[i];
        return answer;
    }
    // 参数：l、r 为 1-based 合法闭区间。
    // 返回：ll，该区间和。
    // 效果：只读，不修改结构。
    ll rangeSum(int l, int r) const { return sumPrefix(r) - sumPrefix(l - 1); }
};

struct PointXY { ll x, y; };
struct QueryXY { ll x, y; int id; };
// 参数：points 为二维点，queries 的 id 必须为 0..queries.size()-1。
// 返回：vector<ll>，各询问的支配点数。
// 效果：只读，不修改结构。
vector<ll> countDominated(const vector<PointXY>& points, vector<QueryXY> queries) {
    vector<ll> ys;
    ys.reserve(points.size() + queries.size());
    for (const auto& p : points) ys.push_back(p.y);
    for (const auto& q : queries) ys.push_back(q.y);
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    // 参数：y 是输入点或查询的纵坐标。
    // 返回：int，离散化后 1-based 的位置。
    // 效果：只读，不修改结构；不修改 ys。
```

```cpp
    auto rankOf = [&](ll y) -> int {
        return int(lower_bound(ys.begin(), ys.end(), y) - ys.begin()) + 1;
    };
    vector<PointXY> sortedPoints = points;
    // 参数：a、b 为两个平面点。
    // 返回：a.x < b.x。
    // 效果：只读比较；同 x 点任意顺序均可。
    sort(sortedPoints.begin(), sortedPoints.end(),
         [](const PointXY& a, const PointXY& b) { return a.x < b.x; });
    // 参数：a、b 为两个二维询问。
    // 返回：a.x < b.x。
    // 效果：只读比较；同 x 的点会在回答前全部加入。
    sort(queries.begin(), queries.end(),
         [](const QueryXY& a, const QueryXY& b) { return a.x < b.x; });
    Fenwick fw((int)ys.size());
    vector<ll> answer(queries.size());
    size_t ptr = 0;
    for (const auto& q : queries) {
        // <= 保证 px==q.x 的点先进入；重复点会被重复计数。
        while (ptr < sortedPoints.size() && sortedPoints[ptr].x <= q.x) {
            fw.add(rankOf(sortedPoints[ptr].y), 1);
            ++ptr;
        }
        int upto = int(upper_bound(ys.begin(), ys.end(), q.y) - ys.begin());
        answer[q.id] = fw.sumPrefix(upto);
    }
    return answer;
}

struct RangeQuery { int l, r, id; };
// 参数：a 是 1-based 数组，a[0] 不参与；queries 中 id 连续且区间合法。
// 返回：vector<int>，每个闭区间的不同元素数。
// 效果：排序 queries 副本并维护局部 BIT；不修改输入数组。
vector<int> distinctInRanges(const vector<int>& a, vector<RangeQuery>
queries) {
    int n = (int)a.size() - 1;
    // 按右端点升序处理；右端点相等时按左端点升序只是稳定的次序选择。
    // 参数：u、v 是两个区间查询。
    // 返回：bool，按 r 升序、再按 l 升序比较。
    // 效果：比较器只读查询；sort 据此原地排序 queries。
    sort(queries.begin(), queries.end(),
         [](const RangeQuery& u, const RangeQuery& v) {
             return u.r != v.r ? u.r < v.r : u.l < v.l;
         });
    Fenwick fw(n);
    map<int, int> last; // 用 map 避免哈希碰撞；值的范围无需预先知道。
    vector<int> answer(queries.size());
    int right = 0;
    for (const auto& q : queries) {
        while (right < q.r) {
            ++right;
            auto it = last.find(a[right]);
            if (it != last.end()) fw.add(it->second, -1);
            fw.add(right, 1);
            last[a[right]] = right;
        }
        answer[q.id] = (int)fw.rangeSum(q.l, q.r);
    }
    return answer;
}
// 参数：a 是 0-based 输入数组。
// 返回：ll，满足 i<j 且 a[i]>a[j] 的逆序对数量。
// 效果：只读，不修改输入 a。
ll countInversions(const vector<ll>& a) {
```

```
    vector<ll> values = a;
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    Fenwick fw((int)values.size());
    ll answer = 0;
    for (auto it = a.rbegin(); it != a.rend(); ++it) {
        int rank = int(lower_bound(values.begin(), values.end(), *it) - values.begin()) + 1;
        answer += fw.sumPrefix(rank - 1); // 已见元素中严格小于当前值，正好构成当前左端点的逆序对。
        fw.add(rank, 1);
    }
    return answer;
}
```

调用示例：

```
vector<PointXY> points = {{2, 2}, {2, 2}, {3, 1}};
auto dominated = countDominated(points, {{2, 2, 0}});
assert(dominated[0] == 2);
vector<int> a = {0, 7, 3, 7};
auto distinct = distinctInRanges(a, {{2, 3, 0}});
assert(distinct[0] == 2);
auto inversions = countInversions({3, 1, 2});
assert(inversions == 2);
```

5. 要改哪里

二维偏序若 x 条件严格小于，把加入条件 `point.x <= q.x` 改为 `<`；若 y 条件严格小于，把 `upper_bound` 改为 `lower_bound`。

区间不同数必须先删旧最后位置，再加新位置；不能给每个位置永久加一。逆序对统计严格 `>` 时查 `rank-1`；若要统计相等关系，先明确题意再调整边界。

看到“所有点左下方有多少点”可考虑二维偏序；看到区间有多少种数可考虑右端点排序；看到 `i<j` 且大小关系可考虑离散化加 BIT。进阶可学三维偏序、CDQ 分治，并比较区间不同数用莫队的取舍。

6. 自检

第 4 部分调用中的重复点 `(2,2)` 应计两次；数组 `[7,3,7]` 的 `[2,3]` 区间有 2 个不同值；`[3,1,2]` 有 2 个严格逆序对。把所有值改成相等时，逆序对应为 0，任意非空区间的不同数为 1。

分层练习： 入门：静态单点修改、区间和与逆序对；进阶：二维矩形左下计数、区间不同数；提高：三维偏序、带时间维度的修改查询和 CDQ 分治。要特别测试空询问列表、相同 `x/y` 的重复点、全相同数组和全逆序数组。

7. 坑

等号由扫描条件及 `lower_bound/upper_bound` 决定；重复点按重数计，查询 id 和闭区间范围须合法。

组合应用：二维偏序还能扩展到 CDQ 分治、三维偏序和扫描线；区间不同数可与莫队比较取舍；逆序对可作为归并排序、Fenwick 或线段树的练习。值域很大时先坐标压缩，重复值必须保留相同 rank。

8. 一句话原理

排序把一个维度变成扫描进度，树状数组维护剩余维度的前缀统计。

5.22 莫队算法

1. 它是做什么的

给定一个数组，有很多次询问，每次给出一个区间，要求统计其中有多少种不同的数。例如：

```
数组：[1, 2, 1, 3, 2]
询问 [1,3]：里面是 [1,2,1]，不同的数有 1、2，答案是 2。
询问 [2,5]：里面是 [2,1,3,2]，不同的数有 1、2、3，答案是 3。
```

最直接的办法是每次重新扫描询问区间，统计出现过哪些数。如果数组和询问都很多，这些扫描会反复处理同一批元素。

莫队的做法是：**保留上一个区间的统计结果，移动左右端点，把它调整成下一个区间。** 从 `[1,3]` 调整到 `[2,5]`，只需要加入位置 4、5，再移除位置 1，不必重新扫描整个 `[2,5]`。

如果询问顺序很乱，左右端点仍可能来回横跨整个数组。莫队因此会先重新排列询问，让相邻询问尽量共享已有的统计结果。全部处理完后，再按原来的询问顺序输出答案。先读入全部询问、再统一计算，称为“离线处理”。本节处理过程中数组保持不变。

2. 变量在做什么

理解莫队时，先分清两个东西：**当前维护的是哪个区间，以及这个区间里有什么。**

名称	含义
`left`、`right`	当前已经统计好的闭区间 `[left,right]`。
`frequency[v]`	值编号 `v` 在当前区间内出现了多少次。
`distinct`	当前区间里有多少种不同的值，也就是多少个频次大于 0。
`compressed[i]`	原数组位置 `i` 上的值经过离散化后的编号。
`MoQuery::l,r`	某次询问要求的区间，不是当前窗口的端点。
`MoQuery::id`	询问在输入中的编号，用来恢复答案顺序。
`blockLength`	左端点分块时，每块包含的位置数，至少为 1。
`answer[id]`	原编号为 `id` 的询问答案。

其中，`frequency` 按**值编号**访问，`compressed` 按**数组位置**访问：

```
位置 i → compressed[i] 得到值编号 → frequency[值编号] 得到出现次数
```

离散化只是把可能很大的原值换成紧凑编号。例如原值为 `[100,500,100]`，可以编号为 `[0,1,0]`。相同的数仍使用相同编号，所以不同数的数量不会改变。为了便于手算，下面的频次表直接写原值；代码实际使用从 0 开始的离散编号。

3. 函数在做什么

对“区间不同数”这个问题，最重要的是加入、删除时如何更新答案。

加入一个值时，若频次从 0 变成 1，就多了一种数，`distinct` 加一；原来已经出现过，就只增加频次。

删除一个值时，若频次从 1 变成 0，就少了一种数，`distinct` 减一；删除后仍出现至少一次，答案不变。

例如，从 `[1,3]` 移动到 `[2,5]`：

操作	当前区间	非零频次	`distinct`
已经维护好 `[1,3]`	`[1,3]`	`1:2, 2:1`	2
加入位置 4 的值 3	`[1,4]`	`1:2, 2:1, 3:1`	3
加入位置 5 的值 2	`[1,5]`	`1:2, 2:2, 3:1`	3
删除位置 1 的值 1	`[2,5]`	`1:1, 2:2, 3:1`	3

删除位置 1 时，值 1 仍在位置 3 出现，所以答案不能减一。

函数	参数与返回	行为及前提
`MoDistinct(a)`	输入 1-based 数组；构造函数无返回值。	将原值离散化，准备频次数组和块长。

函数	参数与返回	行为及前提
`addValue`	输入值编号和答案的引用；返回 `void`。	增加频次，必要时增加不同数数量；内部调用。
`removeValue`	输入值编号和答案的引用；返回 `void`。	减少频次，必要时减少不同数数量；被删除的贡献必须已经在窗口中。
`solve(queries)`	输入全部询问；返回按原编号排列的答案数组。	清空统计、排序询问，然后移动窗口并记录答案；可以重复调用。

`solve` 开始时令 `left=1,right=0`，表示尚未加入任何元素的空区间。之后始终保持：频次表和 `distinct` 恰好对应当前 `[left,right]` 内的元素。向左扩展、向右扩展都是加入；向左收缩、向右收缩都是删除。本节统计不依赖元素排列，因此两端可以共用增删函数。

询问先按左端点所在块排序，同一块内再按右端点排序；相邻块交替使用升序、降序，减少右端点折返。当前代码使用 `l/blockLength` 划分块，首块稍短不影响复杂度。

设数组长度为 `n`、询问数为 `q`、块长为 `B`，端点移动次数上界为 $O(n^2/B + qB)$：右端点在每个左端点块内沿一个方向扫描，合计约为 `n²/B`；同块内左端点每次最多移动约 `B`，合计约为 `qB`。蛇形排序改善常数，不改变这个上界。

本模板取 `B≈√n`，每次增删为常数时间，再计入离散化和询问排序，总时间为 $O((n+q)\sqrt{n} + n\log(n+1) + q\log(q+1))$，空间为 $O(n+q)$。若维护别的统计需要更昂贵的增删操作，移动部分还须乘上单次增删的代价。

4. 怎么调用

将 `MoQuery` 和 `MoDistinct` 模板放在业务代码之前。数组下标 0 占位；每个查询满足 `1<=l<=r<=n`，`id` 恰好覆盖 `0..q-1`。空查询列表可以直接返回空答案；没有有效数组元素时也不能给出非空区间查询。

```cpp
#include <bits/stdc++.h>
using namespace std;

struct MoQuery { int l, r, id; };

class MoDistinct {
    int n;
    vector<int> compressed;
    int blockLength;
    vector<int> frequency;
    // 参数：value 是已离散化的值编号。
    // 返回：void。
    // 效果：把它加入当前窗口并更新 distinct 的引用值。
    void addValue(int value, int& distinct) {
        if (frequency[value]++ == 0) ++distinct;
    }
    // 参数：value 是已离散化的值编号。
    // 返回：void。
    // 效果：它必须当前至少出现一次，再删除并更新 distinct。
    void removeValue(int value, int& distinct) {
        assert(frequency[value] > 0);
        if (--frequency[value] == 0) --distinct;
    }

public:
    // 参数：a 为 1-based 静态数组。
    // 返回：无（构造函数）。
    // 效果：压缩其值并建立计数表，后续 solve 不修改 a 的逻辑内容。
    explicit MoDistinct(const vector<int>& a) : n((int)a.size() - 1) {
        assert(n >= 0);
        vector<int> values(a.begin() + (n ? 1 : 0), a.end());
        sort(values.begin(), values.end());
```

```
        values.erase(unique(values.begin(), values.end()), values.end());
        compressed.assign(n + 1, 0);
        for (int i = 1; i <= n; ++i)
            compressed[i] = int(lower_bound(values.begin(), values.end(), a[i]) - values.begin());
        frequency.assign(values.size(), 0);
        blockLength = max(1, (int)sqrt(max(1, n)));
    }
    // 参数：queries 的 l、r 合法且 id 连续。
    // 返回：vector<int>，各区间不同数，输入查询顺序不影响结果。
    // 效果：每次调用都会清空计数，因此可重复调用；数组本身必须保持静态。
    vector<int> solve(vector<MoQuery> queries) {
        for (int& x : frequency) x = 0;
        // 先按左端点所在块，再按右端点蛇形排序；返回严格弱序比较结果。
        // 参数：a、b 是两个闭区间查询。
        // 返回：bool，按分块和奇偶蛇形顺序比较。
        // 效果：只读 blockLength；sort 据此原地排序 queries。
        sort(queries.begin(), queries.end(), [this](const MoQuery& a, const MoQuery& b) {
            int ba = a.l / blockLength, bb = b.l / blockLength;
            if (ba != bb) return ba < bb;
            return (ba & 1) ? a.r > b.r : a.r < b.r;
        });
        vector<int> answer(queries.size());
        int left = 1, right = 0, distinct = 0;
        for (const MoQuery& q : queries) {
            while (left > q.l) addValue(compressed[--left], distinct);
            while (right < q.r) addValue(compressed[++right], distinct);
            while (left < q.l) removeValue(compressed[left++], distinct);
            while (right > q.r) removeValue(compressed[right--], distinct);
            answer[q.id] = distinct;
        }
        return answer;
    }
};
```

先用本节贯穿的例子验证调用。下面的大括号仅用于让这一示例拥有独立的局部变量：

```
{
    vector<int> a = {0, 1, 2, 1, 3, 2};
    vector<MoQuery> queries = {{1, 3, 0}, {2, 5, 1}};
    MoDistinct mo(a);
    vector<int> answer = mo.solve(queries);
    assert(answer[0] == 2 && answer[1] == 3);
}
```

业务代码只需要构造对象并调用 `solve`。即使处理顺序发生改变，结果仍写入 `answer[q.id]`。下面保留另一组较短的调用，检查区间 `[1,2]` 和 `[2,4]`：

```
vector<int> a = {0, 1, 2, 1, 3};
MoDistinct mo(a);
auto answer = mo.solve({{1, 2, 0}, {2, 4, 1}});
assert(answer[0] == 2 && answer[1] == 3);
```

5. 要改哪里

换成其他区间统计时，先问：已经知道当前区间的状态，加入或删除一个值时，需要怎样调整？

例如，要求所有值的“出现次数平方和” $S = \sum_v frequency[v]^2$。某个值原来出现 `f` 次，加入后贡献增加 $(f+1)^2 - f^2 = 2f + 1$。所以先执行 `S += 2*f+1`，再增加频次。删除时先减去 `2*f-1`，再减少频次。

窗口移动和排序逻辑可以保留，需要改的是统计状态、增删函数，以及记录答案的位置。平方和可能很大，应使用 `long long`。若要求最高频次，则还需要记录“有多少种值恰好出现 `f` 次”等状态，不能只替换一个答案变量。

适合莫队的模型通常是“静态数组、大量区间询问、单点增删可以维护”。区间不同数也能用离线树状数组解决；遇到难以按右端点单向扫描、但可以双向增删的统计，再考虑莫队。若统计依赖顺序，例如逆序对，还要区分从左端或右端增删的贡献。

数组有修改时，需要学习带修改莫队；树路径查询可以学习树上莫队；删除难做时可继续了解回滚莫队。无法快速维护单点增删的目标，不能直接套这份模板。

6. 自检

先检查几组容易手算的输入：全相等数组 `[1,1,1]` 的任意非空区间答案都是 1；互不相同数组 `[1,2,3]` 的答案等于区间长度；单点区间答案为 1。

本节例子的 `[1,3]`、`[2,5]` 应分别得到 2、3。同一区间重复询问结果一致，打乱询问顺序后按 `id` 还原的答案也应一致。重复调用 `solve` 会清空频次，不应继承上一次状态。

对拍时生成小数组，暴力扫描每个区间，用 `set` 统计不同数，与莫队逐项比较；特别加入相隔很远的两个窗口，检查扩展、收缩顺序。

7. 坑

- 删除一次就减少不同数数量：只有频次从 1 变成 0 时，答案才减少。
- 把位置当作值编号：移动到位置 `i` 时，应更新 `frequency[compressed[i]]`。
- 排序后直接顺序输出：必须通过 `id` 保存答案，且编号不能重复或越界。
- 随意交换四个移动循环：本模板先扩展窗口，再收缩窗口，保证删除的是已经加入的贡献。
- 使用过程中修改原数组的逻辑值：对象保存的是构造时离散化后的数组，后续查询不会自动反映修改。
- 用于必须立即回答、不能重排的询问：这不满足普通莫队的使用条件。

8. 一句话原理

重排询问后复用上一个区间的统计结果，只在左右端点移动时加入或删除元素。

5.23 笛卡尔树

1. 它是做什么的

数组 `[3,1,4,2]` 中，下标 2 的值 1 是整个区间最小值。若对许多区间各自扫描，区间常会重复比较同一批元素。笛卡尔树把“下标从左到右的顺序”和“最小值在上层”的关系放进同一棵树，方便之后沿树处理区间；例如下标 `[2,4]` 中最小值为 1，它恰好位于本例整棵树的根；其他区间的最小值未必是整树根。

这组数据的树形结果如下，节点标的是“下标:值”：

```
      2:1
     /   \
  1:3     4:2
           /
        3:4
```

从左到右中序读下标仍是 `1,2,3,4`；每个父节点的值都不大于孩子。

2. 变量在做什么

节点编号是原数组下标，节点的大小比较才看 `a[u]`。这两件事不能混：中序顺序由下标保证，堆序由值保证；值相同的节点也仍是不同位置。区间最小值并列时，模板规定靠左位置优先作为该区间的最小值归属点。

名称	含义
`root`	树根的原数组下标，范围 `1..n`。
`left[u]`,`right[u]`	下标 `u` 的左右孩子；0 表示没有孩子。

名称	含义
`parent[u]`	下标 `u` 的父亲；根和不存在的结点均为 0。
`stack`	从底到顶对应值非递减的候选祖先下标。
`last`	本轮最后弹出的子树根；它成为新节点的左儿子。

3. 函数在做什么

按 `[3,1,4,2]` 手算建树，栈里记录下标：

读入	栈顶处理	新连边与栈
下标 1，值 3	无可弹出	栈 `[1]`。
下标 2，值 1	弹出下标 1，令 `last=1`	`left[2]=1`；栈 `[2]`，2 是根候选。
下标 3，值 4	1 不大于 4，不弹	`right[2]=3`；栈 `[2,3]`。
下标 4，值 2	弹出 3，因为 `4>2`	`last=3`，令 `right[2]=4`、`left[4]=3`；栈 `[2,4]`。

新值比栈顶小时，栈顶对应的整棵候选子树都要转到新节点左侧；栈里剩下的最近节点则把新节点接到右边。每次只改这一条右侧边，因此中序顺序保持原下标顺序。末尾栈底是根。对例子中序得到下标 `1,2,3,4`，堆序也成立。

函数	参数与返回	行为及前提
`buildMinCartesian(a)`	`a` 是下标 1..n 的非空数组；返回 `CartesianTree`。	建立中序下标为 1..n、满足最小堆序的树；严格用 `>` 弹栈，所以相等时左侧节点优先。

每个下标入栈一次、最多出栈一次，建树时间 $O(n)$，空间 $O(n)$。

4. 怎么调用

完整模板如下。代码中每个函数的参数、返回和副作用契约见其定义前注释。

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct CartesianTree {
    int root = 0;
    vector<int> left, right, parent;
};
// 参数：a 是下标 1..n 的非空数组。
// 返回：CartesianTree，中序下标为 1..n 且满足最小堆性质的树。
// 效果：相等值不弹栈，因此相等最小值中下标较小者为祖先。
CartesianTree buildMinCartesian(const vector<ll>& a) {
    int n = (int)a.size() - 1;
    assert(n >= 1);
    CartesianTree result;
    result.left.assign(n + 1, 0);
    result.right.assign(n + 1, 0);
    result.parent.assign(n + 1, 0);
    vector<int> stack;
    for (int i = 1; i <= n; ++i) {
        int last = 0;
        while (!stack.empty() && a[stack.back()] > a[i]) {
            last = stack.back();
            stack.pop_back();
        }
        if (!stack.empty()) result.right[stack.back()] = i;
        result.left[i] = last;
        stack.push_back(i);
    }
```

```
    result.root = stack.front();
    for (int u = 1; u <= n; ++u) {
        if (result.left[u]) result.parent[result.left[u]] = u;
        if (result.right[u]) result.parent[result.right[u]] = u;
    }
    return result;
}
```

调用示例：

```
vector<ll> a = {0, 3, 1, 4, 2};
CartesianTree tree = buildMinCartesian(a);
assert(tree.root == 2);
assert(tree.left[2] == 1 && tree.right[2] == 4);
// 对完整验证，另做中序遍历并核对结果为 1,2,3,4。
```

5. 要改哪里

若要让相等最小值中更靠右者优先，把弹栈比较从 `>` 改成 `>=`；等值旧节点就会被弹出，较右节点成为祖先。最大值版本建立最大笛卡尔树并同步调整比较符。查询一般区间 `[l,r]` 时，区间最小值位于两端点 l、r 在笛卡尔树上的最近公共祖先（LCA）：树按下标划分连续区间，父节点又不会比后代更大。应先预处理 LCA，而不能对任意区间直接读取整树根。静态 RMQ 也可用稀疏表；值会在线修改时通常改用线段树，因为本树不支持值修改。

对每个节点 `i`，设左右子树大小为 `L`、`R`。最小值归给 `i` 的连续子数组，可以从 `i` 向左跨过左子树中任意 `0..L` 个位置，并向右跨过右子树中任意 `0..R` 个位置，因此有 `(L+1)(R+1)` 种。相等值按左侧优先归属，避免一个区间重复记给两个相等最小值。所有子数组最小值之和为 `sum(a[i]*(L+1)*(R+1))`；乘积和总和需选够宽的整数类型。

识别线索是：问题既要求保留原下标顺序，又反复用区间最值作分治枢纽或统计子数组贡献。

6. 自检

数组 `[3,1,4,2]` 的根为下标 2，左儿子为 1、右儿子为 4；中序遍历须恢复下标 `[1,2,3,4]`。全相等数组在严格 `>` 弹栈规则下形成向右的链，根为最左位置。

分层练习： 入门：验证中序和堆性质；进阶：用笛卡尔树加 LCA 做静态 RMQ；提高：所有子数组最小值之和及重复最值的贡献分配。

7. 坑

比较条件决定相等值归属；本模板严格用 `>`，较左相等最小值优先；数组不可为空。

组合应用：笛卡尔树的父子关系可让某些子数组贡献转为子树贡献；建树后给每个节点做 DFS 序，再用 LCA 回答 RMQ。大数组递归 DFS 可能爆栈，应改写为显式栈或使用安全的递归深度策略。

8. 一句话原理

单调栈把原序列中序次序和最值堆序编码为一棵树。

5.24 李超线段树

1. 它是做什么的

已加入直线 `y=x`、`y=-x+4`，要回答 `x=0,2,5` 的最大值，依次是 `4,2,5`。每次都逐条代入会在直线很多、查询很多时反复计算。

李超树把横坐标范围切成小段，每段留下当前的代表直线；可能在某一侧胜出的竞争线只往那一侧传。这样以后只沿查询坐标所在的路径比较少量直线。

2. 变量在做什么

`left/right/mid` 是排序去重后坐标的下标，不是原始横坐标值；真正代入直线计算的是 `xs[index]`。在分配到该节点的候选线中，节点保留中点胜出的直线；其他候选线可能只在一侧更优。

名称	含义
`Line{k,b}`	斜率与截距，均为 `long long`；代表 `y=kx+b`。
`xs`	排序去重后的可查询横坐标；查询只能使用其中的值。
`tree[node]`	节点当前保留的直线。
`used[node]`	节点是否已经有直线。
`left/right/mid`	离散坐标下标的区间和中点。
`position`	查询横坐标在 `xs` 中的下标。

3. 函数在做什么

继续看 `x=0,2,5`，一步步插入和查询：

1. 插入 `y=x`：根节点原来为空，直接保存它。
2. 插入 `y=-x+4`：中点对应 `x=2`，两条线都等于 2，根保留原线。
3. 判断另一条线的去向：左端 `x=0` 时新线值 4 大于原线值 0；右端 `x=5` 时新线值 −1 小于原线值 5。因此只把它送入左孩子。
4. 查询 `x=0`：沿根到左叶的路径比较，答案是 4。查询 `x=2` 或 `x=5` 时，同样只比较各自路径上的线，得到 2、5。

插入时交换以后，节点线在中点不劣，待下传线在中点不可能胜出。两条直线至多相交一次，若待下传线还可能获胜，只会在某个端点仍胜出的那一侧；代码按左右端点判断并至多递归一个孩子。这就是 `swap` 后必须用新的 `tree[node]` 和剩余 `line` 判断方向的原因。

函数	参数与返回	行为及前提
`value`	横坐标；返回 `i128` 函数值。	精确计算当前直线，不修改它。
`LiChaoMax`	可查询坐标集合；构造空树。	排序去重坐标，分配节点空间；空集合也可构造。
`insert`	节点区间和待插线；返回 void。	中点保留较优线，竞争线递归到可能胜出的一侧。
`indexOf`	合法查询坐标；返回离散下标。	二分查找；坐标不在集合时断言失败。
`queryNode`	节点区间和查询位置；返回可选最大值。	汇总根到叶路径上的直线值；空路径结果为空。
`addLine`	斜率、截距；返回 void。	加入整条线；空坐标集合时忽略。
`query`	已声明坐标；返回 `optional<i128>`。	查询所有已加入直线的最大值；没有直线时为空。

令 `m` 为不同查询坐标数，插线和单点查询均为 $O(\log m)$，空间 $O(m)$。当前横坐标需预先给出；任意整数域在线查询需改成动态节点树。

4. 怎么调用

完整模板如下。代码中每个函数的参数、返回和副作用契约见其定义前注释。

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct LiChaoMax {
    struct Line {
        ll k = 0, b = 0;
        // 参数：x 是查询横坐标；k、b 是当前直线字段。
```

```
        // 返回：i128，直线在 x 处的精确值 k*x+b。
        // 效果：只读，不修改直线。
        i128 value(ll x) const { return (i128)k * x + b; }
    };

    vector<ll> xs;
    vector<Line> tree;
    vector<char> used;
    // 参数：xs 是允许查询的横坐标集合，可以为空。
    // 返回：无（构造函数）。
    // 效果：保存排序去重后的所有合法查询横坐标，并建立空节点树。
    explicit LiChaoMax(vector<ll> xs) : xs(std::move(xs)) {
        sort(this->xs.begin(), this->xs.end());
        this->xs.erase(unique(this->xs.begin(), this->xs.end()), this->xs.end());
        tree.assign(max(1, 4 * (int)this->xs.size()), Line{});
        used.assign(tree.size(), false);
    }

private:
    // 参数：node 对应树节点；left、right 是其离散下标区间；line 是待插入直线。
    // 返回：void。
    // 效果：修改节点树，将 line 存入当前节点或唯一可能继续获胜的子树。
    void insert(int node, int left, int right, Line line) {
        if (!used[node]) {
            tree[node] = line;
            used[node] = true;
            return;
        }
        int mid = (left + right) / 2;
        if (line.value(xs[mid]) > tree[node].value(xs[mid]))
            swap(line, tree[node]);
        if (left == right) return;
        // swap 后 tree[node] 在中点不劣，line 才是需要继续寻找胜点的候选。
        bool lineBetterAtLeft = line.value(xs[left]) > tree[node].value(xs[left]);
        bool lineBetterAtRight = line.value(xs[right]) > tree[node].value(xs[right]);
        if (lineBetterAtLeft)
            insert(node * 2, left, mid, line);
        else if (lineBetterAtRight)
            insert(node * 2 + 1, mid + 1, right, line);
    }
    // 参数：x 必须属于构造时给出的查询坐标集合。
    // 返回：int，x 在去重坐标数组 xs 中的下标。
    // 效果：二分查找 xs；参数不在集合中时断言失败。
    int indexOf(ll x) const {
        auto it = lower_bound(xs.begin(), xs.end(), x);
        assert(it != xs.end() && *it == x);
        return int(it - xs.begin());
    }
    // 参数：node、left、right 是当前节点及其区间；position 是查询离散下标。
    // 返回：optional<i128>，该路径上直线的最大值；没有直线时为空。
    // 效果：只读，不修改结构。
    optional<i128> queryNode(int node, int left, int right, int position) const {
        optional<i128> answer;
        if (used[node]) answer = tree[node].value(xs[position]);
        if (left == right) return answer;
        int mid = (left + right) / 2;
        optional<i128> child = position <= mid
            ? queryNode(node * 2, left, mid, position)
            : queryNode(node * 2 + 1, mid + 1, right, position);
        if (!child) return answer;
```

```
        if (!answer || *child > *answer) return child;
        return answer;
    }

public:
    // 参数：k、b 分别是新直线 y=kx+b 的斜率和截距。
    // 返回：void。
    // 效果：空横坐标集合时忽略插入，其他情况修改树。
    void addLine(ll k, ll b) {
        if (!xs.empty()) insert(1, 0, (int)xs.size() - 1, Line{k, b});
    }
    // 参数：x 必须属于构造时坐标集合。
    // 返回：optional<i128>，所有已加入直线在 x 处的最大值，空集合返回 nullopt。
    // 效果：只读，不修改结构。
    optional<i128> query(ll x) const {
        if (xs.empty()) return nullopt;
        return queryNode(1, 0, (int)xs.size() - 1, indexOf(x));
    }
};
```

调用示例：

```
LiChaoMax lc({0, 2, 5});
lc.addLine(1, 0);
lc.addLine(-1, 4);
auto best = lc.query(2);
assert(best.has_value() && *best == 2);
auto missing = lc.query(0);
assert(missing.has_value() && *missing == 4);
```

5. 要改哪里

斜率优化 DP 常能整理为 `dp[i]=max_j(m_j*x_i+b_j)`。例如 `dp[i]=max_j{dp[j]+a[j]*x_i+c[j]}`，把已求出的状态 j 变成斜率 `a[j]`、截距 `dp[j]+c[j]` 的直线，在 `x_i` 查询。先算出 `dp[i]`，再插入它供后续使用，避免错误使用尚未得到的状态。

求最小值时，可以对每条线的斜率、截距取反，用最大值树查询，再将结果取反。求负数前先转成 `i128`，避免 `LLONG_MIN` 取反溢出；当前字段仍是 `long long`，取反后的系数还必须能放回该类型，否则应同时扩展 `Line` 字段和 `addLine` 参数。

若斜率和查询横坐标都单调，可进一步比较凸包优化的实现。若查询坐标无法预先给出，需改成动态坐标域；若一条线只在某段横坐标有效，需增加区间插线逻辑，不能直接使用本节整线插入。

6. 自检

手算小例子： 先加入 `y=x`，再加入 `y=-x+4`，查询 `x=0,2,5` 得到 `4,2,5`。两线在 `x=2` 相等，李超树在中点保留哪一条都不影响答案；若再加入平行线 `y=x+3`，它会在所有查询点压过 `y=x`，而无需特殊分支。

分层练习： 入门：固定横坐标集合上的直线最大值；进阶：斜率优化 DP、平行线和负坐标；提高：区间加线、持久化李超树和动态坐标域。

7. 坑

- 查询 x 必须在构造时给出的坐标集中；离散化保持坐标大小关系，但求值仍要用原坐标。
- 中点交换后要用交换后的节点线和剩余线判断左右方向，不能继续用旧比较结果。
- 本模板只插入直线，不支持删除；空坐标集会忽略插入，查询返回空值。
- 斜率、截距和坐标是 `long long`，仅求值提升为 `__int128`；这并不表示构造出的 DP 系数可以任意大。
- 平行线由比较自然保留截距较大者；负数和重复坐标可以输入。
- `optional` 为空表示尚无有效直线，不能直接解引用；输出 `i128` 要另写十进制转换，不能直接交给 `cout`。

8. 一句话原理

两条直线至多交一次，节点保留中点胜者并沿可能获胜的方向下传另一条线。

5.25 差分 + GCD 线段树

1. 它是做什么的

给数组 `[6,10,14,15]` 的第 2、3 项各加 5，再查询第 2 到第 4 项的 GCD。更新后是 `[6,15,19,15]`，答案是 `gcd(15,19,15)=1`。

直接做法要逐项改区间，再逐项求 GCD；长区间每次都要走很多项。差分把区间加变成最多两个位置的变化；查询时只需还原左端原值，并合并后续差分的 GCD。

2. 变量在做什么

`a[i]` 是原数组的实际值，`diff[i]` 不是另一个待查询数组的原值，而是相邻两个原值的差。原值靠差分前缀和还原；区间内变化则由这些差值的 GCD 概括。

名称	含义
`n`	原数组长度；所有公开区间是 1-based 闭区间。
`diff[i]`	`b[1]=a[1]`，`b[i]=a[i]-a[i-1]`。
`bit`	Fenwick 树，维护 `diff` 前缀和；`sumBit(i)` 还原 `a[i]`。
`seg`	GCD 线段树，叶子为 `|diff[i]|`，内部为儿子 GCD。
`p,l,r`	线段树节点及其管理的闭区间；`pos` 是差分修改位置。

3. 函数在做什么

先解释为什么 GCD 能换成差值。对任意整数 `x,y`，`gcd(x,y)=gcd(x,y-x)`：两边的公因数集合相同，因为能整除 `x,y` 就能整除 `x,y-x`，反过来也成立。

反复使用这一性质，可以把后一项减去前一项而不改变 GCD，得到 `gcd(a[l],a[l+1]-a[l],...,a[r]-a[r-1])`，也就是 `gcd(a[l],diff[l+1],...,diff[r])`。差分求前缀和时，相邻项正负抵消，得到 `diff[1]+...+diff[i]=a[i]`，所以 BIT 可还原 `a[i]`。

以本节数组和操作手算：

1. 初始 `diff=[6,4,4,1]`。
2. `[2,3]` 加 5，只改变 `diff[2]+=5`、`diff[4]-=5`，得到 `[6,9,4,-4]`；BIT 同步修改这两个点。
3. 查询 `[2,4]`，用 `sumBit(2)=6+9=15` 得到左端原值。
4. GCD 树查询 `diff[3..4]` 得 4，再合并 `gcd(15,4)=1`。

函数	参数与返回	行为及前提
`DifferenceGcdTree(a)`	a 为非空 1-based 初值数组；返回无（构造函数）。	复制差分，建立 BIT 和 GCD 树；不修改 a。
`addBit(i,delta)`	i 为 1..n 差分下标，delta 为增量；返回 void。	只更新 BIT；须与差分树的对应修改配合使用。
`sumBit(i)`	i 为 0..n 前缀右端；返回 diff[1..i] 的和；i=0 返回 0，其余还原 a[i]。	只读 BIT。
`build(p,l,r)`	p 为管理闭区间 [l,r] 的节点；返回 void。	从 diff 构建 seg；由构造函数调用。
`pointAdd`	p 管理 [l,r]，pos 为区间内差分下标，delta 为增量；返回 void。	修改 diff[pos] 和祖先 GCD；不更新 BIT。
`query(p,l,r,ql,qr)`	p 管理 [l,r]，目标闭区间 [ql,qr] 与其相交；返回相交部分差分绝对值的 GCD。	只读 seg；公开入口保证目标位于 1..n。

函数	参数与返回	行为及前提
`rangeAdd(l,r,x)`	1<=l<=r<=n，x 为增量；返回 void。	同步修改边界差分与 BIT；差分、前缀和、-x 和绝对值均须可用 ll 表示。
`rangeGcd(l,r)`	1<=l<=r<=n；返回当前 a[l..r] 的非负 GCD。	只读结构；单点区间返回该值的绝对值。

BIT 单点修改或前缀查询需要 $O(\log n)$，线段树点修改或区间查询会处理 $O(\log n)$ 个节点。每个节点还要计算 GCD。构造时间 $O(n \log V)$，单次区间加或 GCD 查询 $O(\log n \log V)$，空间 $O(n)$；`V` 是参与计算的数值绝对值的上界，64 位数的欧几里得算法迭代次数有固定小上限。

4. 怎么调用

将本节模板放在业务代码之前，再按下面的输入调用。

```
vector<long long> a = {0, 6, 10, 14, 15};
DifferenceGcdTree st(a);
st.rangeAdd(2, 3, 5);
long long answer = st.rangeGcd(2, 4);
assert(answer == 1);
```

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct DifferenceGcdTree {
    int n;
    vector<ll> diff, bit, seg;

    // 参数：a 为非空 1-based 初值数组。
    // 返回：无（构造函数）。
    // 效果：复制差分，建立 BIT 和 GCD 树；不修改 a。
    explicit DifferenceGcdTree(const vector<ll>& a) : n((int)a.size() - 1), diff(n + 1, 0), bit(n + 1, 0), seg(4 * n + 4, 0) {
        assert(n >= 1);
        for (int i = 1; i <= n; ++i) diff[i] = a[i] - (i == 1 ? 0 : a[i - 1]);
        for (int i = 1; i <= n; ++i) bit[i] += diff[i];
        for (int i = 1; i <= n; ++i) if (i + (i & -i) <= n) bit[i + (i & -i)] += bit[i];
        build(1, 1, n);
    }

    // 参数：i 为 1..n 差分下标，delta 为增量。
    // 返回：void。
    // 效果：只更新 BIT；须与差分树的对应修改配合使用。
    void addBit(int i, ll delta) {
        for (; i <= n; i += i & -i) bit[i] += delta;
    }

    // 参数：i 为 0..n 前缀右端。
    // 返回：diff[1..i] 的和；i=0 返回 0，其余还原 a[i]。
    // 效果：只读 BIT。
    ll sumBit(int i) const {
        ll result = 0;
        for (; i > 0; i -= i & -i) result += bit[i];
        return result;
    }

    // 参数：p 为管理闭区间 [l,r] 的节点。
    // 返回：void。
    // 效果：从 diff 构建 seg；由构造函数调用。
    void build(int p, int l, int r) {
```

```
40          if (l == r) { seg[p] = llabs(diff[l]); return; }
41          int m = (l + r) / 2;
42          build(p * 2, l, m); build(p * 2 + 1, m + 1, r);
43          seg[p] = gcd(seg[p * 2], seg[p * 2 + 1]);
44      }
45
46      // 参数: p 管理 [l,r], pos 为区间内差分下标, delta 为增量。
47      // 返回: void。
48      // 效果: 修改 diff[pos] 和祖先 GCD; 不更新 BIT。
49      void pointAdd(int p, int l, int r, int pos, ll delta) {
50          if (l == r) { diff[pos] += delta; seg[p] = llabs(diff[pos]);
return; }
51          int m = (l + r) / 2;
52          if (pos <= m) pointAdd(p * 2, l, m, pos, delta);
53          else pointAdd(p * 2 + 1, m + 1, r, pos, delta);
54          seg[p] = gcd(seg[p * 2], seg[p * 2 + 1]);
55      }
56
57      // 参数: p 管理 [l,r], 目标闭区间 [ql,qr] 与其相交。
58      // 返回: 相交部分差分绝对值的 GCD。
59      // 效果: 只读 seg; 公开入口保证目标位于 1..n。
60      ll query(int p, int l, int r, int ql, int qr) const {
61          if (ql <= l && r <= qr) return seg[p];
62          int m = (l + r) / 2; ll result = 0;
63          if (ql <= m) result = gcd(result, query(p * 2, l, m, ql, qr));
64          if (qr > m) result = gcd(result, query(p * 2 + 1, m + 1, r, ql,
qr));
65          return result;
66      }
67
68      // 参数: 1<=l<=r<=n, x 为增量。
69      // 返回: void。
70      // 效果: 同步修改边界差分与 BIT; 差分、前缀和、-x 和绝对值均须可用 ll 表示。
71      void rangeAdd(int l, int r, ll x) {
72          assert(1 <= l && l <= r && r <= n);
73          assert(x != LLONG_MIN);
74          addBit(l, x); pointAdd(1, 1, n, l, x);
75          if (r < n) { addBit(r + 1, -x); pointAdd(1, 1, n, r + 1, -x); }
76      }
77
78      // 参数: 1<=l<=r<=n。
79      // 返回: 当前 a[l..r] 的非负 GCD。
80      // 效果: 只读结构; 单点区间返回该值的绝对值。
81      ll rangeGcd(int l, int r) const {
82          assert(1 <= l && l <= r && r <= n);
83          ll inside = (l == r ? 0 : query(1, 1, n, l + 1, r));
84          return gcd(sumBit(l), inside);
85      }
86  };
```

5. 要改哪里

- 若更新只发生在单点 `[l,l]`，仍须在存在时修改 `diff[l+1]`，才能抵消对后续元素的影响；只有右端点等于 `n` 时跳过 `diff[n+1]`。
- 如果需要查询区间和，应另维护前缀和结构或换用懒标记线段树；GCD 摘要无法直接支持区间加。例如 `[6,10]` 的 GCD 是 2，整体加 1 后 `[7,11]` 的 GCD 是 1，并非 `2+1`。
- 若改为区间赋值，差分会影响边界内的大量差分项，不能只调用两次 `pointAdd`。

6. 自检

原数组 `[6,10,14,15]` 经 `[2,3] += 5` 后为 `[6,15,19,15]`，故 `rangeGcd(2,4)==1`；`rangeGcd(3,3)==19`。`r=n` 的更新后查询不可触及 `n+1`。

7. 坑

构造器接收非空的 1-based 数组，区间必须满足 `1<=l<=r<=n`。元素、差分、前缀和、`-x` 以及绝对值中间结果都必须能用 `long long` 表示；特别不能出现 `LLONG_MIN`，其绝对值无法表示。负数数组可以输入，GCD 结果为非负值。

- GCD 必须按非负绝对值计算；不能直接把有符号差分送给 `llabs` 并忽略 `LLONG_MIN`。
- 更新 `[l,r]` 时忘记在 `r+1` 减去 x，会让区间右侧整体偏移。
- 区间 GCD 由 `gcd(a[l], gcd(diff[l+1..r]))` 得出，不能错误查询 `diff[l..r]` 后再与 `a[l]` 合并。
- `rangeAdd` 断言范围，不代表算术溢出会被检测；调用者仍须保证所有中间值安全。

8. 一句话原理

区间加只改变差分数组的两个端点，而区间 GCD 可由左端原值与内部差分的 GCD 合成。

5.26 扫描线 + 线段树

1. 它是做什么的

两个正方形 `A=[0,2)×[0,2)`、`B=[1,3)×[1,3)` 的面积各是 4，但并集面积是 7，因为重叠部分不能算两次。按 x 方向切成窄片，每片只需知道当前 y 方向覆盖了多长，再乘片宽并相加。

矩形的左右边界会改变当前覆盖状态；线段树负责汇总这时 y 轴上被覆盖的总长度。

2. 变量在做什么

压缩后的 `ys[i]` 是边界坐标，线段树叶子表示相邻边界之间的间隔 `[ys[i],ys[i+1])`，不是坐标点。点本身宽度为零，真正有面积的是间隔；所以矩形 `[y1,y2)` 要更新从 `index(y1)` 到 `index(y2)-1` 的叶子。

名称	含义
`Rectangle`	四边界定义的半开矩形。
`Event`	横坐标处的加入 `+1` 或移除 `-1`。
`ys`	有效矩形的 y 边界，排序去重。
`cover[p]`	节点区间被整段覆盖的层数。
`length[p]`	节点实际被覆盖的 y 长度；根值供面积累加。
`area,previousX`	已算面积与上批事件横坐标。

3. 函数在做什么

手算矩形 A、B。`x=0` 加入 A 后覆盖长度是 2；到 `x=1` 先累加 `2*(1-0)=2`，再加入 B，覆盖 y 变成 `[0,3)`、长度 3；到 `x=2` 累加 `3*(2-1)=3`，再移除 A，剩下长度 2；到 `x=3` 累加 `2*(3-2)=2`。合计 `2+3+2=7`。

扫描到一个 x 时先用旧覆盖状态结算 `[previousX,x)`，再处理同 x 的全部事件。更新线段树时，完整覆盖节点增加或减少 `cover`；若 `cover>0`，节点长度就是 `y[r+1]-y[l]`，否则叶子长度为零、内部长度取两个孩子之和。该规则让重叠矩形只计一次。

函数	参数与返回	行为及前提
`CoveredLengthTree`	至少两个严格递增 y 坐标；构造对象。	建立空覆盖树。
`pull`	线段树节点区间；返回 void。	按覆盖计数和子节点摘要重算覆盖长度。
`update`（递归）	节点区间、目标间隔范围、`delta`；返回 void。	加减覆盖数；删除必须对应先前添加。
`update(y1,y2,delta)`	已压缩边界与 `+1/-1`；返回 void。	将半开 y 区间换成叶子范围后更新。
`covered`	无；返回当前覆盖 y 长度。	读取根节点摘要。
`rectangleUnionArea`	矩形列表；返回并面积。	忽略反向或退化矩形，不修改输入。

过滤后有 `m` 个矩形、至多 `2m` 个不同 y 坐标时，时间 $O(m \log m)$、空间 $O(m)$。

4. 怎么调用

将本节模板放在业务代码之前，再按下面的输入调用。

```
vector<Rectangle> rectangles = {{0, 0, 2, 2}, {1, 1, 3, 3}};
long long area = rectangleUnionArea(rectangles);
assert(area == 7);
```

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Rectangle { ll x1, y1, x2, y2; };

struct CoveredLengthTree {
    vector<ll> y;
    vector<int> cover;
    vector<ll> length;

    // 参数：ys 含至少两个严格递增坐标。
    // 返回：无（构造函数）。
    // 效果：移动参数副本到 y，初始化空覆盖树。
    explicit CoveredLengthTree(vector<ll> ys) : y(move(ys)), cover(4 * max(1, (int)y.size()) + 4, 0), length(cover.size(), 0) {
        assert(y.size() >= 2);
    }

    // 参数：p 管理间隔下标闭区间 [l,r]。
    // 返回：void。
    // 效果：根据 cover 和孩子摘要重算 length[p]。
    void pull(int p, int l, int r) {
        if (cover[p] > 0) length[p] = y[r + 1] - y[l];
        else if (l == r) length[p] = 0;
        else length[p] = length[p * 2] + length[p * 2 + 1];
    }

    // 参数：p 管理 [l,r]，目标为间隔下标 [ql,qr]，delta 为 +1 或 -1。
    // 返回：void。
    // 效果：更新覆盖计数与长度；减覆盖必须对应先前加入的同一区间。
    void update(int p, int l, int r, int ql, int qr, int delta) {
        if (ql <= l && r <= qr) { cover[p] += delta; pull(p, l, r); return; }
        int m = (l + r) / 2;
        if (ql <= m) update(p * 2, l, m, ql, qr, delta);
        if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, delta);
        pull(p, l, r);
    }

    // 参数：y1<y2 为已有压缩坐标，delta 为 +1 或 -1。
    // 返回：void。
    // 效果：将 [y1,y2) 映射为间隔下标后更新树；删除须与添加配对。
    void update(ll y1, ll y2, int delta) {
        auto leftIt = lower_bound(y.begin(), y.end(), y1);
        auto rightIt = lower_bound(y.begin(), y.end(), y2);
        assert(y1 < y2 && leftIt != y.end() && rightIt != y.end() && *leftIt == y1 && *rightIt == y2);
        int l = leftIt - y.begin();
        int r = rightIt - y.begin() - 1;
        if (l <= r) update(1, 0, (int)y.size() - 2, l, r, delta);
    }

    // 参数：无。
    // 返回：当前 y 轴覆盖长度。
    // 效果：只读根摘要。
    ll covered() const { return length[1]; }
```

```
};

// 参数：rectangles 为轴对齐矩形列表。
// 返回：矩形并面积；空输入或全无效时为 0。
// 效果：不修改输入；忽略反向或退化矩形，坐标差、面积乘积及累加均须可用 ll 表示。
ll rectangleUnionArea(const vector<Rectangle>& rectangles) {
    struct Event { ll x, y1, y2; int delta; };
    vector<Event> events; vector<ll> ys;
    for (const Rectangle& rec : rectangles) {
        if (rec.x1 >= rec.x2 || rec.y1 >= rec.y2) continue;
        events.push_back({rec.x1, rec.y1, rec.y2, +1});
        events.push_back({rec.x2, rec.y1, rec.y2, -1});
        ys.push_back(rec.y1); ys.push_back(rec.y2);
    }
    if (events.empty()) return 0;
    sort(ys.begin(), ys.end()); ys.erase(unique(ys.begin(), ys.end()),
ys.end());
    // 参数：a、b 为扫描事件。
    // 返回：a.x < b.x。
    // 效果：只读事件；同 x 事件统一处理。
    sort(events.begin(), events.end(), [](const Event& a, const Event& b) {
        return a.x < b.x;
    });
    CoveredLengthTree tree(ys); ll area = 0, previousX = events.front().x;
    size_t i = 0;
    while (i < events.size()) {
        ll x = events[i].x;
        // 先用旧覆盖长度累积 [previousX,x)，再处理同 x 的所有事件。
        area += tree.covered() * (x - previousX);
        while (i < events.size() && events[i].x == x) {
            tree.update(events[i].y1, events[i].y2, events[i].delta); ++i;
        }
        previousX = x;
    }
    return area;
}
```

5. 要改哪里

- 若要计算矩形并周长，单靠根覆盖长度不够，还需维护覆盖段数及左右端覆盖状态，且同一 x 的加入/删除要按周长规则处理。
- 要求输出覆盖次数至少 k 的长度时，把节点摘要从 `cover>0` 的长度扩展为覆盖层数分布。
- 坐标或面积可能超出 `long long` 时，统一将差、乘积、面积改为更宽类型；只改最终返回类型不够。

6. 自检

矩形 `[0,2)×[0,2)` 与 `[1,3)×[1,3)` 的并面积为 7；单个矩形面积为宽乘高；空输入和全部反向或退化矩形返回 0。

7. 坑

坐标可为负数，输入矩形用半开边界；反向或宽、高为零的矩形不贡献面积。每个 x 差、y 覆盖跨度、`coveredY*(x-previousX)` 中间乘积及面积累加都必须能放入 `long long`，否则相关变量和计算均需改用更宽整数。

- 叶子代表坐标间隔 `[ys[i],ys[i+1])`，不是坐标点；更新右端点映射到 `index(y2)-1`。
- 面积必须用到达 x 前的覆盖长度乘 `x-previousX`，再处理该 x 的事件。
- 同 x 的事件顺序不影响面积，但不得把边界本身计为正面积。
- 面积乘积可能先溢出再累加；返回类型足够不等于中间乘法安全。

8. 一句话原理

按 x 切片累加面积，并在线段树上维护当前被矩形覆盖的 y 间隔总长度。

5.27 线段树 + 矩阵乘法

1. 它是做什么的

假设一个计数过程有两种状态，当前方案数为 x、y。每个位置规定一次状态转移，我们会修改某个位置的规则，并询问连续一段规则执行后的结果。

例如规则 A 将 `(x,y)` 变成 `(x+y,y)`，规则 B 将它变成 `(x,x+y)`。从 `(1,1)` 开始，先 A 后 B，依次得到 `(2,1)`、`(2,3)`。如果每次查询都逐个执行区间内的规则，就会反复做许多相同转移。

每条规则都能写成矩阵，相邻规则也能合成一个矩阵。线段树保存每一段规则的合成结果，于是查询可以复用已经合成的部分；修改一条规则时，只重算它到根的路径。

2. 变量在做什么

状态按列向量写。规则 A 的新 x 是 `1*x+1*y`，新 y 是 `0*x+1*y`，所以两行系数构成矩阵 `[[1,1],[0,1]]`；同理 B 为 `[[1,0],[1,1]]`。

位置顺序和乘法书写顺序要分清：先 A 后 B，结果是 `B*(A*v)=(B*A)*v`。因此区间 `[l,r]` 的总转移为 `M_r…M_l`，右位置的矩阵乘在左边。乘法满足结合律，但不能随意交换位置。

名称	含义
`Matrix::n`	状态维数，也是方阵的行数和列数；示例为 2。
`Matrix::a`	每一条转移的系数，运算时规范到 `[0,MATRIX_MOD)`。
`MATRIX_MOD`	计数使用的模数，当前为 1000000007。
`initial[i]`	第 i 个位置的转移矩阵；数组下标 0 占位。
`MatrixCompose`	定义左右两个区间怎样合并，返回 `right*left`。
`MatrixSegmentTree`	复用 5.15 通用线段树的矩阵树类型，支持点修改和区间查询。

单位矩阵表示“不做任何变化”，用作空区间的合并单位元。

3. 函数在做什么

沿用规则 A、B 和初始状态 `(1,1)`：

1. 两个叶子分别保存 A 和 B，表示两个位置各自的转移。
2. 父节点保存 `B*A=[[1,1],[1,2]]`。乘初始列向量，得到 `(2,3)`，与逐条执行相同。
3. 若误写为 `A*B=[[2,1],[1,1]]`，得到 `(3,2)`，答案不同，说明乘法次序不能交换。
4. 把第二条规则改成单位矩阵，只需更新第二个叶子和祖先。区间总转移变成 A，状态变成 `(2,1)`。

线段树不改变规则的先后顺序，只改变相乘的分组。结合律保证先合成左段或右段都得到相同总转移。

函数	参数与返回	行为及前提
`Matrix`	非负维数；构造零矩阵。	0 阶仅作占位，参与运算的矩阵须正阶。
`identityMatrix`	正阶数；返回单位矩阵。	供线段树空区间使用。
`normalizedMatrix`	方阵副本；返回规范化副本。	把元素调整到模数范围，不改调用方对象。
`multiplyMatrix`	同阶正方阵；返回 `left*right`。	模 `MATRIX_MOD` 计算，不修改输入。
`MatrixCompose::operator`	左右相邻区间摘要；返回合并摘要。	以 `right*left` 表示先左后右。
`matrixRange`	1-based 矩阵数组与闭区间；返回总转移。	归一化副本后建临时树，不改原数组。
`replaceAndQuery`	矩阵树、合法位置、新矩阵和查询区间；返回区间摘要。	先替换归一化后的叶子，再查询。

设状态维数为 k，一次矩阵乘法需 $O(k^3)$；建树 $O(nk^3)$，点改和区间查询 $O(k^3 \log n)$，空间 $O(nk^2)$。若每次使用 `matrixRange`，会重新建一棵临时树，单次总成本为 $O(nk^3 + k^3 \log n)$；多次修改查询时应像第 4 部分一样持有 `MatrixSegmentTree` 对象。逐个应用一个向量需 $O((r-l+1)k^2)$，逐项构造完整区间矩阵需 $O((r-l+1)k^3)$。

4. 怎么调用

将本节模板放在业务代码之前，再按下面的输入调用。

```
Matrix a(2), b(2);
a.a = {{1, 1}, {0, 1}};
b.a = {{1, 0}, {1, 1}};
vector<Matrix> initial = {Matrix(2), a, b};
MatrixSegmentTree tree(initial, identityMatrix(2), MatrixCompose{});
Matrix product = tree.rangeQuery(1, 2); // b*a
assert((product.a == vector<vector<ll>>{{1, 1}, {1, 2}}));
tree.pointSet(2, identityMatrix(2));
assert(tree.rangeQuery(1, 2).a == a.a);
```

```
// 依赖：先放入 5.15 节的 PointSegmentTree<T, Merge> 定义。
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
constexpr ll MATRIX_MOD = 1000000007;

struct Matrix {
    int n;
    vector<vector<ll>> a;
    // 参数：n 为非负阶数；0 只用于占位。
    // 返回：无（构造函数）。
    // 效果：分配 n×n 零矩阵。
    explicit Matrix(int n = 0) : n(n), a(n, vector<ll>(n, 0)) {}
};

// 参数：n 为正矩阵阶数。
// 返回：n 阶单位矩阵。
// 效果：只创建局部结果。
Matrix identityMatrix(int n) {
    assert(n > 0);
    Matrix result(n);
    for (int i = 0; i < n; ++i) result.a[i][i] = 1;
    return result;
}

// 参数：matrix 为正阶方阵副本。
// 返回：元素规范到 [0,MATRIX_MOD) 的矩阵。
// 效果：只修改参数副本。
Matrix normalizedMatrix(Matrix matrix) {
    assert(matrix.n > 0);
    for (auto& row : matrix.a) for (ll& value : row) {
        value %= MATRIX_MOD;
        if (value < 0) value += MATRIX_MOD;
    }
    return matrix;
}

// 参数：left、right 同阶且正阶，元素已按模数规范。
// 返回：模 MATRIX_MOD 的 left*right。
// 效果：不修改输入；返回新矩阵。
Matrix multiplyMatrix(const Matrix& left, const Matrix& right) {
    assert(left.n == right.n && left.n > 0);
    Matrix result(left.n);
    for (int i = 0; i < left.n; ++i)
```

```cpp
        for (int k = 0; k < left.n; ++k) if (left.a[i][k])
            for (int j = 0; j < left.n; ++j)
                result.a[i][j] = (result.a[i][j] + left.a[i][k] * right.a[k][j]) % MATRIX_MOD;
    return result;
}

struct MatrixCompose {
    // 参数：left、right 为相邻区间同阶矩阵摘要。
    // 返回：right*left。
    // 效果：只读输入；对应从左到右作用于列向量。
    Matrix operator()(const Matrix& left, const Matrix& right) const {
        return multiplyMatrix(right, left);
    }
};

// 矩阵线段树类型：复用 5.15 的 PointSegmentTree，合并顺序由 MatrixCompose 指定。
using MatrixSegmentTree = PointSegmentTree<Matrix, MatrixCompose>;

// 参数：initial[1..n] 为同阶正阶矩阵，1<=l<=r<=n。
// 返回：区间总转移 M_r…M_l。
// 效果：归一化全部矩阵副本，再建立临时树查询；不修改 initial。
Matrix matrixRange(const vector<Matrix>& initial, int l, int r) {
    assert(l >= 1 && l <= r && r < (int)initial.size());
    vector<Matrix> normalized(initial.size());
    for (int i = 1; i < (int)initial.size(); ++i) normalized[i] = normalizedMatrix(initial[i]);
    MatrixSegmentTree tree(normalized, identityMatrix(normalized[1].n), MatrixCompose{});
    return tree.rangeQuery(l, r);
}

// 参数：tree 为矩阵树，pos 为合法下标，updated 同阶，[l,r] 为合法闭区间。
// 返回：替换后的区间总转移。
// 效果：归一化 updated 后修改 tree 的对应叶子并查询。
Matrix replaceAndQuery(MatrixSegmentTree& tree, int pos, Matrix updated, int l, int r) {
    tree.pointSet(pos, normalizedMatrix(move(updated)));
    return tree.rangeQuery(l, r);
}
```

5. 要改哪里

- 若区间转移次序相反，应同步改 `MatrixCompose` 的乘法顺序，并用两个不交换矩阵验证。
- 若更新字符或状态，替换对应叶矩阵后查询受影响区间；不必重建整个树。
- 若改模数，更新 `MATRIX_MOD` 并确认乘法中间值仍不溢出；当前 `long long` 乘积要求模数范围受控。

6. 自检

用两个不交换矩阵 `A,B` 检查区间 `[1,2]` 结果是 `B*A`，不是 `A*B`；单元素区间应返回该叶矩阵，替换一个叶后其余区间查询不变。

7. 坑

- 矩阵乘法通常不交换；本代码有意组合为 `right*left`，不可随手交换。
- `PointSegmentTree` 是 5.15 的依赖，单独编译此片段前需合并依赖定义。
- 传入矩阵维数必须一致且元素先归一化；构造时 `initial[1]` 必须存在。
- 模乘中间乘积需能容纳 `MATRIX_MOD` 下两个元素的乘积。

8. 一句话原理

矩阵乘法满足结合律，因此可将一段连续状态转移按顺序合并成线段树摘要。

5.28 线段树上二分

1. 它是做什么的

数组 `[2,-4,7,1,7]` 中，问位置 2 到 5 第一个不小于 7 的数在哪。答案是位置 3。

逐个检查时先看位置 2，再看位置 3；重复查询会重复扫描。线段树把一段数组汇总成“这一段的最大值”。如果整段最大值都不到 7，就能一次跳过整段；否则继续找左边可能有答案的部分。

数组不要求有序，找答案靠的是每个节点能否证明整段不合格。找到位置后即可停止。

2. 变量在做什么

`tree[p]` 是一段区间的最大值，回答“这段有没有机会达到阈值”；它不是原数组中的某个固定位置。`ql,qr` 是要查的下标范围，`need` 是数值门槛，两者不能混为一谈。

名称	含义
`n`	初始 1-based 数组长度。
`tree[p]`	节点管理区间的最大值；若小于 `need`，该段不可能有答案。
`initial`	构造时传入的 1-based 初值，复制到树中。
`ql,qr,need`	搜索区间闭端点和目标下界。
`pos,value`	`pointSet` 的 1-based 修改位置和值。

3. 函数在做什么

沿用数组 `[2,-4,7,1,7]`，查询 `[2,5]`、门槛 7：

1. 根节点管 `[1,5]`，与查询相交，最大值 7，不能剪掉。
2. 看左半 `[1,3]`，它和查询只交于 `[2,3]`；继续看 `[1,2]`，该节点保存的整段最大值为 2，不足 7，可以直接跳过；不需要临时计算交集摘要。
3. 检查 `[3,3]`，值为 7，返回下标 3；右侧无需再看。

若查询改成 `[4,5]`，`[4,4]` 最大值为 1，可剪掉，再到 `[5,5]` 返回 5。这些节点访问说明最左优先和整段剪枝如何配合。

函数	参数与返回	行为及前提
`FirstAtLeastTree`	initial 为非空 1-based 初值数组；返回无（构造函数）。	建立区间最大值树；不修改 initial。
`build(p,l,r,initial)`	p 管理闭区间 [l,r]，initial 为初值；返回 void。	递归写入 tree 的最大值摘要。
`pointSet`	p 管理 [l,r]，pos 位于该区间，value 为新值；返回 void。	修改叶子并重算祖先最大值。
`search(p,l,r,ql,qr,need)`	p 管理 [l,r]，目标为闭区间 [ql,qr]，need 为阈值；返回区间内首个值至少为 need 的下标；没有则 –1。	只读树；先剪枝再搜索左孩子。
`pointSet(pos,value)`	pos 为 1..n 下标，value 为新值；返回 void。	从根调用点赋值辅助函数。
`firstAtLeast(l,r,need)`	1<=l<=r<=n，need 为阈值；返回[l,r] 内首个值至少为 need 的位置；没有则 –1。	只读结构；从根执行搜索。

建树 $O(n)$，单点修改和一次搜索均为 $O(\log n)$，空间 $O(n)$。搜索先跳过不相交节点，再剪去最大值不足的整段；在找到的第一个合格节点中，只沿最左答案的路径下降。查询边界分解成的节点数和下降的树高都为对数级，因此不会遍历所有合格叶子。

4. 怎么调用

将本节模板放在业务代码之前，再按下面的输入调用。

```
vector<long long> initial = {0, 1, 2, 3, 4, 5};
FirstAtLeastTree tree(initial);
tree.pointSet(3, 8);
int pos = tree.firstAtLeast(2, 5, 7);
assert(pos == 3);
```

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct FirstAtLeastTree {
    int n;
    vector<ll> tree;

    // 参数: initial 为非空 1-based 初值数组。
    // 返回: 无（构造函数）。
    // 效果: 建立区间最大值树; 不修改 initial。
    explicit FirstAtLeastTree(const vector<ll>& initial) : n((int)initial.size() - 1), tree(4 * n + 4, LLONG_MIN) {
        assert(n >= 1); build(1, 1, n, initial);
    }

    // 参数: p 管理闭区间 [l,r], initial 为初值。
    // 返回: void。
    // 效果: 递归写入 tree 的最大值摘要。
    void build(int p, int l, int r, const vector<ll>& initial) {
        if (l == r) { tree[p] = initial[l]; return; }
        int m = (l + r) / 2;
        build(p * 2, l, m, initial); build(p * 2 + 1, m + 1, r, initial);
        tree[p] = max(tree[p * 2], tree[p * 2 + 1]);
    }

    // 参数: p 管理 [l,r], pos 位于该区间, value 为新值。
    // 返回: void。
    // 效果: 修改叶子并重算祖先最大值。
    void pointSet(int p, int l, int r, int pos, ll value) {
        if (l == r) { tree[p] = value; return; }
        int m = (l + r) / 2;
        if (pos <= m) pointSet(p * 2, l, m, pos, value);
        else pointSet(p * 2 + 1, m + 1, r, pos, value);
        tree[p] = max(tree[p * 2], tree[p * 2 + 1]);
    }

    // 参数: p 管理 [l,r], 目标为闭区间 [ql,qr], need 为阈值。
    // 返回: 区间内首个值至少为 need 的下标; 没有则 -1。
    // 效果: 只读树; 先剪枝再搜索左孩子。
    int search(int p, int l, int r, int ql, int qr, ll need) const {
        if (r < ql || qr < l || tree[p] < need) return -1;
        if (l == r) return l;
        int m = (l + r) / 2;
        int left = search(p * 2, l, m, ql, qr, need);
        return left != -1 ? left : search(p * 2 + 1, m + 1, r, ql, qr, need);
    }

    // 参数: pos 为 1..n 下标, value 为新值。
    // 返回: void。
    // 效果: 从根调用点赋值辅助函数。
    void pointSet(int pos, ll value) { assert(1 <= pos && pos <= n); pointSet(1, 1, n, pos, value); }

    // 参数: 1<=l<=r<=n, need 为阈值。
    // 返回: [l,r] 内首个值至少为 need 的位置; 没有则 -1。
    // 效果: 只读结构; 从根执行搜索。
```

```
    int firstAtLeast(int l, int r, ll need) const {
        assert(1 <= l && l <= r && r <= n);
        return search(1, 1, n, l, r, need);
    }
};
```

5. 要改哪里

- 若查“最后一个满足条件的位置”，改为先搜右儿子再搜左儿子。
- 若摘要为最小值，寻找首个 `a[i]<=x` 时将剪枝条件改为区间最小值仍大于 x。
- 若用区间和找首个前缀达到阈值，必须保证元素非负；有负数时前缀不单调，不能安全剪枝。

6. 自检

初值 `{0,2,-4,7,1,7}` 查询 `[2,5]`、need=7 应返回 3；改位置 3 为 0 后应返回 5；need=10 无解返回 −1。

7. 坑

- 搜索必须先检查当前节点最大值是否小于 need；不满足时下探会失去剪枝收益。
- 先搜左子树才能保证返回最小下标。
- `tree` 以 `LLONG_MIN` 初始化，构造器要求非空数组。
- 区间求和只有在非负元素保证前缀单调时才能做同类搜索。

8. 一句话原理

若一个节点的区间最大值仍小于阈值，节点内所有位置都不可能满足条件，可以整段跳过。

5.29 倍增 + 单调栈

1. 它是做什么的

数组 `[2,1,4,3,5]` 从位置 1 出发，每次跳到右边第一个更大的数：位置 1 跳到 3，再跳到 5。跳两次的答案是位置 5。

每次从当前点向右重新找，会在许多查询中反复扫描相同位置。先一次性算出每个位置的一步去处，再把两步、四步等结果预先接起来，就能快速走长链。

数组固定不变；这里的“更大”指严格大于。

2. 变量在做什么

`up[0][i]` 是位置 `i` 的下一个更大元素位置；沿着它反复跳构成一条后继链。它只保留“每一步的第一个更大”，不是把所有更大元素按大小排名，也不是区间第 k 大。

`up[j][i]` 表示从 `i` 沿这条链跳 `2^j` 次的位置。`mono` 是建第一层后继时保留的候选下标栈，`k` 是链上跳数而非数值门槛。

名称	含义
`n`	1-based 数组长度。
`log`	倍增层数，表下标为 `0..log-1`；链上最多有 `n-1` 次有效跳转。
`up[j][i]`	从 i 沿 next-greater 链跳 `2^j` 次的位置；0 表示链已断。
`mono`	从右往左扫描时的单调栈，保存可能成为 next greater 的下标。
`start,k,right`	起点、正跳数和查询右边界；`k` 为 `long long`。

3. 函数在做什么

对 `[2,1,4,3,5]` 从右向左扫描，栈内写下标：

当前位置	弹出不大于当前值的候选	一步后继	入栈后的下标栈
5，值 5	无	0，无后继	`[5]`
4，值 3	无	5	`[5,4]`
3，值 4	弹出 4，因其值 3 不大于 4	5	`[5,3]`
2，值 1	无	3	`[5,3,2]`
1，值 2	弹出 2，因其值 1 不大于 2	3	`[5,3,1]`

被弹出的候选更远且不更大；对将来位于更左侧的位置而言，当前元素至少与它一样有用，所以可以永久淘汰。栈顶因此给出最近的严格更大位置。

从位置 1 出发，一步到 3，再一步到 5。倍增表直接保存 `up[1][1]=5`，跳两次只查这一项。若跳数为 5，二进制为 `4+1`，就组合四步和一步的结果；途中遇到哨兵 0 表示链已经结束。

重复值要从候选栈弹掉，因条件是严格更大。倍增关系为 `up[j][i]=up[j-1][up[j-1][i]]`，把跳数拆成二进制位依次走。

函数	参数与返回	行为及前提
`NextGreaterDoubling(a)`	a 为非空 1-based 数组；返回无（构造函数）。	预处理严格更大后继和倍增表；不修改 a。
`kthGreater(start,k)`	start 为起点，k 为后继链上的跳数；返回跳 k 次后的下标；非法起点、k<=0 或不存在时为 0。	只读倍增表；并非区间第 k 大。
`firstGreaterWithin`	start 为起点，right 为合法数组右界；返回[start+1,right] 首个严格更大位置；不存在或起点非法则 0。	O(1) 读取一步后继，不修改状态。

单调栈中每个位置至多入栈、出栈一次，第一层为 $O(n)$；倍增表建表时间和空间都是 $O(n \log n)$。查询只沿二进制跳数所需的层走，时间 $O(\log(k+1))$；有效链长最多 `n-1`。

4. 怎么调用

将本节模板放在业务代码之前，再按下面的输入调用。

```
vector<long long> a = {0, 2, 1, 4, 3, 5};
NextGreaterDoubling jumps(a);
int first = jumps.firstGreaterWithin(1, 4);
int second = jumps.kthGreater(1, 2);
assert(first == 3 && second == 5);
```

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct NextGreaterDoubling {
    int n, log;
    vector<vector<int>> up;

    // 参数：a 为非空 1-based 数组。
    // 返回：无（构造函数）。
    // 效果：预处理严格更大后继和倍增表；不修改 a。
    explicit NextGreaterDoubling(const vector<ll>& a) : n((int)a.size() -
1), log(1) {
        assert(n >= 1);
        while ((1LL << log) <= n) ++log;
        up.assign(log, vector<int>(n + 1, 0));
        vector<int> mono;
        for (int i = n; i >= 1; --i) {
            while (!mono.empty() && a[mono.back()] <= a[i]) mono.pop_back();
            up[0][i] = mono.empty() ? 0 : mono.back();
```

```
            mono.push_back(i);
        }
        for (int j = 1; j < log; ++j)
            for (int i = 1; i <= n; ++i)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }

    // 参数：start 为起点，k 为后继链上的跳数。
    // 返回：跳 k 次后的下标；非法起点、k<=0 或不存在时为 0。
    // 效果：只读倍增表；并非区间第 k 大。
    int kthGreater(int start, ll k) const {
        if (start < 1 || start > n || k <= 0) return 0;
        int node = start;
        for (int bit = 0; k > 0 && node != 0; ++bit, k >>= 1)
            if (k & 1) node = bit < log ? up[bit][node] : 0;
        return node;
    }

    // 参数：start 为起点，right 为合法数组右界。
    // 返回：[start+1,right] 首个严格更大位置；不存在或起点非法则 0。
    // 效果：O(1) 读取一步后继，不修改状态。
    int firstGreaterWithin(int start, int right) const {
        if (start < 1 || start > n || right <= start) return 0;
        int next = up[0][start];
        return next != 0 && next <= right ? next : 0;
    }
};
```

5. 要改哪里

- 要求“下一个大于等于”时，构栈弹出条件需从 `<=` 改为 `<`，并验证重复值链。
- 若只需单次 next greater 查询，不必建倍增表，单调栈的 `up[0]` 已足够。
- 若问独立阈值 x 在 `[l,r]` 的首个满足位置，本节按跳数查询的接口不能直接回答；可使用 5.28 的区间最大值剪枝，或另设计按值判断的后继链倍增查询。

6. 自检

数组 `{0,2,1,4,3,5}` 中 `firstGreaterWithin(1,4)==3`，`kthGreater(1,2)==5`；严格关系下从值 4 不会跳到相等值 4。

7. 坑

- 严格更大使用弹栈条件 `<=`；改成 `<` 后语义会包含相等值。
- `up[0][i]` 是下一个更大位置，不是更大值；哨兵 0 不能当数组下标访问。
- `kthGreater` 沿后继链跳 k 次，不等同于区间第 k 大。
- `firstGreaterWithin` 的 `right` 必须是有效数组边界。

8. 一句话原理

单调栈找出每个位置的一步更大后继，倍增表将后继链上的多步跳跃拆成二进制块。

5.30 可撤销并查集

1. 它是做什么的

四个点开始时互不连通。先记下此时的状态，再合并 `1-2`、`2-3`，询问 1 和 3 是否连通，答案是“是”。回到刚才记录的状态，再问则是“否”。

普通并查集擅长合并，却不直接支持恢复先前状态。可撤销版本记录每次合并改了哪些字段，回退时倒着恢复。

这里的快照像书签，只能沿当前历史向过去回滚；它不会自动保存所有历史版本。

2. 变量在做什么

`parent[x]` 指向父节点，根的父节点是自己；`size[root]` 只记录根所代表集合的大小。一次合并成功会让一个根成为另一个根的孩子。`history` 记录这次变化，`components` 是当前集合数。

快照 `snap` 是历史栈长度，不是连通分量数；它只在同一对象当前仍保留的历史分支上有效；回滚后产生新分支，不能凭长度相同就把旧分支的书签重新当作有效状态。即使两个点已连通，重复 `unite` 也压入一条标记为空操作的历史，保证每次尝试都能对应一次回退。

名称	含义
`parent[x]`	并查集父指针；根满足 `parent[x]==x`。
`size[root]`	根所代表集合的大小，只对根有意义。
`history`	每次 `unite` 一条记录；成功记录旧根大小，失败记录 `merged=false`。
`components`	当前连通分量数。
`Change`	`child`、被挂接的 `parentRoot`、合并前 `oldSize` 及是否成功。

3. 函数在做什么

取 4 个点，初始分量数为 4：

操作	history 长度	`components`	连通关系
`snapshot()`	0	4	四个单点集合
`unite(1,2)`	1	3	1 与 2 连通
`unite(2,3)`	2	2	1、2、3 连通
`rollback(0)`	0	4	恢复四个单点集合

为了让每次合并只改少量字段，本模板不做路径压缩，只按集合大小挂接根。回滚逐条弹出历史，恢复被挂接根的父指针和接收根的旧大小，同时恢复分量数；重复合并的空记录只弹出，不改集合状态。`find` 只沿父链向上走，不做路径压缩。

函数	参数与返回	行为及前提
`RollbackDSU(n)`	n>=1 为点数，编号 1..n；返回无（构造函数）。	建立 n 个单点集合，清空历史。
`find(x)`	x 为 1..n 元素编号；返回所属集合根。	只读父链，不进行路径压缩。
`snapshot()`	无；返回当前 history 长度，作为回滚快照。	只读历史；长度必须可由 int 表示。
`unite(u,v)`	u、v 为合法元素编号；返回两个不同集合合并成功时 true，已同组时 false。	按大小合并；每次尝试均压入一条历史，成功时修改父指针、大小和分量数。
`rollback(snap)`	snap 为当前历史分支上仍有效的较早或当前快照；返回 void。	弹出历史至 snap，恢复父指针、大小与分量数；不能恢复已丢弃的未来分支。
`same(u,v)`	u、v 为合法元素编号；返回两点当前是否连通。	只读父链和历史。

按大小合并使树高为 $O(\log n)$，因此 `find` 和成功合并为 $O(\log n)$，取快照为 $O(1)$。回滚是每条弹出记录 $O(1)$，总耗时按本次弹出记录数计算；若累计尝试 q 次合并，每次最多保留一条记录，空间为 $O(n+q)$。

4. 怎么调用

将本节模板放在业务代码之前，再按下面的输入调用。

```
RollbackDSU dsu(4);
```

```
int snap = dsu.snapshot();
dsu.unite(1, 2);
bool connected = dsu.same(1, 2);
assert(connected);
dsu.rollback(snap);
assert(!dsu.same(1, 2));
```

```
#include <bits/stdc++.h>
using namespace std;

struct RollbackDSU {
    struct Change { int child, parentRoot, oldSize; bool merged; };
    vector<int> parent, size;
    vector<Change> history;
    int components;

    // 参数：n>=1 为点数，编号 1..n。
    // 返回：无（构造函数）。
    // 效果：建立 n 个单点集合，清空历史。
    explicit RollbackDSU(int n) : parent(n + 1), size(n + 1, 1),
components(n) {
        assert(n >= 1);
        iota(parent.begin(), parent.end(), 0);
    }

    // 参数：x 为 1..n 元素编号。
    // 返回：所属集合根。
    // 效果：只读父链，不进行路径压缩。
    int find(int x) const {
        while (parent[x] != x) x = parent[x];
        return x;
    }

    // 参数：无。
    // 返回：当前 history 长度，作为回滚快照。
    // 效果：只读历史；长度必须可由 int 表示。
    int snapshot() const { return (int)history.size(); }

    // 参数：u、v 为合法元素编号。
    // 返回：两个不同集合合并成功时 true，已同组时 false。
    // 效果：按大小合并；每次尝试均压入一条历史，成功时修改父指针、大小和分量数。
    bool unite(int u, int v) {
        int ru = find(u), rv = find(v);
        if (ru == rv) { history.push_back({0, 0, 0, false}); return false; }
        if (size[ru] < size[rv]) swap(ru, rv);
        history.push_back({rv, ru, size[ru], true});
        parent[rv] = ru; size[ru] += size[rv]; --components;
        return true;
    }

    // 参数：snap 为当前历史分支上仍有效的较早或当前快照。
    // 返回：void。
    // 效果：弹出历史至 snap，恢复父指针、大小与分量数；不能恢复已丢弃的未来分支。
    void rollback(int snap) {
        assert(0 <= snap && snap <= (int)history.size());
        while ((int)history.size() > snap) {
            Change change = history.back(); history.pop_back();
            if (!change.merged) continue;
            parent[change.child] = change.child;
            size[change.parentRoot] = change.oldSize;
            ++components;
        }
    }

```

```
57      // 参数：u、v 为合法元素编号。
58      // 返回：两点当前是否连通。
59      // 效果：只读父链和历史。
60      bool same(int u, int v) const { return find(u) == find(v); }
61  };
```

5. 要改哪里

- 若上层需要每次尝试都能按次数回滚，保留失败合并的空记录；删除该记录时，上层快照仍可用，但快照语义变为历史修改数而非尝试数。
- 若要维护额外集合摘要（如点权和），需在成功合并记录旧摘要并在 rollback 中恢复。
- 需要在线路径压缩并查集时，不适合本结构；路径压缩会产生多处需记录的父指针变化。

6. 自检

4 个点先取快照，合并 1-2、2-3 后 `same(1,3)` 为真且分量数为 2；回滚快照后该查询为假、分量数为 4。重复合并也应能完整回滚。

7. 坑

- 不能做路径压缩，否则 `rollback` 没有记录那些被压缩的父指针。
- 快照是 history 长度，必须来自同一对象且不得大于当前长度。
- 快照后的修改必须按后进先出回滚；不能保留后续状态再回滚到更早快照。
- `find/same` 仅在点编号合法时安全；代码不会逐点检查范围。

8. 一句话原理

只按大小合并且不压缩路径，使每次合并只需记录少量字段便能按历史逆序恢复。

5.31 线段树分治（动态连通性）

1. 它是做什么的

有四个时刻：加边 `1-2`、询问连通、删边 `1-2`、再询问。答案分别是“是”和“否”。

如果每个时刻都从头重建图，会重复处理仍然有效的边。离线时可以先找出每条边在哪些时间段存在，再将它们放到时间线的线段树上；沿树走到一个时刻时，只需合并覆盖它的边。

离开一个时间段就撤销该段引入的边，这样兄弟时间段不会互相影响。

2. 变量在做什么

`operations[t]` 是时刻 `t` 的输入；`tree[p]` 则存放整段时间都活跃、可由节点 `p` 一并加入的边。时间树的区间按半开形式 `[l,r)` 表示，`starts` 为每条规范化无向边保存尚未删除的添加时刻。

`answers` 只收集问号操作的结果，`dsu` 是 5.30 的可撤销并查集。边实例按 LIFO 匹配：重复添加会各自活跃，删除关闭最近一次仍未关闭的实例。

名称	含义
`Operation`	单个时间点操作：`type` 为 `A`、`D` 或 `?`，`u,v` 是 1..n 端点。
`operations`	按时间下标 0..q-1 保存的输入操作。
`tree[p]`	时间线线段树节点所负责时段内始终活跃的边。
`answers`	依查询出现顺序写入的 0/1 连通结果；每次 `run()` 前清空。
`dsu`	依赖 5.30 的无路径压缩可撤销并查集。
`starts`	构造阶段按规范化无向边保存未删除添加时刻的栈。

3. 函数在做什么

操作序列的时刻表如下：

时刻	操作	查询结果	操作后边是否活跃
0	`A(1,2)`	—	是
1	`?(1,2)`	连通	是
2	`D(1,2)`	—	否
3	`?(1,2)`	不连通	否

边在添加时刻开始生效、删除时刻不再生效，因此活跃区间恰为 `[0,2)`。时间树根管 `[0,4)`，其左孩子正好覆盖 `[0,2)`，整条边只需放在该节点；它不会进入右孩子 `[2,4)`。

DFS 进入 `[0,2)` 时先取快照，再合并节点上的边，访问时刻 0 和 1 两个叶子；离开该节点时回滚到快照，恢复无边状态。于是两次查询答案为 `{1,0}`。

函数	参数与返回	行为及前提
构造函数	n>=1；operations 按时间给出 A/D/?，端点为 1..n；返回无（构造函数）。	保存操作副本，按 LIFO 配对重复边并构造半开活跃时间段。
`normalized(u,v)`	u、v 为无向边端点；返回 (min(u,v),max(u,v))。	不修改参数或对象。
`addInterval`	p 管理 [l,r)，边 e 活跃于 [ql,qr)；返回 void。	将 e 存入被活跃时间段完全覆盖的节点；不相交时返回。
`buildIntervals()`	无；返回 void。	仅构造时调用；按 LIFO 配对 A/D 写入时间树，未匹配删除忽略。
`solve(p,l,r)`	p 管理非空时间段 [l,r)；返回 void。	临时合并节点边，叶子回答查询，退出前回滚至进入时快照。
`run()`	无；返回按时间顺序排列的 0/1 连通结果。	清空答案并从空边 DSU 状态重跑；支持重复调用，q=0 返回空向量。

若有 `q` 个操作和 `s` 个活跃区间，每个区间分解进 $O(\log(q+1))$ 个节点；配对 map 扫描 $O(q\log(q+1))$。总时间安全上界 $O(q\log(q+1)+s\log(q+1)\log(n+1)+q\log(n+1))$，空间 $O(q+s\log(q+1)+n)$。合并因按大小而为对数代价，回滚按实际弹出的历史记录计费。

4. 怎么调用

将本节模板放在业务代码之前，再按下面的输入调用。

```
vector<Operation> ops = {{'A', 1, 2}, {'?', 1, 2}, {'D', 1, 2}};
OfflineDynamicConnectivity solver(3, ops);
vector<int> answers = solver.run();
assert(answers.size() == 1 && answers[0] == 1);
```

```
// 依赖：先放入 5.30 节的 RollbackDSU；本节不重复定义它。
#include <bits/stdc++.h>
using namespace std;

struct Operation { char type; int u, v; };

struct OfflineDynamicConnectivity {
    struct Edge { int u, v; };
    int n, q;
    vector<Operation> operations;
    vector<vector<Edge>> tree;
    vector<int> answers;
    RollbackDSU dsu;

    // 参数：n>=1；operations 按时间给出 A/D/?，端点为 1..n。
    // 返回：无（构造函数）。
    // 效果：保存操作副本，按 LIFO 配对重复边并构造半开活跃时间段。
    OfflineDynamicConnectivity(int n, vector<Operation> operations)
```

```
19          : n(n), q((int)operations.size()), operations(move(operations)),
    tree(4 * max(1, q) + 4), dsu(n) {
20          assert(n >= 1);
21          buildIntervals();
22      }
23
24      // 参数：u、v 为无向边端点。
25      // 返回：(min(u,v),max(u,v))。
26      // 效果：不修改参数或对象。
27      pair<int, int> normalized(int u, int v) const { return {min(u, v),
    max(u, v)}; }
28
29      // 参数：p 管理 [l,r)，边 e 活跃于 [ql,qr)。
30      // 返回：void。
31      // 效果：将 e 存入被活跃时间段完全覆盖的节点；不相交时返回。
32      void addInterval(int p, int l, int r, int ql, int qr, Edge e) {
33          if (qr <= l || r <= ql) return;
34          if (ql <= l && r <= qr) { tree[p].push_back(e); return; }
35          int m = (l + r) / 2;
36          addInterval(p * 2, l, m, ql, qr, e);
37          addInterval(p * 2 + 1, m, r, ql, qr, e);
38      }
39
40      // 参数：无。
41      // 返回：void。
42      // 效果：仅构造时调用；按 LIFO 配对 A/D 写入时间树，未匹配删除忽略。
43      void buildIntervals() {
44          if (q == 0) return;
45          map<pair<int, int>, vector<int>> starts;
46          for (int t = 0; t < q; ++t) {
47              if (operations[t].type == 'A') {
48                  starts[normalized(operations[t].u,
    operations[t].v)].push_back(t);
49              } else if (operations[t].type == 'D') {
50                  auto key = normalized(operations[t].u, operations[t].v);
51                  auto it = starts.find(key);
52                  if (it != starts.end() && !it->second.empty()) {
53                      int begin = it->second.back(); it->second.pop_back();
54                      addInterval(1, 0, q, begin, t, {key.first, key.second});
55                  }
56              }
57          }
58          for (const auto& item : starts) for (int begin : item.second)
59              addInterval(1, 0, q, begin, q, {item.first.first,
    item.first.second});
60      }
61
62      // 参数：p 管理非空时间段 [l,r)。
63      // 返回：void。
64      // 效果：临时合并节点边，叶子回答查询，退出前回滚至进入时快照。
65      void solve(int p, int l, int r) {
66          int snap = dsu.snapshot();
67          for (const Edge& edge : tree[p]) dsu.unite(edge.u, edge.v);
68          if (r - l == 1) {
69              const Operation& op = operations[l];
70              if (op.type == '?') answers.push_back(dsu.same(op.u, op.v));
71          } else {
72              int m = (l + r) / 2;
73              solve(p * 2, l, m); solve(p * 2 + 1, m, r);
74          }
75          dsu.rollback(snap);
76      }
77
78      // 参数：无。
```

```
79      // 返回：按时间顺序排列的 0/1 连通结果。
80      // 效果：清空答案并从空边 DSU 状态重跑；支持重复调用，q=0 返回空向量。
81      vector<int> run() {
82          answers.clear(); dsu.rollback(0);
83          if (q > 0) solve(1, 0, q);
84          return answers;
85      }
86  };
```

5. 要改哪里

- 若删除匹配最早尚未删除的添加实例，把每条边的起点容器从栈改为队列，并按 FIFO 取出。
- 若要回答组件大小等问题，可扩展 5.30 的 DSU 查询与回滚记录；时间区间分解框架保持不变。
- 若查询要求在线回答，未来操作尚未知，无法预先构造活跃时间段，应选择在线动态连通结构。

6. 自检

操作序列 A(1,2), ?(1,2), D(1,2), ?(1,2) 的答案应为 `{1,0}`；空操作列表 `run()` 返回空向量；重复添加后删除一次，另一个实例仍活跃。

7. 坑

- 边在添加时刻生效、删除时刻失效，因此保存为半开区间 `[add,del)`；未删边为 `[add,q)`，空区间不放进树。
- 重复边按规范化端点和 LIFO 实例配对；未匹配删除被忽略。
- 类型为 `?` 的操作必须提供合法 u、v；无关字段虽不读取，也应初始化。
- `run()` 会重置答案并回滚 DSU；不要在递归 DFS 中使用带路径压缩的 DSU。
- 空操作列表不会调用 `solve`，因此时间线根上的空区间不会被访问。

8. 一句话原理

把边的活跃时间段分解到时间线线段树节点，DFS 子树期间临时合并，离开时回滚。

5.32 P2 进阶：树上莫队、Segment Tree Beats 与 LCT

本节分别介绍三个 P2 专题的模型、维护信息和实现流程，不含可直接提交的完整模板。

5.32.1 树上莫队

1. 它是做什么的

在树 `1-2, 1-3, 2-4` 上，颜色依次为 `A,B,C,A`，路径 `4→3` 上有三种颜色。

每次都沿路径重新数颜色，会在大量询问间反复走过同一批点。树上莫队把路径对应成括号序的一段，让相邻询问复用当前窗口，只对窗口变化的部分增删点。

2. 变量在做什么

普通 DFS 序中一个顶点只出现一次，树路径通常不是连续区间；括号序则让每个顶点进出各出现一次。某点在窗口中出现奇数次时保留，偶数次时抵消，选好窗口后，奇数出现的点构成要维护的集合；某些路径还需临时补入最近公共祖先。

名称	含义
`tin[u]`	点 `u` 第一次进入 DFS 时在长度 `2n` 括号序中的位置。
`tout[u]`	点 `u` 退出 DFS 时在括号序中的位置；每点一进一出。
`euler[pos]`	括号序位置 `pos` 对应的顶点编号。
`active[u]`	当前窗口奇数出现集合是否包含点 `u`；每次遇到都翻转。
`L,R`	当前维护的闭区间端点。

名称	含义
`z`	路径两端的最近公共祖先；某些区间需临时补入。
`block`	莫队排序的分块长度。

3. 函数在做什么

括号序为 `[1,2,4,4,2,3,3,1]`。询问 `4→3` 时，窗口 `[4,6]` 对应 `[4,2,3]`，奇数出现集合正是这三个点；最近公共祖先 1 不在其中，回答前临时补入 1，答案读完马上撤回。询问 `2→4` 用 `[2,3]`，窗口 `[2,4]` 已是完整路径，不补祖先。

先按 `tin` 排序端点并求 `z=LCA(u,v)`。若 `z=u`，取 `[tin[u],tin[v]]`；否则取 `[tout[u],tin[v]]` 并记下额外点 `z`。窗口移动时，每个位置都执行 toggle：若点当前不活跃则加入统计，否则删除，最后翻转活跃标记。

以下是建议的流程名，不是仓库现成 API；颜色统计函数需按题目实现。

函数	参数与返回	行为及前提
`toggle(pos)`	括号序位置；无返回。	对 `euler[pos]` 加入或删除颜色，再翻转 `active`；增删必须互逆。
`moveWindow(L,R,ql,qr)`	旧区间和目标闭区间；无返回。	移动端点并逐位置 toggle；初始状态对应旧区间。
`pathInterval(u,v)`	两个树点；返回区间和可选补点。	排序端点并求 LCA，按祖先关系选择上述区间。
`answerPath(u,v)`	两个树点；返回维护统计量。	移窗，临时补祖先、读答案后撤销。

LCA 倍增预处理 $O(n \log n)$，排序询问 $O(q \log q)$，每问求 LCA $O(\log n)$。块长约 `sqrt(2n)` 且加入/删除为 $O(1)$ 时，指针移动项为 $O((n+q)\sqrt{n})$，空间 $O(n \log n + q)$；若增删更贵，移动项还要乘其代价。

4. 怎么调用

下面是把询问转换和维护顺序写清楚的流程伪代码，不是可直接提交的实现。仓库没有本节可直接提交的树上莫队模板；`buildEulerTour`、`lca`、`addColor`、`removeColor`、`readAnswer` 均需按所选统计量实现。示例树为 `1-2,1-3,2-4`，查询 `4→3` 时区间 `[4,6]` 需补入点 `1`。

```
树：1-2，1-3，2-4；询问：path(4, 3)
(2n 括号序, tin, tout) = buildEulerTour(tree, root=1)
(u, v) = (4, 3)，若 tin[u] > tin[v] 则交换
z = lca(u, v) = 1
若 z == u：目标区间 = [tin[u], tin[v]]，extra = none
否则：目标区间 = [tout[u], tin[v]]，extra = z
按莫队顺序移动当前窗口到目标区间；每移动一格都 toggle(position)
若 extra != none：addColor(value[extra])
result = readAnswer()
若 extra != none：removeColor(value[extra])
assert 当前窗口统计状态仍只对应目标区间
```

5. 要改哪里

- 先练习手画括号序并验证窗口：覆盖祖先端点、非祖先端点、`u==v`、端点交换四种情况。
- 把普通莫队的 `add/remove` 改为对 `euler[pos]` 执行的 `toggle`，用随机小树与暴力逐点枚举路径对拍。
- 选择可增可删的路径统计量实现 `addColor/removeColor/readAnswer`；例如不同颜色数或频率平方和。若颜色有修改，再把修改时间作为第三维排序和回滚。
- 练习统计路径颜色对数时，重点校验补 LCA 后状态的恢复；树结构若动态变化，括号序和预处理均失效，应换动态树结构。参考 OI Wiki：树上莫队。

6. 自检

对树 `1-2,1-3,2-4` 和括号序 `[1,2,4,4,2,3,3,1]`：查询 `4→3` 的 `[4,6]` 窗口奇数集合应为 `{4,2,3}`，临时补入 `1` 后为 `{4,2,1,3}`；查询 `2→4` 的 `[2,3]` 窗口为 `{2,4}`，不补 LCA。完成一次带补点的回答后，`active` 必须恢复为仅表示窗口的状态。

7. 坑

- 窗口采用闭区间；非祖先情形的左端是 `tout[u]`，常见错误是误用 `tin[u]`。
- 补入 LCA 只用于读取当前答案，读取后必须撤销；否则后续询问状态被污染。
- `toggle` 必须严格按奇偶切换；把两次出现当成两次独立加入会重复计数。
- LCA 预处理和括号序针对静态树；改边后不能沿用旧序列。
- 上述复杂度依赖常数时间加入/删除及约 `sqrt n` 的块长；状态代价变化时需要重新估算。

8. 一句话原理

括号序中路径以外的顶点成对出现并抵消，树路径询问因此转化为区间奇偶集合的莫队维护。

5.32.2 Segment Tree Beats

1. 它是做什么的

数组 `[8,8,5,2]` 的总和为 23。把每项压到不超过 6 后是 `[6,6,5,2]`，总和变 19。

若每次更新都逐项扫描，大区间会反复访问很多元素。Segment Tree Beats 在条件满足时能整段压低最大值层；条件不够时才继续拆开看孩子。

2. 变量在做什么

`mx1` 是最大值，`mx2` 是严格次大值，也就是小于最大值的第二大不同值；`cnt` 是最大值出现次数。只有 `sum` 和 `mx1` 不够，因为不知道哪些元素会被压低；即使知道 `cnt`，也要知道压低后会不会碰到下一层。

名称	含义
`sum`	节点区间所有元素之和。
`mx1`	当前区间最大值。
`mx2`	当前区间严格次大值；全相等或单元素时为负无穷哨兵。
`cnt`	当前区间中等于 `mx1` 的元素个数。
`x`	本次 `chmin` 上限，元素改成 `min(a[i],x)`。
`left/right`	节点对应的数组闭区间。

3. 函数在做什么

对 `[8,8,5,2]`，初始 `(sum,mx1,mx2,cnt)=(23,8,5,2)`。上限为 6 时，只有最大值层的两个 8 改变：和减去 `(8-6)×2=4`，新和 19，最大值变 6，个数仍为 2。再压到 5，边界碰到原次大值，值变为 `[5,5,5,2]`，和为 17，最大值个数变成 3。

因此只有 `mx2 < x < mx1` 才能直接改：执行 `sum -= (mx1-x)*cnt` 并设 `mx1=x`。若 `x>=mx1` 不变；若 `x<=mx2` 则往孩子递归。向下前还要把父节点的最大值上限传给超标孩子，再重新合并摘要。

以下是标准流程接口约定，不代表仓库提供了现成 API。

函数	参数与返回	行为及前提
`applyChmin(node,x)`	节点摘要与上限；无返回。	仅在 `mx2 < x < mx1` 时直接覆盖。
`push(node)`	内部节点和孩子；无返回。	将父节点最大值约束传给最大值更大的孩子。
`pull(node)`	内部节点和孩子；无返回。	重算 `sum,mx1,mx2,cnt`，正确处理两侧最大值相等与否。

函数	参数与返回	行为及前提
`rangeChmin`	节点区间、目标区间和上限；无返回。	不相交或 `x>=mx1` 时结束；可直接覆盖时应用，否则下推递归并合并。
`rangeSum(node,l,r,ql,qr)`	节点及目标区间；返回区间和。	全覆盖取 `sum`，部分覆盖先下推再合并相交孩子。
`rangeMax`	输入查询区间；返回其中的最大值。	完全覆盖读 `mx1`，部分覆盖先下推再取相交孩子的最大值。
`rangeMaxCount`	输入查询区间；返回其中最大值的出现次数。	部分覆盖时合并“最大值、次数”二元组：最大值相同则次数相加，否则只保留较大者的次数。

标准单向区间 `chmin` 加求和/最值查询，总时间摊还 $O((n+q)\log n)$、空间 $O(n)$；这是整体界，不保证每次更新都是 $O(\log n)$。势能分析依赖这组摘要。扩展 `chmax`、区间加或历史最值会改变字段、懒标记和证明。数组须非空，哨兵不能和合法值冲突，乘积及区间和不得溢出。

4. 怎么调用

这是调用次序和节点分支的流程伪代码，不是可直接提交的实现；仓库没有本节模板。示例数组 `[8,8,5,2]`，将全区间压到 `6` 后总和应为 `19`，最大值为 `6`；再压到 `5` 后总和为 `17`，最大值个数为 `3`。

```
数组 = [8, 8, 5, 2]
build(数组)
rangeChmin([0,3], 6)
assert rangeSum([0,3]) == 19
assert rangeMax([0,3]) == 6
rangeChmin([0,3], 5)
assert rangeSum([0,3]) == 17
assert rangeMaxCount([0,3]) == 3

rangeChmin(node, ql, qr, x):
    若 node 与 [ql,qr] 不相交 或 x >= node.mx1: 返回
    若 node 完全包含于 [ql,qr] 且 node.mx2 < x: applyChmin(node,x); 返回
    push(node)
    递归更新左右孩子
    pull(node)
```

5. 要改哪里

- 入门先写逐元素暴力 `chmin` 和区间和；再用 `[8,8,5,2]` 手算为什么需要严格次大值及最大值个数。
- 实现练习从单向 `chmin + sum/max` 开始，建立节点摘要、快速覆盖、下推与合并；不要直接扩成多种修改。
- 对拍重点包含全相等、单元素、`x=mx1`、`x=mx2`、负数、反复收紧、部分覆盖与空查询边界。
- 若题目更新后某种势能量快速下降（例如区间取模），可能也能用摊还分析，但它们不是本节模板的直接变体；先证明自己的势能界。

6. 自检

对 `[8,8,5,2]`，初始摘要为 `(23,8,5,2)`；`chmin(6)` 后和为 `19`、最大值为 `6`、最大值个数为 `2`；继续 `chmin(5)` 后和为 `17`、最大值个数为 `3`。用随机小数组对拍时，应逐次与逐元素执行 `min(a[i],x)` 的暴力结果核对区间和与最大值。

7. 坑

- `mx2` 必须是严格次大值；若把最大值重复项误作次大值，快速覆盖条件会错误。
- 快速覆盖只在 `mx2 < x < mx1` 成立；`x==mx2` 必须递归，`x==mx1` 则无变化。
- 下推时父节点的最大值上限要作用于孩子，否则父摘要与孩子摘要不一致。
- 复杂度是总操作摊还界，不能解释为每个更新最坏 $O(\log n)$。
- 哨兵、数值范围与累加乘积溢出检查属于正确性前提；扩展操作后需要重新推导复杂度。

8. 一句话原理

利用最大值与严格次大值之间的间隙，在只改变最大值层时批量更新；跨过间隙就递归，并以摊还分析控制总递归量。

参考：

- OI Wiki：区间最值操作

5.32.3 Link-Cut Tree

1. 它是做什么的

森林有边 `1-2,2-3`，点权是 `[4,7,1]`，路径 `1→3` 的和为 12。删去 `2-3` 后两点断开；再加边 `1-3`，路径 `2→3` 变成 `2→1→3`，和仍为 12。

如果边会在线变化，固定的树上路径拆分会失效。Link-Cut Tree（LCT）动态组织路径信息，从而支持连边、删边与路径查询。

2. 变量在做什么

先知道 Splay 的基本作用：它通过旋转把指定节点调整到一棵二叉树的根。LCT 用这种辅助树组织路径顺序，树中孩子的左右关系不按原树点号大小决定。

原森林的边描述真实图；辅助 Splay 树的孩子关系用于组织路径，不等于原树父子关系。`fa[x]` 因此可能是辅助父指针，也可能是路径上的父关联，必须用 `isRoot` 区分。路径反转时辅助树左右孩子互换。

名称	含义
`ch[x][0], ch[x][1]`	辅助树左、右孩子。
`fa[x]`	辅助父指针或路径父关联。
`rev[x]`	延迟翻转标记。
`val[x]`	顶点自身权值。
`sum[x]`	辅助树代表路径的摘要；求和例中为路径和。
`x,y`	动态森林顶点编号。

3. 函数在做什么

对路径 `1→3`：先把 1 变成表示上的根，再暴露到 3 的路径，最后让 3 成为辅助 Splay 根。此时 3 的摘要覆盖 `1,2,3`，所以和为 `4+7+1=12`。删边 `2-3` 后查询应先发现不连通；重连 `1-3` 后再暴露 `2→3`，路径是 `2,1,3`，摘要为 12。

执行 `makeRoot(x); access(y); splay(y)` 后读取 `sum[y]`。以下是标准操作约定，不是仓库已有 API。

函数	参数与返回	行为及前提
`isRoot(x)`	顶点 x；返回布尔值。	判断 x 是否为辅助 Splay 根。
`push(x)`	节点 x；无返回。	下传翻转等延迟标记。
`pull(x)`	节点 x；无返回。	用自身权值和孩子摘要重算维护量。
`splay(x)`	节点 x；无返回。	先下传标记，再旋转使 x 成为辅助树根。
`access(x)`	顶点 x；无返回。	暴露原树根到 x 的路径。
`makeRoot(x)`	顶点 x；无返回。	暴露并翻转，使 x 成为表示上的根。
`connected(x,y)`	两点；返回是否连通。	不连通时不得执行路径聚合。
`link(x,y)`	两点；返回是否成功。	仅连接不连通点，避免成环。
`cut(x,y)`	两点；返回是否成功。	仅删除原树中直接相邻的一条边。
`pathQuery(x,y)`	同一树两点；返回路径摘要。	暴露路径并读取 `y` 的摘要。

若要增加点权修改，先 `access(x); splay(x)` 暴露目标点，再改 `val[x]` 并执行 `pull(x)`，使辅助树摘要与新点权一致。

标准 Splay 型 LCT 的 link、cut、暴露路径和查询为摊还 $O(\log n)$，空间 $O(n)$；单次操作不保证最坏 $O(\log n)$。维护虚子树贡献或非交换路径摘要还需额外状态；反转时要同步交换正、反向摘要。

4. 怎么调用

下面给出操作顺序的流程伪代码，不是可直接提交的实现；仓库没有本节现成 LCT 模板。必须先实现并验证 Splay 的旋转、`push/pull` 和连通判断。点权 `[4,7,1]`，边为 `1-2,2-3`，示例中的路径和断言对应前述手算。

```
初始化 3 个孤立点，点权分别为 [4,7,1]
assert link(1,2) == true
assert link(2,3) == true
assert connected(1,3) == true
assert pathQuery(1,3) == 12
assert cut(2,3) == true
assert connected(1,3) == false
assert link(1,3) == true
assert pathQuery(2,3) == 12
assert pathQuery(1,3) == 5

pathQuery(x,y):
    若 not connected(x,y)：返回“不连通”
    makeRoot(x)
    access(y)
    splay(y)
    返回 sum[y]
```

5. 要改哪里

- 学习顺序：先掌握 Splay 旋转、伸展、标记下传，再实现 LCT；先只维护路径异或或和，验证路径暴露后再扩展。
- 基础练习手算原树边与辅助 Splay 树边的区别；实现练习随机执行合法森林的连边、删边与路径查询，用 BFS 暴力对拍。
- 组合练习再加入路径懒标记；若维护非交换运算，要为翻转维护正向和反向摘要。
- 子树聚合或虚子树贡献需要额外维护与证明，不能把路径摘要直接挪用；任意图动态连通也不是动态森林问题。参考 OI Wiki：Link Cut Tree。

6. 自检

初始边 `1-2,2-3`、权值 `[4,7,1]` 时，`pathQuery(1,3)==12`。成功 `cut(2,3)` 后 `connected(1,3)==false`；随后 `link(1,3)`，路径 `2→1→3` 的和为 `12`，路径 `1→3` 的和为 `5`。每次 `link` 都应拒绝已经连通的端点，每次 `cut` 都应拒绝非直接相邻顶点。

7. 坑

- `fa[x]` 可能是原树路径父关联而非辅助树父子边，旋转前必须正确判断 `isRoot(x)`。
- 访问/旋转前要按从辅助根到目标的顺序下传翻转等懒标记；否则孩子方向与摘要可能不一致。
- `link` 必须先检查不连通，`cut` 必须验证目标边是直接边；否则森林不变量被破坏。
- LCT 的 $O(\log n)$ 是摊还界，不是单次最坏界；任意图动态连通与动态森林路径查询也不是同一个问题。
- 子树信息、非交换路径摘要需要额外维护；反向路径查询必须正确交换正反向摘要。

8. 一句话原理

通过 Splay 动态重排辅助路径，把原树两点间路径暴露为一棵辅助树上的连续摘要，并用摊还旋转支持森林边的在线变化。

第六章

图论

本章默认点编号为 `1..n`。无向边必须加入两个方向；权值和距离使用 `long long`。若题目允许重边，应保留全部边，或在矩阵建图时取最小权值。DFS 与 BFS 基础统一见第 3.8、3.9 节。

6.1 拓扑排序

1. 它是做什么的

对 1-based 有向图，拓扑排序返回一个排列，使每条边 `u -> v` 的 `u` 都排在 `v` 前面。适用于依赖顺序和 DAG 动态规划；若图有环则无拓扑序，函数返回空向量。邻接表存边数 `m`，运行时间为 $O(n+m)$，额外空间为 $O(n)$；连同输入图计算，存储为 $O(n+m)$。本实现用队列返回任意一个合法顺序，不保证字典序最小。

2. 变量在做什么

名称	含义
`graph[u]`	从 `u` 出发的有向边终点列表；重边应逐条保留。
`indegree[v]`	输入中进入 `v` 的边数，包含重边。调用时传入副本，函数内部递减。
`n`	`graph.size()-1`，点编号为 `1..n`，下标 0 不用。
`q`	当前剩余入度为 0、可加入结果的点。
`order`	已取出的拓扑序前缀；长度小于 `n` 表示有环。

3. 函数在做什么

函数	契约
`topologicalSort(graph, indegree)`	`graph` 是 1-based 邻接表，`indegree` 长度至少 `n+1` 且按每条边准确统计；返回含全部点的拓扑序，或有环时返回空向量。入度按值传递，不改调用者的向量；不校验端点与入度是否一致。

4. 怎么调用

建边时同时统计终点入度，再调用函数。示例边为 `1->2, 1->3, 2->3`，队列处理顺序固定为 `1,2,3`；另用自环检查环检测：

```
vector<vector<int>> graph(4);
vector<int> indegree(4, 0);
graph[1].push_back(2); ++indegree[2];
graph[1].push_back(3); ++indegree[3];
graph[2].push_back(3); ++indegree[3];
vector<int> order = topologicalSort(graph, indegree);
assert(order == vector<int>({1, 2, 3}));

vector<vector<int>> cyclic(2);
cyclic[1].push_back(1);
vector<int> cyclicIndegree = {0, 1};
assert(topologicalSort(cyclic, cyclicIndegree).empty());
```

模板如下：

```

```

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数 graph 为 1-based 有向邻接表，indegree 是逐边统计的入度副本；返回拓扑序，有环返回空表。
vector<int> topologicalSort(
    const vector<vector<int>>& graph,
    vector<int> indegree
) {
    // graph[u] 只保存有向出边 u->v；indegree 必须逐条统计所有入边（含重边）。
    int n = (int)graph.size() - 1;
    queue<int> q;
    for (int u = 1; u <= n; ++u) {
        if (indegree[u] == 0) q.push(u);
    }

    // 不变量：队列中的点在“删去已输出点及其出边后”入度为 0。
    vector<int> order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (int v : graph[u]) {
            if (--indegree[v] == 0) q.push(v);
        }
    }

    if ((int)order.size() != n) return {};  // 有向环导致剩余点无法降至入度 0
    return order;
}
```

5. 要改哪里

- 若要求字典序最小拓扑序，将队列换为小根堆，其余入度处理不变，复杂度变为 $O((n+m)\log n)$。
- 若要计算 DAG 上的 DP，可在弹出 `u` 时处理 `u`，再沿出边更新后继状态；必须保持拓扑顺序。
- 重边需在建图和入度统计中都逐条处理；若输入图可能无效，应在调用前验证点编号和入度一致性。

6. 自检

三点 DAG 的答案应是 `[1,2,3]`，且每条边起点在终点之前。单点无边图返回 `[1]`；单点自环返回空向量。平行边 `1->2` 两次时，入度应为 2，处理 1 时也应递减两次。

7. 坑

- DAG 却返回空：入度漏算或与邻接表中的重边数量不符。
- 环图结果长度不足：这正是检测信号，不能把部分序列当作完整答案。
- 把出度当入度：入度为 0 的点才可先输出。
- 需要唯一顺序却直接使用结果：拓扑序可能不唯一；若要判唯一，应在每一步检查队列中是否恰有一个点。

8. 一句话原理

反复输出当前没有未处理前驱的点，并删去它的所有出边；剩余点无法继续时便含有环。

参考：

- OI Wiki：拓扑排序

6.2 Dijkstra 单源最短路

1. 它是做什么的

Dijkstra 求 1-based 有向或无向非负权图中，指定起点到各点的最短距离，并记录一组前驱以恢复一条最短路。不可达点距离为 `INF`，其 `previous` 为 `-1`；起点前驱也为 `-1`。负权边不适用。邻接表加堆实现的时

间为 $O((n+m)\log(n+m))$，空间为 $O(n+m)$；堆中允许同一点存在多条旧记录，至多有 $O(m)$ 条成功松弛记录。

2. 变量在做什么

名称	含义
`Edge{to,weight}`	邻接表项，表示当前点到 `to` 的有向边及非负权值。无向边要加入两个方向。
`distance[v]`	已知最短距离；初始为 `INF`，起点为 0。有限最短路必须小于 `INF`，计算时距离加边权不能溢出。
`previous[v]`	当前最短路树中的前驱点；不可达点和起点为 `-1`。
`pq`	小根堆，存入堆时的 `(距离,点)`；距离与当前 `distance` 不相同的记录已过期。
`n`	`graph.size()-1`，有效点号为 `1..n`。

3. 函数在做什么

函数	契约
`dijkstra(graph,start)`	`graph` 为 1-based 邻接表，端点合法、权值非负，`start` 在 `1..n`；返回距离和前驱数组，不修改图。允许重边。
`restorePath(start,target,previous)`	使用同一次 Dijkstra 产生的前驱表和合法点号；返回起点到终点的点序列，不可达时返回空向量；不改 `previous`。

4. 怎么调用

有向图只加入给定方向；无向边要同时加入反向边。例中最短路为 `1->2->3->4`，长度 5，点 5 不可达：

```
vector<vector<Edge>> graph(6);
graph[1].push_back({2, 2}); graph[1].push_back({3, 10});
graph[2].push_back({3, 1}); graph[3].push_back({4, 2});
DijkstraResult result = dijkstra(graph, 1);
assert(result.distance[4] == 5);
assert(result.distance[5] == INF);
assert(restorePath(1, 4, result.previous) == vector<int>({1, 2, 3, 4}));
assert(restorePath(1, 5, result.previous).empty());
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

struct Edge {
    // 邻接表中的一项表示有向边“当前点 -> to”；无向边需手动加入反向项。
    int to;
    ll weight;
};

struct DijkstraResult {
    vector<ll> distance;  // INF 表示从 start 不可达
    vector<int> previous; // 最短路树中的前驱；start 和不可达点均为 -1
};

// 参数 graph 为 1-based 非负权邻接表、start 为合法起点；返回各点距离和前驱，不修改图。
DijkstraResult dijkstra(
    const vector<vector<Edge>>& graph,
    int start
```

```
) {
    // 仅适用于所有边权 >=0；允许重边。点编号为 1..n。
    int n = (int)graph.size() - 1;
    vector<ll> distance(n + 1, INF);
    vector<int> previous(n + 1, -1);
    // 小根堆保存“入堆时距离,点”；同一点可能有多条新旧记录。
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
int>>> pq;

    distance[start] = 0;
    pq.push({0, start});
    while (!pq.empty()) {
        auto [currentDistance, u] = pq.top();
        pq.pop();
        if (currentDistance != distance[u]) continue;  // 跳过已经失效的旧记录

        for (const Edge& edge : graph[u]) {
            int v = edge.to;
            // u 被取出时 distance[u] 已是当前最优；这里松弛一条出边 u->v。
            if (distance[v] > distance[u] + edge.weight) {
                distance[v] = distance[u] + edge.weight;
                previous[v] = u;
                pq.push({distance[v], v});
            }
        }
    }
    return {distance, previous};
}

// 参数 start/target 为点号，previous 是最短路前驱表；返回路径点序，不可达时空表；不改
输入。
    vector<int> restorePath(int start, int target, const vector<int>&
previous) {
    // 返回 start 到 target 的点序列；前驱链无法到 start 时返回空数组。
    vector<int> path;
    for (int v = target; v != -1; v = previous[v]) {
        path.push_back(v);
        if (v == start) break;
    }
    if (path.empty() || path.back() != start) return {};
    reverse(path.begin(), path.end());
    return path;
}
```

5. 要改哪里

- 无向图建边时为每条输入边插入两个 `Edge`；有向图只插入题面方向。
- 若要计算到某个终点，可在堆中取出终点的当前有效记录时提前结束；若还需要完整 `distance` 数组则不能提前停止。
- 存在负边时应改用 Bellman–Ford 或其他适用算法；仅过滤掉从起点暂时没走到的负边不够。
- 多源最短路可将所有源点以距离 0 入堆；前驱恢复需据题意处理多个起点。

6. 自检

示例距离 `distance[2..4]` 为 `2,3,5`，`distance[5]` 保持 `INF`，恢复路径应为 `[1,2,3,4]`。单点图起点距离为 0、路径为 `[1]`；零权边合法，重边保留也会由松弛选出较短结果。不可达距离不可直接参与普通加法。

7. 坑

- 有负权时答案错误：Dijkstra 的贪心确定性依赖所有边权非负，应换算法。
- 前驱链或距离异常：无向边只加了一个方向，或起点、终点编号越界。
- 堆反复处理旧状态：必须跳过 `currentDistance != distance[u]` 的记录。

- `INF` 输出成实际距离或参与加法：先判断是否等于 `INF`，且保证所有有限距离运算安全。

8. 一句话原理

每次确定当前距离最小的未过期堆记录，并用它松弛出边，非负权保证该距离不再变小。

参考：

- OI Wiki：最短路

6.3 Bellman–Ford 与负环判断

1. 它是做什么的

Bellman–Ford 在有向带权图上求单源最短距离，允许负边，并检测从起点可达的负环。若检测到可达负环，返回布尔值 `true`，同时距离数组全部填为 `INF`；否则不可达点为 `INF`，其余为有限最短距离。时间最坏 $O(nm)$，空间为 $O(n)$（另有输入边表 $O(m)$）；无更新时提前结束。有限结果须能表示为 `long long` 且严格小于 `INF`。实现用 `__int128` 保存中间值，但其范围有限，输入仍须保证中间运算及 `BIG=2^120` 哨兵安全。

2. 变量在做什么

名称	含义
`DirectedEdge{from,to,weight}`	一条有向边 `from -> to`；无向边要显式加入两个方向。
`work[v]`	`__int128` 暂存距离；`BIG` 表示当前不可达。
`distance[v]`	最终 `long long` 距离；不可达或检测到负环时为 `INF`。
`round`	当前边表扫描轮次；至多扫描 `n` 次。
`changed`	本轮是否发生松弛；若为 false，算法可结束。
`n`, `start`	点数和起点；点编号为 `1..n`。

只有 `work[from]` 可达时才尝试松弛。第 `n` 轮仍有更新表示存在起点可达的负环；不保证报告与起点不连通的负环。

3. 函数在做什么

函数	契约
`bellmanFord(n,start,edges)`	`n>=1`，`start` 和所有边端点在 `1..n`；边表按有向边解释。返回 `{distance, hasReachableNegativeCycle}`，不修改 `edges`。若负环标志为真，距离数组全部为 `INF`；否则返回可达最短距离和不可达哨兵。

4. 怎么调用

准备点数、起点和有向边表；无向边按双向各放一条。示例中 `1->2->3->4` 的距离为 5，点 5 不可达；第二组边含起点可达负自环：

```
vector<DirectedEdge> edges = {{1, 2, 4}, {1, 3, 5}, {2, 3, -2}, {3, 4, 3}};
auto result = bellmanFord(5, 1, edges);
assert(!result.second && result.first[4] == 5 && result.first[5] == INF);

vector<DirectedEdge> negative = {{1, 2, 1}, {2, 2, -1}};
auto cycle = bellmanFord(2, 1, negative);
assert(cycle.second && cycle.first[1] == INF && cycle.first[2] == INF);
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;
```

```cpp

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

struct DirectedEdge {
    // 边方向固定为 from -> to；无向边若需要须显式加入两个方向。
    int from, to;
    ll weight;
};

// 参数 n/start/edges 为点数、起点和有向边表；返回距离及起点可达负环标志，不改边表。
pair<vector<ll>, bool> bellmanFord(
    int n,
    int start,
    const vector<DirectedEdge>& edges
) {
    // 用更宽的中间值避免 long long 松弛溢出；有限答案须能放入 ll 且小于 INF。
    const __int128_t BIG = (__int128_t)1 << 120;
    vector<__int128_t> work(n + 1, BIG);
    work[start] = 0;

    // 每轮扫描所有边；原地松弛可一次推进多条边，但 n 轮判环结论仍成立。
    for (int round = 1; round <= n; ++round) {
        bool changed = false;
        for (const auto& edge : edges) {
            if (work[edge.from] == BIG) continue; // 不可达点不能参与松弛
            __int128_t candidate = work[edge.from] + edge.weight;
            if (candidate < work[edge.to]) {
                work[edge.to] = candidate;
                changed = true;
                if (round == n) return {vector<ll>(n + 1, INF), true};
            }
        }
        if (!changed) break;
    }

    vector<ll> distance(n + 1, INF);
    for (int v = 1; v <= n; ++v) {
        if (work[v] != BIG) {
            // 调用方需保证所有可达最短距离均在 ll 范围内且严格小于 INF。
            distance[v] = (ll)work[v];
        }
    }
    return {distance, false};
}
```

5. 要改哪里

- 若需检测全图任意位置的负环，可新增超级源向所有原点连零权边，或改实现把所有初始距离设为 0；当前接口只判断 `start` 可达的负环。
- 若需输出负环影响的具体点或恢复路径，需要额外保存前驱并追踪第 `n` 轮更新；当前只返回布尔标志，且负环时抹去所有距离。
- 无向负边会形成可达的二向负环（同一边往返），应根据题目语义确认是否允许。
- 边权或路径极大时，调整数值类型与哨兵设计并证明中间值范围；`__int128` 也不是无限精度。

6. 自检

第一组中距离依次为 `dist[1]=0, dist[2]=4, dist[3]=2, dist[4]=5`，点 5 为 `INF`。第二组负自环可从 1 到达，返回负环标志且所有距离为 `INF`。负环若位于与起点断开的分量，不应触发该接口的标志；单点无边图返回距离 0 且无负环。

7. 坑

- `INF` 被负边松弛成有限值：跳过不可达的 `from`，不要对哨兵做加法。

- 把任意负环都当成检测结果：当前只检测从 `start` 可达的负环。
- 负环时读取部分距离：函数明确将整张距离数组置为 `INF`，此时只有布尔标志可用。
- 以为 `__int128` 无溢出风险：它只扩大范围，仍需约束点数、边权和累加值。

8. 一句话原理

反复松弛所有边至多 `n-1` 轮求有限最短路，第 `n` 轮仍能改进就说明起点可达负环。

参考：

- OI Wiki：Bellman–Ford 算法

6.4 Floyd 全源最短路

1. 它是做什么的

Floyd–Warshall 在点数较小的有向加权图上计算所有点对的最短距离，可含负边，但要求不存在负环。输入和结果都是 1-based 距离矩阵；`INF` 表示不可达。时间为 $O(n^3)$，矩阵空间为 $O(n^2)$。算法原地修改矩阵；每次只在两段都可达时相加，但仍要求有限边权及存储的有限距离小于 `INF`，所有候选加法不溢出 `long long`。

2. 变量在做什么

名称	含义
`distance[u][v]`	当前已知 `u` 到 `v` 的最短距离；初值对角线为 0、其他为 `INF`。
`k`	当前允许作为中转点的编号；必须位于最外层循环。
`i`, `j`	当前尝试改进的起点和终点。
`INF`	不可达哨兵；不能直接参与加法。

每轮 `k` 后，`distance[i][j]` 已考虑编号不超过 `k` 的点作为中转。完成后若某个 `distance[i][i] < 0`，说明存在负环。

3. 函数在做什么

函数	契约
`makeDistanceMatrix(n)`	为 `n` 个点创建 `(n+1)×(n+1)` 矩阵；点编号 `1..n`，对角线 0、其余 `INF`。返回新矩阵。
`addDirectedEdge(distance,u,v,weight)`	对矩阵单元取当前值与 `weight` 的较小值；原地修改 `distance[u][v]`，不自动添加反向边；端点须合法。
`floydWarshall(distance)`	参数为合法 1-based 方阵且无负环；原地覆盖所有点对距离，无返回值。调用后可检查对角线发现负环，但此时最短距离不具有限定义。

4. 怎么调用

先创建矩阵，再加入有向边；无向边对两个方向各调用一次。例中最短路 `1->2->3->4` 长度为 4，反向不可达：

```
vector<vector<ll>> distance = makeDistanceMatrix(4);
addDirectedEdge(distance, 1, 2, 3);
addDirectedEdge(distance, 2, 3, -1);
addDirectedEdge(distance, 1, 3, 5);
addDirectedEdge(distance, 3, 4, 2);
floydWarshall(distance);
assert(distance[1][4] == 4);
assert(distance[1][3] == 2);
assert(distance[4][1] == INF);
assert(distance[2][2] == 0);
```

模板如下：

```cpp

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

// 参数 n 为点数；返回 n+1 方阵，1..n 对角为 0、其余 INF；分配新矩阵。
    vector<vector<ll>> makeDistanceMatrix(int n) {
    // 矩阵使用 1..n；distance[u][v] 表示有向距离，初始仅自身到自身为 0。
    vector<vector<ll>> distance(n + 1, vector<ll>(n + 1, INF));
    for (int i = 1; i <= n; ++i) distance[i][i] = 0;
    return distance;
}

// 参数为距离矩阵、端点和权值；返回 void，以较小权值写 distance[u][v]；不自动添加反向边。
    void addDirectedEdge(vector<vector<ll>>& distance, int u, int v, ll weight) {
    // 保留同方向重边中的最小值；无向边需再调用一次 (v,u,weight)。
    distance[u][v] = min(distance[u][v], weight);
}

// 参数是 1-based 方阵；返回 void，原地覆盖为全源最短距离，须无负环且加法不溢出。
    void floydWarshall(vector<vector<ll>>& distance) {
    int n = (int)distance.size() - 1;
    // k 必须最外层：完成第 k 轮后，只允许 1..k 作为内部中转点。
    for (int k = 1; k <= n; ++k) {
        for (int i = 1; i <= n; ++i) {
            if (distance[i][k] == INF) continue;
            for (int j = 1; j <= n; ++j) {
                if (distance[k][j] == INF) continue;
                // 两段都可达才相加，避免 INF 参与运算产生伪路径。
                distance[i][j] = min(
                    distance[i][j],
                    distance[i][k] + distance[k][j]
                );
            }
        }
    }
}
```

5. 要改哪里

- 无向边需调用两次 `addDirectedEdge`，并确保两个方向权值一致。
- 检测负环时在算法结束后检查所有 `distance[i][i] < 0`；存在负环的相关最短距离不能当作有限答案使用。
- 点数较大时，改用单源算法逐个起点运行；Floyd 的立方时间和平方空间增长很快。
- 若需记录路径，额外维护前驱或下一跳矩阵，并在每次成功松弛时同步更新；当前模板只返回距离。

6. 自检

示例中 `distance[1][4]` 应为 4，`distance[1][3]` 应为 2，`distance[4][1]` 保持 `INF`，对角线仍为 0。无边的单点图距离为 0；重边应保留较小权值。含负边但无负环的路径应正常缩短；增加可达负环后至少一个对角线会变为负数。

7. 坑

- 结果漏掉中转路径：`k` 不在最外层，破坏动态规划依赖顺序。
- `INF` 相加产生伪距离：相加前分别检查 `distance[i][k]`、`distance[k][j]` 可达。
- 平行边覆盖成较大权值：加边应取最小值。
- 有负环仍输出最短距离：负环可无限降低路径代价，检测到后距离不再有有限定义。

8. 一句话原理

逐步开放每个点作为中转，尝试用 `i->k->j` 改进所有点对距离。

参考：

- OI Wiki：Floyd 算法

6.5 Kruskal 最小生成树

1. 它是做什么的

Kruskal 为 1-based 连通无向加权图求最小生成树；若图不连通，则返回最小生成森林及其总权。每条无向边在输入边表中只出现一次，权值可为负，重边允许。排序时间为 $O(m \log m)$，并查集合并额外耗时 $O(m\alpha(n))$；整体为 $O(n + m\log(m+1))$，空间为 $O(n+m)$，含按值复制的边表和结果边集。至少要求 `n>=1` 且边端点合法。

2. 变量在做什么

名称	含义
`WeightedEdge{u,v,weight}`	一条无向边；边表中只放一次。
`DSU.parent,DSU.sizeOf`	并查集父节点与根集合大小，用来判断加入边是否成环。
`edges`	`kruskal` 按值接收并排序的边表副本；调用方原向量不变。
`chosen`	被选入生成树/森林的边；最多 `n-1` 条。
`connected`	是否选出 `n-1` 条边；`n=1` 时空边集也算连通生成树。
`totalWeight`	`chosen` 中边权之和；不连通时是森林总权，不是生成树权。

3. 函数在做什么

函数	契约
`DSU(n)`	初始化点 `1..n` 为单元素集合；内部用于 Kruskal。
`find(x)`	返回元素 `x` 的根并压缩路径；要求 `x` 在 `1..n`。
`unite(a,b)`	合并两点所在集合；成功返回 `true`，已连通返回 `false`，并更新根大小。
`kruskal(n,edges)`	参数为正点数和合法端点的无向边表，边只存一次；返回 `{connected,totalWeight,chosen}`，不修改调用方的 `edges`。同权边顺序任意。

内部排序比较器按 `weight` 升序排列；同权时不承诺选边顺序。

4. 怎么调用

准备边表后调用 `kruskal`，根据 `connected` 判断是否存在生成树。例中最轻边权为 –1、1、2，总权 2：

```
vector<WeightedEdge> edges = {{1, 2, 1}, {2, 3, 2}, {1, 3, 4}, {3, 4, -1}};
KruskalResult result = kruskal(4, edges);
assert(result.connected && result.totalWeight == 2);
assert(result.chosen.size() == 3);
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct DSU {
```

```cpp
    // 只有根的 sizeOf 有效；路径压缩和按大小合并保证近常数均摊复杂度。
    vector<int> parent, sizeOf;
    // 参数 n 是点编号上界；建立 1..n 单元素集合并初始化内部父节点和大小。
    explicit DSU(int n) : parent(n + 1), sizeOf(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    // 参数 x 是 1..n 元素；返回集合代表并压缩访问路径。
    int find(int x) {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }
    // 参数 a/b 是 1..n 元素；成功合并返回 true，否则 false；修改代表和集合大小。
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sizeOf[a] < sizeOf[b]) swap(a, b);
        parent[b] = a;
        sizeOf[a] += sizeOf[b];
        return true;
    }
};

struct WeightedEdge {
    // 无向边在边表中只保存一次；允许负权、重边和自环。
    int u, v;
    ll weight;
};

struct KruskalResult {
    bool connected;
    ll totalWeight;
    vector<WeightedEdge> chosen;
};

// 参数 n 为点数、edges 为每条无向边只存一次的边表副本；返回生成森林信息，不改调用者边表。
    KruskalResult kruskal(int n, vector<WeightedEdge> edges) {
    // 按权从小到大扫描；不成环的最轻边一定可安全加入当前生成森林。
// 比较器参数为两条边；权值较小者排前，同权顺序任意；不修改边对象。
        sort(edges.begin(), edges.end(), [](const auto& a, const auto& b) {
        return a.weight < b.weight;
    });

    DSU dsu(n);
    ll totalWeight = 0;
    vector<WeightedEdge> chosen;
    for (const auto& edge : edges) {
        if (!dsu.unite(edge.u, edge.v)) continue;  // 同一连通块：加入会成环
        totalWeight += edge.weight;
        chosen.push_back(edge);
        if ((int)chosen.size() == n - 1) break;
    }
    return {(int)chosen.size() == n - 1, totalWeight, chosen};
}
```

5. 要改哪里

- 只需要最小生成森林总权时，可不保存或返回 `chosen`，减少结果边集空间。
- 若要输出树边或计算其他边集合性质，可沿用 `chosen`；边权相同可能有多种合法最优树，不要依赖固定选边顺序。
- 有向图的最小生成树不是该问题定义，不能直接使用本算法。
- 若需要最大生成树，可按权值降序排序；其余并查集判环逻辑不变。

6. 自检

示例选中 `3-4(-1)`、`1-2(1)`、`2-3(2)`，总权为 2 且有 3 条边。单点无边图应连通且总权 0。四点仅有 `1-2(-2)`、`3-4(3)` 时，应返回不连通、两条森林边、总权 1。自环不会被选入；平行边只会选择更有利的连接。

7. 坑

- 把不连通结果当生成树：检查 `connected`；否则 `totalWeight` 只表示森林权值。
- 无向边重复放两个方向：边表要求每条无向边只保存一次，重复反向项会增加冗余但不会改变并查集正确性。
- 忽略负权：Kruskal 按升序照常处理负边。
- 自环被计入：两端根相同，`unite` 返回 `false`，该边跳过。

8. 一句话原理

从轻到重加入连接不同连通块的边，跳过会成环的边，直到得到生成树或扫描完边表。

参考：

- OI Wiki：最小生成树

6.6 二分图判定

1. 它是做什么的

二分图判定检查无向图的每条边能否连接两个不同组，等价于判断是否含奇环。模板逐个连通块做 BFS 染色，图可不连通；时间为 $O(n+m)$，额外空间为 $O(n)$（颜色与队列），输入邻接表另占 $O(n+m)$。成功时返回一组 `0/1` 染色；失败时立即返回 `false`，此时颜色可能只填了一部分。自环会判为非二分图。

2. 变量在做什么

名称	含义
`graph[u]`	无向邻接表；每条边应在两个端点列表中各出现一次。
`color[u]`	`-1` 表示未访问，`0` 或 `1` 表示所在组。函数会先清空并重新赋值。
`start`	当前尚未染色的连通块起点。
`q`	当前 BFS 队列；弹出 `u` 时为所有未染色邻点赋予 `color[u]^1`。
`n`	`graph.size()-1`，点编号为 `1..n`。

3. 函数在做什么

函数	契约
`colorBipartite(graph,color)`	参数为 1-based 无向图和颜色输出向量；返回是否二分图，并将 `color` 重置为长度 `n+1`。成功时每个点均为 0/1 且相邻点异色；失败时可能提前返回，颜色不保证完整。

4. 怎么调用

先将无向边双向加入邻接表，再调用并检查返回值。示例有两条非连通链，所有相邻点应异色；另以自环检查失败：

```
vector<vector<int>> graph(6);
graph[1].push_back(2); graph[2].push_back(1);
graph[2].push_back(3); graph[3].push_back(2);
graph[4].push_back(5); graph[5].push_back(4);
vector<int> color;
assert(colorBipartite(graph, color));
assert(color[1] != color[2] && color[2] != color[3]);
assert(color[4] != color[5]);
```

```

vector<vector<int>> loopGraph(2);
loopGraph[1].push_back(1);
assert(!colorBipartite(loopGraph, color));
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;

// 参数 graph 为 1-based 无向邻接表；返回是否二分图，并覆盖 color 为染色结果（失败时可能不完整）。
    bool colorBipartite(const vector<vector<int>>& graph, vector<int>& color) {
    // graph 必须是无向图的双向邻接表；-1 未访问，0/1 表示两侧颜色。
    int n = (int)graph.size() - 1;
    color.assign(n + 1, -1);

    // 必须枚举所有起点，才能覆盖非连通图中的每个连通块。
    for (int start = 1; start <= n; ++start) {
        if (color[start] != -1) continue;
        queue<int> q;
        color[start] = 0;
        q.push(start);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : graph[u]) {
                if (color[v] == -1) {
                    // 每条无向边两端必须异色；异或 1 得到另一种颜色。
                    color[v] = color[u] ^ 1;
                    q.push(v);
                } else if (color[v] == color[u]) {
                    return false;
                }
            }
        }
    }
    return true;
}
```

5. 要改哪里

- 若输入是有向图但问题问底层无向图是否二分，先按题意忽略方向并构造无向邻接表；若方向本身重要，本模板不适用。
- 若要恢复两侧顶点集合，成功后按 `color[u]` 分组即可。
- 若只需要奇环存在性，仍可使用返回值；要输出具体奇环则需额外记录 BFS 父节点并在冲突边处恢复环。

6. 自检

示例中两条链都可交替染色，函数返回 `true`。单点无边图也为二分图；三角形 `1-2-3-1` 返回 `false`；自环返回 `false`。重边两端仍须异色，不会改变结论。注意非连通图每个分量都必须处理。

7. 坑

- 只有部分分量被染色：只从一个点开始 BFS；应遍历所有 `color[start] == -1` 的点。
- 把 `color == 0` 当未访问：未访问标记是 `-1`。
- 有向边漏掉反向：接口要求无向邻接表双向存边。
- 失败后使用完整染色：发现冲突会立即返回，只有成功时颜色才是完整划分。

8. 一句话原理

从每个未染色分量起步，沿边交替染色；同一条边两端同色就存在奇环。

参考：

- OI Wiki：二分图

6.7 二分图最大匹配（增广路）

1. 它是做什么的

给定左右两部和左点到右点的可选边，增广路算法求最大匹配，并在 `matchRight` 中记录每个右点的左搭档。左右点各自编号 `1..leftCount`、`1..rightCount`，两套编号独立。适合中等规模二分图；时间为 $O(LE)$，其中 `L=leftCount`、`E` 为邻接边数，空间为 $O(L+R+E)$。更大规模可考虑 Hopcroft–Karp。该对象应先建完图，再只调用一次 `maximumMatching()`；当前实现不支持安全地在同一对象上重复求解或增量加边后继续求解，应重新建对象。

2. 变量在做什么

名称	含义
`graphLeft[u]`	左点 `u` 可连接的右点列表。
`matchRight[v]`	与右点 `v` 匹配的左点；0 表示未匹配。
`seen[v]`	右点在当前增广尝试中是否访问过，以 `visitToken` 区分轮次。
`leftCount`, `rightCount`	左右部点数；编号从 1 开始。
`visitToken`	每尝试一个新左点递增，避免每轮清空 `seen`。

3. 函数在做什么

函数	契约
`BipartiteMatcher(L,R)`	创建有 `L` 个左点、`R` 个右点的空图和空匹配；点数应非负。
`addEdge(left,right)`	添加一条左到右的可匹配边，无返回值；参数需分别在 `1..L`、`1..R`。建图应在求解前完成。
`augment(u)`	递归尝试为左点 `u` 找增广路；成功时修改 `matchRight` 并返回 `true`，失败返回 `false`。内部辅助函数，不应由调用者单独启动。
`maximumMatching()`	返回最大匹配大小，并写入 `matchRight`。应在静态图上对新对象调用一次；重复调用会再次尝试已匹配左点，可能造成同一左点占据多个右点。

4. 怎么调用

先确定左右部、加入所有边，再调用一次匹配函数。下例中左点 1 可连两个右点，左点 2 只可连右点 1；算法会为左点 1 改配到右点 2：

```
BipartiteMatcher matcher(2, 2);
matcher.addEdge(1, 1);
matcher.addEdge(1, 2);
matcher.addEdge(2, 1);
assert(matcher.maximumMatching() == 2);
assert(matcher.matchRight[1] == 2 && matcher.matchRight[2] == 1);
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;

struct BipartiteMatcher {
```

```
    int leftCount, rightCount;
    // graphLeft[u] 只存“左部 u -> 右部 v”，左右编号属于两个独立空间。
    vector<vector<int>> graphLeft;
    vector<int> matchRight; // matchRight[v]=u; 0 表示右点 v 当前未匹配
    vector<int> seen;       // 时间戳：本轮增广中右点是否已尝试
    int visitToken = 0;

    // 参数 leftCount/rightCount 为左右部点数；创建空二分图并初始化匹配及访问时间戳。
    BipartiteMatcher(int leftCount, int rightCount)
        : leftCount(leftCount), rightCount(rightCount),
          graphLeft(leftCount + 1), matchRight(rightCount + 1, 0),
          seen(rightCount + 1, 0) {}

    // 参数 left/right 分别为左右部合法点号；返回 void，将兼容边加入 graphLeft。
    void addEdge(int left, int right) {
        graphLeft[left].push_back(right);
    }

    // 参数 u 是当前待匹配左点；返回是否找到增广路，成功时修改 matchRight 的匹配关系。
    bool augment(int u) {
        // 不变量：尝试为左点 u 找交错路；成功时沿递归链整体改配。
        for (int v : graphLeft[u]) {
            if (seen[v] == visitToken) continue;
            seen[v] = visitToken;
            // v 空闲，或能把 v 的旧搭档改配到别处，就可令 u 占用 v。
            if (matchRight[v] == 0 || augment(matchRight[v])) {
                matchRight[v] = u;
                return true;
            }
        }
        return false;
    }

    // 无参数；从新对象的空匹配求最大匹配数并写 matchRight；只调用一次，重算须重建对象。
    int maximumMatching() {
        int matching = 0;
        for (int u = 1; u <= leftCount; ++u) {
            ++visitToken; // 每个左点必须拥有独立访问轮次，不能共用 seen 状态
            if (augment(u)) ++matching;
        }
        return matching;
    }
};
```

5. 要改哪里

- 若左右两部点数不同，分别传入真实大小；编号都从 1 开始，不要把右部编号偏移后再传给 `addEdge`。
- 若要输出左点匹配，可遍历 `matchRight[v]`，反向构造左到右关系；当前没有单独的 `matchLeft` 数组。
- 若图在求解后还要修改，创建新对象并重新加入完整边集；当前匹配状态与递归访问状态没有重置接口。
- 极大图可换 Hopcroft–Karp，简单 DFS 增广的 $O(LE)$ 可能过慢。

6. 自检

示例最终匹配是 `1-2`、`2-1`，大小为 2；若只有边 `1-1`，匹配大小为 1；无边图大小为 0。验证每个非零 `matchRight[v]` 均为合法左点，且不同右点不应被误认为同一左点有多份匹配。求解后不要再次调用同一对象。

7. 坑

- 匹配数偏小：每个新左点前没有新的 `visitToken`，旧的 `seen` 阻断了重试。
- 左右编号混淆：`graphLeft` 的行号是左点，列表元素是右点。
- 重复调用后匹配非法：函数会重新尝试已有匹配的左点，但不先释放旧右点；每次求解使用新对象。

- 误以为 `matchRight[v]==0` 表示左点未匹配：它只描述右部状态，左侧匹配关系需从该数组反查。

8. 一句话原理

逐个尝试给左点找路，必要时沿交替路改配旧搭档，从而增加一个匹配边。

参考：

- OI Wiki：二分图最大匹配

6.8 强连通分量（Tarjan）

1. 它是做什么的

Tarjan 算法把 1-based 有向图划分成强连通分量：同一分量内任意两点互相可达，且该集合极大。模板为每个点赋 `component[u]` 编号并统计分量数，时间 $O(n+m)$，额外数组和递归栈空间 $O(n)$；邻接表另占 $O(n+m)$。结果可用于缩点，分量间有向边形成 DAG。递归深度最坏为 n。对象只可 `run()` 一次；图变更或需重新计算时创建新对象。

2. 变量在做什么

名称	含义
`graph[u]`	从 `u` 出发的有向边列表；重边不影响分量划分。
`dfn[u]`	首次访问时间；0 表示尚未访问。
`low[u]`	从 DFS 子树沿树边和仍在栈中的返边可到达的最小 `dfn`。
`st`, `inStack`	Tarjan 栈及成员标记；只存尚未归入分量的活跃点。
`component[u]`	点所属编号；初始 `-1`，最终范围 `0..componentCount-1`。具体编号顺序不应当作语义。
`timer`, `componentCount`	DFS 时间计数及已弹出的分量数。

当 `low[u] == dfn[u]` 时，`u` 是一个分量的根；从栈顶弹到 `u` 的点属于同一分量。访问到已访问点时，只有它仍在栈中才更新 `low`，并使用 `dfn[v]`。

3. 函数在做什么

函数	契约
`TarjanSCC(graph)`	接收 1-based 邻接表的常量引用并初始化状态；图必须比对象活得久，且构造后到 `run()` 前保持不变。
`dfs(u)`	内部递归处理尚未访问的点，写入 `dfn`、`low`、`component`、栈状态；通常不由调用者直接调用。
`run()`	无参数、无返回值；遍历所有未访问点并写入分量编号和数量。只能在对象尚未运行时调用；需重算时新建对象。

4. 怎么调用

先创建图并加入有向边，再构造 solver 并运行。示例中的 `{1,2}` 和 `{3,4}` 分别互相可达，点 5 独立：

```
vector<vector<int>> graph(6);
graph[1].push_back(2); graph[2].push_back(1);
graph[2].push_back(3);
graph[3].push_back(4); graph[4].push_back(3);
TarjanSCC solver(graph);
solver.run();
assert(solver.componentCount == 3);
assert(solver.component[1] == solver.component[2]);
assert(solver.component[3] == solver.component[4]);
assert(solver.component[1] != solver.component[3]);
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;

struct TarjanSCC {
    // graph 是 1-based 有向邻接表；只按题目方向加边，重边不影响正确性。
    const vector<vector<int>>& graph;
    int n, timer = 0, componentCount = 0;
    // dfn：首次访问序；low：经 DFS 树边和栈内返边能到达的最小 dfn。
    vector<int> dfn, low, component, st;
    vector<char> inStack;

    // 参数 graph 是 1-based 有向邻接表；保存其引用并分配 SCC 工作数组，图须长于对象生命周期。
    explicit TarjanSCC(const vector<vector<int>>& graph)
        : graph(graph), n((int)graph.size() - 1),
          dfn(n + 1, 0), low(n + 1, 0), component(n + 1, -1),
          inStack(n + 1, false) {}

    // 参数 u 是未访问顶点；返回 void，递归遍历其可达区域并更新 dfn/low/栈/分量编号。
    void dfs(int u) {
        // st 只保存尚未归入任何 SCC 的活跃点，inStack 与之同步。
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        inStack[u] = true;

        for (int v : graph[u]) {
            if (dfn[v] == 0) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (inStack[v]) {
                // 已访问且仍在栈中才可能属于当前 SCC；此处必须使用 dfn[v]。
                low[u] = min(low[u], dfn[v]);
            }
        }

        if (low[u] == dfn[u]) {
            // u 是一个 SCC 的 DFS 根，弹到 u 为止的点恰属于同一分量。
            while (true) {
                int v = st.back();
                st.pop_back();
                inStack[v] = false;
                component[v] = componentCount;
                if (v == u) break;
            }
            ++componentCount;
        }
    }

    // 无参数；从所有未访问点启动 DFS 并写 component/componentCount；对象仅应运行一次。
    // 无参数；遍历全部连通区域并填充 SCC 编号及数量；须在对象尚未运行时调用。
    void run() {
        for (int u = 1; u <= n; ++u) {
            if (dfn[u] == 0) dfs(u);
        }
    }
};
```

5. 要改哪里

- 缩点时遍历原边 `u->v`；当 `component[u] != component[v]` 时加入分量图边，按后续算法需要去重。

- 需要拓扑序或 DAG DP 时，在缩点图上处理；原图的分量编号只用于分组，不应假定编号本身就是可依赖的拓扑顺序。
- 图更新或换一张图后重新计算，创建新 solver 并传入新图；该类保存图引用，且 `run` 不清空时间戳、栈和分量状态。
- 若递归深度可能过大，应改成迭代 Tarjan 或选用适合环境的实现；当前函数是递归 DFS。

6. 自检

示例应得到 3 个分量：`{1,2}`、`{3,4}`、`{5}`；编号具体值不固定，只检查相等关系。单点无边图有一个分量；单向边 `1->2` 的两个点属于不同分量。每个点最终 `component[u] >= 0`，重边不改变结果。

7. 坑

- 不同分量被合并：遇到已访问且仍在栈中的点时错用 `low[v]`；应以 `dfn[v]` 更新。
- 已弹栈的点影响 `low`：只对 `inStack[v]` 为真的已访问邻点更新。
- 重新运行得到旧结果：`run()` 不重置状态；重算需新对象。
- solver 引用失效：传入的图对象必须在 solver 生命周期内保持存在。

8. 一句话原理

用 `low` 判断 DFS 子树能否回到更早的活跃点，无法回到时就从栈中切出一个完整强连通分量。

参考：

- OI Wiki：强连通分量

6.9 割点与桥（Tarjan）

1. 它是做什么的

在 1-based 无向图中，割点是删除后连通块数增加的顶点，桥是删除后连通块数增加的边。模板可处理非连通图、重边和自环；运行时间为 $O(n+m)$，额外数组和递归栈为 $O(n)$，邻接表为 $O(n+m)$。桥以端点对返回，平行边中只有真正会断开的边才被判为桥。DFS 递归深度最坏为 n。

2. 变量在做什么

名称	含义
`Edge{to,id}`	邻接项；无向边的两个方向共用一个 `id`。
`dfn[u]`	点 `u` 的首次访问时间，0 表示未访问。
`low[u]`	从 `u` 的 DFS 子树经树边和返边能到达的最小访问时间。
`isCutVertex[u]`	是否为割点；根节点使用至少两个 DFS 子树的特殊条件。
`bridges`	桥端点对 `(u,v)`；方向取决于 DFS 父子关系。
`parentEdge`	进入当前点的边编号；只跳过这一条边，从而保留通向同一父点的平行边。
`timer`, `edgeCount`	访问时间计数和下一条无向边的编号。

非根点若存在孩子 `v` 满足 `low[v] >= dfn[u]`，则 `u` 是割点；若 `low[v] > dfn[u]`，父子边是桥。根点割点条件另算。

3. 函数在做什么

函数	契约
`UndirectedLowlink(n)`	分配 `1..n` 的图和工作数组；点数应非负。
`addEdge(u,v)`	参数为合法点号；添加一条无向边并自动插入双向邻接项，共享边编号；无返回值。每条输入边只调用一次。
`dfs(u,parentEdge)`	内部递归填充 `dfn/low` 并追加割点、桥；根的边编号传 `-1`，通常不由调用者直接调用。

函数	契约
`run()`	无参数、无返回值；遍历所有连通块并写结果。须在加完边后对新对象调用一次；重算需新建对象。

4. 怎么调用

先加入所有无向边，再运行一次。例中三角形含一条平行边，尾部链上的两条边为桥，3 和 4 为割点，6 是孤立点：

```
UndirectedLowlink solver(6);
solver.addEdge(1, 2); solver.addEdge(1, 2);
solver.addEdge(2, 3); solver.addEdge(3, 1);
solver.addEdge(3, 4); solver.addEdge(4, 5);
solver.run();
assert(solver.isCutVertex[3] && solver.isCutVertex[4]);
assert(!solver.isCutVertex[1] && !solver.isCutVertex[6]);
assert(solver.bridges.size() == 2);
bool has34 = false, has45 = false;
for (auto [u, v] : solver.bridges) {
    if ((u == 3 && v == 4) || (u == 4 && v == 3)) has34 = true;
    if ((u == 4 && v == 5) || (u == 5 && v == 4)) has45 = true;
}
assert(has34 && has45);
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;

struct UndirectedLowlink {
    // 同一条无向边的两个邻接项共享 id；这是正确处理父子重边的关键。
    struct Edge { int to, id; };

    int n, timer = 0, edgeCount = 0;
    vector<vector<Edge>> graph;
    vector<int> dfn, low;
    vector<char> isCutVertex;
    vector<pair<int, int>> bridges;

    // 参数 n 为点数；初始化无向邻接表及割点/桥工作数组。
    explicit UndirectedLowlink(int n)
        : n(n), graph(n + 1), dfn(n + 1), low(n + 1),
          isCutVertex(n + 1, false) {}

    // 参数 u/v 是 1..n 端点；返回 void，加入一条带共享边编号的无向边。
    void addEdge(int u, int v) {
        // 每条无向边只调用一次，内部自动加入 u->v 与 v->u 两个方向。
        graph[u].push_back({v, edgeCount});
        graph[v].push_back({u, edgeCount});
        ++edgeCount;
    }

    // 参数 u 为当前点、parentEdge 为进入该点的边 id（根传 -1）；返回 void，写 lowlink 并追加割点/桥。
    void dfs(int u, int parentEdge) {
        // low[u]: 从 u 的 DFS 子树经至多一条非树边能到达的最小 dfn。
        dfn[u] = low[u] = ++timer;
        int childCount = 0;

        for (const Edge& edge : graph[u]) {
            int v = edge.to;
            if (edge.id == parentEdge) continue; // 只跳过来时那一条边，不跳所有父点边
```

```
            if (dfn[v] == 0) {
                ++childCount;
                dfs(v, edge.id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridges.push_back({u, v}); // 子树无法绕回 u 或更高
                // 非根 u：v 子树无法越过 u 时，删除 u 会把 v 子树分离。
                if (parentEdge != -1 && low[v] >= dfn[u]) isCutVertex[u] = true;
            } else {
                low[u] = min(low[u], dfn[v]);
            }
        }

        // DFS 根没有父侧，只有至少两个独立 DFS 子树时才是割点。
        if (parentEdge == -1 && childCount >= 2) isCutVertex[u] = true;
    }

    // 无参数；遍历无向图所有连通块并追加割点/桥结果；请在 addEdge 完成后调用一次。
    void run() {
        for (int u = 1; u <= n; ++u) {
            if (dfn[u] == 0) dfs(u, -1);
        }
    }
};
```

5. 要改哪里

- 若要统计删除点后连通块数量，需另外累计各 DFS 子树的分离贡献；当前只给割点布尔标记。
- 若要按边编号输出桥，可在结果中同时保存树边 `id`；当前 `bridges` 仅保存端点。
- 若图会修改或要重新计算，创建新对象并重新加入边；`run()` 不清空 `timer`、结果数组或图。
- 有向图的强连通性不是本模板的问题，应改用 SCC 算法。

6. 自检

示例中平行的 `1-2` 边都不是桥，三角形任删一边仍连通；只有 `3-4` 和 `4-5` 是桥，删除 3 或 4 会使连通块增加。单独一条边的两端都不是割点但该边是桥；孤立点不是割点。重边测试可确认 DFS 只跳过父边编号。

7. 坑

- 割点或桥判定边界错：割点非根用 `>=`，桥用严格 `>`。
- 根节点误判：根至少有两个 DFS 树孩子才是割点。
- 平行边被误判为桥：只跳过进入点的父边 ID，不能跳过所有通向父节点的边。
- 非连通部分漏掉：`run()` 必须从每个未访问点启动 DFS。
- 重跑得到旧状态：添加边后重新计算应新建对象并重新加边。

8. 一句话原理

`low` 记录子树能否绕回祖先；无法绕回时，相应树边是桥，无法越过当前点时当前点是割点。

参考：

- OI Wiki：割点和桥

6.10 最近公共祖先（倍增 LCA）

1. 它是做什么的

倍增 LCA 在静态、连通、无向树上预处理祖先表，支持最近公共祖先查询及向上提升。点编号为 `1..n`，根深度为 0。预处理时间和空间均为 $O(n \log n)$；每次 `lca` 或 `lift` 为 $O(\log n)$。构造器通过常量引用读取邻接表，不修改也不保存输入图引用。输入必须是无环、无重边、无自环的单棵树；不支持森林中跨树查询。

2. 变量在做什么

名称	含义
`depth[u]`	根到 `u` 的边数；根深度 0。
`up[j][u]`	`u` 的第 `2^j` 个祖先；越过根统一到虚拟点 0。
`levels`	足以表示任意树内向上距离的祖先层数。
`q`	构造时 BFS 定根的队列。
`root`	调用者选择的合法根；更换根会改变深度和祖先关系。

3. 函数在做什么

函数	契约
`BinaryLiftingLCA(tree,root)`	`tree` 是 1-based 连通无向树，`root` 在 `1..n`；建立深度和祖先表，不改 `tree`。
`lift(u,steps)`	返回从 `u` 向上 `steps` 条边到达的祖先；要求 `steps` 在 `0..depth[u]`。
`lca(u,v)`	返回同一棵树中两点的最近公共祖先；不修改预处理表。

4. 怎么调用

先双向建树，再指定根构造对象；之后重复查询。示例中路径为 `1-2-4, 1-2-5, 1-3`：

```
vector<vector<int>> tree(6);
tree[1] = {2, 3}; tree[2] = {1, 4, 5};
tree[3] = {1}; tree[4] = {2}; tree[5] = {2};
BinaryLiftingLCA lcaSolver(tree, 1);
assert(lcaSolver.lca(4, 5) == 2);
assert(lcaSolver.lca(4, 3) == 1);
assert(lcaSolver.lift(4, 1) == 2);
assert(lcaSolver.depth[4] + lcaSolver.depth[3] - 2 *
lcaSolver.depth[lcaSolver.lca(4, 3)] == 3);
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;

struct BinaryLiftingLCA {
    int n, levels;
    vector<int> depth;        // depth[root]=0；模板只接受一棵连通树
    // up[j][u] 是 u 的第 2^j 个祖先；越过根后统一落到虚拟点 0。
    vector<vector<int>> up;

    // 参数 tree 为 1-based 连通无向树、root 为合法根；预处理祖先表/深度，不修改
tree。
    BinaryLiftingLCA(const vector<vector<int>>& tree, int root) {
        // tree 是 1-based 无向树，必须双向加边且不能含环、重边或自环。
        n = (int)tree.size() - 1;
        levels = 1;
        while ((1 << levels) <= n) ++levels;
        depth.assign(n + 1, -1);
        up.assign(levels, vector<int>(n + 1, 0));

        queue<int> q;
        depth[root] = 0;
        q.push(root);
        // BFS 定根：up[0][v] 保存父节点；这里只跳过父边，不是一般图的 visited 检查。
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : tree[u]) {
```

```cpp
                if (v == up[0][u]) continue; // 仅跳父边；因此输入必须真的是树
                up[0][v] = u;
                depth[v] = depth[u] + 1;
                q.push(v);
            }
        }

        for (int j = 1; j < levels; ++j) {
            for (int u = 1; u <= n; ++u) {
                // 先跳 2^(j-1)，再从中间祖先跳同样距离。
                up[j][u] = up[j - 1][up[j - 1][u]];
            }
        }
    }

    // 参数 u 为树点、steps 在 0..depth[u]；返回向上 steps 步的祖先，不改预处理表。
    int lift(int u, int steps) const {
        // 按 steps 的二进制位拆分向上跳；调用者应保证 steps<=depth[u]。
        for (int j = 0; j < levels; ++j) {
            if ((steps >> j) & 1) u = up[j][u];
        }
        return u;
    }

    // 参数 u/v 为同一棵树内的点；返回最近公共祖先点号，不修改预处理表。
    int lca(int u, int v) const {
        // 先对齐深度，再从最高位向下保持“两点祖先仍不同”。
        if (depth[u] < depth[v]) swap(u, v);
        u = lift(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int j = levels - 1; j >= 0; --j) {
            if (up[j][u] != up[j][v]) {
                u = up[j][u];
                v = up[j][v];
            }
        }
        // 此时 u、v 不同但父节点相同，该父节点就是最近公共祖先。
        return up[0][u];
    }
};
```

5. 要改哪里

- 计算无权路径边数可用 `depth[u]+depth[v]-2*depth[lca(u,v)]`；若要路径点数再加 1。
- 若需处理森林，可为每个连通块分别建根并记录所属分量，跨分量查询需定义失败状态；当前类假设单棵树。
- 若 `n` 可能接近 `int` 左移上界，构造 `levels` 时应使用足够宽的移位类型，避免 `1 << levels` 溢出。
- 更换根需重新构造对象和祖先表；旧表中的深度、父亲关系对应旧根。

6. 自检

示例中 `lca(4,5)=2`，`lca(4,3)=1`，`lift(4,1)=2`，4 到 3 的路径有 3 条边。单点树的 `lca(root,root)` 应为根，`lift(u,0)` 应返回 `u`；根自身不能向上提升正步数。确认所有输入点均与根连通。

7. 坑

- 输入有环导致重复入队：构造仅跳过父点，要求输入真的是树。
- 祖先跨过根后出现非法下标：虚拟点 0 的祖先表保持为 0。
- 深度差没先对齐：先提升较深点，再同步倍增。
- 用于森林或不同分量：接口没有分量标记，跨树 LCA 无定义。
- 极大 `n` 移位溢出：`1 << levels` 使用 `int`，需按规模改用 `1LL`。

8. 一句话原理

先把两点深度对齐，再从大步到小步同步上跳，停在最近公共祖先的下一层。

参考：

- OI Wiki：最近公共祖先

6.11 树的直径

1. 它是做什么的

树的直径是任意两点间最长简单路径的长度。本模板适用于 `n>=1` 的 1-based 连通无向树，边权须非负；无权树令每条边权为 1。两次从起点遍历找最远点，返回直径长度和一对端点。时间 $O(n)$，额外空间 $O(n)$；每次遍历树边一次。一般图或含负权的树不能直接套用。

2. 变量在做什么

名称	含义
`TreeEdge{to,weight}`	邻接项，表示无向边一端及非负权值；每条边在两端都存一项。
`distance[u]`	当前遍历起点到 `u` 的唯一路径长度；`-1` 表示未访问，故边权必须非负。
`vertex`	当前发现的最远点；距离相同时返回哪个点取决于编号扫描顺序。
`a, b`	直径端点候选；从 1 找到 `a`，再从 `a` 找到 `b`。
`diameter`	`a` 到最远点 `b` 的距离，即直径长度。

3. 函数在做什么

函数	契约
`farthestVertex(tree,start)`	`tree` 为 1-based 连通非负权树，`start` 是合法点；返回 `{最远点,距离}`。遍历数组为局部变量，不修改图。
`treeDiameter(tree)`	要求 `n>=1` 的 1-based 连通非负权树；返回 `{直径长度, 端点 a,端点 b}`，不修改图。若存在并列直径，端点不保证唯一。

4. 怎么调用

将无向边双向加入邻接表，再调用 `treeDiameter`。示例中 1 到 3 距离为 7，是最长路径：

```
vector<vector<TreeEdge>> tree(5);
tree[1].push_back({2, 3}); tree[2].push_back({1, 3});
tree[2].push_back({3, 4}); tree[3].push_back({2, 4});
tree[2].push_back({4, 2}); tree[4].push_back({2, 2});
auto [length, a, b] = treeDiameter(tree);
assert(length == 7);
assert((a == 1 && b == 3) || (a == 3 && b == 1));
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct TreeEdge {
    // 无向树边需在两端邻接表各存一份；两次最远点法要求 weight>=0。
    int to;
    ll weight;
};

```

```cpp
// 参数 tree 为 1-based 非负边权树、start 为树点；返回最远点及距离，使用局部遍历数组。
    // 参数 tree 为 1-based 非负权连通树、start 为合法点；返回最远点和距离，仅分配局部数组。
    // 参数 tree 为 1-based 非负权连通树、start 为合法点；返回最远点和距离，仅分配局部数组。
pair<int, ll> farthestVertex(
    const vector<vector<TreeEdge>>& tree,
    int start
) {
    // 点编号为 1..n；distance=-1 同时作为未访问标记，因此边权必须非负。
    int n = (int)tree.size() - 1;
    vector<ll> distance(n + 1, -1);
    stack<int> st; // 树上起点到每点路径唯一，无需 Dijkstra 即可累计带权距离
    st.push(start);
    distance[start] = 0;

    while (!st.empty()) {
        int u = st.top();
        st.pop();
        for (const auto& edge : tree[u]) {
            if (distance[edge.to] != -1) continue; // 跳过父节点，避免沿无向边返回
            distance[edge.to] = distance[u] + edge.weight;
            st.push(edge.to);
        }
    }

    int vertex = start;
    for (int u = 1; u <= n; ++u) {
        if (distance[u] > distance[vertex]) vertex = u;
    }
    return {vertex, distance[vertex]};
}

// 参数为 1-based 连通非负带权树且 n>=1；返回 {直径长度,端点a,端点b}，不改树。
    tuple<ll, int, int> treeDiameter(const vector<vector<TreeEdge>>& tree) {
    // 第一次找直径某端点 a，第二次从 a 找另一端 b；返回 {长度,a,b}。
    auto [a, ignored] = farthestVertex(tree, 1);
    auto [b, diameter] = farthestVertex(tree, a);
    return {diameter, a, b};
}
```

5. 要改哪里

- 无权树可把每条边的 `weight` 设为 1，返回长度即边数。
- 若需输出具体直径路径，在第二次遍历时记录父节点，再从 `b` 回溯到 `a`；当前只返回端点和长度。
- 若边权为负，两次最远点法及 `-1` 未访问标记均不适用，应根据题意改用其他树形 DP。
- 输入是森林时，先定义目标是最大分量直径还是全局路径；当前函数固定从点 1 出发且要求连通树。

6. 自检

示例直径长度为 `3+4=7`，端点为 1 和 3。单点树返回长度 0、两个端点均为 1；含零权边仍能正确遍历，零距离点也会标记为已访问。带负边时不要使用本模板。

7. 坑

- 把一般图交给两次遍历：树的路径唯一，带环图中该过程不再计算最短/最长简单路径。
- 负边或断开图导致错误：要求连通且所有权值非负。
- 无向边只存一个方向：每条树边须在两端邻接表都加入。
- 误认为端点唯一：存在多个等长直径时，遍历顺序会影响返回的端点。

8. 一句话原理

从任意点走到最远端点，再从该端点走到最远点，后一次距离就是树的直径。

参考：

- OI Wiki：树的直径

6.12 最大流（Dinic）

1. 它是做什么的

Dinic 在非负容量有向网络中求源点到汇点的最大流，支持以 `limit` 截断本次增广量。点号为 `1..n`，源汇必须不同。算法反复对正残量边 BFS 建层次图，再用当前弧 DFS 增广；一般上界为 $O(V^2E)$，残量图空间为 $O(V+E)$，DFS 递归深度可达 $O(V)$。默认 `limit=INF`，因此返回值至多为 `INF`；若真实最大流可能更大，应传入可表示且足够大的上限。每次调用返回本次新增的流量，残量图会被修改。

2. 变量在做什么

名称	含义
`Edge{to,reverseIndex,capacity}`	残量边；`capacity` 是当前剩余容量，不是原始容量。正反边互相记录索引。
`graph[u]`	残量网络邻接表；每次 `addEdge` 增加一条正向边和一条初始容量 0 的反向边。
`level[u]`	BFS 层数；`-1` 表示从当前源点在残量网络不可达。
`current[u]`	当前层次图中 `u` 尚未尝试的边位置，用于避免重复扫描失败边。
`flow,limit`	本次调用新增流量及上限。

3. 函数在做什么

函数	契约
`Dinic(n)`	创建 `n` 个点的空残量网络；点号范围 `1..n`。
`addEdge(from,to,capacity)`	加一条有向边及反向残量边；容量须非负，端点合法；原地修改残量图。平行边与自环可加入。
`buildLevelGraph(source,sink)`	沿正残量边重建 `level`；返回汇点是否可达。会覆盖旧层次。
`sendFlow(u,sink,pushed)`	内部 DFS，至多发送 `pushed`，成功时修改正反残量；通常由 `maxFlow` 调用。
`maxFlow(source,sink,limit)`	要求源汇合法且不同、`limit>=0`；返回当前残量图上最多新增的流量，上限为 `limit`。残量状态被修改，可再次调用继续增广；重算新实例需重新加边。

4. 怎么调用

按题意加入有向容量边，再调用 `maxFlow`。下面网络最大流为 5；分两次限额增广分别得到 3 和 2，之后没有剩余增广量：

```
Dinic flow(4);
flow.addEdge(1, 2, 3); flow.addEdge(1, 3, 2);
flow.addEdge(2, 3, 1); flow.addEdge(2, 4, 2);
flow.addEdge(3, 4, 3);
assert(flow.maxFlow(1, 4, 3) == 3);
assert(flow.maxFlow(1, 4) == 2);
assert(flow.maxFlow(1, 4) == 0);
assert(!flow.buildLevelGraph(1, 4));
```

模板如下：

```

#include <bits/stdc++.h>
using namespace std;
```

```cpp

using ll = long long;
const ll INF = numeric_limits<ll>::max() / 4;

struct Dinic {
    struct Edge {
        int to, reverseIndex;
        ll capacity; // 当前残量，不是原始容量；必须始终非负
    };

    int n;
    vector<vector<Edge>> graph; // graph[u][i] 与其 reverseIndex 指向的残量边互为反边
    vector<int> level;          // 残量网络中从 source 出发的 BFS 层号，-1 为不可达
    vector<int> current;        // 当前弧：本轮分层图中每点尚未尝试的首条边

    // 参数 n 为点数；初始化空残量网络、层次和当前弧数组。
    explicit Dinic(int n)
        : n(n), graph(n + 1), level(n + 1), current(n + 1) {}

    // 参数为端点和非负容量；返回 void，加入正向边及零容量反向边，修改残量图。
    void addEdge(int from, int to, ll capacity) {
        // 加有向容量边 from->to；反向边初始容量为 0，只用于撤回已经发送的流。
        // 自环的两条边都进入同一 vector，正向边的反向下标要跳过自身。
        Edge forward{to, (int)graph[to].size() + (from == to), capacity};
        Edge backward{from, (int)graph[from].size(), 0};
        graph[from].push_back(forward);
        graph[to].push_back(backward);
    }

    // 参数 source/sink 为合法点号；返回汇点是否可达，并重建 level 数组。
    bool buildLevelGraph(int source, int sink) {
        // 只沿正残量边 BFS；后续 DFS 只能走 level 恰好增加 1 的边。
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[source] = 0;
        q.push(source);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (const Edge& edge : graph[u]) {
                if (edge.capacity <= 0 || level[edge.to] != -1) continue;
                level[edge.to] = level[u] + 1;
                q.push(edge.to);
            }
        }
        return level[sink] != -1;
    }

    // 参数 u/sink 为点号、pushed 为非负上限；返回实际发送量，并原地修改正反残量边。
    ll sendFlow(int u, int sink, ll pushed) {
        // 返回本次从 u 实际送到 sink 的流量；pushed 是上游允许的上限。
        if (u == sink || pushed == 0) return pushed;
        // i 使用引用：失败边以后无需重扫，这就是当前弧优化的不变量。
        for (int& i = current[u]; i < (int)graph[u].size(); ++i) {
            Edge& edge = graph[u][i];
            if (edge.capacity <= 0 || level[edge.to] != level[u] + 1) continue;
            ll sent = sendFlow(edge.to, sink, min(pushed, edge.capacity));
            if (sent == 0) continue;
            // 正向残量减少、反向残量增加，保证之后可以撤销/改道这部分流。
            edge.capacity -= sent;
            graph[edge.to][edge.reverseIndex].capacity += sent;
```

```
65                 return sent;
66             }
67             return 0;
68         }
69
70         // 参数为不同源汇和非负流量上限；返回本次从当前残量网可增广流量，残量图被修改。
71         ll maxFlow(int source, int sink, ll limit = INF) {
72             // 要求 source!=sink、点编号 1..n；返回不超过 limit 的最大增广量。
73             assert(source != sink);
74             ll flow = 0;
75             while (flow < limit && buildLevelGraph(source, sink)) {
76                 fill(current.begin(), current.end(), 0); // 新分层图必须重置当前弧
77                 while (flow < limit) {
78                     ll sent = sendFlow(source, sink, limit - flow);
79                     if (sent == 0) break;
80                     flow += sent;
81                 }
82             }
83             return flow;
84         }
85     };
```

5. 要改哪里

- 无向容量关系通常需要按题意分别调用两次 `addEdge`；这表示两个独立方向各有容量，不能随意当成一条共享容量边。
- 二分图匹配可建超级源到左点、兼容边、右点到超级汇，容量均设为 1；匹配数等于最大流。
- 求最小割必须让流跑到无增广路，不能用有限 `limit` 提前截断。此时再次 `buildLevelGraph` 返回 false，`level[v]>=0` 是源侧；割容量应从原始输入边表统计，不能把残量反边当原边。
- 若要重算另一组网络，创建新对象并重新加边；重复 `maxFlow` 是在旧残量状态上求新增量。

6. 自检

示例总流为 5：源点两条出边容量和为 5，汇点两条入边容量和也为 5；三次调用新增量分别为 3、2、0。单边容量为 0 时流为 0；只有反向于源汇方向的边时也无可发送流。源汇相同时不满足接口前提，代码会触发断言。

7. 坑

- 反向边被当原始边：反向边初始容量为 0，只用于撤回或改道已发送的流。
- 更新正向残量却没更新反向：会破坏流量守恒及后续重路由。
- 认为每次调用返回累计流：函数返回当前调用新增量；多次调用需自行累加。
- 限额后直接取最小割：尚有可增广流时层次图仍可达，不能得到最小割。
- 边容量或总流超过类型/默认上限：使用 `long long` 且默认上限为 `INF`，需按约束检查。

8. 一句话原理

在残量网络中只沿层数递增的最短增广路推流，直到汇点不可达或达到调用上限。

参考：

- OI Wiki：最大流

6.13 欧拉路径与回路

1. 它是做什么的

Hierholzer 算法为无向多重图构造一条每条边恰好使用一次的欧拉点序。输入边表中每条无向边只出现一次，点编号为 `1..n`；平行边用内部边编号区分，自环也按一条边计。若忽略孤立点后不连通，或奇度点数不是 0 或 2，函数返回空向量；无边图也按接口约定返回空向量。时间与空间均为 $O(n+m)$，返回序列成功时长 `m+1`。

2. 变量在做什么

名称	含义
`edges[id]`	第 `id` 条输入无向边；每条边只有一个 ID。
`g[u]`	`(邻点,边 ID)` 邻接项；同一无向边在两端各存一次。
`deg[u]`	度数；自环会增加 2。
`used[id]`	该边是否已被消费，双向邻接项共享同一标记。
`it[u]`	`u` 当前扫描到的邻接位置，跳过已用边避免反复检查。
`st, path`	Hierholzer 显式栈及回退生成的逆序答案；返回前反转。

存在欧拉回路时从首个非孤立点出发；有两个奇度点时从扫描到的一个奇度点出发。具体点序依赖邻接顺序。

3. 函数在做什么

函数	契约
`eulerTrailUndirected(n,edges)`	`n` 为点数，端点须在 `1..n`，边表每条无向边只出现一次；返回覆盖全部边的点序，失败或无边时返回空向量，不改输入边表。

函数会检查度数条件，并在构造后核对 `path.size()==m+1`；长度不足表示有边未被连通遍历。

4. 怎么调用

准备无向边表，每条边写一次，再调用函数。三角形的插入顺序会得到点序 `1,2,3,1`：

```
vector<pair<int, int>> edges = {{1, 2}, {2, 3}, {3, 1}};
vector<int> path = eulerTrailUndirected(3, edges);
assert(path == vector<int>({1, 2, 3, 1}));
assert(eulerTrailUndirected(3, {}).empty());
```

模板如下：

```

// 参数 n 为点数、edges 为每条无向边一次的端点表；返回全边欧拉点序，无欧拉遍历或非连通时为空。
vector<int> eulerTrailUndirected(
    int n, const vector<pair<int, int>>& edges) {
    // Hierholzer 回溯入答案，末尾需 reverse；每条边只消费一次。
    vector<vector<pair<int, int>>> g(n + 1);
    vector<int> deg(n + 1), it(n + 1);
    vector<char>used(edges.size());
    for (int id = 0; id < (int)edges.size(); ++id) {
        auto [u, v] = edges[id];
        g[u].push_back({v,id});
        g[v].push_back({u,id});
        ++deg[u];
        ++deg[v];
    }
    int start = 0, odd = 0;
    for (int u = 1; u <= n; ++u) {
        if (deg[u] == 0) {
            continue;
        }
        if (!start) {
            start = u;
        }
        if (deg[u] & 1) {
            ++odd;
            start = u;
        }
    }
```

```
    if (!start || (odd != 0 && odd != 2)) {
        return {};
    }
    vector<int> st{start}, path;
    while (!st.empty()) {
        int u = st.back();
        while (it[u] < (int)g[u].size()
               && used[g[u][it[u]].second]) {
            ++it[u];
        }
        if (it[u] == (int)g[u].size()) {
            path.push_back(u);
            st.pop_back();
        } else {
            auto [v, id] = g[u][it[u]++];
            if (!used[id]) {
                used[id] = true;
                st.push_back(v);
            }
        }
    }
    if (path.size() != edges.size() + 1) {
        return {};
    }
    reverse(path.begin(), path.end());
    return path;
}
```

5. 要改哪里

- 有向图需要按每条边只加入一次，并改用入度/出度条件及有向 Hierholzer；不能把有向边当无向边重复加入。
- 若需返回边序而不只是点序，应在每次消费边时同时记录边 ID，并在回退时将边 ID 与点序同步整理。
- 若题目允许无边图输出单点路径，可修改空边处理；当前接口统一返回空向量。
- 若要指定起点，需检查起点满足欧拉路径端点条件，并从该点启动，而不是使用当前自动选点规则。

6. 自检

三角形中每个点度数为 2，返回 4 个点且首尾相同；空边表返回空。单边图有两个奇度点，返回两个端点构成的 2 点序列；两个互不相连的单边会因未覆盖全部边而返回空。自环度数按 2 计，并且边 ID 只消费一次。

7. 坑

- 欧拉条件满足但返回空：非孤立部分不连通，检查路径长度是否为 `m+1`。
- 平行边漏掉或重复使用：以边 ID 标记，不能仅按端点去重。
- 回溯顺序颠倒：`path` 是逆序生成，最终必须反转。
- 把点序长度当边数：成功时点数为 `m+1`，边数为 `m`。

8. 一句话原理

沿尚未使用的边不断深入，走不动时回退并记录节点，最后反转回退序列得到覆盖全部边的路线。

6.14 2-SAT

1. 它是做什么的

2-SAT 判断变量取真/假时，一组二文字析取约束是否可满足，并在有解时返回一组赋值。每个子句形如 `(x==xv) OR (y==yv)`；节点编号将变量 `1..n` 的假、真文字分别映射到 `1..2n`。建蕴含图后调用第 6.8 节的 Tarjan SCC，时间与空间均为 $O(n+m)$，其中 `m` 为子句数。无解返回 `nullopt`；有解返回长度 `n+1` 的 `vector<char>`，下标 1..n 为布尔值。代码依赖 6.8 的 `TarjanSCC` 实现。

2. 变量在做什么

名称	含义
`vars`	布尔变量数，变量编号 `1..vars`。
`g`	蕴含图，文字节点编号为 `1..2*vars`。
`node(x,value)`	变量 `x` 的文字节点；false 为 `2*x-1`，true 为 `2*x`。
`scc`	引用成员 `g` 的 Tarjan 求解器；成员声明顺序保证先构造 `g` 再构造它。
`answer[x]`	有解时变量 `x` 的取值；`char` 的 0 表示 false，非 0 表示 true。

析取 `(x==xv) OR (y==yv)` 转成两条蕴含：`¬(x==xv) -> (y==yv)` 和 `¬(y==yv) -> (x==xv)`。若同一变量的真、假文字落在同一 SCC，则无解。

3. 函数在做什么

函数	契约
`TwoSAT(n)`	创建 `n` 个变量及 `2n` 个文字节点；`n` 应非负。内部 `scc` 保存对 `g` 的引用，因此不要复制或移动构造后的对象。
`node(x,value)`	要求 `x` 在 `1..vars`；返回对应文字节点号，不修改状态。
`imply(a,b)`	参数是合法文字节点号；向 `g` 添加边 `a->b`，无返回值。若调用者以它表示逻辑约束 `a=>b`，还必须加入逆否边 `¬b=>¬a`；通常由 `addOr` 成对调用。
`addOr(x,xv,y,yv)`	加入一个二文字析取子句，更新蕴含图，无返回值；变量号须合法。所有子句应在 `solve` 前加入。
`solve()`	运行内部 Tarjan 并返回可满足赋值或 `nullopt`。求解后不要再添加约束；若要修改/重算，创建新对象。

4. 怎么调用

先添加全部子句，再调用 `solve()`；示例约束为 `x OR y` 与 `¬x OR y`，因此 `y` 必须为真。该片段依赖 6.8 节的 `TarjanSCC`：

```
TwoSAT solver(2);
solver.addOr(1, true, 2, true);
solver.addOr(1, false, 2, true);
auto answer = solver.solve();
assert(answer.has_value());
assert((*answer)[1] || (*answer)[2]);
assert(!(*answer)[1] || (*answer)[2]);
```

模板如下：

```

struct TwoSAT {
    // 每个变量 x 有 false/true 两个文字节点；蕴含边记在 g 中。
    int vars;
    vector<vector<int>> g;
    TarjanSCC scc; // 复用 6.8 节实现；它保存 g 的引用，故 g 必须先声明

    // 参数 n 为变量数；分配文字蕴含图及引用该图的 SCC 求解器。
    explicit TwoSAT(int n) : vars(n), g(2 * n + 1), scc(g) {}

    // 参数 x 属于 1..vars、value 指定真假文字；返回对应 1-based 蕴含图节点号。
    int node(int x, bool value) const {
        return 2 * (x - 1) + (value ? 1 : 0) + 1;
    }
    // 参数 a/b 为文字节点号；返回 void，向蕴含图添加有向边 a->b。
```

```
    void imply(int a, int b) {
        g[a].push_back(b);
    }
    // 参数给出两个变量及各自文字真假值；返回 void，加入析取子句对应的两条蕴含边。
    void addOr(int x, bool xv, int y, bool yv) {
        // (x==xv) OR (y==yv) 等价于 ¬x->y 与 ¬y->x。
        int a = node(x, xv), b = node(y, yv);
        imply(node(x, !xv), b);
        imply(node(y, !yv), a);
    }
    // 无参数；返回可满足赋值（下标 1..vars）或 nullopt；运行 SCC 并写内部分量状态，仅可运行一次。
    optional<vector<char>> solve() {
        scc.run();
        vector<char> answer(vars + 1);
        for (int x = 1; x <= vars; ++x) {
            int f = node(x, false), t = node(x, true);
            if (scc.component[f] == scc.component[t]) return nullopt;
            // TarjanSCC 按弹栈顺序编号，较小分量先赋值为 true。
            answer[x] = scc.component[t] < scc.component[f];
        }
        return answer;
    }
};
```

5. 要改哪里

- 单文字约束 `x` 可写成 `addOr(x,true,x,true)`；`¬x` 可写成 `addOr(x,false,x,false)`。
- 若输入是一般三文字子句（3-SAT），本模板不适用；只有题目结构能化成二文字约束时才可使用。
- 若换用不同 SCC 实现，必须根据其分量编号方向重新核对 `component[t] < component[f]` 的赋值比较。
- 修改约束后需用新 `TwoSAT` 实例重新建图；内部 SCC 状态不会因新增边而重置。

6. 自检

示例的两个子句共同强制 `y=true`，返回赋值须满足两式。单变量 `x OR x` 可满足且要求 `x=true`；同时加入 `x OR x` 与 `¬x OR ¬x` 则应返回 `nullopt`。变量下标 0 不属于输出赋值范围；检查时读取 `answer[1..vars]`。

7. 坑

- 文字节点偏一位：假文字为 `2*x-1`，真文字为 `2*x`，图节点从 1 开始。
- 蕴含边方向反了：子句 `a OR b` 要加 `¬a->b` 与 `¬b->a`；单加逻辑蕴含 `a=>b` 时还要加逆否边。
- 改用 SCC 后赋值取反：分量编号顺序由实现决定，需验证比较方向。
- 求解后继续加子句：SCC 缓存已完成状态，不会重新处理新边。
- 复制/移动 `TwoSAT`：`scc` 的引用不会自动改指向副本的 `g`；直接在原对象中建图和求解。

8. 一句话原理

把每个析取转成两条蕴含，再用 SCC 判断真假文字是否矛盾并据分量顺序构造赋值。

参考：

- 依赖第 6.8 节的 Tarjan 强连通分量模板。

6.15 最小费用最大流

1. 它是做什么的

最小费用最大流在容量网络中，按最小单位费用发送尽可能多的流；`run(s,t,limit)` 最多新增 `limit` 单位并返回本次 `{flow,cost}`。点号为 `1..n`，容量非负，源汇不同。每次用 SPFA 在残量图中找最短增广路；SPFA 单次最坏可达 $O(VE)$，整体由增广次数和每次最短路成本决定，没有固定的 $O(m \log n)$ 保证，适合中小规模。空间为 $O(V+E)$。要求过程中不存在从源点可达的负费用环，且距离、反向费用、`add*distance`

及总代价均能安全表示为 `long long`；`cost` 不能是无法取负的最小整数。用于最短路的有限费用距离还须小于内部 `INF`。代码使用别名 `ll`，需在定义前声明 `using ll = long long;`。

2. 变量在做什么

名称	含义
`E{to,rev,cap,cost}`	残量边；`cap` 为剩余容量，`cost` 为单位费用。正反边费用互为相反数。
`g[u]`	残量邻接表；`rev` 指向目标点邻接表中对应反边。
`d[v]`	本轮从 `s` 到 `v` 的最短费用距离；不可达为 `INF`。
`pv[v],pe[v]`	最短路前驱点和前驱边下标，用于回溯增广路径。
`in[v]`	当前是否在 SPFA 队列中，避免重复入队。
`flow,cost`	本次调用新增的流量和费用；重复调用从当前残量状态继续，返回本次增量。

3. 函数在做什么

函数	契约
`MinCostMaxFlow(nodeCount)`	创建 `1..nodeCount` 的空网络；点数应为正。
`addEdge(u,v,cap,cost)`	加入有向容量边及容量 0、费用 `-cost` 的反向残量边；容量非负、端点合法，且 `-cost` 可表示；原地修改 `g`。
`run(s,t,limit)`	要求 `s!=t`、端点合法、`limit>=0` 且无可达负费用环；返回本次新增流量和费用，修改残量图。若流量不足上限，返回实际流量，不另设失败标记。

4. 怎么调用

先定义 `ll` 别名、建图，再按所需上限调用。示例先发送两单位走低费用通道，第二次在残量图上继续发送剩余一单位：

```
MinCostMaxFlow flow(4);
flow.addEdge(1, 2, 2, 1); flow.addEdge(2, 4, 2, 1);
flow.addEdge(1, 3, 1, 0); flow.addEdge(3, 4, 1, 5);
auto first = flow.run(1, 4, 2);
assert(first.first == 2 && first.second == 4);
auto second = flow.run(1, 4);
assert(second.first == 1 && second.second == 5);
auto exhausted = flow.run(1, 4);
assert(exhausted.first == 0 && exhausted.second == 0);
```

模板如下：

```

struct MinCostMaxFlow {
    // 残量网络逐次找最短路，parent 回溯增广；反向边费用为负。
    struct E {
        int to, rev;
        ll cap, cost;
    };
    int n;
    vector<vector<E>> g;
    // 参数 nodeCount 为点数；初始化空残量网络。
    explicit MinCostMaxFlow(int nodeCount)
        : n(nodeCount), g(nodeCount + 1) {
    }
    // 参数为端点、非负容量和单位费用；返回 void，加入正向边与负费用反向残量边。
    void addEdge(int u, int v, ll cap, ll cost) {
        E a{v, (int)g[v].size() + (u == v), cap, cost};
        E b{u, (int)g[u].size(), 0, -cost};
        g[u].push_back(a);
```

```
19          g[v].push_back(b);
20      }
21      // 参数 s/t 为不同源汇、limit 为非负流量上限；返回本次 {流量,费用}，并修改残量图；须无可达负费用环。
22      pair<ll, ll> run(int s, int t, ll limit = LLONG_MAX / 4) {
23          assert(s != t);
24          ll flow = 0;
25          ll cost = 0;
26          const ll INF = numeric_limits<ll>::max() / 4;
27          while (flow < limit) {
28              vector<ll> d(n + 1, INF);
29              vector<int> pv(n + 1), pe(n + 1);
30              vector<char> in(n + 1);
31              queue<int> q;
32              d[s]=0;
33              q.push(s);
34              in[s]=1;
35              while (!q.empty()) {
36                  int u = q.front();
37                  q.pop();
38                  in[u]=0;
39                  for (int i = 0; i < (int)g[u].size(); ++i) {
40                      auto& e = g[u][i];
41                      if (e.cap && d[e.to] > d[u] + e.cost) {
42                          d[e.to] = d[u] + e.cost;
43                          pv[e.to] = u;
44                          pe[e.to] = i;
45                          if (!in[e.to]) {
46                              in[e.to] = 1;
47                              q.push(e.to);
48                          }
49                      }
50                  }
51              }
52              if (d[t] == INF) break;
53              ll add = limit - flow;
54              for (int v = t; v != s; v = pv[v]) add = min(add, g[pv[v]][pe[v]].cap);
55              flow += add;
56              cost += add * d[t];
57              for (int v = t; v != s; v = pv[v]) {
58                  auto& e = g[pv[v]][pe[v]];
59                  e.cap -= add;
60                  g[v][e.rev].cap += add;
61              }
62          }
63          return {flow,cost};
64      }
65  };
```

5. 要改哪里

- 若只允许发送指定流量 `F`，调用 `run(s,t,F)` 后检查返回的 `flow==F`；不足时网络无法满足需求。
- 若有可达负费用环，应先按题意处理环流，或换用能处理负环的建模/算法；当前 SPFA 增广过程可能无法终止。
- 大网络可改用势能配合 Dijkstra 的实现；势能初始化仍需正确处理负费用边。
- 多次调用可逐步追加流量，但返回的是本次增量；要从头重算应新建对象并重新加边。

6. 自检

示例第一次发送 2 单位，每单位费用 2，费用为 4；第二次发送剩余 1 单位，费用为 5；第三次无路可增，返回 `{0,0}`。若只有容量 1、费用 3 的单边且请求上限 2，应返回 `{1,3}`，调用方需识别未满足全部需求。容量 0 的边不可用；自环和反向残量边不能误当成独立原始容量。

7. 坑

- 反向边费用符号错误：撤回一单位流要抵消原费用，因此费用为 `-cost`。
- 把返回流量当累计总流：每次 `run` 的局部计数从 0 开始，结果仅是本次增量。
- 请求固定流量却不检查返回值：算法可能因无增广路提前停止，需比较实际流量和目标。
- 有可达负费用环：SPFA 会持续改进距离，违反算法前提，可能不终止。
- 溢出：检查负费用取反、最短距离、`add*d[t]` 和总费用的范围；`long long` 不是无限精度。

8. 一句话原理

每次在残量图中找费用最低的增广路并推送瓶颈流量，同时用反向边保留改道能力。

6.16 树上差分 + LCA

1. 它是做什么的

一棵静态连通树，多次给路径上的所有点或边增加权值，最后统一读取各点或各边的总增量。

将每次路径修改变成常数个差分标记，最后一次自底向上汇总。

输入 1-based 无向树、根和若干 `(u,v,w)`；输出 `node[u]` 为点 `u` 总增量，`edge[u]` 为父边 `(parent[u],u)` 总增量，根的父边值为 0。允许负增量和重复路径；不改变原始权值。

复杂度：预处理时间、空间 $O(n \log n)$，每次加路径 $O(\log n)$，一次汇总 $O(n)$；q 次更新再统一汇总共 $O((n+q)\log n)$。逐次汇总查询会增加到 $O(nq)$，不适用于高频在线查询。

从数组差分开始。数组区间加可以只在左端点记开始、右端点之后记结束，最后前缀和还原。树没有唯一的“右端点”，但每个非根节点都有唯一父亲。我们改为让标记向父亲汇总：在 `u` 记 `+w`，就会让 `u` 到根的整条祖先链收到 `w`。负标记负责抵消不应该被修改的祖先。

为什么要用 LCA。记 $z = \mathrm{lca}\ (u, v)$。分别在 `u`、`v` 加 `w`，上行到 `z` 时两路汇合，`z` 及其以上祖先暂时收到两份贡献。对点路径，`z` 应收到一份而其父亲以上应收到零份，所以：

$$d_u \mathrel{+}= w,\quad d_v \mathrel{+}= w,\quad d_z \mathrel{-}= w,\quad d_{\mathrm{parent}\ (z)} \mathrel{-}= w.$$

父亲为虚拟点 0 时，最后一项直接跳过。对边路径，把每条边寄存在较深端点：路径上不包含 `z` 的父边，故在 `z` 减去两份：

$$e_u \mathrel{+}= w,\quad e_v \mathrel{+}= w,\quad e_z \mathrel{-}= 2w.$$

最后按孩子先于父亲的顺序执行 `d[parent[u]] += d[u]`。它是在计算每个点的差分子树和，不是沿 DFS 序直接做数组前缀和。点和边要用两套公式，不能混用。

手算。树边为 `1-2`，`1-3`，`2-4`，`2-5`。给路径 `4→2→5` 加 3，LCA 为 2。点差分先记 `d[4]=3,d[5]=3,d[2]=-3,d[1]=-3`；汇总后点 4、5、2 都是 3，点 1、3 是 0。边差分在点 2 记 −6，汇总后 `e[4]=e[5]=3,e[2]=0`，因此父边 `1-2` 没有被误加。若 `u=v`，点路径只含一个点，边路径为空，公式仍成立。

2. 变量在做什么

名称	含义
`lca`	6.10 节的倍增 LCA，`up[0][u]` 为父亲；根的父亲为 0。
`order`	从根开始、父亲先于孩子的遍历序；逆序汇总时孩子已完成计算。
`nodeDiff[u]`	点路径的差分标记，尚未汇总时不能当作节点答案。
`edgeDiff[u]`	边路径的差分标记，汇总后由较深端点代表父边。
`node[u]`、`edge[u]`	`collect()` 返回的总增量副本；根对应的 `edge` 值为 0。

名称	含义
`z`、`p`	更新路径的 LCA 及其父亲；点差分在 `p=0` 时跳过抵消。

3. 函数在做什么

函数	参数、返回与作用
`TreePathDifference(tree,root)`	输入非空的 1-based 连通无向树及合法根，建立 LCA、遍历序，清零两套差分，不修改原树。
`addNodes(u,v,w)`	给包含两端的点路径记录增量 `w`，返回 `void`，只改 `nodeDiff`。
`addEdges(u,v,w)`	给路径上的边记录增量 `w`，返回 `void`，只改 `edgeDiff`；同点路径没有边。
`collect()`	无参数，返回两套 1-based 总增量数组；在副本上汇总，不清空原标记，可重复调用或继续更新。

4. 怎么调用

下方复用 6.10 节的 `BinaryLiftingLCA`，先复制其定义。输入须为非空的合法树。所有差分值、子树累计值与 `2*w` 均须能放入 `long long`。构造时 BFS 得到父先于子的序列，反向处理即可避免这部分的递归栈。

```cpp
// 依赖：6.10 的 BinaryLiftingLCA，以及 <bits/stdc++.h> 和 using namespace std。
struct TreePathDifference {
    BinaryLiftingLCA lca;
    vector<int> order;
    vector<long long> nodeDiff, edgeDiff;

    // tree 为 1-based 非空连通无向树，root 在 1..n；建立 LCA/遍历序并清零差分，不修改 tree。
    TreePathDifference(const vector<vector<int>>& tree, int root = 1)
        : lca(tree, root), nodeDiff(tree.size()), edgeDiff(tree.size()) {
        order.push_back(root);
        for (size_t i = 0; i < order.size(); ++i) {
            int u = order[i];
            for (int v : tree[u]) if (v != lca.up[0][u]) order.push_back(v);
        }
    }

    // u,v 为 1..n 点号，w 为增量；返回 void，记录点路径（包含两端）的加法，暂不汇总。
    void addNodes(int u, int v, long long w) {
        int z = lca.lca(u, v), p = lca.up[0][z];
        nodeDiff[u] += w;
        nodeDiff[v] += w;
        nodeDiff[z] -= w;
        if (p) nodeDiff[p] -= w;
    }

    // u,v 为 1..n 点号，w 为增量；返回 void，记录路径所有边的加法；u==v 时总效果为零。
    void addEdges(int u, int v, long long w) {
        int z = lca.lca(u, v);
        edgeDiff[u] += w;
        edgeDiff[v] += w;
        edgeDiff[z] -= 2 * w;
    }

    // 无参数；返回 {点增量数组,父边增量数组}，均 1-based；不修改差分，可重复调用或继续添加路径。
    pair<vector<long long>, vector<long long>> collect() const {
        auto node = nodeDiff, edge = edgeDiff;
        for (int i = (int)order.size() - 1; i > 0; --i) {
            int u = order[i], p = lca.up[0][u];
            node[p] += node[u];
            edge[p] += edge[u];
```

```
        }
        edge[order[0]] = 0;
        return {node, edge};
    }
};
```

下面对树 `1-2,1-3,2-4,2-5` 的路径 `4→2→5` 分别增加点、边贡献。两套标记互不影响，需要两类结果时分别调用。

```
vector<vector<int>> tree = {{}, {2,3}, {1,4,5}, {1}, {2}, {2}};
TreePathDifference solver(tree, 1);
solver.addNodes(4, 5, 3);
solver.addEdges(4, 5, 3);
auto [node, edge] = solver.collect();
assert((node == vector<long long>{0,0,3,0,3,3}));
assert((edge == vector<long long>{0,0,0,0,3,3}));
assert(solver.collect() == make_pair(node, edge));
```

5. 要改哪里

- 原始点权或边权非零时，在最终增量上叠加原值；模板不会替调用者保存原权值。
- 统计路径经过次数时令 `w=1`，再找经过次数最多的边；若要求总运输成本，可将父边权乘以 `edge[u]` 后求和。
- 路径拥堵次数、批量负载累加适合本法；每次修改后立即查询任意路径和，应改用 HLD 等动态维护方法。差分不能直接维护路径最小值。
- 改根或换树时新建对象；`collect()` 允许重复调用，但每次都是 $O(n)$，不能据此把批量算法当成高效在线算法。

6. 自检

基础：按上例手算差分与子树和，再检查链、星形树、根为 LCA 的情况。`u=v` 时，点更新只影响一个点，边更新总效果为零。

实现：将随机路径逐点枚举的暴力结果与模板比较；检查负增量、重复路径、没有更新、重复汇总以及汇总后再更新。组合：计算所有路径的总边权贡献，并与逐条路径暴力求和比较。

7. 坑

1. 点路径沿用边差分公式，会把 LCA 的点贡献减掉；先明确维护点还是边。
2. 将 DFS 序做前缀和不能恢复树上差分；应按孩子先于父亲的顺序累计子树和。
3. 调用 `addNodes` 不会更新边标记；两类操作与结果分别维护。
4. 根没有父边，不能把 `edge[root]` 当成真实边答案。
5. `2*w`、差分和累计值都可能溢出，须分别检查 `long long` 范围。

8. 一句话原理

端点标记产生向根传播的贡献，在 LCA 及其父亲处抵消多余部分，再用一次子树汇总还原整批路径增量。

参考： OI Wiki：前缀和与差分（含树上差分）。

第七章

数学

本章默认整数为 `long long`，模数记为 `mod`。若无特别说明，模运算结果规范到 $[0, mod)$；乘法可能超过 `long long` 时用 `__int128`。

7.1 GCD 与扩展 GCD

1. 它是做什么的

最大公约数适用于整数；扩展欧几里得算法还会给出一组 Bézout 系数。本模板 `exgcd` 返回非负 gcd，并写出 `a*x+b*y=gcd`；`norm` 将余数规范到 `[0,mod)`。输入及递归系数乘积须能由 `long long` 表示，且不能含 `LLONG_MIN`。`exgcd` 为 $O(\log(\max(2, |a|, |b|)))$ 时间和递归栈空间；`norm` 为 $O(1)$。

2. 变量在做什么

名称	含义
`a, b`	当前递归的两个整数；C++ 余数规则决定递归参数。
`x, y`	输出引用；函数覆盖它们，使原始输入满足 Bézout 等式。
`x1, y1`	递归子问题 `b*x1+(a%b)*y1=g` 的系数。
`g`	子问题返回的非负 gcd。
`mod`	`norm` 的正模数；必须大于 0。

`exgcd(0,0)` 返回 0，并选择系数 `(1,0)`；这只是众多解中的一种。模余数的输出范围是 `[0,mod)`。

3. 函数在做什么

函数	参数与返回、副作用
`exgcd(a,b,x,y)`	输入 `a,b` 满足上述范围；返回非负 gcd，覆盖 `x,y` 为一组 Bézout 系数。
`norm(x,mod)`	要求 `mod>0`；返回 `x` 的规范余数，不修改调用者状态。

4. 怎么调用

调用扩展 GCD 后检查返回值和系数等式；`norm` 单独用于负余数归一化。

```
long long x, y;
long long g = exgcd(30, 18, x, y);
assert(g == 6 && 30 * x + 18 * y == g);
assert(norm(-7, 5) == 3);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 输入可为负数；返回非负 gcd，并令 a*x+b*y=gcd(a,b)。系数本身不取模。
// 参数 a,b 及递归系数中间值须可由 long long 表示，不能含 LLONG_MIN；返回 gcd>=0，并覆盖 x,y 为 Bézout 系数。
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        // 选 x 的符号是为了即使 a<0，右侧仍为非负的 |a|。
```

```
        x = (a >= 0 ? 1 : -1);
        y = 0;
        return llabs(a);
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    // 由 b*x1+(a%b)*y1=g 反代；若输入接近 long long 上界，乘法可能溢出。
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// 要求 mod>0；C++ 的负数余数可能为负，所以统一规范到 [0,mod)。
// 参数 mod>0；返回 x 的规范模余数 [0,mod)，不修改实参。
ll norm(ll x, ll mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
```

5. 要改哪里

求方程 `a*x+b*y=c` 的整数解时，先检查 `g!=0` 且 `c%g==0`，再将 Bézout 系数乘以 `c/g`；这一步仍需检查乘法范围。若只需要 gcd，使用 `std::gcd` 并确认输入范围符合其约束。模逆元问题需额外要求 gcd 为 1，并将系数用 `norm` 归一化。需要处理 `LLONG_MIN` 或更大中间值时，须改用更宽整数或大整数实现。

6. 自检

`exgcd(30,18)` 应返回 6，且 `30*x+18*y==6`。`exgcd(0,0)` 返回 0；`norm(-7,5)==3`，`norm(7,5)==2`。对任意正模数，规范化结果必须落在 `[0,mod)`。

7. 坑

1. 返回 gcd 却忽略系数输出：调用方要用引用参数 `x,y`。
2. 将负余数直接当数组下标：先调用 `norm`。
3. 输入包含 `LLONG_MIN`：`llabs` 无法表示其绝对值，必须拒绝或换更宽类型。
4. 系数反代溢出：`(a/b)*y1` 也可能超出 `long long`，输入可表示并不自动保证中间值安全。

8. 一句话原理

欧几里得递归先求余数对的 gcd，再用 `a%b=a-(a/b)b` 反代恢复原输入的一组系数。

参考：

- OI Wiki：最大公约数

7.2 快速幂

1. 它是做什么的

`qpow` 计算非负指数的模幂，采用二进制平方，把乘方次数降到对数级。要求 `e>=0`、`mod>0`；负底数会规范到 `[0,mod)`，`mod==1` 时返回 0，`0^0` 按空乘积约定返回 `1%mod`。预处理无，单次时间 $O(\log e)$，额外空间 $O(1)$。`mul_mod` 用 `__int128` 保存乘积后取模；其范围仍有限，但两个 long long 的乘积可容纳在有符号 128 位范围内。

2. 变量在做什么

名称	含义
`a`	循环中的底数幂，始终规范到 `[0,mod)`。
`e`	尚未处理的指数；每轮右移一位。
`ans`	已处理指数位对应的累乘结果，初值 `1%mod`。

名称	含义
`mod`	正模数。
`a`、`b`（`mul_mod`）	乘数；返回 C++ 有符号余数，负操作数时结果也可能为负。

3. 函数在做什么

函数	参数与返回、副作用
`mul_mod(a,b,mod)`	要求 `mod>0`；以 `__int128` 计算乘积并返回余数，不改实参。
`qpow(a,e,mod)`	要求 `e>=0`、`mod>0`；返回 `a^e mod mod` 的规范值，不修改外部状态；每次乘法调用 `mul_mod`。

4. 怎么调用

直接给底数、非负指数和正模数；以下片段包含负底数、零指数和模数 1 的核对：

```
assert(qpow(-2, 5, 13) == 7);
assert(qpow(9, 0, 7) == 1);
assert(qpow(123, 8, 1) == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 要求 mod>0，且 a、b 已在 long long 内；用 __int128 避免乘法先溢出再取模。
// 参数 mod>0；返回 a*b 的 C++ 余数，乘积在 __int128 中求值，不改实参。
ll mul_mod(ll a, ll b, ll mod) {
    // 提升后再乘，避免两个 long long 先溢出；要求 mod>0。
    return (ll)((__int128)a * b % mod);
}

// 参数 e>=0、mod>0；返回规范化的 a^e mod mod，内部不持有可变共享状态。
ll qpow(ll a, long long e, ll mod) {
    // e 必须非负；若题目出现负指数，应先求 a 的模逆元再调用。
    a %= mod;
    if (a < 0) a += mod;
    ll ans = 1 % mod;
    while (e > 0) {
        // 不变量：ans * a^e 与原始 a^原始e 在模 mod 下同余。
        if (e & 1) ans = mul_mod(ans, a, mod);
        a = mul_mod(a, a, mod);
        e >>= 1;
    }
    return ans;
}
```

5. 要改哪里

- 负指数不能传入当前循环；若要算模逆幂，先求底数的模逆元，再用非负指数调用。
- 若乘数范围超出 long long，当前接口无法接收；需改参数和乘法策略。
- 需要多次对同一模数求幂时可重复调用，函数无共享可变状态；若要减少模运算，可按题目批量预处理幂表。

6. 自检

`qpow`(a,0,m)==1%m 对所有 m>0 成立；`qpow`(−2,5,13)==7；模数 1 时任何合法输入都返回 0。检查指数为 1、偶数和奇数，结果都应落在 [0,m)。

7. 坑

1. 把负指数传入：循环不会实现模逆，先求逆元或拒绝该输入。

2. 直接用 `a*b%mod`：乘法可能在取模前溢出，使用 `mul_mod`。
3. 模数为 0 或负数：取模无效，调用前保证 `mod>0`。
4. 误认为 `mul_mod` 对负乘数返回规范余数：其结果遵循 C++ 余数规则；`qpow` 通过规范化底数保证幂运算状态非负。

8. 一句话原理

按指数二进制位累乘当前底数幂，并逐轮平方底数、右移指数。

参考：

- OI Wiki：快速幂

7.3 素数筛

1. 它是做什么的

`PrimeSieve(n)` 用线性筛预处理 `[0,n]`，同时生成递增质数表、最小质因子、最小质因子指数和正因数个数。构造时间与表空间均为 $O(n)$；`isPrime` 和 `countDivisors` 查询为 $O(1)$。适用于已知上界内的批量查询；只分解一个很大的数时，筛完整范围通常不划算。`countDivisors` 只接受 `1<=x<=n`；`isPrime` 对越界及小于 2 的输入返回 false。

2. 变量在做什么

成员	含义
`primes`	不超过 n 的递增质数。
`minPrime[x]`	x 的最小质因子；0、1 处为 0。
`minPrimeExponent[x]`	`minPrime[x]` 在 x 中的指数；0、1 处为 0。
`divisorCount[x]`	x 的正因数个数；`divisorCount[1]=1`，下标 0 仅占位。
`i,p,product`	当前整数、试用质数及合数 `i*p`。

当 p 等于 i 的最小质因子时，指数增加一并依公式修正因数个数，然后停止内层循环；否则 p 是新因子，因数个数加倍。

3. 函数在做什么

接口	参数、返回与副作用
`PrimeSieve(n)`	要求 `n>=0`；构造并填充所有表。对象成员保存结果。
`isPrime(x)`	任意整数输入；越界或 `x<2` 返回 false，不修改对象。
`countDivisors(x)`	要求 `1<=x<=n`；返回 x 的正因数个数，只读表。

4. 怎么调用

先构造上界足够大的筛对象，再查询。示例确认素数、越界和约数个数：

```
PrimeSieve sieve(12);
assert(sieve.isPrime(11));
assert(!sieve.isPrime(1) && !sieve.isPrime(13));
assert(sieve.countDivisors(12) == 6);
assert(sieve.countDivisors(1) == 1);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;

struct PrimeSieve {
    // 数组下标就是被筛整数；0、1 的 minPrime 和 minPrimeExponent 为 0，
divisorCount[0] 仅作占位。
```

```
    vector<int> primes, minPrime, minPrimeExponent, divisorCount;

    // 若 i=p^e*m 且 gcd(p,m)=1，则 d(i)=(e+1)d(m)。已有最小质因子时 d(ip)=d(i)/(e+1)*(e+2)，新增质因子时 d(ip)=2d(i)；前者先做精确整除。
    // 参数 n>=0；构造 [0,n] 筛表，填充素数、最小质因子、其指数和正因数个数；n=0/1 也有效。
    explicit PrimeSieve(int n)
        : minPrime(n + 1, 0), minPrimeExponent(n + 1, 0), divisorCount(n + 1, 0) {
        if (n >= 1) divisorCount[1] = 1;
        for (int i = 2; i <= n; ++i) {
            if (minPrime[i] == 0) {
                minPrime[i] = i;
                minPrimeExponent[i] = 1;
                divisorCount[i] = 2;
                primes.push_back(i);
            }
            for (int p : primes) {
                // 只用 p<=minPrime[i]：保证 i*p 恰由其最小质因子筛到一次。
                if (p > minPrime[i] || 1LL * i * p > n) break;
                int product = i * p;
                minPrime[product] = p;
                if (p == minPrime[i]) {
                    // 同一最小质因子指数由 e 增到 e+1，其余质因子的计数因子不变。
                    minPrimeExponent[product] = minPrimeExponent[i] + 1;
                    divisorCount[product] = divisorCount[i] /
                        (minPrimeExponent[i] + 1) *
(minPrimeExponent[product] + 1);
                    break;
                }
                // p<minPrime[i] 时新增一个一次因子 p，tau 值乘 2。
                minPrimeExponent[product] = 1;
                divisorCount[product] = divisorCount[i] * 2;
            }
        }
    }

    // 参数 x 为任意整数；返回 x 是否在筛范围内且为素数，越界返回 false，不改对象。
    bool isPrime(int x) const {
        return x >= 2 && x < (int)minPrime.size() && minPrime[x] == x;
    }

    // 参数 x 满足 1<=x<minPrime.size()；返回 x 的正因数个数，读取预处理表且不修改对象。
    int countDivisors(int x) const {
        return divisorCount[x];
    }
};
```

5. 要改哪里

要增加其他积性函数表，需在合数生成处按最小质因子的相同/不同情况推导递推式，并保持每个合数只被其最小质因子生成。若只需要判素数，可删除指数和约数数目两张表以节省空间。若要查询超出 n 的值，必须提高上界重建对象；当前查询不会动态扩表。

6. 自检

`n=0` 和 `n=1` 均可构造；筛到 10 时质数应为 2、3、5、7，`countDivisors(1)=1`、`countDivisors(12)=6`。`isPrime(0)`、`isPrime(1)` 及越界值均为 false。

7. 坑

1. 把下标 0 的约数计数解释为数学值：它仅是占位 0。
2. 用筛表判断超出上界的数：接口按设计返回 false，不代表它在数学上不是素数。

3. 把 `i*p` 先用 int 相乘：代码先以 long long 比较上界，避免乘积比较溢出。
4. 随意移除内层停止条件：会破坏线性筛每个合数只生成一次的性质。

8. 一句话原理

每个合数只由最小质因子生成一次，同时根据这个因子是否重复更新其指数与约数数目。

参考：

- OI Wiki：筛法

7.4 质因数分解

1. 它是做什么的

`factorize(n)` 对单个正整数试除，返回升序的“质因子、指数”对；要求 `n>=1`，输入按值传递，不改变调用者变量。`n=1` 返回空向量。最坏试除时间为 $O(\sqrt{n})$，输出空间为不同质因子个数；额外工作空间同阶于输出。大量有共同上界的数可改用最小质因子筛。

2. 变量在做什么

名称	含义
`n`	剩余待分解部分；每取出一个质因子的全部幂次就缩小。
`p`	从 2 递增尝试的除数，满足 `p<=n/p`。
`e`	当前质因子 p 的指数。
`fac`	按 p 递增追加的 `{p,e}` 结果；最后剩余的大于 1 的数是一个质因子。

试除上界随剩余 n 缩小，因此质因子取出后可减少后续扫描。

3. 函数在做什么

`factorize(ll n)` 要求正整数输入；返回 `vector<pair<ll,int>>`，每对按质因子升序排列。函数只修改自己的参数副本，不修改调用者变量；1 的分解为空。

4. 怎么调用

返回值可直接遍历，下面断言 360 的分解为 `2^3*3^2*5`，并检查 1：

```
auto f = factorize(360);
assert((f == vector<pair<long long,int>>{{2,3},{3,2},{5,1}}));
assert(factorize(1).empty());
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 n>=1；返回升序 {质因子,指数}，n==1 返回空表，不改调用者数据。
vector<pair<ll, int>> factorize(ll n) {
    // 要求 n>=1；返回质因子递增的 {质因子, 指数}，不记录因子 1。
    vector<pair<ll, int>> fac;
    // 用 p<=n/p 代替 p*p<=n，避免 long long 乘法溢出；n 会随试除缩小。
    for (ll p = 2; p <= n / p; ++p) {
        if (n % p != 0) continue;
        int e = 0;
        while (n % p == 0) {
            n /= p;
            ++e;
        }
        fac.push_back({p, e});
```

```
    }
    // 此时若仍有余数，它不可能再含两个质因子，因此本身就是最后一个质因子。
    if (n > 1) fac.push_back({n, 1});
    return fac;
}
```

5. 要改哪里

若需要约数个数，计算各指数加一的乘积；若需约数和，可按每个质因子对应的等比和相乘，并注意乘积类型范围。需要分解很多个不超过固定上界的整数时，预处理最小质因子通常更合算。输入为 0 或负数不在本函数定义域，不能直接套用。

6. 自检

`factorize(1)` 为空；`factorize(13)` 为 `{{13,1}}`；`factorize(360)` 的质因子依次为 2、3、5，指数为 3、2、1。对返回结果重乘各因子幂，应还原原输入。

7. 坑

1. 循环写 `p*p<=n`：乘法可能溢出；使用 `p<=n/p`。
2. 试除时不缩小 `n`：不仅效率较差，也可能误用原数的剩余状态。
3. 漏掉循环后 `n>1` 的剩余项：它是最后一个质因子。
4. 对 `n=0` 或负数调用：超出定义域，调用前验证正数。

8. 一句话原理

递增试除并一次取尽每个因子的幂次，最后剩余的大于 1 的部分必为质数。

参考：

- OI Wiki：质因数分解

7.5 欧拉函数

1. 它是做什么的

欧拉函数 `phi(n)` 统计 `1..n` 中与 `n` 互质的正整数个数。`eulerPhi` 单次试除，时间 $O(\sqrt{n})$、额外空间 $O(1)$；`phiSieve` 一次生成 `[0,n]` 整张表，时间和空间均为 $O(n)$。单次接口要求 `n>=1`，并约定 `phi(1)=1`；筛表接受 `n>=0`，`n=0` 时返回仅含占位 `phi[0]=0` 的向量。

2. 变量在做什么

名称	含义
`n`	当前待处理整数；试除版本会移除已处理质因子。
`ans`	起始为原 `n`，依次乘每个不同质因子的 `(p-1)/p`。
`phi[x]`	筛表中整数 `x` 的欧拉函数；0 号位为占位。
`primes`、`composite`	线性筛的质数表与合数标记。
`p`	当前试用质数；若整除 `i`，生成 `i*p` 后停止内层循环。

3. 函数在做什么

函数	参数、返回与副作用
`eulerPhi(n)`	要求 `n>=1`；返回单个欧拉函数值，不修改外部状态。
`phiSieve(n)`	要求 `n>=0`；返回长度 `n+1` 的新表，局部筛数组不会泄漏到外部。

4. 怎么调用

单点和整表接口都可独立使用；下例核对 `phi(9)=6`、`phi(1)=1` 和筛表的 `phi(10)=4`：

```
assert(eulerPhi(9) == 6);
assert(eulerPhi(1) == 1);
auto phi = phiSieve(10);
assert(phi.size() == 11 && phi[10] == 4 && phi[0] == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 n>=1；返回 phi(n)，不修改 n 或外部状态。
ll eulerPhi(ll n) {
    // 要求 n>=1；ans 始终等于原 n 乘上已处理质因子的 (p-1)/p。
    ll ans = n;
    for (ll p = 2; p <= n / p; ++p) {
        if (n % p != 0) continue;
        while (n % p == 0) n /= p;
        // p 一定整除当前 ans，先除后乘既保持精确，也降低中间溢出风险。
        ans = ans / p * (p - 1);
    }
    if (n > 1) ans = ans / n * (n - 1);
    return ans;
}

// 参数 n>=0；返回下标 [0,n] 的 phi 表，分配局部筛状态，不修改外部对象。
vector<int> phiSieve(int n) {
    // phi[x] 对应整数 x；线性筛中每个合数只由最小质因子产生一次。
    vector<int> phi(n + 1), primes;
    vector<bool> composite(n + 1, false);
    if (n >= 1) phi[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (!composite[i]) {
            primes.push_back(i);
            phi[i] = i - 1;
        }
        for (int p : primes) {
            if (1LL * i * p > n) break;
            composite[i * p] = true;
            if (i % p == 0) {
                // p 已整除 i：新增一个 p 因子，phi(i*p)=phi(i)*p，随后必须停止。
                phi[i * p] = phi[i] * p;
                break;
            }
            // gcd(i,p)=1，乘入新质因子 p 时欧拉函数乘以 p-1。
            phi[i * p] = phi[i] * (p - 1);
        }
    }
    return phi;
}
```

5. 要改哪里

只需单个值时用 `eulerPhi`；需要大量有界查询时用 `phiSieve`。若要计算互质计数以外的积性函数，需重推筛内递推，不能只换答案变量。欧拉定理 `a^phi(n) ≡ 1 (mod n)` 还要求 `gcd(a,n)=1`，不能把它当作对所有底数成立。

6. 自检

`phi(1)=1`，素数 p 的 `phi(p)=p-1`，`phi(9)=6`、`phi(10)=4`。筛到 0 返回长度 1 且 `phi[0]=0`；筛到 1 应有 `phi[1]=1`。

7. 坑

1. 把 `phi(1)` 设为 0：定义下它等于 1。

2. 同一质因子重复更新：先取尽 p 的所有次幂，再执行一次 `ans=ans/p*(p-1)`。
3. 乘除顺序颠倒：先整除再乘，保持精确并降低中间值风险。
4. 无条件使用欧拉定理：底数与模数必须互质。

8. 一句话原理

每个不同质因子 p 都将当前计数按比例乘以 `(p-1)/p`，线性筛则同时传播这一规则。

参考：

- OI Wiki：欧拉函数

7.6 模逆元

1. 它是做什么的

`modInverse(a,mod)` 求模乘法逆元，支持质数或合数模；逆元存在当且仅当 `gcd(a,mod)=1`。要求 `mod>1`，a 可带符号，函数先规范底数。返回 `{true,x}` 时 `x∈[0,mod)` 且 `a*x≡1 (mod mod)`；不存在则返回 `{false,0}`。时间 $O(\log mod)$，递归栈空间 $O(\log mod)$。

2. 变量在做什么

名称	含义
`a`	先取模并规范为非负值，再作为 exgcd 的一侧输入。
`mod`	大于 1 的模数。
`x,y`	扩展 GCD 写出的 Bézout 系数，满足 `a*x+mod*y=gcd(a,mod)`。
`g`	最大公约数；仅当 `g==1` 时 x 才是逆元。

3. 函数在做什么

`modInverse(a,mod)` 返回 `pair<bool,ll>`。第一项表示是否存在，第二项在成功时为规范逆元，失败时固定为 0。内部根调用为 `exgcd(a,mod,x,y)`，其中 a 已规范到 [0,mod)、b=mod；函数返回 gcd 并覆盖 x、y 为 Bézout 系数。递归系数中间值须可由 `long long` 表示。

4. 怎么调用

先检查存在标志，再使用逆元；以下核对模 64 的逆元及不互质情况：

```
auto inv = modInverse(35, 64);
assert(inv.first && inv.second == 11);
assert((35 * inv.second) % 64 == 1);
auto missing = modInverse(6, 12);
assert(!missing.first && missing.second == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 a,b 为非负且递归系数运算须可表示；返回 gcd 并写 Bézout 系数 x,y。
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
```

```
    return g;
}

// 参数 mod>1、a 可带符号；返回 {是否存在,规范逆元}，不互质时为 {false,0}。
pair<bool, ll> modInverse(ll a, ll mod) {
    // 要求 mod>1；返回 false 表示 gcd(a,mod)!=1，此时任何模除法都无定义。
    a %= mod;
    if (a < 0) a += mod;
    ll x, y;
    ll g = exgcd(a, mod, x, y);
    if (g != 1) return {false, 0};
    // Bézout 等式给出 a*x+mod*y=1，所以 x 就是逆元；规范到 [0,mod)。
    x %= mod;
    if (x < 0) x += mod;
    return {true, x};
}
```

5. 要改哪里

若要计算 `b/a mod m`，先求 `a` 的逆元，存在时再乘 b 并取模。若使用费马小定理 `a^(p-2)`，必须确认模数 p 为质数且 a 不被 p 整除；当前扩展 GCD 方法也支持合数模。若需要更宽输入或中间系数，需换成大整数实现。

6. 自检

`modInverse(35,64)` 应为 11；`modInverse(6,12)` 不存在；任何成功结果都应满足乘积模 m 等于 1。`a=0` 或 a 是模数倍数时无逆元。

7. 坑

1. 不检查存在标志就做除法：不互质时返回的 0 不是逆元。
2. 合数模数也套费马小定理：费马形式要求质数模数。
3. 结果未规范化：成功结果已保证落在 `[0,mod)`。
4. 模数取 1 或非正数：不满足函数前提。

8. 一句话原理

扩展 GCD 给出 `a*x+mod*y=1` 时，系数 x 正是 a 的模逆元。

参考：

- OI Wiki：模逆元

7.7 线性同余方程

1. 它是做什么的

求解线性同余式 `a*x≡b (mod m)`，要求 `m>0`，系数和常数可为负。函数返回 `{exists,x0,step}`；有解时所有整数解为 `x=x0+k*step`，其中 `x0∈[0,step)`、`step=m/gcd(a,m)`。无解返回 `false,0,0`。扩展 GCD 时间和递归栈空间为 $O(\log m)$，其余工作为常数空间。

2. 变量在做什么

名称	含义
`a,b`	先按 m 规范化的系数与常数。
`m`	正模数；保持方程的同余意义。
`g`	`gcd(a,m)`；可解条件是 `g|b`。
`step`	解的周期 `m/g`。
`x,y`	扩展 GCD 得到的 Bézout 系数。
`x0`	约去 g 后求得并规范到 `[0,step)` 的一个解。

3. 函数在做什么

`solveCongruence(a,b,m)` 返回 `CongruenceSolution`。`m` 必须为正；a、b 可以带符号。`exists=false` 时其余字段为 0；成功时代表一个模 step 的剩余类。内部根调用为 `exgcd(a,m,x,y)`，其中 a 已规范到 [0,m)、b=m；函数返回 gcd 并覆盖 x、y 为 Bézout 系数，递归系数运算须可由 `long long` 表示。

4. 怎么调用

调用后先检查 exists；以下方程 `6x≡8 (mod 14)` 的代表解为 6、周期为 7：

```
auto s = solveCongruence(6, 8, 14);
assert(s.exists && s.x0 == 6 && s.step == 7);
assert((6 * s.x0 - 8) % 14 == 0);
auto no = solveCongruence(4, 3, 6);
assert(!no.exists && no.x0 == 0 && no.step == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 a,b 为非负；返回 gcd 并覆盖 x,y 为 Bézout 系数。
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}

struct CongruenceSolution {
    bool exists;
    // exists=true 时全部解为 x=x0+k*step，且 x0 位于 [0,step)。
    ll x0, step;
};

// 参数 m>0、a,b 可带符号；返回解存在标志及 x0、step，不解时三字段为 false,0,0。
CongruenceSolution solveCongruence(ll a, ll b, ll m) {
    // 要求 m>0；负系数和负常数先规范化，方程语义不变。
    a %= m;
    b %= m;
    if (a < 0) a += m;
    if (b < 0) b += m;
    ll x, y;
    ll g = exgcd(a, m, x, y);
    // ax≡b (mod m) 可解当且仅当 gcd(a,m) 整除 b。
    if (b % g != 0) return {false, 0, 0};
    ll step = m / g;
    // 约去 g 后 x 是 a/g 在模 m/g 下的逆元；乘法用 __int128 防溢出。
    ll x0 = (ll)((__int128)x * (b / g) % step);
    if (x0 < 0) x0 += step;
    return {true, x0, step};
}
```

5. 要改哪里

枚举模 m 内全部解时，取 `x0+k*step`，`0<=k<g`。若需最小非负解，当前 `x0` 已满足。改求其他同余方程组时可将此接口作为一步合并器；模数必须保持为正。涉及超出 long long 的乘积时，应替换 __int128/long long 的数据类型。

6. 自检

`6x≡8 (mod 14)` 有 2 个模 14 解：6、13，返回代表类 6 mod 7。`4x≡3 (mod 6)` 无解，因为 gcd 为 2 不整除 3。若 `a=0`，方程只有在 b 是 m 的倍数时才有解。

7. 坑

1. 未检查 `b%g==0`：不满足时不能继续求逆元。
2. 约去 g 后仍以 m 为周期：正确周期是 `m/g`。
3. 只检查一个代表解：返回值描述整个周期类，不是唯一整数解。
4. 忽略负输入：函数会规范化 a、b，但 m 仍须为正。

8. 一句话原理

先用 gcd 判定常数能否整除，再约去 gcd，把问题化为一个互质模数下的逆元求解。

参考：

- OI Wiki：线性同余方程

7.8 中国剩余定理（非互质模数）

1. 它是做什么的

合并多条同余式 `x≡rem[i] (mod mod[i])`，允许模数不互质。要求两向量等长、每个模数为正，且逐步最小公倍数可由 `long long` 表示。成功返回 `{true,r,m}`，表示全部解 `x=r+k*m` 且 `r∈[0,m)`；矛盾返回 `{false,0,0}`。空输入按 `0 mod 1` 返回。k 条式子的时间约为 $O(k \log M)$，其中 M 是过程中模数的量级；递归栈空间 $O(\log M)$，循环变量额外空间 $O(1)$。

2. 变量在做什么

名称	含义
`rem[i],mod[i]`	第 i 条同余式的余数和正模数。
`r,m`	已合并方程组的代表余数和周期，初值为 `0 mod 1`。
`b`	当前余数规范到 `[0,mod[i])` 后的值。
`g`	`gcd(m,mod[i])`；必须整除两余数之差才能合并。
`q`	新模数除以 g；新周期为 `m*q`。
`t`	使新解满足当前同余式的步数参数。

3. 函数在做什么

`crt(rem,mod)` 要求向量等长且模数均为正；返回 `CRTResult`，矛盾时 `exists=false`。成功后 remainder 已规范，modulus 是逐步 lcm。每轮内部根调用为 `exgcd(m,mod[i],x,y)`，两个参数均为正；函数返回 gcd 并覆盖 x、y 为 Bézout 系数。递归栈占用 $O(\log M)$。中间 lcm 以 __int128 计算后转回 long long，因此输入必须保证结果可表示。

4. 怎么调用

传入并行的余数、模数数组。示例两式兼容，另外一组不兼容，空组具有约定结果：

```
auto ok = crt(vector<long long>{2, 5}, vector<long long>{6, 9});
assert(ok.exists && ok.remainder == 14 && ok.modulus == 18);
auto bad = crt(vector<long long>{1, 2}, vector<long long>{4, 6});
assert(!bad.exists);
auto empty = crt(vector<long long>{}, vector<long long>{});
assert(empty.exists && empty.remainder == 0 && empty.modulus == 1);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```cpp

// 参数 a,b 为正；返回 gcd 并覆盖 x,y 为 Bézout 系数，要求递归乘积不溢出。
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g;
}

struct CRTResult {
    bool exists;
    // exists=true 时全部解为 x=remainder+k*modulus，余数已规范化。
    ll remainder, modulus;
};

// 参数 rem/mod 等长、每个模数正且逐步 lcm<=LLONG_MAX；返回规范解类或矛盾标志，不改输入。
CRTResult crt(const vector<ll>& rem, const vector<ll>& mod) {
    // 要求 rem.size()==mod.size() 且每个 mod[i]>0；空方程组返回 x≡0 (mod 1)。
    ll r = 0, m = 1;
    for (int i = 0; i < (int)rem.size(); ++i) {
        ll b = rem[i] % mod[i];
        if (b < 0) b += mod[i];
        ll x, y;
        ll g = exgcd(m, mod[i], x, y);
        ll diff = b - r;
        // 令新解为 r+m*t；只有 gcd(m,mod[i]) 能整除 b-r 时才可合并。
        if (diff % g != 0) return {false, 0, 0};

        ll q = mod[i] / g;
        // x 是 m/g 对模 q 的逆元；t 取最小非负剩余类即可。
        ll t = (ll)((__int128)(diff / g) * x % q);
        if (t < 0) t += q;
        __int128 newMod = (__int128)m * q;
        __int128 newR = (__int128)r + (__int128)m * t;
        // 若题目不保证最终 lcm 可放入 long long，需改用大整数。
        m = (ll)newMod;
        r = (ll)(newR % m);
        if (r < 0) r += m;
    }
    return {true, r, m};
}
```

5. 要改哪里

若模数保证两两互质，可使用经典 CRT，但当前算法无需该限制。若逐步 lcm 可能超出 long long，需将返回类型和中间运算全部改成大整数；只把乘法临时提升后立即窄化并不能防止最终溢出。需要报告某条冲突约束时，可扩展结果类型保存冲突位置。

6. 自检

`x≡2 (mod 6), x≡5 (mod 9)` 合并为 `14 mod 18`；`x≡1 (mod 4), x≡2 (mod 6)` 不相容。空方程组返回 `0 mod 1`。对成功结果，代回每条输入同余式均应成立。

7. 坑

1. 直接套互质模数公式：不互质时须先检查差值能否被 gcd 整除。
2. 把模数乘积当作新周期：一般新周期是 lcm，即 `m*(mod[i]/g)`。
3. 余数为负未规范：代码会先规范当前输入余数及合并结果。
4. 忽略范围约束：最终 lcm 和余数更新都不能超出 long long 可表示范围。

8. 一句话原理

逐条合并时先用 gcd 检查相容，再在约去公因数后的模数上解一个线性同余式。

参考：

- OI Wiki：中国剩余定理

7.9 组合数

1. 它是做什么的

`Combinations(N,mod)` 预处理组合数阶乘与逆阶乘，适合固定质数模数下反复查询。要求 `mod` 为大于 1 的质数且 `0<=N<mod`。构造时间 $O(N+\log mod)$，空间 $O(N)$；表内一次 `C(n,k)` 查询 $O(1)$。`n>N`、`k<0` 或 `k>n` 按约定返回 0。

2. 变量在做什么

成员	含义
`mod`	所有运算使用的质数模数。
`fact[i]`	`i! mod mod`。
`invFact[i]`	阶乘 `i!` 的模逆。
`n,k`	查询的组合数参数；有效范围为 `0<=k<=n<=N`。

逆阶乘从 `N!` 的逆元向下推，依赖模数为质数且 `N<mod`，此时表内阶乘都非零。

3. 函数在做什么

函数	契约
`qpow(a,e,mod)`	内部辅助，要求 `0<=a<mod`、`e>=0`、质数 `mod>1`；返回规范模幂。
`Combinations(N,mod)`	建立并持有阶乘表；要求 `0<=N<mod`、mod 为质数。
`C(n,k)`	返回 `C(n,k) mod mod`；非法 k 或超出预处理上界返回 0，不修改表。

4. 怎么调用

同一对象可回答多次查询；示例使用质数模 7 且预处理到 6：

```
Combinations comb(6, 7);
assert(comb.C(5, 2) == 3); // 10 mod 7
assert(comb.C(6, 0) == 1);
assert(comb.C(3, 4) == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 0<=a<mod、e>=0、mod>1；返回规范的 a^e 模余数，供阶乘逆元计算。
ll qpow(ll a, ll e, ll mod) {
    ll ans = 1;
    while (e > 0) {
        if (e & 1) ans = (ll)((__int128)ans * a % mod);
        a = (ll)((__int128)a * a % mod);
        e >>= 1;
    }
    return ans;
}

struct Combinations {
    ll mod;
```

```
    // fact[i]=i!, invFact[i]=(i!)^{-1}; 数组下标覆盖 [0,n]。
    vector<ll> fact, invFact;

    // 参数 0<=n<mod、mod 为质数; 构造并拥有 0..n 阶乘和逆阶乘表。
    Combinations(int n, ll mod) : mod(mod), fact(n + 1), invFact(n + 1) {
        // 前提: mod 为质数且 0<=n<mod，否则 fact[n] 可能为 0 且没有逆元。
        fact[0] = 1;
        for (int i = 1; i <= n; ++i)
            fact[i] = (ll)((__int128)fact[i - 1] * i % mod);
        // 费马小定理只在质数模下成立; 改成任意模数时需替换整套求逆方案。
        invFact[n] = qpow(fact[n], mod - 2, mod);
        for (int i = n; i >= 1; --i)
            invFact[i - 1] = (ll)((__int128)invFact[i] * i % mod);
    }

    // 参数 n,k 为整数; 返回 C(n,k) mod mod，越出预处理范围或 k 不合法时返回 0，不改表。
    ll C(int n, int k) const {
        // 返回 C(n,k) mod mod; 非法 k 或超出预处理上界时约定返回 0。
        if (k < 0 || k > n || n >= (int)fact.size()) return 0;
        ll ans = (ll)((__int128)fact[n] * invFact[k] % mod);
        return (ll)((__int128)ans * invFact[n - k] % mod);
    }
};
```

5. 要改哪里

若 N 可能达到或超过 mod，阶乘将为 0，当前逆阶乘方案失效；需换 Lucas 定理或其他适合模数/范围的算法。若模数是合数，费马小定理求逆不可用，不能只把 mod 参数换掉。小 N 且任意模数时，可用 Pascal 递推，但单次/整体复杂度不同。

6. 自检

`C(5,2)=10`，模 7 得 3；`C(n,0)=C(n,n)=1`；`k>n` 返回 0。若 `N=0` 且模数满足前提，`C(0,0)=1`。

7. 坑

1. 将模板用于合数模：阶乘未必可逆。
2. 取 `N>=mod`：预处理阶乘可能为 0。
3. 对非法 k 访问数组：接口返回 0，应保留参数检查。
4. 每次查询重新构造对象：重复预处理会浪费时间，应复用同一表。

8. 一句话原理

用阶乘和逆阶乘把组合数公式拆成三次数组读取与常数次乘法。

参考：

- OI Wiki：组合数学

7.10 容斥原理

1. 它是做什么的

`countDivisible(n,a)` 统计 `[1,n]` 中至少被一个给定正除数整除的整数个数。要求 `n>=0`，输入除数非负，且有效条件数少于 64；0 会忽略，重复项和冗余倍数会删除。枚举所有非空子集，设原输入长度为 `t`，清理后有效除数数目为 k。预处理时间为 $O(t \log t + tk)$，其中排序去重后再检查冗余倍数；枚举子集时含 gcd 的时间为 $O(2^k k \log(n+1))$，额外空间为 $O(t)$，含按值传入的副本。超出 n 的 lcm 子集贡献为 0。

2. 变量在做什么

名称	含义
`a`	按值传入的除数副本；排序、去重和删冗余不会改调用者向量。
`b`	清理后的有效除数集合。
`mask`	当前非空子集，位 i 表示是否选中 `b[i]`。
`lcm`	当前子集交集对应的最小公倍数。
`g`	更新 lcm 时的 gcd。
`ans`	容斥累计值；奇数项加、偶数项减。
`n`	闭区间 `[1,n]` 的右端点。

如果已有较小除数 y 整除 x，则被 x 整除的数已经包含在 y 的集合里，x 可删。

3. 函数在做什么

`countDivisible(n,a)` 返回并集计数；`n>=0`、除数非负、有效数量少于 64 是调用前提。它接收向量副本并在本地清理，不修改调用者状态。lcm 超过 n 时跳过该子集。

4. 怎么调用

示例统计 1..20 中 2 或 3 的倍数；集合大小为 10+6-3=13：

```
assert(countDivisible(20, vector<long long>{2, 3}) == 13);
assert(countDivisible(10, vector<long long>{2, 4, 0, 2}) == 5);
assert(countDivisible(0, vector<long long>{2, 3}) == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 n>=0、a 含非负除数且有效项数<64；返回 [1,n] 并集计数，传值副本可排序删项。
ll countDivisible(ll n, vector<ll> a) {
    // 统计闭区间 [1,n]；模板假设 n>=0、除数为非负数，且有效条件数 k<64。
    sort(a.begin(), a.end());
    a.erase(remove(a.begin(), a.end(), 0), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    vector<ll> b;
    for (ll x : a) {
        bool redundant = false;
        // 若 x 是已有 y 的倍数，“能被 x 整除”已包含于“能被 y 整除”，可删除。
        for (ll y : b) if (x % y == 0) {
            redundant = true;
            break;
        }
        if (!redundant && x <= n) b.push_back(x);
    }

    ll ans = 0;
    int k = (int)b.size();
    for (unsigned long long mask = 1; mask < (1ULL << k); ++mask) {
        // 一个子集的交集就是被这些除数的 lcm 整除；奇数个集合加，偶数个减。
        ll lcm = 1;
        bool tooLarge = false;
        for (int i = 0; i < k; ++i) if (mask >> i & 1ULL) {
            ll g = std::gcd(lcm, b[i]);
            // 一旦 lcm>n，本子集贡献为 0；先除后比较同时避免 lcm 乘法溢出。
            if (lcm > n / (b[i] / g)) { tooLarge = true; break; }
            lcm *= b[i] / g;
        }
        if (tooLarge) continue;
```

```
35          if (__builtin_popcountll(mask) & 1) ans += n / lcm;
36          else ans -= n / lcm;
37      }
38      return ans;
39  }
```

5. 要改哪里

若要统计闭区间 [L,R]，先计算不超过 R 的计数减去不超过 L-1 的计数，注意 L 的范围及减法溢出。若条件数较大，子集枚举不可行，应利用数论结构或改用其他算法。负除数不在输入约定内；先取绝对值时还要处理最小有符号整数的溢出风险。

6. 自检

`n=20`、除数 2 和 3 的答案为 13；除数集合含 2、4、0、2 时，清理后只剩 2，`n=10` 答案为 5。`n=0` 返回 0；空除数集合也应返回 0。

7. 坑

1. 容斥符号统一相加：子集大小奇数加、偶数减。
2. 直接相乘求 lcm：先约 gcd 并检查是否超过 n，避免溢出。
3. 忘记冗余项：正确性仍可成立，但会扩大指数枚举规模。
4. 有效条件数达到 64：左移掩码不再满足实现前提，且复杂度早已不可接受。

8. 一句话原理

把多个整除集合的并集按子集交集逐项加减，交集由所选除数的 lcm 描述。

参考：

- OI Wiki：容斥原理

7.11 矩阵快速幂

1. 它是做什么的

`Matrix` 支持固定模数下的矩阵乘法，`matPow` 对方阵计算非负整数次幂。维度 d 的方阵幂时间 $O(d^3 \log e)$，结果及中间矩阵空间 $O(d^2)$；每次乘法为 $O(r*c*k)$，其中维度为 r×k 与 k×c。适合小维线性递推和状态转移。模数须为正，矩阵乘法要求内维匹配且模数一致；幂指数非负，零次幂返回单位矩阵。写入矩阵的元素应先规范到 `[0,mod)`，否则 C++ 负余数可能产生负代表。

2. 变量在做什么

成员/变量	含义
`rows,cols`	行数和列数；元素下标从 0 开始。
`mod`	矩阵使用的正模数。
`a[i][j]`	第 i 行第 j 列元素，构造时为 0。
`x,y`	矩阵乘法左右操作数；要求 `x.cols==y.rows` 且模数相同。
`ans`	幂运算累积矩阵，初始为单位阵。
`e`	尚未处理的指数位，每轮右移。

3. 函数在做什么

接口	参数、返回与副作用
`Matrix(r,c,mod)`	要求非负维度、正模数；建立零矩阵。
`Matrix::identity(n,mod)`	返回 n 阶单位矩阵；不改外部状态。

接口	参数、返回与副作用
`operator*(x,y)`	返回新乘积，不修改输入；需维度及模数匹配，代码未做运行时检查。
`matPow(base,e)`	按值接收矩阵并返回幂；要求方阵和 `e>=0`，不改调用者的矩阵。

4. 怎么调用

创建 2×2 Fibonacci 转移矩阵并计算五次幂，检查关键元素与零次幂单位阵：

```
Matrix a(2, 2, 1000);
a.a[0][0] = 1; a.a[0][1] = 1;
a.a[1][0] = 1; a.a[1][1] = 0;
Matrix p = matPow(a, 5);
assert(p.a[0][0] == 8 && p.a[0][1] == 5);
Matrix id = matPow(a, 0);
assert(id.a[0][0] == 1 && id.a[0][1] == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Matrix {
    int rows, cols;
    // 本模板所有元素都在同一正模数 mod 下运算；不同矩阵相乘时模数须一致。
    ll mod;
    vector<vector<ll>> a;

    // 参数 r,c>=0、mod>0；构造并拥有 r*c 个零元素。
    Matrix(int r, int c, ll mod) : rows(r), cols(c), mod(mod), a(r,
vector<ll>(c)) {}

    // 参数 n>=0、mod>0；返回 n 阶模乘单位阵，不修改外部状态。
    static Matrix identity(int n, ll mod) {
        Matrix id(n, n, mod);
        for (int i = 0; i < n; ++i) id.a[i][i] = 1 % mod;
        return id;
    }
};

// 参数矩阵满足 x.cols==y.rows 且模数相同为正；返回新乘积矩阵，不改 x,y。
Matrix operator*(const Matrix& x, const Matrix& y) {
    // 前提 x.cols==y.rows；i-k-j 顺序可跳过零项，乘积用 __int128 防溢出。
    Matrix z(x.rows, y.cols, x.mod);
    for (int i = 0; i < x.rows; ++i)
        for (int k = 0; k < x.cols; ++k) if (x.a[i][k])
            for (int j = 0; j < y.cols; ++j)
                z.a[i][j] = (z.a[i][j] +
                    (__int128)x.a[i][k] * y.a[k][j]) % x.mod;
    return z;
}

// 参数 base 为正模方阵、e>=0；按值接收底矩阵并返回 base^e，实参不变。
Matrix matPow(Matrix base, long long e) {
    // 要求 base 为方阵、e>=0；列向量转移通常写 state(t)=base^t*state(0)。
    Matrix ans = Matrix::identity(base.rows, base.mod);
    while (e > 0) {
        // 不变量：ans * base^e 等于原矩阵的原指数次幂。
        if (e & 1) ans = ans * base;
        base = base * base;
        e >>= 1;
    }
```

```
    return ans;
}
```

5. 要改哪里

列向量状态满足 `state(t+1)=A*state(t)` 时使用 `A^t*state(0)`；行向量约定需反转乘法方向。若不需要模运算，应改乘法和数据类型并重新评估溢出；若维数大，稠密 $O(d^3)$ 乘法可能不合适，应利用稀疏结构或其他算法。不同模数的矩阵不能直接相乘。

6. 自检

`A^0` 必须为单位阵；`A^1=A`。对 Fibonacci 矩阵，上例五次幂左上角为 8、右上角为 5；`1×1` 矩阵也应符合普通整数模幂。

7. 坑

1. 转移方向不一致：先确定列向量或行向量约定。
2. 忘记维度/模数前提：模板没有断言，应由调用者保证。
3. 对负指数调用：循环不支持矩阵逆。
4. 使用 long long 直接乘矩阵元素：实现已提升乘积至 __int128，但模数和加和仍须满足表示范围。

8. 一句话原理

矩阵乘法组合状态转移，二进制快速幂把重复转移压缩为对数次方阵乘法。

参考：

- OI Wiki：矩阵

7.12 高斯消元

1. 它是做什么的

`gauss` 使用部分主元消元求解有限实数系数的线性方程组。输入增广矩阵至少有一行且每行同宽；返回 0 无解、1 唯一解、2 无穷多解。`answer` 在有解时给出一组解，自由变量取 0；无解时不可使用。时间 $O(mn\min(m,n))$，空间 $O(mn)$，包括按值复制的矩阵和辅助标记。固定绝对 `EPS` 适用于尺度适合的数据，病态问题需更稳健的数值算法。

2. 变量在做什么

名称	含义
`a[i][j]`	增广矩阵；最后一列 `a[i][n]` 为常数项。
`m`,`n`	方程数和未知数数目。
`where[col]`	列 `col` 主元所在行；-1 表示自由变量。
`row`,`pivot`	下一主元行和当前列的最大绝对值候选行。
`answer`	输出解向量；初始化为零，故自由变量取 0。
`EPS`	固定阈值 1e-12，用于判零及残差检查。

3. 函数在做什么

`gauss(a,answer)` 要求 `a` 至少有一行且每行 `n+1` 列，返回 0/1/2 状态码。函数按值接收矩阵副本并覆盖 `answer`；无解时 `answer` 不可用。输入矩阵不被调用修改。

4. 怎么调用

最后一列放常数项。下面解 x+y=3、x-y=1，另检查矛盾方程：

```
vector<vector<long double>> a = {{1, 1, 3}, {1, -1, 1}};
vector<long double> answer;
assert(gauss(a, answer) == 1);
```

```
assert(fabsl(answer[0] - 2) < 1e-12L);
assert(fabsl(answer[1] - 1) < 1e-12L);
vector<vector<long double>> bad = {{1, 1, 1}, {1, 1, 2}};
assert(gauss(bad, answer) == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;

// 0: 无解; 1: 唯一解; 2: 无穷多解。
// 参数 a 为有限数值且形状 m>0、每行 n+1; 覆盖 answer，返回 0无解/1唯一/2多解。无解时 answer 不可用。
int gauss(vector<vector<long double>> a, vector<long double>& answer) {
    // a 是 m 行、n+1 列的增广矩阵; 模板要求 m>0 且各行长度一致。
    // EPS 应按输入尺度调整: 数值很大或病态矩阵可能需要更稳健的方法。
    const long double EPS = 1e-12L;
    int m = (int)a.size();
    int n = (int)a[0].size() - 1;
    vector<int> where(n, -1);
    // where[col] 是该未知数主元所在行; -1 表示自由变量，最终默认取 0。
    int row = 0;

    for (int col = 0; col < n && row < m; ++col) {
        int pivot = row;
        // 部分主元: 选绝对值最大项，减少除以小数造成的误差放大。
        for (int i = row; i < m; ++i)
            if (fabsl(a[i][col]) > fabsl(a[pivot][col])) pivot = i;
        // 整列近似为 0 时不能制造主元，这一列成为自由变量候选。
        if (fabsl(a[pivot][col]) < EPS) continue;
        swap(a[pivot], a[row]);
        where[col] = row;

        long double div = a[row][col];
        for (int j = col; j <= n; ++j) a[row][j] /= div;
        // 消去所有其他行，直接得到简化行阶梯形; 只消下方则需额外回代。
        for (int i = 0; i < m; ++i) if (i != row) {
            long double factor = a[i][col];
            if (fabsl(factor) < EPS) continue;
            for (int j = col; j <= n; ++j)
                a[i][j] -= factor * a[row][j];
        }
        ++row;
    }

    answer.assign(n, 0);
    for (int col = 0; col < n; ++col)
        if (where[col] != -1) answer[col] = a[where[col]][n];

    for (int i = 0; i < m; ++i) {
        // 用得到的一组解回代等价方程; 不满足说明出现 0=非0，即无解。
        long double lhs = 0;
        for (int j = 0; j < n; ++j) lhs += a[i][j] * answer[j];
        if (fabsl(lhs - a[i][n]) > EPS) return 0;
    }
    for (int col = 0; col < n; ++col)
        if (where[col] == -1) return 2;
    return 1;
}
```

5. 要改哪里

若需输出所有解的参数表达式，应额外输出自由变量与主元变量的关系；当前接口只给自由变量置零的一组解。模方程组不可直接用浮点消元，应采用模逆元并处理不可逆主元。数值尺度变化时需重新评估 `EPS`，复杂病态问题应换用更稳健的分解算法。

6. 自检

x+y=3、x-y=1 唯一解为 (2,1)。x+y=1 与 x+y=2 无解。x+y=2 与 2x+2y=4 有无穷多解，返回的答案仍须代回成立。空矩阵不满足接口前提。

7. 坑

1. 无主元列并不代表无解：它是自由变量，仍需检查 0=非 0。
2. 忽略返回状态：状态 0 时答案向量不可用。
3. 用精确等于 0 检查浮点数：实现使用绝对 `EPS`，尺度影响判断。
4. 输入行数为 0 或行长度不一致：调用前须保证形状合法。

8. 一句话原理

每列选择绝对值最大的可用主元并消去其他行，再把自由变量置零恢复一组解。

参考：

- OI Wiki：高斯消元

7.13 公平组合游戏与 SG 函数

1. 它是做什么的

`SGGame` 用记忆化 `mex` 计算无环公平组合游戏的 Sprague–Grundy 值；对多个彼此独立的子游戏，用 SG 值异或判断先手胜负。规则须为正常玩法，图为 0-based DAG。遍历所有可达状态的总时间为 $O(V+E)$：每条边的后继值标记一次，每个点扫描至多出度加一的 `mex` 候选。图和 `memo` 空间 $O(V+E)$，递归栈最坏 $O(V)$。

2. 变量在做什么

名称	含义
`nextState[u]`	从状态 `u` 一步可达的后继状态编号。
`memo[u]`	-1 表示未计算，否则存储 SG(`u`)。
`values`	后继状态的 SG 值，可包含重复。
`seen`	标记 `mex` 候选值是否出现，长度为后继数加一。
`total`	独立起始局面的 SG 值异或。

终局没有后继，`mex` 空集为 0；DAG 是递归终止条件。

3. 函数在做什么

接口	参数、返回、副作用
`SGGame(graph)`	按值接收 0-based DAG 邻接表并移入对象，初始化 `memo`；边端点必须合法。
`sg(u)`	要求合法状态且图无环；返回 SG 值并递归填充 `memo`。
`firstPlayerWins(game,starts)`	`starts` 中状态合法且子游戏独立；返回 SG 异或是否非零，并修改 `game.memo`。

4. 怎么调用

将每种局面作为状态、合法操作作为有向边。2 是终局，0 可到 1、2：

```
SGGame game({{1, 2}, {2}, {}});
assert(game.sg(2) == 0);
assert(game.sg(1) == 1);
assert(game.sg(0) == 2);
```

```
assert(firstPlayerWins(game, vector<int>{0}));
assert(!firstPlayerWins(game, vector<int>{2}));
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;

struct SGGame {
    // nextState[u] 列出局面 u 一步可达的所有局面；状态图必须是 DAG。
    vector<vector<int>> nextState;
    // memo[u]=-1 表示尚未计算，否则保存 SG(u)，使每个状态只展开一次。
    vector<int> memo;

    // 参数 graph 是 0-based DAG 邻接表；移动其内容初始化游戏和 -1 memo。
    explicit SGGame(vector<vector<int>> graph)
        : nextState(std::move(graph)), memo(nextState.size(), -1) {}

    // 参数 u 是合法状态编号；返回 SG(u)，并递归填充本对象 memo，要求图无环。
    int sg(int u) {
        if (memo[u] != -1) return memo[u];
        vector<int> values;
        values.reserve(nextState[u].size());
        for (int v : nextState[u]) values.push_back(sg(v));

        // k 个后继的 mex 至多为 k，因此只需标记 [0,k]，更大的 SG 值可忽略。
        vector<bool> seen(values.size() + 1, false);
        for (int x : values)
            if (x < (int)seen.size()) seen[x] = true;
        int mex = 0;
        while (mex < (int)seen.size() && seen[mex]) ++mex;
        return memo[u] = mex;
    }
};

// 参数 starts 是合法且互相独立的起始状态；返回 SG 异或非零，并通过 game 填充共享
memo。
bool firstPlayerWins(SGGame& game, const vector<int>& starts) {
    // starts 表示彼此独立的子游戏；普通玩法下异或和非 0 当且仅当先手必胜。
    int total = 0;
    for (int s : starts) total ^= game.sg(s);
    return total != 0;
}
```

5. 要改哪里

多个独立棋堆可把各自初始状态放进 `starts`；有相互作用时应构造一个联合局面，而非分开异或。若状态图含环、玩法含随机步骤，或无合法操作者获胜，需按对应博弈模型重新分析，当前 SG 递归不能直接使用。

6. 自检

终局值为 0；只有一个终局后继的状态值为 1；后继值集合 {0,1} 的 `mex` 为 2。两个相同 SG 值的独立子游戏异或为 0，先手不胜；空起始列表也返回 false。

7. 坑

1. 有环图递归：可能无限递归，先证明状态图无环。
2. 把多个子游戏的 SG 值相加：组合规则是异或。
3. 把 SG=0 当先手必胜：正常玩法下它表示后手胜。
4. 子游戏不独立仍异或：独立性是定理的前提。

8. 一句话原理

单个局面取后继 SG 值的 `mex`，独立局面组合取 SG 值的异或。

参考：

- OI Wiki：公平组合游戏

7.14 质因数分解、约数个数与约数和

1. 它是做什么的

该节提供质因数分解、约数个数/约数和及正约数枚举，要求正整数 `n`，统计结果和中间幂能由 long long 表示。质因数分解最坏时间 $O(\sqrt{n})$，额外空间为不同质因子数；约数统计在分解后再逐项累加等比级数，时间仍由试除主导、额外空间 $O(\omega(n))$。枚举约数扫描至平方根并排序，时间 $O(\sqrt{n}+d\log d)$、空间 $O(d)$，`d` 为约数数目。`n=1` 时分别返回空分解、{1,1} 统计、{1} 列表。

2. 变量在做什么

名称	含义
`f`	升序质因子与指数对 (p,e)。
`p,e`	当前质因子及其在 `n` 中的指数。
`count,sum`	约数个数、约数和，初始乘法单位元均为 1。
`term,power`	当前质因子的等比和 1+p+…+p^e。
`d,i`	枚举列表及试除因子；`i` 和 `n/i` 成对。

试除循环用 `i<=n/i` 避免 `i*i` 溢出；若 `i` 是平方根，配对值相同，只加入一次。

3. 函数在做什么

函数	参数、返回与副作用
`factorize(n)`	要求 n>=1；返回升序 {p,e} 向量，`n=1` 返回空，不改外部状态。
`divisorCountSum(n)`	要求 n>=1 且结果及中间幂可放入 long long；返回 {约数个数,约数和}。
`enumerateDivisors(n)`	要求 n>=1；返回升序约数，1 返回 {1}。

4. 怎么调用

以 `n=12` 为例，约数个数为 6、约数和为 28：

```
auto stats = divisorCountSum(12);
assert(stats.first == 6 && stats.second == 28);
assert((enumerateDivisors(12) == vector<long long>{1,2,3,4,6,12}));
auto one = divisorCountSum(1);
assert(one.first == 1 && one.second == 1);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 参数 n>=1；返回按质因子升序的 {p,e}，1 返回空向量，不改外部状态。
vector<pair<ll,int>> factorize(ll n) {
    // 试除到 p*p>n；剩余 n>1 必为大质因子。
    vector<pair<ll, int>> f;
    for (ll p = 2; p <= n / p; ++p) {
        if (n % p != 0) continue;
        int e=0;
        do {
            n/=p;
            ++e;
        }
```

```
        while (n % p == 0);
        f.push_back({p,e});
    }
    if(n>1)f.push_back({n,1});
    return f;
}
// 参数 n>=1 且约数个数、约数和及中间幂可放进 ll；返回 {个数,总和}。
pair<ll,ll> divisorCountSum(ll n) {
    // 假设结果可放入 ll
    ll count = 1, sum = 1;
    for (auto [p, e] : factorize(n)) {
        ll term = 1, power = 1;
        for (int i = 0; i < e; ++i) {
            power *= p;
            term += power;
        }
        count*=e+1;
        sum*=term;
    }
    return {count,sum};
}
// 参数 n>=1；返回全部正约数升序列表，1 返回 {1}，不修改输入。
vector<ll> enumerateDivisors(ll n) {
    vector<ll>d;
    for (ll i = 1; i <= n / i; ++i) {
        if (n % i != 0) continue;
        d.push_back(i);
        if(i!=n/i)d.push_back(n/i);
    }
    sort(d.begin(),d.end());
    return d;
}
```

5. 要改哪里

需要模约数和/个数时，应在每个乘法和等比项累积处及时取模，并确保除法步骤仍然成立。大量有界整数查询可使用最小质因子筛；任意 64 位大数分解可改 Pollard–Rho。若结果可能超出 long long，需换大整数，不能只提升最后一次乘法。

6. 自检

`n=1` 的统计应为 (1,1)，约数列表为 {1}；`n=12` 的约数依次为 1、2、3、4、6、12，和为 28。`n=36` 的平方根 6 只出现一次，列表无重复；各质因子指数的 `(e+1)` 乘积等于枚举结果个数。

7. 坑

1. 使用 `p*p<=n`：平方可能溢出，写 `p<=n/p`。
2. 平方数重复插入根：仅当 `i!=n/i` 时加入配对约数。
3. 忽略结果溢出：输入可表示并不代表约数和或中间幂也能表示。
4. 输入 `n=0`：0 有无限多个正约数，接口仅定义于 `n>=1`。

8. 一句话原理

由质因数指数分解的乘法结构，分别计算约数数量、几何级数和，或组合出所有约数。

第八章

字符串

本章字符串统一使用 0-based 下标，区间统一写成左闭右开 `[l,r)`。长度为 `n` 的字符串合法字符下标为 `0..n-1`；各模板用 `int` 保存长度和下标，因此输入长度及下标表达式必须可由 `int` 表示。未特别说明时字符按字节处理，不直接支持 UTF-8 的“单个汉字”语义。

8.1 字符串哈希

1. 它是做什么的

`StringHash` 为字节串预处理前缀双哈希，之后可在常数时间取得任意子串哈希，用于快速比较子串。区间采用 0-based 半开形式 `[l,r)`，长度为 `r-l`，合法范围是 `0<=l<=r<=s.size()`；空区间哈希为 `(0,0)`。相等比较必须同时确认长度相同和双哈希相同。

构造时扫描字符串一次，计算两个模数下的前缀值与幂数组，时间 $O(n)$、空间 $O(n)$；`get(l,r)` 为 $O(1)$。固定双哈希仍有碰撞概率，不适合要求数学上零误判或可被对抗性构造的输入。该实现按 `unsigned char` 处理 `string` 字节，UTF-8 多字节字符不是单个字符。

2. 变量在做什么

名称	含义
`MOD1`、`MOD2`、`BASE`	全部待比较对象必须一致的两个模数和哈希底数。
`h1[i]`、`h2[i]`	前缀 `s[0..i)` 的哈希值；长度数组索引，`h[0]=0`。
`p1[i]`、`p2[i]`	`BASE^i` 在相应模数下的值；`p[0]=1`。
`value`	当前字节无符号值加 1，避免将字节 0 映射为零贡献。
`l`、`r`	子串左闭右开边界，哈希移除左侧前缀时使用幂 `BASE^(r-l)`。
`x1`、`x2`	从两个前缀哈希中减去左侧贡献后得到的模哈希。

两个模数分别运算，`get` 返回一对余数。不同起点的子串哈希可直接比较，但不同长度的字符串即便哈希对相同，也不能据此判等。

3. 函数在做什么

函数	参数与返回	行为及前提
`StringHash(s)`	`const string&` 字节串；构造对象。	建立前缀哈希与幂数组，不修改输入；对象持有之后查询所需的全部数据。
`get(l,r)`	合法半开区间 `[l,r)`；返回 `pair<ll,ll>`。	返回该子串的双模哈希，不修改对象；调用者负责检查边界和比较长度。

两个不同对象之间也可比较，但必须使用代码中相同的底数和模数；本模板的静态常量自动保持一致。

4. 怎么调用

为字符串构造对象，对每个子串调用 `get(l,r)`；比较时同时比较长度和哈希对。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```cpp
using ll = long long;

struct StringHash {
    // 两个模数与 BASE 必须在所有待比较对象中完全一致；哈希相等仍有极小碰撞概率。
    static constexpr ll MOD1 = 1000000007;
    static constexpr ll MOD2 = 1000000009;
    static constexpr ll BASE = 911382323;
    vector<ll> h1, h2, p1, p2;

    // 参数 s 按字节读取；构造并拥有前缀哈希与幂数组，不修改 s。
    explicit StringHash(const string& s) {
        int n = (int)s.size();
        // h[i] 表示前缀 s[0..i) 的哈希，p[i]=BASE^i；两数组都按长度而非字符下标。
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        p1.assign(n + 1, 1); p2.assign(n + 1, 1);
        // 每轮把一个字节扩展进两个前缀哈希，并同步预存下一次查询所需的幂。
        for (int i = 0; i < n; ++i) {
            int value = (unsigned char)s[i] + 1;
            h1[i + 1] = (h1[i] * BASE + value) % MOD1;
            h2[i + 1] = (h2[i] * BASE + value) % MOD2;
            p1[i + 1] = p1[i] * BASE % MOD1;
            p2[i + 1] = p2[i] * BASE % MOD2;
        }
    }

    // 返回 0-based 左闭右开子串 s[l..r) 的双哈希；要求 0<=l<=r<=s.size()。
    // 参数是 s 的 0-based 半开区间 [l,r)；返回双模哈希，调用者须保证边界合法；不改状态。
    pair<ll, ll> get(int l, int r) const {
        // h[r]-h[l]*BASE^(r-l) 消掉左侧前缀，使不同起点的等长子串可直接比较。
        ll x1 = (h1[r] - h1[l] * p1[r - l]) % MOD1;
        ll x2 = (h2[r] - h2[l] * p2[r - l]) % MOD2;
        if (x1 < 0) x1 += MOD1;
        if (x2 < 0) x2 += MOD2;
        return {x1, x2};
    }
};
```

`abracadabra` 中 `[0,4)` 与 `[7,11)` 都是 `abra`，长度与双哈希均相同：

```cpp
string s = "abracadabra";
StringHash hash(s);
auto first = hash.get(0, 4);
auto second = hash.get(7, 11);
int lenFirst = 4;
int lenSecond = 11 - 7;
assert(lenFirst == lenSecond && first == second);
```

5. 要改哪里

- 若比较对象可能来自不同实现或参数的哈希对象，必须统一底数与模数；参数不同的哈希对不能直接比较。
- 若使用场景要求零碰撞误判，用直接比较、KMP、后缀数组等确定性方法；增加模数只能降低碰撞概率，不能消除它。
- 若要按 Unicode 字符而不是字节比较，先解码成码点序列，并相应改为对整数序列建哈希。
- 若只处理单个固定长度模式，可按需只预处理相关结构；当前类为整串任意子串查询保存四个长度 `n+1` 的数组。

6. 自检

`abracadabra` 的 `[0,4)` 和 `[7,11)` 都为 `abra`，双哈希应相等；`[0,3)` 和 `[7,10)` 都为 `abr`，也应相等。空区间 `[k,k)` 返回 `(0,0)`。边界 `[0,s.size())` 对应整个字符串。对子串判等时若长度不同，即使哈希碰巧相同也必须判为不同。

7. 坑

1. 把 `[l,r)` 当作闭区间；长度和幂次都应使用 `r-l`，末端 `r` 不包含。
2. 只比较哈希对而不比较长度；不同长度哈希可能相同，判等前必须核对长度。
3. 忽略固定哈希的碰撞可能；双哈希不是绝对正确性保证，对抗场景换确定性算法。
4. `char` 当有符号值使用；实现转为 `unsigned char` 后再加 1，保持字节值一致。
5. 越界查询导致数组访问错误；`get` 不做边界检查，调用方须保证 `0<=l<=r<=s.size()`。

8. 一句话原理

前缀哈希减去左侧前缀乘以对应幂次，就能在常数时间得到任意半开子串的双模哈希。

参考：

- OI Wiki：字符串哈希

8.2 前缀函数与 KMP

1. 它是做什么的

`prefixFunction` 计算字符串每个前缀的最长相等真前后缀长度；`kmpSearch` 使用该表，在文本中查找模式串的所有出现位置，允许重叠。字符串按 `std::string` 字节处理，结果位置为 0-based 起点。空模式按模板约定返回空列表；若题目另有空模式定义，需单独处理。

长度为 `m` 的模式前缀表计算时间 $O(m)$、空间 $O(m)$。在长度为 `n` 的文本中搜索总时间 $O(n+m)$，使用前缀表及输出位置占 $O(m+k)$ 空间，其中 `k` 为匹配次数；输入不修改。

2. 变量在做什么

名称	含义
`s`	`prefixFunction` 的字节串输入。
`pi[i]`	子串 `s[0..i]` 的最长相等真前后缀长度；是长度，不是末尾下标。
`text`、`pattern`	待搜索文本和模式字节串。
`j`	当前已匹配的模式前缀长度；扫描文本字符前满足 `0<=j<m`。
`i`	当前扫描到的文本下标。
`positions`	找到的 0-based 匹配起点，按扫描顺序递增。

失配时将 `j` 缩短为当前边界的次长候选 `pi[j-1]`，文本位置不回退。完整匹配后也回退到模式串最长真边界，使后续搜索保留可能的重叠部分。

3. 函数在做什么

函数	参数与返回	行为及前提
`prefixFunction(s)`	字节串常量引用；返回 `vector<int>`。	返回长度与 `s` 相同的前缀表；空串返回空表，不修改输入。
`kmpSearch(text,pattern)`	文本和模式字节串常量引用；返回 `vector<int>`。	返回所有匹配的 0-based 起点，包含重叠；空模式返回空向量，不修改输入。内部调用前缀函数。

UTF-8 字符串会按字节匹配；若要按 Unicode 码点查找，应先解码成符号序列再应用同样算法。

4. 怎么调用

需要前缀表时调用 `prefixFunction`；查找所有出现时调用 `kmpSearch`。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

```

```cpp
// 参数 s 是字节串；返回与 s 等长的 pi 长度数组，空串返回空数组，不改输入。
vector<int> prefixFunction(const string& s) {
    int n = (int)s.size();
    // pi[i] 是 s[0..i] 的最长相等真前后缀“长度”，不是末尾下标。
    vector<int> pi(n, 0);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        // 失配后只可能尝试当前边界的次长边界；因此跳 pi[j-1]，文本位置 i 不回退。
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

// 参数 text/pattern 是字节串；返回 pattern 的全部 0-based 起点（含重叠），空模式返回空表。
vector<int> kmpSearch(const string& text, const string& pattern) {
    // 返回 pattern 在 text 中所有 0-based 起点；空模式的语义由题目决定，这里返回空。
    vector<int> positions;
    if (pattern.empty()) return positions;
    vector<int> pi = prefixFunction(pattern);
    // 扫描 text[0..i] 前，j 表示已匹配的 pattern 前缀长度，始终有 0<=j<m。
    int j = 0;
    for (int i = 0; i < (int)text.size(); ++i) {
        while (j > 0 && text[i] != pattern[j]) j = pi[j - 1];
        if (text[i] == pattern[j]) ++j;
        if (j == (int)pattern.size()) {
            positions.push_back(i - j + 1);
            // 匹配后回到最长真边界，才能继续寻找允许重叠的下一次出现。
            j = pi[j - 1];
        }
    }
    return positions;
}
```

文本 `aaaaa` 中模式 `aaa` 出现于 0、1、2；`ababa` 的前缀表为 `[0,0,1,2,3]`：

```cpp
vector<int> pi = prefixFunction("ababa");
assert((pi == vector<int>{0, 0, 1, 2, 3}));
vector<int> positions = kmpSearch("aaaaa", "aaa");
assert((positions == vector<int>{0, 1, 2}));
```

5. 要改哪里

- 若只想判断是否出现，可在找到第一次匹配时提前返回；当前函数会扫描完整文本并收集所有位置。
- 若只要第一次出现，可返回首个起点或 `-1`，需调整返回类型和匹配后逻辑。
- 若要统计出现次数且允许重叠，可用 `positions.size()`；匹配后回退不可省略。
- 若要忽略大小写，可先将文本和模式统一大小写再运行 KMP。通配符匹配通常不满足前缀失配复用所需的相等关系，不能只替换比较符，应采用专门算法或增加相应状态。
- 若处理 Unicode 字符，先转成码点序列；当前 `string` 接口以字节为单位。

6. 自检

`prefixFunction("ababa")` 应为 `[0,0,1,2,3]`。`kmpSearch("aaaaa","aaa")` 返回 `[0,1,2]`，验证重叠匹配。空文本或比文本长的非空模式返回空向量；空模式也按此模板返回空向量。单字符模式在重复字符文本中应列出每个对应位置。

7. 坑

1. 把 `pi[i]` 当末尾下标；它存真前后缀长度，失配回退到 `pi[j-1]`。
2. 失配后将文本下标回退；KMP 只回退模式状态 `j`，文本扫描位置继续前进。
3. 找到匹配后把 `j` 清零；应回退到 `pi[j-1]`，否则会漏掉重叠匹配。

4. 对空模式访问 `pattern[0]`；模板先检测空模式并返回空结果。
5. 将 UTF-8 的字节位置当字符位置；若题目按字符计数，应先解码。

8. 一句话原理

前缀函数记录模式自身可复用的边界，失配时沿这些边界回退而不重扫文本，从而在线性时间内找到所有匹配。

参考：

- OI Wiki：前缀函数与 KMP 算法

8.3 Z 函数（扩展 KMP）

1. 它是做什么的

`zFunction(s)` 对每个位置 `i` 计算从 `s[i..]` 开始的后缀与整串的最长公共前缀长度；非空串约定 `z[0]=s.size()`，空串返回空数组。`zSearch(text,pattern)` 利用 Z 函数找出模式串在文本中的所有匹配起点，位置为 0-based，允许重叠；空模式按本模板约定返回空表。

字符串按 `std::string` 字节处理。Z 数组计算对长度 `N` 的字符串耗时 $O(N)$、空间 $O(N)$；匹配会构造长度 `|pattern|+1+|text|` 的拼接串并计算 Z 数组，时间和额外空间均为 $O(n+m)$。代码使用 `\0` 作为分隔符，因此 `text` 和非空 `pattern` 都不能包含 NUL 字节。

2. 变量在做什么

名称	含义
`z[i]`	`s` 与后缀 `s[i..]` 的 LCP 长度，范围不超过 `s.size()-i`。
`l`、`r`	当前最靠右的匹配盒 `[l,r)`；盒内子串等于从 0 开始的前缀。
`i`	当前计算 Z 值的位置。
`text`、`pattern`	待搜索文本和模式字节串。
`joined`	`pattern + '\0' + text`；文本位置 `i` 在拼接串中的位置为 `m+1+i`。
`m`	模式串字节长度。
`positions`	所有匹配的文本 0-based 起点，按从小到大排列。

若 `i<r`，可以从镜像位置 `i-l` 的 Z 值开始复用，但最多延伸到盒子的右端 `r`；之后再直接比较扩展。拼接后某文本位置对应的 `z` 值至少为 `m`，表示整个模式与文本该后缀匹配。

3. 函数在做什么

函数	参数与返回	行为及前提
`zFunction(s)`	字节串常量引用；返回 `vector<int>`。	返回长度为 `s.size()` 的 Z 数组；空串返回空数组，非空时 `z[0]=size`；不修改输入。
`zSearch(text,pattern)`	不含 NUL 字节的文本和模式；返回 `vector<int>`。	返回所有匹配起点；空模式返回空向量，不修改输入。内部构造拼接串并调用 Z 函数。

4. 怎么调用

需要后缀 LCP 信息时调用 `zFunction`；要找模式串所有位置时调用 `zSearch`。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数 s 是字节串；返回各位置的 Z 长度，非空时 z[0]=|s|，空串返回空数组。
```

```cpp
vector<int> zFunction(const string& s) {
    int n = (int)s.size();
    // z[i]=LCP(s, s[i..n)); 所有字符与区间均为 0-based、左闭右开。
    vector<int> z(n, 0);
    if (n == 0) return z;
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        // [l,r) 是目前右端点最远的前缀匹配段，且 s[l..r)==s[0..r-l)。
        // i 在盒内时先复用镜像 z[i-l]，但不能越过尚未验证的右端点 r。
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}

// 参数 text/pattern 为字节串且均不含 '\0' 分隔符；返回 0-based 匹配起点，空模式返回空表。
vector<int> zSearch(const string& text, const string& pattern) {
    // 返回所有 0-based 起点；若字符集可能含 '\0'，需把分隔符换成确定不存在的值。
    vector<int> positions;
    if (pattern.empty()) return positions;
    string joined = pattern + '\0' + text;
    vector<int> z = zFunction(joined);
    int m = (int)pattern.size();
    // joined 中 text[i] 的下标是 m+1+i；z 值至少为 m 才覆盖整个模式串。
    for (int i = 0; i < (int)text.size(); ++i)
        if (z[m + 1 + i] >= m) positions.push_back(i);
    return positions;
}
```

`aaaaa` 中模式 `aaa` 可重叠地出现于 0、1、2；串 `aaaa` 的 Z 数组为 `[4,3,2,1]`：

```cpp
vector<int> z = zFunction("aaaa");
assert((z == vector<int>{4, 3, 2, 1}));
vector<int> positions = zSearch("aaaaa", "aaa");
assert((positions == vector<int>{0, 1, 2}));
```

5. 要改哪里

- 若字符集可能包含 `\0`，将分隔符换成保证不出现在两个输入中的字符；若不存在可靠字符分隔符，可将字节映射到整数序列并用超出字节范围的整数作分隔符。
- 若要按 Unicode 字符而不是字节匹配，先解码字符串再对符号序列计算 Z 值。
- 若只需要判断是否存在匹配，可在首次发现 `z>=m` 时提前结束；当前接口收集全部起点。
- 空模式返回空表是本模板约定。若题目把空模式视为在每个边界匹配，应调整空模式分支并明确输出位置数量。

6. 自检

`zFunction("aaaa")` 为 `[4,3,2,1]`。`zSearch("aaaaa","aaa")` 为 `[0,1,2]`，验证重叠匹配。模式长于文本时结果为空；空文本和非空模式结果为空；空模式按当前约定也返回空。含 NUL 字节的输入违反该匹配函数前提。

7. 坑

1. 维护区间 `[l,r)` 却按半开代码更新；本实现用 `[l,r)`，盒内条件是 `i<r`。
2. 混淆 `z[0]` 约定；本实现非空时令其等于整串长度。
3. 拼接分隔符出现在输入串中；这会让跨分隔符的 LCP 造成错误匹配，应使用输入中不存在的分隔符。
4. 忘记拼接串中文本偏移为 `m+1`；检查位置时访问 `z[m+1+i]`。

5. 误将 UTF-8 字节索引当字符索引；需要字符语义时先解码。

8. 一句话原理

用最靠右的前缀匹配盒复用已知 LCP，再向右扩展，从而在线性时间得到所有后缀与整串的匹配长度。

参考：

- OI Wiki：Z 函数（扩展 KMP）

8.4 Manacher

1. 它是做什么的

`manacher(s)` 在线性时间求出字节串每个位置为中心的最长奇回文半径，以及每个字符间隙为中心的最长偶回文半径。输入下标按 0-based 字节位置，空串合法；返回的两个数组都与输入等长。可由半径换算最长回文区间或长度，但函数不直接返回全局最长回文。

每个位置只会被线性次扩展或复用，时间 $O(n)$；奇偶半径数组各需 $O(n)$ 空间。`std::string` 按字节比较，UTF-8 多字节字符不视为一个字符。输入不修改。

2. 变量在做什么

名称	含义
`odd[i]`	以字符 `i` 为中心的奇回文半径，包含中心，至少为 1。总长度 `2*odd[i]-1`，区间 `[i-odd[i]+1,i+odd[i])`。
`even[i]`	以缝隙 `i-1|i` 为中心的偶回文半径，可为 0。总长度 `2*even[i]`，区间 `[i-even[i],i+even[i])`。
`l`、`r`	当前最右回文边界；奇回文循环中区间为闭区间 `[l,r]`。
`k`	当前中心半径的临时扩展值；先复用镜像半径，再比较边界外字符。
`i`	原串位置或偶回文中心缝隙编号，范围 `0..n-1`。
`ManacherResult`	包含 `odd`、`even` 两个半径向量。

奇回文中心落在字符上；偶回文中心落在字符之间。`even[i]` 对应的是 `i-1` 与 `i` 之间，不是字符 `i`。空串时循环不执行，两数组为空。

3. 函数在做什么

函数/类型	参数与返回	行为及前提
`ManacherResult`	字段 `odd`、`even`，类型均为 `vector<int>`。	分别存奇、偶中心半径。
`manacher(s)`	字符串常量引用；返回 `ManacherResult`。	返回每个字节位置与间隙的最长回文半径；空串返回两个空数组，不修改输入。

4. 怎么调用

调用后可扫描两个半径数组求全局最大长度，或用公式取得中心对应的半开区间。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

struct ManacherResult {
    // odd/even 均以原串 0-based 字符位置为中心索引，数值是单侧半径而非总长度。
    vector<int> odd, even;
};

// 参数 s 是字节串；返回等长 odd/even 半径数组，空串对应两个空数组，不改输入。
```

```cpp
ManacherResult manacher(const string& s) {
    int n = (int)s.size();
    vector<int> odd(n), even(n);

    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        // [l,r] 是当前最右奇回文；盒内位置先复用关于中心的镜像半径。
        int k = (i > r ? 1 : min(odd[l + r - i], r - i + 1));
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
        odd[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        // even[i] 的中心是缝隙 i-1|i；镜像下标因此比奇回文多一个 +1。
        int k = (i > r ? 0 : min(even[l + r - i + 1], r - i + 1));
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
        even[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }
    return {odd, even};
}
```

`abba` 的最长回文是偶回文 `[0,4)`，其中心缝隙编号 2 的半径为 2：

```cpp
string s = "abba";
ManacherResult result = manacher(s);
assert((result.even == vector<int>{0, 0, 2, 0}));
assert(2 * result.even[2] == 4);
string empty;
ManacherResult none = manacher(empty);
assert(none.odd.empty() && none.even.empty());
```

5. 要改哪里

- 若只需最长回文长度，在所有 `odd[i]` 和 `even[i]` 中分别换算长度并取最大；当前返回完整半径表，未替调用者汇总。
- 若要判断给定半开区间 `[l,r)` 是否回文，应根据长度奇偶计算中心，并检查对应半径是否覆盖该区间；边界需与上述公式一致。
- 若需要按 Unicode 字符而非字节判断回文，先将字符串解码成码点序列，再对该序列应用算法。
- 若要输出最长回文内容，记录最大长度和对应区间后从原串截取；当前接口只返回半径。

6. 自检

`abba` 的 `even` 为 `[0,0,2,0]`，最大偶回文长度 4。`aba` 的中心位置 1 的奇半径为 2，对应区间 `[0,3)`；全相同串 `aaaa` 的中心半径应逐步覆盖最长回文。空串返回两空向量，单字符的奇半径为 1、偶半径为 0。

7. 坑

1. 将奇半径直接当总长度；应换算为 `2*k-1`。
2. 认为偶半径中心是字符 `i`；它实际位于 `i-1` 与 `i` 之间。
3. 把 `even[i]` 最小值写成 1；偶回文可能不存在，最小半径为 0。
4. 区间换算把右端当闭区间；本节给出的回文区间均为半开形式 `[l,r)`。
5. 用 UTF-8 字节长度解释用户可见字符数；本实现逐字节比较，字符语义需先解码。

8. 一句话原理

利用当前最右回文的镜像中心复用已知半径，再向两侧扩展，从而在线性时间求出所有奇偶回文中心半径。

参考：

- OI Wiki：Manacher

8.5 Trie 字典树

1. 它是做什么的

`Trie` 维护只含小写 ASCII 字母 `a..z` 的字符串多重集合，支持插入、查询完整词出现次数和查询某前缀下的字符串总数。重复插入按多重集合累计；空串也可插入并在根节点计数。字符不在 `a..z` 时实现不会检查，访问孩子数组可能越界。

插入或查询长度为 `L` 的串需 $O(L)$ 时间。每个节点为 26 个字符预留孩子槽位；若插入串总长度为 `T`，节点数至多 `1+T`，空间为 $O(26T)$（字母表大小固定时写作 $O(T)$）。`pass` 和 `end` 是 `int` 计数，需保证插入数量不超过其范围。

2. 变量在做什么

名称	含义
`node`	节点池，根固定编号 0；新节点从 1 起分配。
`Node::next[c]`	当前节点沿字母 `c` 的孩子编号；0 表示该边不存在。
`Node::pass`	有多少次插入经过该节点，重复词按次数累计。根的 `pass` 是所有插入字符串总数。
`Node::end`	有多少次插入恰好在该节点结束。
`u`	当前遍历到的 Trie 节点编号。
`c`	字符映射 `ch-'a'`，合法范围 `0..25`。

`pass` 与 `end` 不同：例如插入 `app` 两次、`apple` 一次，`app` 节点的 `pass` 为 3、`end` 为 2。节点号 0 既是根，也是“无孩子”哨兵，因此不能把新节点编号分配为 0。

3. 函数在做什么

函数	参数与返回	行为及前提
`insert(s)`	只含 `a..z` 的字符串；返回 `void`。	创建缺失路径，并递增根及路径节点的 `pass`，终点 `end` 递增；重复插入累加计数。
`findNode(s)`	小写 ASCII 字符串；返回节点编号或 `-1`。	查找整条前缀路径；空串返回根编号 0，不修改 Trie。
`countWord(s)`	小写 ASCII 字符串；返回 `int`。	返回完整词插入次数；路径不存在时为 0。
`countPrefix(s)`	小写 ASCII 字符串；返回 `int`。	返回以该串为前缀的插入次数；空串返回总插入数。

该实现没有删除接口。Trie 对象可连续插入和查询；计数反映所有历史插入。

4. 怎么调用

先插入词，再按需查询完整词或前缀次数。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Trie {
    struct Node {
        // next[c]=0 表示无边；根恰为 0，所以实际新节点编号必须从 1 开始。
```

```
        array<int, 26> next{};
        // pass: 经过该节点的已插入字符串数; end: 恰在该节点结束的字符串数。
        int pass = 0;
        int end = 0;
    };
    vector<Node> node{1};

    // 参数 s 只含 a..z; 把一个词加入本 Trie, 重复插入会增加 pass/end 计数。
    void insert(const string& s) {
        // 字符映射固定为 'a'..'z'; 改题换字符集时同时修改 26 与 ch-'a'。
        int u = 0;
        // 根的 pass 也计数, 因此 countPrefix("") 会返回插入串总数。
        ++node[u].pass;
        for (char ch : s) {
            int c = ch - 'a';
            if (node[u].next[c] == 0) {
                node[u].next[c] = (int)node.size();
                node.push_back(Node{});
            }
            u = node[u].next[c];
            ++node[u].pass;
        }
        ++node[u].end;
    }

    // 参数 s 只含 a..z; 返回前缀终点节点号, 缺边返回 -1; 不修改 Trie。
    int findNode(const string& s) const {
        // 返回前缀终点编号; -1 是查询失败状态, 与合法根节点 0 区分。
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (node[u].next[c] == 0) return -1;
            u = node[u].next[c];
        }
        return u;
    }

    // 参数 s 只含 a..z; 返回完整词插入次数, 未出现为 0; 不修改 Trie。
    int countWord(const string& s) const {
        int u = findNode(s);
        return u == -1 ? 0 : node[u].end;
    }

    // 参数 s 只含 a..z; 返回以 s 为前缀的插入次数; 空串返回总插入次数。
    int countPrefix(const string& s) const {
        int u = findNode(s);
        return u == -1 ? 0 : node[u].pass;
    }
};
```

插入 app 两次、apple 一次后，完整词 app 出现 2 次，前缀 app 共匹配 3 次：

```
Trie trie;
trie.insert("app");
trie.insert("app");
trie.insert("apple");
assert(trie.countWord("app") == 2);
assert(trie.countPrefix("app") == 3);
assert(trie.countPrefix("") == 3);
assert(trie.countWord("ap") == 0);
```

5. 要改哪里

- 若字符集不是小写英文字母，需同时调整 `array<int,26>` 大小和 `ch-'a'` 映射；若字符范围大且稀疏，可用映射表保存孩子。

- 若要支持删除, 需沿路径递减 `pass` 和终点 `end`, 并确保计数不会变负; 是否回收无用节点需另行实现。
- 若只需存在性而不需重复次数，可将计数转成布尔标记；当前实现保留插入多重性。
- 若计数可能超过 `int`，将 `pass`、`end` 和查询返回类型改为更宽整数。
- 若要支持大小写或任意字节，必须定义完整字符映射，不能直接使用 `ch-'a'`。

6. 自检

插入 `app` 两次、`apple` 一次后，`countWord("app")=2`，`countPrefix("app")=3`，`countPrefix("")=3`。`countWord("ap")` 为 0，因为它只是前缀，不是已插入完整词。未插入的前缀查询返回 0。插入空串后，`countWord("")` 为 1，根节点的前缀计数总数也增加 1。

7. 坑

1. 把 `pass` 当成完整词计数；它统计经过节点的所有插入，完整词次数看 `end`。
2. 新节点编号从 0 开始；0 已表示根/无边，新节点必须从 1 开始。
3. 输入含非 `a..z` 字符；映射结果会越界，调用前校验或扩展字符映射。
4. 忽略重复插入；本模板统计次数，重复词会增加 `pass` 和 `end`。
5. 把前缀存在误认为完整词存在；路径存在不代表 `end>0`，应调用 `countWord`。

8. 一句话原理

把字符串逐字符映射为节点边, 并沿路径维护经过次数和终点次数, 从而按字符串长度完成插入与前缀查询。

参考：

- OI Wiki：字典树

8.6 AC 自动机

1. 它是做什么的

AC 自动机在同一文本中查找多个模式串的出现次数, 适用于小写 ASCII 字符串。先插入全部非空模式, 保存各自的终止节点，再调用一次 `build()`，最后扫描文本统计；匹配允许重叠，重复模式对应同一终止节点并得到相同次数。空模式不受支持，文本和模式字符都必须在 `a..z`。

令 `S` 为 Trie 节点数，`T` 为文本长度，`K` 为查询的模式终止节点数。插入总模式长度为 `P` 时插入耗时 $O(P)$；固定 26 个字母下，`build()` 为 $O(26S)$，文本扫描和 fail 传播为 $O(T+S)$，返回查询结果另需 $O(K)$。节点转移空间为 $O(26S)$，计数、BFS 顺序和输出额外需 $O(S+K)$。构建会原地把缺失 Trie 边补成自动机转移，因此仅能在所有模式插入后调用一次。

2. 变量在做什么

名称	含义
`node[u].next[c]`	字符 `c` 的转移；构建前是 Trie 边，构建后缺失边也被补为自动机跳转。
`node[u].fail`	状态 `u` 的最长真后缀状态；根为 0。
`bfsOrder`	按 Trie 深度递增记录的非根状态；逆序遍历时可先把深层访问次数汇入 fail 状态。
`terminal`	调用者保存的模式终止节点编号列表，顺序与要取回答案的模式顺序一致。
`visits[u]`	扫描中恰好落在状态 `u` 的次数；fail 逆序传播后，也包含落在其 fail 后缀的次数。
`u`	文本扫描时当前自动机状态。

查询先计数每个文本前缀结束时所在的状态，再沿 fail 链从深到浅累加；因此以某状态为后缀的模式也计入该状态的访问。终止节点必须来自非空模式，不能把根节点当作模式终点。

3. 函数在做什么

函数	参数与返回	行为及前提
`insert(s)`	非空小写 ASCII 模式串；返回终止节点编号。	向 Trie 加入路径；必须在 `build()` 前调用。
`build()`	无参数；返回 `void`。	BFS 计算 fail 并补齐转移，改变 `next` 的语义；所有模式插入后只调用一次。
`countOccurrences`	小写文本及有效非根终点列表；返回 `vector<long long>`。	按 terminal 输入顺序返回各模式计数，包含重叠；可在同一已构建自动机上多次查询，局部访问计数每次重置。

`terminal` 中有重复节点时结果向量也保留重复项。`countOccurrences` 不修改自动机；`insert` 和 `build` 会修改内部节点状态。

4. 怎么调用

插入所有模式并保存终点，构建一次，然后扫描文本。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

struct AhoCorasick {
    struct Node {
        // build 前 next 是 Trie 边；build 后缺边被补为自动机转移，不能再继续 insert。
        array<int, 26> next{};
        // fail 指向当前字符串的最长真后缀状态，根及根的孩子失败时回到根 0。
        int fail = 0;
    };
    vector<Node> node{1};
    vector<int> bfsOrder;

    // 参数 s 是非空小写模式串；加入 Trie 并返回终止节点号，须在 build 前调用。
    int insert(const string& s) {
        // 返回模式串终止节点，调用者须保存它；字符集固定为小写 'a'..'z'。
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (node[u].next[c] == 0) {
                node[u].next[c] = (int)node.size();
                node.push_back(Node{});
            }
            u = node[u].next[c];
        }
        return u;
    }

    // 无参数；补全本对象转移并填 bfsOrder，改变 Trie 边语义；插入结束后仅调用一次。
    void build() {
        // 只调用一次；BFS 保证计算子节点 fail 前，其 fail 父节点的完整转移已就绪。
        queue<int> q;
        for (int c = 0; c < 26; ++c) {
            int v = node[0].next[c];
            if (v != 0) q.push(v);
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            bfsOrder.push_back(u);
            for (int c = 0; c < 26; ++c) {
                int v = node[u].next[c];
                if (v != 0) {
```

```
                    // u 经 c 到 v，则 v 的最长可用后缀是 fail[u] 经同一字符的转移。
                    node[v].fail = node[node[u].fail].next[c];
                    q.push(v);
                } else {
                    // 补全缺边后，文本扫描每个字符只做一次数组跳转，不需 while 回退。
                    node[u].next[c] = node[node[u].fail].next[c];
                }
            }
        }
    }

    // 参数 text 是小写文本，terminal 是有效的非根模式终点列表；返回逐项出现次数（允许重叠），不改自动机。
    vector<long long> countOccurrences(
        const string& text, const vector<int>& terminal
    ) const {
        // visits[u] 先统计扫描时恰好落到 u 的次数；terminal 中存各模式的终止节点。
        vector<long long> visits(node.size(), 0), answer;
        int u = 0;
        for (char ch : text) {
            u = node[u].next[ch - 'a'];
            ++visits[u];
        }
        // fail 树上由深到浅累加：到达 u 也代表其每个 fail 后缀模式出现一次。
        for (int i = (int)bfsOrder.size() - 1; i >= 0; --i) {
            int v = bfsOrder[i];
            visits[node[v].fail] += visits[v];
        }
        for (int v : terminal) answer.push_back(visits[v]);
        return answer;
    }
};
```

模式 `a` 和 `aa` 在文本 `aaa` 中分别出现 3 次和 2 次，重叠匹配也计数：

```
AhoCorasick ac;
int terminalA = ac.insert("a");
int terminalAA = ac.insert("aa");
ac.build();
vector<long long> count = ac.countOccurrences("aaa", {terminalA, terminalAA});
assert((count == vector<long long>{3, 2}));
```

5. 要改哪里

- 若字符集不是小写英文字母，调整节点孩子数组大小和字符映射；插入与扫描必须使用一致映射。
- 若要支持空模式，需在输入层定义空串出现次数（通常为文本边界数），不能把根节点计数直接当成普通非空模式结果。
- 若要查询是否出现，只需将计数与 0 比较；若需报告所有匹配位置，需在扫描过程中维护当前位置并沿输出链枚举终点，当前接口只返回次数。
- 新增模式或改变模式集合时，需重新构造自动机、插入全部模式并重新 `build()`；构建后的补全转移已覆盖原 Trie 缺边语义，不能继续插入。
- 计数类型为 `long long`，若文本/匹配数量可能超过范围，应改计数类型。

6. 自检

插入模式 `a`、`aa` 并扫描 `aaa`，计数应分别为 3 和 2；两个 `aa` 匹配重叠于起点 0 和 1。未匹配模式的终点计数为 0。若两次分别查询不同文本，第二次结果应只依赖第二个文本，确认 `visits` 是每次新建。插入空模式或非小写字符违反模板前提。

7. 坑

1. 插入模式后再调用 `build()` 多次或继续插入；构建会改写缺失转移，顺序必须是全部插入后只构建一次。
2. 统计时只数扫描直接落到的状态；后缀模式需通过 fail 逆序传播访问次数。
3. 正向传播 fail 计数；必须深层先累计到浅层，使用 BFS 顺序的逆序。
4. 忘记重叠匹配；按状态逐字符扫描并传播计数会自然计入重叠，不能跳过匹配后缀。
5. 终点列表包含根节点或字符越界；只传非空模式的终止节点，并校验所有输入字符为 `a..z`。

8. 一句话原理

Trie 把多模式共享前缀合并，fail 链表示可继续匹配的后缀，再把文本状态访问次数沿 fail 关系汇总给每个模式终点。

参考：

- OI Wiki：AC 自动机

8.7 后缀数组

1. 它是做什么的

`suffixArray(s)` 将字符串所有后缀按字典序排列，返回它们在原串中的 0-based 起点。`lcpArray(s,sa)` 返回排序后相邻后缀的最长公共前缀长度。输入按 `std::string` 字节比较；空串的后缀数组和 LCP 数组均为空。

后缀数组用倍增法，每轮按两个 rank 组成的键对 `n` 个后缀进行比较排序；轮数 $O(\log n)$、每轮排序 $O(n \log n)$，总时间 $O(n \log^2 n)$、空间 $O(n)$。Kasai 算法构造相邻 LCP 为 $O(n)$ 时间和 $O(n)$ 空间。此实现适合中等长度；大规模输入可改用计数排序优化的倍增或线性/近线性构造法。

2. 变量在做什么

名称	含义
`sa[i]`	字典序第 `i` 小的后缀在 `s` 中的起点，范围 `0..n-1`。
`rk[i]`	当前轮后缀 `s[i..]` 的排序等级。
`tmp[i]`	按新 rank 对后缀排序后生成的下一轮等级。
`k`	当前倍增长度；按 `(rk[i], rk[i+k])` 排序，越界第二关键字视作 `-1`。
`rankPair(i)`	lambda：返回后缀起点 `i` 对应的两个排序键。
`lcp[i]`	`sa[i]` 与 `sa[i+1]` 的 LCP 长度，因此数组长度为 `max(0,n-1)`。
`rk`（LCP 中）	后缀起点到其 `sa` 排名的逆映射。
`h`	Kasai 复用前一个起点的 LCP 下界；每轮结束后至多减 1。

初始 rank 是单字节值（按 unsigned byte 读取）；每轮把已知长度扩展到约两倍。`lcpArray` 先建立逆排名，再扫描原串起点并复用前一个 LCP，避免每对后缀从头比较。

3. 函数在做什么

函数	参数与返回	行为及前提
`suffixArray(s)`	字符串常量引用；返回 `vector<int>`。	返回后缀起点的字典序排列；空串返回空向量，不修改输入。
内部 `rankPair(i)` lambda	后缀起点 `0<=i<n`；返回当前轮的两个 rank 键。	用于 `sort` 比较，捕获当前 `rk`、`k`，不修改它们。
`lcpArray(s,sa)`	字符串及其后缀数组；返回 `vector<int>`。	要求 `sa` 是 `[0,n)` 的合法排列且按后缀字典序排列；返回相邻后缀

函数	参数与返回	行为及前提
		LCP，输入不修改。空串或单字符字符串均返回空 LCP 表。

4. 怎么调用

先构造 `sa`，再把同一字符串与 `sa` 传给 LCP 函数。模板如下：

```cpp
#include <bits/stdc++.h>
using namespace std;

// 参数 s 是字节串；返回按字典序排列的后缀起点，空串返回空表，不改输入。
vector<int> suffixArray(const string& s) {
    // 倍增按两个 rank 排序，越界第二关键字视为 -1。
    int n = s.size();
    vector<int> sa(n), rk(n), tmp(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; ++i) rk[i] = (unsigned char) s[i];
    // k 逐次翻倍，rank 对应长度 2k 的前缀；所有后缀按新 rank 重排。
    for (long long k = 1; k < n; k <<= 1) {
        // 参数 i 是 0..n-1 的后缀起点；返回当前 2k 长度排序键，只读捕获的 rk/k。
        auto rankPair = [&](int i) {
            return pair{rk[i], i + k < n ? rk[i + k] : -1};
        };
        // 比较器参数是两个后缀起点；按 rankPair 严格升序比较，不改捕获状态。
        sort(sa.begin(), sa.end(), [&](int i, int j) {
            return rankPair(i) < rankPair(j);
        });
        if (n) tmp[sa[0]] = 0;
        for (int i = 1; i < n; ++i) {
            int a = sa[i - 1], b = sa[i];
            tmp[b] = tmp[a] + (rankPair(a) < rankPair(b));
        }
        rk=tmp;
        if (n && rk[sa.back()] == n - 1) break;
    }
    return sa;
}
// 参数 sa 必须是按后缀字典序排列的 [0,|s|) 起点排列；返回相邻后缀 LCP，长度 max(0,n-1)，不改输入。
vector<int> lcpArray(const string& s, const vector<int>& sa) {
    int n = s.size();
    vector<int> rk(n), lcp(max(0, n - 1));
    for (int i = 0; i < n; ++i) rk[sa[i]] = i;
    // Kasai 复用相邻后缀的公共前缀，向右移动起点后最多只需少比较一个字符。
    for (int i = 0, h = 0; i < n; ++i) {
        int r = rk[i];
        if (r == n - 1) {
            h = 0;
            continue;
        }
        int j = sa[r + 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
        lcp[r] = h;
        if (h) --h;
    }
    return lcp;
}
```

`banana` 的后缀起点排序为 `[5,3,1,0,4,2]`，相邻 LCP 为 `[1,3,0,0,2]`：

```cpp
string s = "banana";
vector<int> sa = suffixArray(s);
assert((sa == vector<int>{5, 3, 1, 0, 4, 2}));
```

```
vector<int> lcp = lcpArray(s, sa);
assert((lcp == vector<int>{1, 3, 0, 0, 2}));
assert(suffixArray(string{}).empty());
```

5. 要改哪里

- 若要查找模式串出现位置，可在后缀数组上二分比较模式与后缀前缀；当前两个函数只构造 SA 和相邻 LCP。
- 若要最长重复子串，可取 `lcp` 最大值并结合对应相邻起点恢复区间；当前接口只返回 LCP 数组。
- 若输入规模导致 `O(n log² n)` 超时，需将每轮比较排序替换为基数/计数排序等线性轮次排序；仅优化 LCP 不会降低主要构造成本。
- 若按 Unicode 字符而非字节排序，先解码成整数符号序列并相应调整实现；当前后缀位置是字节下标。
- `lcpArray` 不校验传入 `sa`；若排序数组不合法，结果没有定义，应使用与同一 `s` 对应的正确排列。

6. 自检

`banana` 的后缀序列依次为 `a, ana, anana, banana, na, nana`，起点 `[5,3,1,0,4,2]`，相邻 LCP 为 `[1,3,0,0,2]`。空串返回空 SA 和 LCP；单字符字符串 SA 为 `[0]`、LCP 为空；重复字符 `aaaa` 的 SA 应为 `[3,2,1,0]`，LCP 为 `[1,2,3]`。

7. 坑

1. 混淆后缀起点与排名；`sa[rank]` 给起点，逆映射 `rk[start]` 才给排名。
2. 把 `lcp[i]` 当作后缀 `sa[i]` 与原串的 LCP；它表示排序相邻的 `sa[i]` 和 `sa[i+1]`。
3. 用倍增比较排序却按 $O(n \log n)$ 估算；本实现每轮 `sort`，实际为 $O(n \log^2 n)$。
4. 给 `lcpArray` 传入未经排序或不是排列的 `sa`；该函数要求正确后缀数组，不会验证。
5. 把字节位置当 Unicode 字符位置；`string` 中的排序和 LCP 都按字节进行。

8. 一句话原理

倍增排序逐轮扩展已知后缀前缀排名，Kasai 再复用相邻后缀 LCP 的重叠部分在线性时间生成 LCP 表。

第九章

计算几何

本章要求浮点输入为有限值，并采用两套明确的数值约定：含交点、距离、投影时使用 `long double` 和绝对误差 `EPS = 1e-12L`；坐标尺度变化明显时应按题目精度调整 EPS，或采用相对误差。整数输入只判断方向、求凸包或平方距离时，输入用 `long long`、运算前提升到 `__int128`；坐标差、乘积、叉积、面积和平方距离的中间量都必须落在 signed `__int128` 范围内，否则需增加坐标限制或使用大整数。浮点数不能直接用于有序容器的“近似相等”比较。

9.1 点、向量与基本运算

1. 它是做什么的

本节用 `Point` 表示二维实数坐标点，也将其当作向量；提供加减、缩放、点积、叉积、长度与逆时针旋转。各运算为 $O(1)$。浮点类型为 `long double`，判号 `sgn` 使用绝对 `EPS=1e-12`；坐标尺度过大时此容差可能不合适。除法标量须非零，`rotate` 的角度单位为弧度。

2. 变量在做什么

名称	含义
`x,y`	`Point` 的横纵坐标；向量按同一坐标差解释。
`EPS`	绝对误差判零阈值。
`PI`	由反余弦计算出的圆周率常量。
`a,b`	点积/叉积中的向量；`cross(a,b)>0` 表示 b 位于 a 的逆时针侧。
`angle`	旋转弧度，正值为逆时针。

3. 函数在做什么

`Point` 的 +、-、*、/ 运算符返回新对象，不改操作数；除法要求除数非零。`dot` 返回点积，`cross` 返回有向面积标量，`norm2` 返回平方长度，`length` 返回非负长度，`rotate` 返回旋转后的新向量。`sgn(x)` 返回 –1/0/1，按绝对 `EPS` 判定。

4. 怎么调用

用两个向量验证点积、叉积和 90 度旋转：

```
Point a{1, 0}, b{0, 1};
assert(dot(a, b) == 0);
assert(sgn(cross(a, b)) == 1);
Point r = rotate(a, PI / 2);
assert(fabsl(r.x) < 1e-12L && fabsl(r.y - 1) < 1e-12L);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;

const ld EPS = 1e-12L;
const ld PI = acosl(-1.0L);

```

```cpp
// 这是绝对误差判号；若坐标尺度极大，应按题目误差要求调整 EPS 或改用相对误差。
// 参数 x 为 long double；按绝对 EPS 返回 -1/0/1，不修改全局容差。
int sgn(ld x) {
    if (x > EPS) return 1;
    if (x < -EPS) return -1;
    return 0;
}

struct Point {
    // 浮点点也作为向量使用；b-a 表示从 a 指向 b，坐标下标不涉及点编号。
    ld x = 0, y = 0;
    // 参数 p 是另一向量；返回逐坐标相加的新点，不改两操作数。
    Point operator+(const Point& p) const { return {x + p.x, y + p.y}; }
    // 参数 p 是被减向量；返回 this-p 的新向量，不改两操作数。
    Point operator-(const Point& p) const { return {x - p.x, y - p.y}; }
    // 参数 k 为实数缩放因子；返回缩放后的新向量，不改 this。
    Point operator*(ld k) const { return {x * k, y * k}; }
    // 参数 k 必须非零；返回逐坐标除以 k 的新向量，不改 this。
    Point operator/(ld k) const { return {x / k, y / k}; }
};

// 参数 a,b 为向量；返回点积标量，不修改输入。
ld dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
// cross(a,b)>0 表示从向量 a 逆时针转到 b；绝对值是平行四边形面积。
// 参数 a,b 为向量；返回有向叉积标量，不修改输入。
ld cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
// 参数 a 为向量；返回平方长度，不开方且不修改输入。
ld norm2(Point a) { return dot(a, a); }
// 参数 a 为向量；返回非负欧氏长度，不修改输入。
ld length(Point a) { return sqrtl(norm2(a)); }

// 参数 a 为向量、angle 为弧度；返回逆时针旋转后的新向量，不改输入。
Point rotate(Point a, ld angle) {
    // angle 使用弧度，正值按数学坐标系逆时针旋转；屏幕 y 轴向下时视觉方向相反。
    ld c = cosl(angle), s = sinl(angle);
    return {a.x * c - a.y * s, a.x * s + a.y * c};
}
```

5. 要改哪里

角度制输入先乘 PI/180 转成弧度。若坐标尺度变化显著，应重新设计 EPS 或使用相对误差判定；不可仅把阈值机械调小。若参与除法的量可能为零，调用前处理退化情况。整数精确判向可使用 9.2 的整点叉积模板。

6. 自检

(1,0) 点乘 (0,1) 为 0，叉积为 1；旋转 (1,0) 逆时针 π/2 后近似为 (0,1)。零向量的长度为 0；对任意向量 v，norm2(v) 应非负。

7. 坑

1. 把角度制直接传入 `rotate`：接口用弧度。
2. 浮点值直接与 0 比较：使用 `sgn` 或明确误差界。
3. 零向量参与除法：先检查长度或除数。
4. 坐标很大仍使用固定绝对 `EPS`：误差尺度不匹配，需调整模型。

8. 一句话原理

把点的坐标差当作向量，用点积、叉积和三角函数表达二维几何关系。

参考：

- OI Wiki：二维计算几何基础

9.2 叉积与方向判断

1. 它是做什么的

整点模板用叉积精确判断左转、右转、共线及点是否落在闭线段上。单次操作 $O(1)$。点坐标是 `long long`，三点叉积先提升到 signed __int128 再相减；坐标差和最终叉积仍须能由 signed __int128 表示。`onSegment` 包含端点。

2. 变量在做什么

名称	含义
`IPoint` 的 `x,y`	整数点坐标，不代表数组下标。
`a,b`	`cross` 的向量，或三点重载中的起点和第二点。
`c`	三点叉积中的第三点，计算 `(b-a)x(c-a)`。
`p`	待测点。
`value`	有向叉积；正左转、负右转、零共线。

3. 函数在做什么

`cross(a,b)` 返回两个整数向量叉积；`cross(a,b,c)` 返回 `(b-a)x(c-a)`，两者均返回 `i128`。`orientation` 将符号映射为 1/0/-1。`onSegment(a,b,p)` 仅在共线且 `p` 落在两坐标轴包围盒内时返回 true，端点包含；均不修改输入。

4. 怎么调用

以水平线段及其两侧点检查方向和闭区间端点：

```
IPoint a{0, 0}, b{4, 0}, p{2, 0}, above{2, 1};
assert(orientation(a, b, above) == 1);
assert(onSegment(a, b, p));
assert(onSegment(a, b, a));
assert(!onSegment(a, b, above));
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct IPoint {
    ll x = 0, y = 0;
};

// 两向量叉积；整数坐标下精确判号，乘法提升到 i128 以避免 long long 溢出。
// 参数 a,b 为整数向量；返回叉积，结果须可由 signed i128 表示。
i128 cross(IPoint a, IPoint b) {
    return (i128)a.x * b.y - (i128)a.y * b.x;
}

// 参数 a,b,c 为整数点；返回 (b-a)x(c-a)，精确结果须在 signed i128 范围内。
i128 cross(IPoint a, IPoint b, IPoint c) {
    // 返回 (b-a)x(c-a)：正为 a->b 到 a->c 左转，负为右转，零为共线。
    i128 abx = (i128)b.x - a.x, aby = (i128)b.y - a.y;
    i128 acx = (i128)c.x - a.x, acy = (i128)c.y - a.y;
    return abx * acy - aby * acx;
}

// 参数为三点；返回左转 1、共线 0、右转 -1，不修改输入。
int orientation(IPoint a, IPoint b, IPoint c) {
    i128 value = cross(a, b, c);
```

```
    return (value > 0) - (value < 0);
}

// 参数 a,b 定义闭线段、p 为查询点；共线且在包围盒内（含端点）时返回 true。
bool onSegment(IPoint a, IPoint b, IPoint p) {
    // 闭线段判定：共线还不够，还要落在两端点的横纵坐标包围盒内；端点算在线。
    return orientation(a, b, p) == 0 &&
           min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
```

5. 要改哪里

浮点坐标需采用 EPS 判号与范围判断，不能直接把浮点值传入整数模板。若要判断线段相交，可由方向值和 `onSegment` 组合；若要计算交点坐标，则使用直线/线段交点算法。坐标范围接近 `long long` 极限时，差值和叉积可能超出 `i128`，应改用更宽/大整数。

6. 自检

`a`=(0,0), `b`=(4,0) 时 above=(2,1) 左转为 1，below=(2,-1) 右转为 −1，共线点 (2,0) 在闭段上，端点也在线段上；共线点 (5,0) 不在线段上。

7. 坑

1. 将共线误认为在线段上：还要检查横纵包围盒。
2. 交换 `b`、`c` 后仍期待原符号：交换顺序会改变方向。
3. 在提升前先做坐标减法：要在 signed `i128` 中求差。
4. 认为 `i128` 没有范围限制：极端坐标下叉积仍会溢出。

8. 一句话原理

叉积符号给出转向，叉积为零时再用坐标范围检查线段包含。

参考：

- OI Wiki：极坐标与叉积

9.3 直线交点与线段相交

1. 它是做什么的

浮点模板求两条无限直线的唯一交点，并判断两条闭线段是否有公共点；端点接触和共线重叠都算相交。每次操作 $O(1)$。直线方向须非零，交点计算以绝对 `EPS=1e-12` 判断分母；绝对误差适用于尺度合适的数据。`lineIntersection` 对平行或重合返回 false 且不改 `out`。

2. 变量在做什么

名称	含义
`p,v`	第一条直线的起点和方向，参数式 `p+t*v`。
`q,w`	第二条直线的起点和方向，参数式 `q+s*w`。
`denominator`	`cross(v,w)`；近零表示平行或重合，无唯一交点。
`t`	第一条直线交点参数。
`a,b,c,d`	两条闭线段的端点。
`c1..c4`	各端点相对另一条有向直线的方向符号。

3. 函数在做什么

`sgn` 按绝对 `EPS` 返回符号；`Point` 的 `+`、`-`、`*` 返回新向量。`cross` 有向量和三点两个重载。`lineIntersection` 要求 `v`、`w` 非零，成功写出 `out` 并返回 true；平行/重合返回 false，`out` 保持原值。`onSegment(a,b,p)` 的参数 `a`、`b` 为线段端点、`p` 为查询点；共线且在包围盒内（含端点）时为 true，退化段按当前容差作点附近

判定。`segmentsIntersect` 返回两闭线段是否共享点，含端点及共线重叠；退化线段按当前容差作点附近判定。

4. 怎么调用

两条交叉直线交于 (0.5,0.5)，两段端点相接也算相交：

```
Point out{9, 9};
bool ok = lineIntersection(Point{0,0}, Point{1,1},
                           Point{0,1}, Point{1,-1}, out);
assert(ok && fabsl(out.x - 0.5L) < 1e-12L);
assert(fabsl(out.y - 0.5L) < 1e-12L);
assert(segmentsIntersect(Point{0,0}, Point{1,0},
                         Point{1,0}, Point{2,0}));
assert(segmentsIntersect(Point{0,0}, Point{2,0},
                         Point{1,0}, Point{3,0}));
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;
const ld EPS = 1e-12L;

// EPS 是绝对误差阈值；整数输入且只判相交时应改用 i128 叉积以得到精确结果。
// 参数 x 为浮点量；按本节绝对 EPS 返回 -1/0/1。
int sgn(ld x) { return (x > EPS) - (x < -EPS); }

struct Point {
    ld x = 0, y = 0;
    // 参数 p 为向量；返回坐标和的新值，不改操作数。
    Point operator+(Point p) const { return {x + p.x, y + p.y}; }
    // 参数 p 为向量；返回 this-p 的新值，不改操作数。
    Point operator-(Point p) const { return {x - p.x, y - p.y}; }
    // 参数 k 为实数；返回按 k 缩放的新向量，不改 this。
    Point operator*(ld k) const { return {x * k, y * k}; }
};

// 参数 a,b 为向量；返回浮点有向叉积，不修改输入。
ld cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
// 参数为三点；返回 (b-a)x(c-a)，供方向判定使用。
ld cross(Point a, Point b, Point c) { return cross(b - a, c - a); }

// 参数给出 p+t*v、q+s*w 且方向非零；成功写 out 返回 true，平行/重合返回 false 且 out 不变。
bool lineIntersection(Point p, Point v, Point q, Point w, Point& out) {
    // 两条无限直线分别是 p+t*v、q+s*w；v、w 必须是非零方向向量。
    ld denominator = cross(v, w);
    // 叉积近零包含“平行不重合”和“整条重合”，二者都没有唯一交点。
    if (sgn(denominator) == 0) return false;
    // 由 (q-p)xw=t*(v x w) 解出参数 t；交换叉积顺序会把符号写反。
    ld t = cross(q - p, w) / denominator;
    out = p + v * t;
    return true;
}

// 参数 a,b 定义闭线段，p 为查询点；按绝对 EPS 判断是否在线段上，包含端点，退化段按容差判定点附近。
bool onSegment(Point a, Point b, Point p) {
    // 使用带容差的闭区间判定；a==b 时坐标乘积仍使用 EPS，不能当作精确点相等。
    return sgn(cross(a, b, p)) == 0 &&
           sgn((p.x - a.x) * (p.x - b.x)) <= 0 &&
           sgn((p.y - a.y) * (p.y - b.y)) <= 0;
}

```

```cpp
// 参数为两条闭线段端点；返回是否共享一点，端点接触和共线重叠都算相交。
bool segmentsIntersect(Point a, Point b, Point c, Point d) {
    // c1、c2 是 c、d 相对有向直线 a->b 的方向；异号才是严格跨越。
    int c1 = sgn(cross(a, b, c)), c2 = sgn(cross(a, b, d));
    int c3 = sgn(cross(c, d, a)), c4 = sgn(cross(c, d, b));
    if (c1 * c2 < 0 && c3 * c4 < 0) return true;
    return (c1 == 0 && onSegment(a, b, c)) ||
           (c2 == 0 && onSegment(a, b, d)) ||
           (c3 == 0 && onSegment(c, d, a)) ||
           (c4 == 0 && onSegment(c, d, b));
}
```

5. 要改哪里

若只判断整数线段相交，改用 9.2 的精确 i128 方向判断。若要区分“严格穿越”与接触/重叠，应在闭段相交结果外单独规定退化与共线规则。坐标尺度变化时调整 `EPS`；需要输出平行重合的关系时，应另增分类结果，当前接口只报告没有唯一交点。

6. 自检

两条非平行直线应写出唯一交点；平行和重合直线都返回 false 且 `out` 不变。线段端点接触为 true，共线不重叠为 false。零长度线段仅在它所在点落在另一段上时相交，但不能作为无限直线方向。

7. 坑

1. 把 `lineIntersection` 的 false 当成没有公共点：它也可能表示直线重合。
2. 只检查端点异侧：共线端点需额外用 `onSegment` 判断。
3. 忽略端点接触：本函数采用闭线段语义。
4. 用零方向求直线交点：方向非零是前提。

8. 一句话原理

叉积符号判断两端相对方向，唯一直线交点则通过解参数方程求出。

参考：

- OI Wiki：直线与线段相关

9.4 距离与投影

1. 它是做什么的

本节计算点到无限直线或闭线段的投影点及距离，单次时间 $O(1)$。坐标采用 `long double`；线段长度平方小于 `EPS*EPS` 时按退化点处理。无限直线接口要求 `a`、`b` 不同，线段接口会把退化段当作点 `a`。`EPS` 为绝对阈值，尺度不合适时要调整。

2. 变量在做什么

名称	含义
`p`	查询点。
`a,b`	直线或线段端点。
`v=b-a`	方向向量。
`length2`	`v` 的平方长度；小于 `EPS*EPS` 时线段退化。
`t`	投影参数 `dot(p-a,v)/|v|²`；截到 [0,1] 后对应线段最近点。

3. 函数在做什么

`dot` 返回点积，`norm2` 返回平方长度，`distance` 返回欧氏距离。`projectionOnLine` 要求 `a!=b`，返回无限直线垂足；`distanceToLine` 返回点到直线距离。`closestPointOnSegment` 对短于 `EPS` 的段返回 `a`，否则将投影参数截到 [0,1]。`distanceToSegment` 返回 `p` 到最近点的距离。

4. 怎么调用

`p`=(2,2)，线段端点 (-1,0),(1,0)，无限直线垂足是 (2,0)，线段最近点为 (1,0)：

```
Point p{2,2}, a{-1,0}, b{1,0};
Point foot = projectionOnLine(p, a, b);
assert(fabsl(foot.x - 2) < 1e-12L && fabsl(foot.y) < 1e-12L);
Point near = closestPointOnSegment(p, a, b);
assert(fabsl(near.x - 1) < 1e-12L && fabsl(near.y) < 1e-12L);
assert(fabsl(distanceToSegment(p, a, b) - sqrtl(5.0L)) < 1e-12L);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;
const ld EPS = 1e-12L;

struct Point {
    ld x = 0, y = 0;
    // 参数 p 为向量；返回坐标和的新值，不改操作数。
    Point operator+(Point p) const { return {x + p.x, y + p.y}; }
    // 参数 p 为向量；返回 this-p 的新值，不改操作数。
    Point operator-(Point p) const { return {x - p.x, y - p.y}; }
    // 参数 k 为缩放因子；返回新向量，不改 this。
    Point operator*(ld k) const { return {x * k, y * k}; }
};

// 参数 a,b 为向量；返回点积标量。
ld dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
// 参数 a 为向量；返回平方长度。
ld norm2(Point a) { return dot(a, a); }
// 参数为两点；返回非负欧氏距离，平方量以 long double 计算。
ld distance(Point a, Point b) { return sqrtl(norm2(a - b)); }

// 参数 p 为查询点、a!=b 定义无限直线；返回垂足，不限制投影参数。
Point projectionOnLine(Point p, Point a, Point b) {
    // 求无限直线参数 t；前提 a!=b，否则 norm2(b-a)=0。线段版本在下方单独处理退化。
    Point v = b - a;
    return a + v * (dot(p - a, v) / norm2(v));
}

// 参数 p 为查询点、a!=b 定义直线；返回到无限直线的非负距离。
ld distanceToLine(Point p, Point a, Point b) {
    return distance(p, projectionOnLine(p, a, b));
}

// 参数 p 为查询点、a,b 为线段端点；返回最近点，EPS² 内退化段返回 a。
Point closestPointOnSegment(Point p, Point a, Point b) {
    Point v = b - a;
    ld length2 = norm2(v);
    // EPS^2 与平方长度同量纲；退化或极短线段按单点 a 处理。
    if (length2 < EPS * EPS) return a;
    ld t = dot(p - a, v) / length2;
    // 无限直线投影参数截到 [0,1]，分别对应端点 a、b 和线段内部。
    t = max((ld)0, min((ld)1, t));
    return a + v * t;
}

// 参数 p 为查询点、a,b 为闭线段端点；返回非负距离，退化段按点 a 计算。
ld distanceToSegment(Point p, Point a, Point b) {
```

```
49      return distance(p, closestPointOnSegment(p, a, b));
50  }
```

5. 要改哪里

若要最近点在射线而非线段上，将投影参数限制为 `t>=0`；无限直线则不截断。若输入为整数且只比较点到直线距离，可避免开方但仍需设计足够宽的精确类型。短线段的 `EPS*EPS` 规则取决于坐标尺度，应与其他几何判定保持一致。

6. 自检

垂足落在线段内部时两种投影一致；垂足在线段外时最近点应为相邻端点。`a=b` 时 `closestPointOnSegment` 返回 `a`，`distanceToSegment` 等于点距；`projectionOnLine` 对同样输入不合法。

7. 坑

1. 将直线投影误当线段最近点：线段参数必须截在 [0,1]。
2. 直接除以 `norm2(v)`：退化线段会除零，直线接口要求端点不同。
3. 用平方长度与线性 `EPS` 比较：退化阈值要用 `EPS*EPS`。
4. 需要比较距离却反复开方：可比较平方距离，但要确认所有量同尺度。

8. 一句话原理

沿方向向量求投影参数，直线保留原参数，线段将其夹到闭区间 [0,1]。

参考：

- OI Wiki：距离

9.5 多边形面积

1. 它是做什么的

`signedDoubleArea` 用鞋带公式计算按边界顺序给出的简单整数多边形有向二倍面积，时间 $O(n)$、额外空间 $O(1)$。输入点坐标为 `long long`；坐标差、单项叉积及总和都必须可由 `signed __int128` 表示。首点无需重复在末尾；空输入、少于三个点或共线退化时返回 0。逆时针结果为正，顺时针为负。

2. 变量在做什么

名称	含义
`IPoint` 的 `x,y`	整数顶点坐标。
`polygon`	按多边形边界顺序排列的顶点；不是无序点集。
`n`	顶点数。
`i`	当前边的起点下标。
`(i+1)%n`	当前边终点；最后一项自动连接回首点。
`sum`	所有边叉积之和，即有向二倍面积。

3. 函数在做什么

`signedDoubleArea(polygon)` 返回 `signed __int128`，不修改输入。顶点按边界顺序给出且不重复首点；空输入、少于三个点或共线退化时返回 0。对简单多边形取绝对值再除以 2 得到普通面积；自交顶点列的有向叉积和不代表通常的覆盖面积。

4. 怎么调用

单位正方形逆时针的有向二倍面积为 2，反向则为 −2：

```
1  vector<IPoint> ccw{{0,0},{1,0},{1,1},{0,1}};
2  vector<IPoint> cw{{0,1},{1,1},{1,0},{0,0}};
```

```
assert(signedDoubleArea(ccw) == 2);
assert(signedDoubleArea(cw) == -2);
assert(signedDoubleArea(vector<IPoint>{}) == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct IPoint { ll x = 0, y = 0; };

// 单项叉积提升到 i128；若坐标约在 1e18 量级，多个叉积之和仍可能超过 i128。
// 参数 a,b 为整数位置向量；返回叉积，signed i128 须容纳乘积差。
i128 cross(IPoint a, IPoint b) {
    return (i128)a.x * b.y - (i128)a.y * b.x;
}

// 参数为按边界顺序且首点不重复的顶点列；返回有向二倍面积，逆时针为正，不改输入。
i128 signedDoubleArea(const vector<IPoint>& polygon) {
    // 顶点须按边界顺序给出且不重复首点；返回有向二倍面积，逆时针为正。
    i128 sum = 0;
    int n = (int)polygon.size();
    // (i+1)%n 自动补上末点到首点的闭合边；n<3 时自然得到退化面积 0。
    for (int i = 0; i < n; ++i)
        sum += cross(polygon[i], polygon[(i + 1) % n]);
    return sum;
}
```

5. 要改哪里

要返回非负面积，可在调用端对结果取绝对值并除以 2；若二倍面积为奇数，真实面积含 .5。输入必须提供真实多边形边界顺序；只有题目要求凸包面积时才先求凸包，不能将一般凹多边形替换成凸包。若坐标范围增大，应重新证明差值、叉积和累加总和均不溢出；提升到 `i128` 并不代表无限精度。

6. 自检

逆时针单位正方形返回 2，顺时针返回 −2；少于 3 点和共线顶点序列返回 0。交换顶点遍历方向会改变符号但绝对值不变。

7. 坑

1. 忘记结果是二倍面积：最后需除以 2。
2. 输入无序点集：鞋带公式要求沿边界顺序。
3. 漏掉末点到首点边：模板通过取模补齐闭合边。
4. 只提升乘法、不考虑总和：单项及累加值都要可放入 `i128`。

8. 一句话原理

沿边界把相邻顶点叉积求和，得到有向二倍面积。

参考：

- OI Wiki：多边形面积

9.6 点在多边形内

1. 它是做什么的

给定按边界顺序排列的简单浮点多边形和查询点，射线奇偶法判断点在外部、边界还是内部。多边形顶点可顺时针或逆时针，首点无需重复；空顶点列返回 `OUTSIDE`。单次时间 $O(n)$，除输入外额外空间 $O(1)$。自

交多边形不适用。代码使用绝对 `EPS=1e-12`，坐标尺度很大或很小时可能需要调整；退化的 1、2 点顶点列不代表通常的简单多边形，应按题意自行定义语义。

2. 变量在做什么

名称	含义
`Point{x,y}`	长双精度坐标点；减法返回向量差。
`polygon[i]`	边界顺序第 `i` 个顶点，向量下标为 0-based。
`a,b`	当前边的起点及下一顶点；最后一点连接回第 0 点。
`p`	查询点。
`cross(a,b)`	两向量有向叉积，符号表示相对方向。
`inside`	射线与边界交点数的奇偶状态；跨过一个右侧交点就翻转。
`EPS`	绝对容差，`sgn` 将 `[-EPS,EPS]` 视为 0。

函数先用 `onSegment` 判断边界，再用半开高度条件检查边是否跨过点的水平射线，避免顶点重复计数。

3. 函数在做什么

函数	契约
`sgn(x)`	返回 `-1/0/1`，按绝对 `EPS` 判断符号。
`Point::operator-(p)`	返回当前点减 `p` 的向量，不改操作数。
`cross(a,b)`	返回两个向量的有向叉积标量。
`onSegment(a,b,p)`	若 `p` 按当前 EPS 判断在线段闭区间上则返回 `true`，含端点；不修改输入。
`pointInPolygon(polygon,p)`	参数为简单多边形顶点列和查询点；返回 `OUTSIDE`、`BOUNDARY` 或 `INSIDE`，不修改输入。

4. 怎么调用

按顺序提供多边形顶点，调用后根据枚举值分支。正方形示例验证内部、边界、外部和空多边形：

```
vector<Point> square = {{0, 0}, {2, 0}, {2, 2}, {0, 2}};
assert(pointInPolygon(square, {1, 1}) == INSIDE);
assert(pointInPolygon(square, {2, 1}) == BOUNDARY);
assert(pointInPolygon(square, {3, 1}) == OUTSIDE);
assert(pointInPolygon({}, {0, 0}) == OUTSIDE);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ld = long double;
const ld EPS = 1e-12L;

// 参数 x 为浮点量；按本节绝对 EPS 返回 -1/0/1。
int sgn(ld x) { return (x > EPS) - (x < -EPS); }
struct Point {
    ld x = 0, y = 0;
    // 参数 p 为向量；返回 this-p 的新值，不改操作数。
    Point operator-(Point p) const { return {x - p.x, y - p.y}; }
};
// 参数 a,b 为向量；返回有向叉积标量。
ld cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }

// 参数 a,b 定义闭边、p 为点；共线且落在包围盒内（含端点）时返回 true。
bool onSegment(Point a, Point b, Point p) {
    // 边界优先单独判定，避免射线与水平边或顶点重合时归属不明确。
    return sgn(cross(b - a, p - a)) == 0 &&
```

```cpp
            sgn((p.x - a.x) * (p.x - b.x)) <= 0 &&
            sgn((p.y - a.y) * (p.y - b.y)) <= 0;
}

enum PointLocation { OUTSIDE, BOUNDARY, INSIDE };

// 参数是按边界顺序的简单多边形和查询点；返回 OUTSIDE/BOUNDARY/INSIDE，不修改输入。
PointLocation pointInPolygon(const vector<Point>& polygon, Point p) {
    // 适用于顶点按边界排列的简单多边形；返回值明确区分边界与内部。
    bool inside = false;
    int n = (int)polygon.size();
    for (int i = 0; i < n; ++i) {
        Point a = polygon[i], b = polygon[(i + 1) % n];
        if (onSegment(a, b, p)) return BOUNDARY;
        // 半开高度条件保证射线经过顶点时只让相邻两边中的一条计数。
        bool straddles = (a.y > p.y) != (b.y > p.y);
        if (straddles) {
            ld intersectionX = a.x + (b.x - a.x) *
                (p.y - a.y) / (b.y - a.y);
            // 只统计查询点右侧交点；每跨过一次边界就翻转内外奇偶性。
            if (intersectionX > p.x + EPS) inside = !inside;
        }
    }
    return inside ? INSIDE : OUTSIDE;
}
```

5. 要改哪里

- 若需整数坐标精确判定，应将叉积及方向运算改为足够宽的整数类型，并在减法前扩宽；`__int128` 也有取值范围。
- 若输入是自交多边形，必须先定义奇偶规则或非零绕数规则，再选对应算法；当前接口假设简单多边形。
- 若坐标范围变化明显，需重新评估 `EPS` 对叉积、坐标乘积和交点比较的影响；该实现使用统一绝对阈值，不具尺度不变性。

6. 自检

正方形 `[0,2]×[0,2]` 中 `(1,1)` 为内部，边 `(2,0)-(2,2)` 上的 `(2,1)` 为边界，`(3,1)` 在外部。空点列返回外部。再检查水平边上的点和顶点本身均返回边界；顶点附近相差小于 EPS 的点可能受容差影响，坐标尺度需与 EPS 匹配。

7. 坑

- 射线过顶点时奇偶翻转两次：使用半开高度条件，不能简单判断两端是否跨过。
- 边界点被判内或外：先调用 `onSegment` 并单独返回 `BOUNDARY`。
- 浮点几何在不同尺度下误判：固定绝对 EPS 不随坐标缩放，按输入尺度调整并测试近边界点。
- 用于自交轮廓：内部定义不唯一或规则不同，需换成题目要求的判定方法。

8. 一句话原理

先识别边界，再数查询点向右射线与简单多边形的交点奇偶性。

参考：

- OI Wiki：判断一个点是否在多边形内部

9.7 凸包

1. 它是做什么的

Andrew 单调链从无序整数点集中构造凸包，返回按逆时针排列且不重复首点的顶点；本实现删除边上中间的共线点。重复输入点可有；空集返回空，单点保留，全共线时只返回两个端点。排序需 $O(n \log n)$，扫

描为 $O(n)$，总时间 $O(n \log n)$；按值接收输入并建立凸包，空间 $O(n)$。坐标差和叉积都必须能由 signed `__int128` 表示；提升到 `__int128` 能避免 `long long` 减法先溢出，但乘积及其差仍有有限范围。

2. 变量在做什么

名称	含义
`IPoint{x,y}`	64 位整数点；按 `x` 再按 `y` 字典序排序，坐标相等视为同一点。
`points`	函数的输入副本；先排序，再去重。
`hull`	当前构造的下凸壳和上凸壳拼接序列。
`lowerSize`	下凸壳完成后的长度，作为逆序扫描上凸壳的边界。
`cross(a,b,c)`	`(b-a)×(c-a)`；正值左转，零共线，负值右转。

返回 vector 是 0-based；首点为字典序最小点，不再在末尾重复首点。判断转向前坐标先扩宽到 `i128`，但若输入极值使叉积超出 signed `__int128` 仍会溢出。

3. 函数在做什么

函数	契约
`IPoint::operator<(p)`	按 `x`、`y` 字典序比较，用于排序。
`IPoint::operator==(p)`	两坐标完全相等时返回 `true`，供 `unique` 去重。
`cross(a,b,c)`	返回有向叉积；坐标差、乘积及最终差均须处于 signed `__int128` 范围。
`convexHull(points)`	按值接收任意顺序整数点（可重复）；返回逆时针凸包，不修改调用者向量。边界共线中间点会删除。

4. 怎么调用

提供点集并调用即可。正方形去重后仍按逆时针返回四角；共线点集只保留极端两点：

```
vector<IPoint> points = {{1, 1}, {0, 0}, {1, 0}, {0, 1}, {1, 1}};
vector<IPoint> hull = convexHull(points);
assert(hull == vector<IPoint>({{0, 0}, {1, 0}, {1, 1}, {0, 1}}));
vector<IPoint> line = convexHull({{0, 0}, {1, 1}, {2, 2}, {1, 1}});
assert(line == vector<IPoint>({{0, 0}, {2, 2}}));
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct IPoint {
    ll x = 0, y = 0;
    // 参数 p 为另一整数点；按 x 再 y 严格字典序比较，用于排序，不改点。
    bool operator<(const IPoint& p) const { return tie(x, y) < tie(p.x, p.y); }
    // 参数 p 为另一整数点；仅两坐标完全相等时返回 true。
    bool operator==(const IPoint& p) const { return x == p.x && y == p.y; }
};

// 参数为整数点；返回 (b-a)x(c-a)，坐标差和乘积差须在 signed i128 范围。
i128 cross(IPoint a, IPoint b, IPoint c) {
    // (b-a)x(c-a)>0 是严格左转；整数 i128 判号不需要 EPS。
    i128 abx = (i128)b.x - a.x, aby = (i128)b.y - a.y;
    i128 acx = (i128)c.x - a.x, acy = (i128)c.y - a.y;
    return abx * acy - aby * acx;
}

```

```cpp
// 参数 points 按值接收，可乱序/重复；返回逆时针凸包且不重复首点，删去边上中间共线点。
vector<IPoint> convexHull(vector<IPoint> points) {
    // 输入可无序且可重复；输出 0-based vector 中按逆时针排列，不重复首点。
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()), points.end());
    if (points.size() <= 1) return points;

    vector<IPoint> hull;
    for (const IPoint& p : points) {
        // 维护下凸壳连续三点严格左转；<=0 会同时删除右转和边上共线中间点。
        // 若题目要求保留边界共线点，改为 <0，并额外处理全部共线的输入。
        while (hull.size() >= 2 &&
               cross(hull[hull.size() - 2], hull.back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }
    size_t lowerSize = hull.size();
    // 逆序扫描构造上凸壳；lowerSize 是边界，防止弹掉已完成的整条下凸壳。
    for (int i = (int)points.size() - 2; i >= 0; --i) {
        const IPoint& p = points[i];
        while (hull.size() > lowerSize &&
               cross(hull[hull.size() - 2], hull.back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }
    // 最后压入的是起点副本，删除它以保持“首点不重复”的返回约定。
    hull.pop_back();
    return hull;
}
```

5. 要改哪里

- 若要保留凸包边上的所有共线点，可将弹栈条件 `<= 0` 改为 `< 0`，并单独处理全部共线输入，避免重复端点。
- 若坐标范围可能超过 `long long` 差值或 `__int128` 叉积范围，需更宽的精确表示或限制输入范围；不能只因用了 `i128` 就视为无限安全。
- 若需要顺时针输出，可在得到凸包后反转非首点部分，保持起点约定。
- 若要算面积或周长，可在返回点列上循环 `(i+1)%hull.size()`；空集和单点要单独处理。

6. 自检

示例正方形应得到 `(0,0),(1,0),(1,1),(0,1)`，首点不重复；重复输入点不改变结果。全共线的 `(0,0),(1,1),(2,2)` 应只返回两端点；单点返回自身，空集返回空。对凸包相邻三点检查叉积应为正；输入有极大坐标时先核对差值与乘积范围。

7. 坑

- 同一点重复进入链：先按字典序排序并去重。
- 共线点保留策略不符：`<=0` 删除中间共线点，改 `<0` 会改变输出且须处理全共线情况。
- 把末点重复首点当作结果：模板已删除重复首点，遍历闭合边时使用 `(i+1)%n`。
- 叉积溢出：减法前转 `i128`，同时保证乘积和最终减法仍在其范围内。

8. 一句话原理

按字典序分别扫描下、上链，遇到非左转就回退，保留包住全部点的最外层转折点。

参考：

- OI Wiki：凸包

9.8 旋转卡壳：凸包直径

1. 它是做什么的

给定按逆时针排列、无重复首点的严格凸包，旋转卡壳求最远点对的欧氏距离平方。输入点型为 `IPoint`，坐标为 `long long`；叉积、绝对值和平方距离均须在 signed `__int128` 范围内。若凸包有 0 或 1 个点返回 0，2 点返回唯一平方距离；其余情况时间 $O(h)$，额外空间 $O(1)$，其中 `h` 为凸包顶点数。若先从 `n` 个原始点构造凸包，总时间为 $O(n \log n)$。不直接接受未排序原始点或含边上共线中间点的轮廓。组合使用 9.7 时，应共用或合并 `IPoint`、`cross` 等公共定义，不能原样拼接两份含同名定义的完整代码块。

2. 变量在做什么

名称	含义
`IPoint{x,y}`	整数坐标点；沿用凸包章节的点字段。
`hull`	逆时针严格凸包，首点不重复；内部下标 0-based。
`i, nextI`	当前凸包边的起点和下一个顶点，`nextI=(i+1)%n`。
`j`	与当前边对应的对踵点指针，只沿凸包前进。
`cross`	边到候选点的有向面积；凸包方向已知，代码取绝对值比较面积。
`answer`	已检查点对的最大平方距离，类型为 `i128`。

3. 函数在做什么

函数	契约
`cross(a,b,c)`	返回 `(b-a)×(c-a)`；差值、乘积和最终减法均须适配 signed `i128`。
`distanceSquared(a,b)`	返回两点欧氏距离平方；坐标差及平方和须适配 signed `i128`。
`abs128(x)`	返回 `|x|`；输入不能是 signed `i128` 最小值。
`convexDiameterSquared(hull)`	输入严格逆时针凸包且首点不重复；返回最大距离平方，0/1 点为 0、2 点直接计算。
`toString(x)`	将非负 `i128` 转成十进制字符串；负数不受支持。

4. 怎么调用

传入严格凸包；若已有原始点，先用 9.7 的 Andrew 凸包算法处理。下例矩形对角线平方为 $4^2+3^2=25$，两点退化输入也直接返回平方距离：

```
vector<IPoint> hull = {{0, 0}, {4, 0}, {4, 3}, {0, 3}};
i128 d2 = convexDiameterSquared(hull);
assert(d2 == 25 && toString(d2) == "25");
assert(convexDiameterSquared({{0, 0}, {3, 4}}) == 25);
assert(convexDiameterSquared({{7, 9}}) == 0);
assert(convexDiameterSquared({}) == 0);
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;

struct IPoint {
    ll x = 0, y = 0;
};

// 参数为整数点；返回有向叉积，调用者保证差值和乘积在 signed i128 范围。
i128 cross(IPoint a, IPoint b, IPoint c) {
    i128 abx = (i128)b.x - a.x, aby = (i128)b.y - a.y;
    i128 acx = (i128)c.x - a.x, acy = (i128)c.y - a.y;
    return abx * acy - aby * acx;
}
```

```cpp

// 参数为两整数点；返回平方距离，坐标差、平方和须可由 signed i128 表示。
i128 distanceSquared(IPoint a, IPoint b) {
    // 差值和平方均在 i128 中完成；返回平方距离，避免开方误差。
    i128 dx = (i128)a.x - b.x;
    i128 dy = (i128)a.y - b.y;
    return dx * dx + dy * dy;
}

// 参数 x 不得是 signed i128 最小值；返回 |x|，不修改外部状态。
i128 abs128(i128 x) { return x < 0 ? -x : x; }

// 参数是逆时针且不重复首点凸包；返回最远平方距离，0/1 点为 0、2 点为点距。
i128 convexDiameterSquared(const vector<IPoint>& hull) {
    // 前提：hull 是逆时针严格凸包且不重复首点；可直接接 convexHull 的输出。
    int n = (int)hull.size();
    if (n <= 1) return 0;
    if (n == 2) return distanceSquared(hull[0], hull[1]);

    i128 answer = 0;
    int j = 1;
    for (int i = 0; i < n; ++i) {
        int nextI = (i + 1) % n;
        // 固定有向边 i->nextI，面积随对踵点前进先增后减，因此 j 全程只向前转。
        // 使用严格 > 可停在面积平台任一端；随后同时检查边的两个端点与 j。
        while (abs128(cross(hull[i], hull[nextI], hull[(j + 1) % n])) >
               abs128(cross(hull[i], hull[nextI], hull[j]))) {
            j = (j + 1) % n;
        }
        answer = max(answer, distanceSquared(hull[i], hull[j]));
        answer = max(answer, distanceSquared(hull[nextI], hull[j]));
    }
    return answer;
}

// 参数 x>=0；返回十进制字符串，仅转换非负距离值，不修改输入。
string toString(i128 x) {
    // 本章仅用它输出非负平方距离；若复用于有向面积，需先处理负号和最小值。
    if (x == 0) return "0";
    string s;
    while (x > 0) {
        s.push_back(char('0' + x % 10));
        x /= 10;
    }
    reverse(s.begin(), s.end());
    return s;
}
```

5. 要改哪里

- 若要返回最远点对，在每次更新 `answer` 时同步保存对应下标；返回点坐标时再映回 `hull`。
- 若要输出实际距离，可将非负平方值转为 `long double` 后开方；若输出整数平方距离，使用 `toString`，不能直接用 `cout` 输出 `i128`。
- 若输入点集未经凸包处理，先求凸包；若凸包保留大量共线边界点，先按本算法前提规整或改写指针逻辑。
- 坐标极大时需证明坐标差、叉积、绝对值和平方和都未超出 `i128`，不能只检查最终答案。

6. 自检

矩形的最远点是对角顶点，平方距离应为 25。两点 `(0,0),(3,4)` 返回 25；单点和空凸包返回 0。对正方形应得到边长平方的两倍；输入应为逆时针且不重复首点。叉积平台出现时，代码以严格 `>` 停在平台端点，再检查当前边两端与该对踵点。

7. 坑

- 直接把无序点集传入：指针单调性依赖凸包按逆时针顺序排列。
- 首点重复或边上保留大量共线点：不符合严格凸包前提，可能破坏指针前进规则。
- 输出整数平方值乱码或编译失败：标准流不直接格式化 `__int128`，使用 `toString`。
- 认为 `i128` 不会溢出：乘法与平方和仍有有限范围；绝对值也不能对最小值取负。

8. 一句话原理

沿凸包边旋转并单调移动对踵点，只检查相邻边端与对踵点构成的候选直径。

参考：

- OI Wiki：旋转卡壳

9.9 圆与直线、圆与圆

1. 它是做什么的

本节提供无限直线与圆、两圆交点计算，坐标和半径为 `long double`，采用绝对 `EPS=1e-12`。每次计算时间 $O(1)$，返回至多两个 `Point`，结果容器空间 $O(1)$。直线由点 `p` 和方向向量 `v` 表示，要求 `dot(v,v)>EPS`；近零方向向回空。圆半径须非负。两圆圆心距离 `d<=EPS` 时（包括重合圆有无穷交点的情形）返回空；相切返回一个点，分离或包含而不相切返回空。浮点误差与坐标尺度有关，阈值需结合输入调整。

2. 变量在做什么

名称	含义
`Circle{o,r}`	圆心 `o` 和非负半径 `r`。
`p, v`	直线上一点及非零方向向量；`v` 不要求单位化。
`vv`	`dot(v,v)`，方向向量长度平方。
`foot`	圆心在直线上的垂足。
`h2`	从垂足到交点的偏移距离平方；小于 `-EPS` 判无交，接近 0 判切点。
`dvec, d`	两圆心的位移向量和距离。
`base, perp, off`	两交点弦中点、单位垂直方向及偏移向量。
`EPS`	绝对误差阈值；代码将长度平方、平方距离差等量也与同一个 EPS 比较。

依赖第 9.1 节的 `Point`、`dot`、`length`、`EPS` 和 `Point` 的加减、数乘操作。直线偏移按 `v * sqrt(h2/dot(v,v))` 计算，因此方向向量长度不限。

3. 函数在做什么

函数	契约
`lineCircleIntersections(p,v,c)`	返回无限直线与圆的 0、1 或 2 个交点；近零方向向量返回空。要求圆半径非负，依赖第 9.1 节几何定义。
`circleIntersections(a,b)`	返回两圆的 0、1 或 2 个交点；半径须非负。圆心近同（`d<=EPS`）时一律返回空，包括重合圆。

相切时两个接口都只返回一个点；返回点顺序按各自计算方向，不应依赖顺序代表特定语义。

4. 怎么调用

先提供第 9.1 节的几何基础定义，再调用函数。浮点结果用误差比较；下例直线与单位圆相交于 `(-1,0)`, `(1,0)`，两个单位圆相切于 `(1,0)`：

```
// near(a,b) 比较两个 long double 的绝对误差是否小于 1e-10，不修改状态。
```

```
auto near = [](long double a, long double b) { return fabsl(a - b) <
1e-10L; };
Circle unit{{0, 0}, 1};
auto lineHits = lineCircleIntersections({-2, 0}, {1, 0}, unit);
assert(lineHits.size() == 2);
assert(near(lineHits[0].x, -1) && near(lineHits[1].x, 1));
auto circleHits = circleIntersections(unit, Circle{{2, 0}, 1});
assert(circleHits.size() == 1 && near(circleHits[0].x, 1));
assert(lineCircleIntersections({0, 0}, {0, 0}, unit).empty());
```

模板如下:

```
struct Circle {
    Point o;
    long double r;
};
// 参数 p,v 定义直线且 dot(v,v)>EPS，圆半径非负；返回 0/1/2 交点，近零方向返回空。
vector<Point> lineCircleIntersections(Point p,Point v,Circle c) {
    long double vv=dot(v,v);
    if (vv <= EPS) return {};
    Point foot=p+v*(dot(c.o-p,v)/vv);
    long double h2=c.r*c.r-dot(foot-c.o,foot-c.o);
    if (h2 < -EPS) return {};
    if (h2 <= EPS) return {foot};
    Point off=v*(sqrtl(max((long double)0,h2)/vv));
    return {foot-off,foot+off};
}
// 参数是半径非负的两圆；返回 0/1/2 交点，`d<=EPS` 的近同心情形（含重合圆）返回空，切点只返一次。
vector<Point> circleIntersections(Circle a,Circle b) {
    // 先判相离、包含、同心等退化，再计算垂足与偏移。
    Point dvec=b.o-a.o;
    long double d=length(dvec);
    if(d<=EPS) {
        return {};
    }
    // 同心：无交点或无穷多交点，均返回空
    if (d > a.r + b.r + EPS || d < fabsl(a.r - b.r) - EPS) return {};
    long double x = (a.r * a.r - b.r * b.r + d * d) / (2 * d);
    long double h2 = a.r * a.r - x * x;
    Point base = a.o + dvec * (x / d);
    if (h2 <= EPS) return {base};
    Point perp{-dvec.y / d, dvec.x / d};
    Point off = perp * sqrtl(max((long double) 0, h2));
    return {base - off, base + off};
}
```

5. 要改哪里

- 若输入的是线段而非无限直线，需在求交后检查交点参数是否落在线段参数区间；当前函数不限制直线参数。
- 若同心且重合圆需表示“无限多个交点”，应换用能区分有限点集和重合状态的返回类型；当前统一返回空。
- 若要处理半径为负的输入，应在调用前拒绝或规范化；模板要求非负半径。
- 若要求更强数值稳定性或坐标尺度跨度大，应改用尺度相关容差/稳健几何判定，并重新审查每个平方量的阈值。

6. 自检

直线 `y=0` 与单位圆的交点横坐标应为 `-1`、`1`。单位圆心距 2 时相切并只返回一点；圆心距大于半径和时无交点，小于半径差且不同心时一圆包含另一圆、返回空。零方向向量返回空；同心相等圆因有无穷多个交点也按接口约定返回空。

7. 坑

- 把零方向向量当成直线：`dot(v,v)<=EPS` 时接口返回空。
- 把直线结果当线段结果：当前没有限制交点参数范围。
- 重合圆返回空却当作无交点：它表示近同心情形统一空，不区分无交与无限交。
- 直接比较浮点坐标或根号内负值：应按 EPS 分类；根号前代码将近零负值截到 0。
- EPS 对不同量纲一概使用：当前实现对长度平方等量也使用绝对 EPS，坐标尺度可能改变判定效果。

8. 一句话原理

先投影求垂足或两圆弦中点，再根据剩余平方距离决定无交、相切或两个对称交点。

9.10 整数几何方向模板

1. 它是做什么的

模板用整数精确判断三点方向、叉积符号，以及点是否落在包含端点的闭线段上。点坐标为 `long long`，中间差值和叉积使用 signed `__int128`；乘积和最终相减仍须在其范围内。三项操作时间均为 $O(1)$，额外空间 $O(1)$，不使用浮点 EPS。距离平方不是本模板接口，另行计算时也必须检查平方和范围。

2. 变量在做什么

名称	含义
`IPoint{x,y}`	整数坐标点，字段为 `long long`。
`a,b,c`	方向判断所用三点；`crossInt(a,b,c)` 计算 `(b-a)×(c-a)`。
`p,a,b`	线段判定中的查询点和两个端点。
`abx,aby,acx,acy`	先扩宽到 `__int128` 后再做坐标差，避免 `long long` 减法溢出。
叉积符号	正值表示 `a->b` 转向 `a->c` 为逆时针，负值为顺时针，0 为共线。

线段判定先要求叉积为 0，再检查两个坐标都落在端点闭区间内。若 `a==b`，只有 `p==a` 时返回真。

3. 函数在做什么

函数	契约
`crossInt(a,b,c)`	返回三点有向叉积；坐标差、乘积和相减结果必须适配 signed `__int128`。
`signInt(x)`	对 signed `__int128` 返回 `-1`、`0` 或 `1`，不修改输入。
`onSegmentInt(p,a,b)`	若 `p` 在线段 `ab` 的闭区间上返回 `true`，包括端点；叉积不得溢出。

4. 怎么调用

整数坐标直接传入，不需 EPS。下例中 `c` 在 `a->b` 左侧，`p` 在水平线段上，而更外侧点不在线段内：

```
IPoint a{0, 0}, b{4, 0}, c{1, 1}, p{2, 0};
assert(signInt(crossInt(a, b, c)) == 1);
assert(onSegmentInt(p, a, b));
assert(!onSegmentInt({5, 0}, a, b));
assert(onSegmentInt({2, 2}, {2, 2}, {2, 2}));
```

模板如下：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct IPoint {
```

```
    ll x,y;
};
// 参数为整数点；返回 (b-a)x(c-a)，坐标差和叉积须在 signed i128 范围。
__int128 crossInt(IPoint a, IPoint b, IPoint c) {
    // 转换发生在减法之前，避免 long long 坐标差先溢出。
    __int128 abx = (__int128)b.x - a.x, aby = (__int128)b.y - a.y;
    __int128 acx = (__int128)c.x - a.x, acy = (__int128)c.y - a.y;
    return abx * acy - aby * acx;
}
// 参数 x 为 signed i128；返回 -1/0/1，不修改输入。
int signInt(__int128 x) {
    return (x>0)-(x<0);
}
// 参数 p 为查询点、a,b 为线段端点；闭线段含端点判定，crossInt 不得溢出。
bool onSegmentInt(IPoint p,IPoint a,IPoint b) {
    return crossInt(a, b, p) == 0
        && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x)
        && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
```

5. 要改哪里

- 要判断严格位于线段内部，可在 `onSegmentInt` 的共线和包围盒条件外再排除 `p==a`、`p==b`。
- 要计算距离平方，应先把坐标转为 `__int128` 再相减、平方；平方和可能超出 signed `__int128`，需按坐标范围另作保证。
- 要判断线段相交，可组合四次方向符号判断，并单独处理共线重叠；本模板没有直接提供相交接口。
- 需要非整数交点时，交点坐标可能是分数，应改用有理数或浮点表示，不能强行存入 `IPoint`。

6. 自检

`(0,0)->(4,0)` 与 `(1,1)` 的叉积为 4，符号为正；`(2,0)` 在线段上，`(5,0)` 共线但在端点范围外。退化线段 `(2,2)-(2,2)` 只包含同一点。坐标取极端 `long long` 时，减法扩宽仍不代表叉积乘积一定安全。

7. 坑

- 先用 `long long` 相减再转换：溢出已发生；必须像模板一样在减法前扩宽。
- 只检查共线不检查范围：会把整条直线上的点都判在线段上。
- 对整数几何套 EPS：整数叉积可直接比较 0，不需要浮点容差。
- 将 `__int128` 当无限精度或直接流输出：它有范围，输出时需自行转换成十进制字符串。

8. 一句话原理

用精确叉积判断共线和转向，再用坐标闭区间检查确认点落在线段范围内。

第十章

题意关键词索引

先用题意关键词定位候选模板，再核对数据范围和前提；关键词相似不代表模型一定相同。

初次学习中级数据结构，先看 学习路线与十二个重点专题，再按下表定位具体推导与模板。

10.1 上机、语言与 STL

题意/现场信号	优先查看	使用前核对
多组数据、第二组错、状态残留	IO、多测与状态	题面是否真的给 T；清空范围与总规模
算法可能超时、二维表可能爆内存	复杂度与内存速判	看最坏范围和总规模
溢出、精度、INF、负数取模	整数、浮点与边界	中间值而非只看最终答案
RE、UB、偶发崩溃	编译、日志与 Sanitizer	第一处错误；失效迭代器与递归栈
连续数组、邻接表、排序去重	`vector` 与 `array`	`reserve` 不改变 `size`；扩容失效
文本处理、字符序列	`string`	下标是字节位置
点权、边、异质记录	`pair`、`tuple`	字段较多时用有名 `struct`
两端插删、栈、队列、BFS	`deque` 与适配器	`pop` 不返回元素；空容器不可读
动态取最大/最小	`priority_queue`	默认最大堆；任意删除不便
前驱后继、有序动态集合	`set`、`multiset`	重复项删除语义；用成员二分
键值映射且要排序	`map`	`operator[]` 会插入默认值
频率表、状态判重、稀疏映射	哈希容器	无顺序保证；最坏退化与扩容
固定位集合、批量可达性	`bitset`	长度编译期确定；移位状态不越界
排序、等值范围、二分边界	排序与二分	区间有序且比较规则一致
去重、排列枚举	去重与排列	`unique` 不缩容；排列数可达 $n!$
区间求和、填充、局部谓词	数值算法与 Lambda	累加初值类型；捕获生命周期
迭代器边界、`end()`、算法迭代器要求	迭代器与半开区间	类别决定可用操作；容器修改可能使其失效

10.2 序列、搜索、DP 与数据结构

题意关键词	候选模板	区分信号
区间和、矩形和、批量区间加	前缀和与差分	静态查询用前缀；批量修改后统一恢复用差分
“最小可行/最大可行”、单调答案	二分答案	必须证明判定谓词单调
有序数组两端、连续区间、窗口	双指针与滑动窗口	指针只向前；和式窗口常要求非负数
值域很大但只关心相对次序	离散化	后续是否仍需原值距离
逆序对、跨左右两段统计	分治与归并计数	合并时计算跨区间贡献
区间最多选若干、按结束时间选	贪心与区间调度	交换论证；别凭局部直觉

题意关键词	候选模板	区分信号
连通块、枚举路径、递归状态	DFS	深度可达 n 时注意栈溢出
最少步数、分层扩展、等权图	BFS	入队即标记；状态空间是否过大
边权只有 0/1、最短路	0-1 BFS	0 边放队首，1 边放队尾
排列/组合枚举、满足约束的方案	回溯与剪枝	恢复现场；估算指数规模
要输出满足限制的对象而非计算最优值	常用构造题思路	先证明可行条件，再核对不变量和小规模样例
$n \approx 40$ 的子集枚举	折半搜索	两半状态能否排序/哈希合并
每件最多一次、容量限制	0-1 背包	容量倒序；恰好装满版无解返回空 optional；组限制需另写转移
每件无限次/有限次数	完全/多重背包	完全背包容量正序；多重拆分
最长严格/非严格上升子序列	LIS	严格用 `lower_bound`，非严格常用 `upper_bound`
两串保序公共序列、编辑操作	LCS / 编辑距离	子序列不要求连续
合并相邻段、括号/回文区间	区间 DP	按区间长度递增
树上选点、子树贡献	树形 DP	父子方向；是否要换根得到全点答案
每个点作根的答案	换根 DP	先子树 DP，再父到子传贡献
n 很小、集合状态	状压 DP	估算状态数与转移枚举
统计 $[0, N]$ 中满足数位限制的数	数位 DP	前导零、上界限制、记忆化维度
依赖无环、路径计数/最优值	DAG DP	拓扑顺序与不可达初值
随机过程的期望步数/收益	概率与期望 DP	线性方程、自环概率与终止条件
左/右首个更大更小、贡献范围	单调栈	严格与非严格比较符号
固长窗口最值、DP 滑窗优化	单调队列	队首过期；队尾维护单调性
动态连通、合并集合	并查集	不能直接处理删除；路径压缩+按秩合并
单点改、前缀和、区间和	树状数组	下标通常从 1 开始
区间加与区间求和	线段树（区间加）	懒标记合并、闭区间端点与建树范围
区间赋值、区间加与区间求和	线段树（区间赋值与加法）	赋值覆盖旧加法；之后的新加法累加到赋值值上
单点修改、自定义区间聚合	单点修改线段树与自定义合并	合并需满足结合律并提供双侧单位元；非交换合并须保持左右顺序
静态区间最值、幂等运算	ST 表	无修改；查询 $O(1)$
差值、相对距离、关系矛盾	带权并查集	先统一 `value[y]-value[x]` 的符号；模关系另作归一化
二维偏序、逆序对、区间不同数	离线树状数组	可重排查询；等值事件先后次序和严格/非严格关系
静态区间频率、平方和、可增删统计	莫队	单点增删是否足够快；数组是否静态
区间第 k 小、历史前缀频次	主席树	两前缀版本相减；离散值还原、排名与内存
区间最值划分、子数组最小值贡献	笛卡尔树	重复值归属；中序和堆序同时满足
多条直线最值、线性 DP 转移	李超线段树	查询坐标域、插线时机、比较乘积溢出
区间加、区间 GCD	差分 + GCD 线段树	首项前缀和与剩余差分 GCD 分开维护

题意关键词	候选模板	区分信号
动态转移、自动机、路径计数	线段树 + 矩阵	列向量的矩阵作用顺序；维数与结合律
第一个达标位置、非负前缀第 k 个	线段树上二分	摘要能否安全排除整段；前缀和版本要求非负
next greater、重复跳后继	倍增 + 单调栈	严格比较与哨兵；后继链排名不是区间第 k 大
区间 chmin/chmax、求和	Segment Tree Beats 入门	严格次大值；分情况修改与摊还分析，非普通懒标记

10.3 图与树

题意关键词	候选模板	使用前核对
依赖、先修、判有向环	拓扑排序	只适用于 DAG；输出数不足即有环
无权/等权最短路	BFS	多源时所有源同时入队
非负边最短路	Dijkstra	负边不可用；跳过堆中旧状态
有负边、判可达负环	Bellman–Ford	第 `n` 轮松弛；负环定义是否限源可达
全源最短路、点数较小	Floyd	$O(n^3)$；中转点必须最外层
无向图最小生成森林	Kruskal	本章模板用排序边集加并查集；不连通时得到森林
左右两类配对、任务分配	二分图最大匹配	左右编号独立；当前对象仅求解一次，重算需新对象
每条边恰走一次	欧拉路径/回路	度数条件、连通性、有向/无向版本
布尔选择与“至少一个”的二元约束	2-SAT	每条约束转为蕴含边；变量与否定编号
有向图互相可达、缩点	强连通分量	Tarjan 栈内标记与低链更新来源
删点/删边使无向图断开	割点与桥	返祖边、重边、DFS 根的特殊规则
子树区间、树上离线统计	DFS 序/子树展平	进入/退出时间及区间端点
子树颜色/频次统计、保留重儿子信息	DSU on Tree	DFS 位置映射回节点；统计结构按颜色/深度等维度维护；计入单次增删代价
合并子树集合、不同颜色、众数	启发式合并	小容器所有权转移；保存答案后再消费子树集合
合并子树频次、第 k 小	线段树合并	相同值域；破坏性合并不得与持久化共享节点混用
树上路径第 k 小	主席树四根公式	`u+v-lca-parent(lca)`；版本 0 为空
多询问、每次少量关键点	虚树	补相邻 DFS 序关键点的 LCA；压缩边代表原树路径
多条路径批量加、经过次数	树上差分 + LCA	点差分与边差分不同；最终子树汇总
祖先、两点树上距离	倍增 LCA	深度对齐；预处理根与倍增层数
树上最远点、最长路径	树的直径	负边权时两次最远点法需重新确认
容量、可同时输送的最大量	Dinic 最大流	反向边下标、残量网络、`long long` 容量
单位费用、在最大流下最小成本	最小费用最大流	负费用、势能/最短路与溢出

题意关键词	候选模板	使用前核对
任意子树/路径修改查询	树链剖分	点权按 pos 映射；边权排除 LCA；非交换合并需补方向
临时合并、撤销、递归恢复	可撤销并查集	禁止路径压缩；快照来自当前记录栈
动态加删边、离线连通性	线段树分治	存活时间半开区间；重复边和无效删除的语义
静态树路径频次、可离线	树上莫队入门	双次欧拉序的奇偶抵消；按情况补 LCA
森林在线连边断边、路径聚合	LCT 入门	原图必须保持森林；Splay 和路径翻转为先修

10.4 数学、字符串与几何

题意关键词	候选模板	使用前核对
最大公约数、线性丢番图	GCD / 扩展 GCD	系数与中间乘积范围
矩形面积并、区间覆盖长度	扫描线 + 线段树	叶子是坐标间隔；先累积面积再处理同 x 事件
大指数取模	快速幂	模数、负底数与乘法溢出
多次判素、列素数或查询 `[1,n]` 正因数个数	素数筛	上界内存；线性筛或埃氏筛；计数查询限定在筛表范围
单个数分解、约数个数/和	质因数分解与约数公式	试除到 $p^2 \le n$；剩余大质因子
1..n 中与 n 互质个数	欧拉函数	分解质因子后乘法顺序
除法取模、解同余	模逆元 / 线性同余	逆元存在当且仅当互质（一般模数）
多个余数条件合并	CRT / 扩展 CRT	模数可否不互质；合并是否无解
组合数、计数取模	阶乘逆元 / Pascal	模数是否质数，n 与模数大小
至少一个、排除重复计数	容斥原理	交集项符号和子集规模
线性递推、走固定步数	矩阵快速幂	状态向量方向、乘法次序、模数
实数线性方程组	高斯消元	浮点 EPS；检查无解与多解
必胜/必败、若干独立游戏	SG 函数 / Nim	是否公平组合游戏，mex 的可达集合
子串相等、回文快速判定	字符串哈希	碰撞风险、下标与模数
单模式匹配、前后缀	KMP / 前缀函数	失配跳转和匹配位置换算
前缀与每个后缀的 LCP	Z 函数	Z-box 右端点约定
最长回文子串	Manacher	插入分隔符后的下标换算
小写字母串前缀查询/匹配	Trie	本章模板字符集为 `a–z`；节点内存与字符集大小
多模式匹配	AC 自动机	先插入全部模式再 build 一次；逆序累计 fail 贡献，允许重叠和重复模式
所有后缀排序、重复子串	后缀数组	rank/LCP 定义与 0/1-based
左右方向、共线、线段相交	叉积与方向判断	整数叉积溢出；浮点 EPS
直线交点、点到线距离/投影	直线与投影	平行、重合和退化线段
多边形面积、点在多边形内	多边形	顶点顺序；边界点如何定义
最小凸多边形包住所有点	凸包	共线点保留策略、重复点
凸包最远点对	旋转卡壳	输入须为按序凸包；退化点数

题意关键词	候选模板	使用前核对
圆与直线/圆与圆交点	圆基础	相切误差、无穷多交点与退化半径

10.5 五秒模型核对

```
1. 数据范围允许这个复杂度吗？多测看总规模。
2. 模板前提满足吗：有序、单调、非负、无环、二分、连通？
3. 题目要的是值、方案、数量，还是字典序最小方案？
4. 下标、区间、无解状态、重边、自环、负权、取模是否对齐？
5. 复制模板后，哪三个位置必须按本题修改？
```